THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal
Spécialité : Nano électronique et Nano technologies
Unité de recherche : CEA /LIST - Laboratoire d'intégration de systèmes et de technologies

Conception d'un Système Mémoire pour Traitement de Données Creuses

Design of a Memory System for Sparse Data Processing

Présentée par :

Valentin ISAAC--CHASSANDE

Direction de thèse :

Frédéric ROUSSEAU PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA	Directeur de thèse
Adrian EVANS CEA	Co-encadrant de thèse

Rapporteurs :

Roselyne Chotin
PROFESSEURE DES UNIVERSITES, Laboratoire LIP6/CIAN
Olivier Sentieys
PROFESSEUR DES UNIVERSITES, IRISA/INRIA

Thèse soutenue publiquement le **11 mai 2026**, devant le jury composé de :

Frédéric ROUSSEAU, PROFESSEUR DES UNIVERSITES, Université Grenoble Alpes	Directeur de thèse
Roselyne Chotin, PROFESSEURE DES UNIVERSITES, Laboratoire LIP6/CIAN	Rapporteure
Olivier Sentieys, PROFESSEUR DES UNIVERSITES, IRISA/INRIA	Rapporteur
Yves Robert, PROFESSEUR DES UNIVERSITES, Ecole Normale Supérieure de Lyon	Examinateur
Benoit Dupont de Dinechin, DIRECTEUR DE RECHERCHE, Société Kalray	Examinateur

Invités :

Adrian Evans
DOCTEUR EN SCIENCES, CEA

Valentin ISAAC--CHASSANDE
Université Grenoble Alpes
CEA Grenoble (DRT/LIST/DSCIN/LSTA)
TIMA Laboratory (MADMax)
ORCID : 0009-0007-9842-5777

THESIS

DESIGN OF A MEMORY SUB-SYSTEM FOR SPARSE DATA PROCESSING

Thesis Adviser: Pr. Frédéric ROUSSEAU
Thesis Co-Supervisors: Dr. Adrian EVANS
Dr. Yves DURAND

Thesis defended publicly on: May 11th, 2026

Rapporteurs: Pr. Roselyne CHOTIN
Pr. Olivier SENTIEYS
Jury Members: Benoit DUPONT DE DINECHIN
Pr. Yves ROBERT
Pr. Frédéric ROUSSEAU

EEATS Doctoral College, NENT Speciality
2022 – 2026 Academic Years

ACKNOWLEDGEMENTS

TODO

"Work. Or school. This is for the money. But what is anything else, for that matter, but required means of enhancing your bare living – better sleep and better food – like spending decades just to turn a little knob higher on a pathetic radio called existence."

Karl Kristian Flores, The Goodbye Song.

ABSTRACT

SPARSE matrices are ubiquitous in data-intensive applications such as scientific computing, graph analytics, and machine learning, yet their irregular memory access patterns pose significant challenges for traditional computing architectures. This thesis addresses the core challenge of designing hardware and architectural mechanisms to efficiently support sparse data processing, focusing on scalability, adaptability, and high performance.

We analyze the fundamental algorithms for sparse matrix multiplication, *Inner-Product*, *Outer-Product*, and *Gustavson*'s, and identify their hardware challenges, separated into the *gather* and *scatter* problems. *Gustavson*'s algorithm emerges as the most promising for hardware implementation due to its balanced trade-offs, while the *Outer-Product* algorithm is well-suited for Matrix-Vector kernels.

Our main contribution is SpDCache, a region-based architecture that optimizes memory transactions for sparse data by leveraging the structural properties of banded matrices. SpDCache partitions cache memory into in-band and out-of-band regions, significantly reducing memory traffic compared to generic caches. Key contributions include:

- Architectural Design and Modeling: A Python-based simulation model and a probabilistic C-based model analyze SpDCache's memory transactions, revealing the importance of coarse-grain matrix structure (*ex.*, banded matrices).
- Microarchitectural Implementation: RTL-level simulations validate SpDCache's effectiveness in reducing memory traffic and improving cache utilization, with virtual partitioning ensuring flexibility and adaptability. Further analysis reveals the importance of fine-grain matrix structure in optimizing performance (*e.g.*, cache-line density).
- Optimizations and Scalability: Extensions to SpMSpM kernels and multi-core architectures position SpDCache as a versatile, scalable solution for modern sparse matrix processing.

This thesis advances the state-of-the-art in cache architectures for sparse data, emphasizing intrinsic understanding and adaptive, structure-aware design for high-performance hardware solutions.

RÉSUMÉ EN FRANÇAIS

LES matrices creuses sont omniprésentes dans les applications gourmandes en données, telles que le calcul scientifique, l'analyse de graphes et l'apprentissage automatique. Pourtant, leurs schémas d'accès mémoire irréguliers posent des défis majeurs pour les architectures d'ordinateurs et de mémoires traditionnelles. Cette thèse aborde le défi central de la conception de mécanismes matériels et architecturaux pour prendre en charge efficacement le traitement des données creuses, en mettant l'accent sur la scalabilité, l'adaptabilité et les hautes performances.

Nous analysons les algorithmes fondamentaux de multiplication de matrices creuses, à savoir *Inner-Product*, *Outer-Product* et *Gustavson*, et identifions leurs défis matériels, séparés en problèmes de *gather* (recherche d'intersections) et de *scatter* (réductions aléatoires). L'algorithme de *Gustavson* s'impose comme le plus prometteur pour une implémentation matérielle en raison de ses compromis équilibrés, tandis que l'algorithme *Outer-Product* est particulièrement adapté aux opérations matrice-vecteur.

Notre contribution principale est SpDCache, une architecture de cache de réduction basée sur des régions qui optimise les transactions mémoire pour les données creuses en exploitant les propriétés structurelles des matrices bandées. SpDCache partitionne la mémoire cache en régions en-bande et hors-bande, réduisant significativement le trafic mémoire par rapport aux caches génériques. Les contributions clés incluent :

- Conception architecturale et modélisation : Un modèle de simulation basé sur Python et un modèle probabiliste en C analysent les transactions mémoire de SpDCache, révélant l'importance de la structure matricielle à gros grain (par exemple, les matrices bandées).
- Implémentation microarchitecturale : Des simulations au niveau RTL valident l'efficacité de SpDCache dans la réduction du trafic mémoire et l'amélioration de l'utilisation du cache, avec un partitionnement virtuel de la mémoire garantissant flexibilité et adaptabilité. Une analyse approfondie révèle l'importance de la structure matricielle à petit grain pour optimiser les performances (en particulier la densité des lignes de cache).
- Optimisations et scalabilité : Des extensions aux multiplications matrice-matrice et aux architectures multi-cœurs positionnent SpDCache comme une solution polyvalente et évolutive pour le traitement moderne des matrices creuses.

Cette thèse fait progresser l'état-de-l'art des architectures de cache pour les données creuses, en mettant l'accent sur une compréhension intrinsèque et une conception adaptative et sensible à la structure pour des solutions matérielles hautes performances.

LIST OF PUBLICATIONS

This thesis is based on the following publications:

I Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. Reference [58]

II Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3-7. Reference [59]

III Valentin Isaac--Chassande, Adrian Evans, and Yves Durand. 2025. Dispositif de calcul informatique à mémoire cache optimisée pour le calcul matriciel. *Commissariat à l'Energie Atomique et aux Energies Alternatives*, N° EP4650972, FR3162293 (Nov. 2025). References [56] and [57]

The papers and patent will be referred to in the thesis using their Roman numerals.

SUMMARY

NOTATIONS

General Notations

$[a, b]$	Real interval, *e.g.*, $[a, b]$ contains any real value in $\mathbb{R}$ ranging between $a \in \mathbb{R}$ and $b \in \mathbb{R}$, both included
$[\![a, b]\!]$	Integer interval, *e.g.*, $[\![a, b]\!]$ contains any integer value in $\mathbb{Z}$ ranging between $a \in \mathbb{Z}$ and $b \in \mathbb{Z}$, both included
$a\%b$	a modulo b
$\{a, b, c\}$	Set, containing a, b and c in this example
0.00 e-0	Scientific notation, equivalent with 0.00×10^{-0}
$a \mathrel{+}= b$	Accumulation, *e.g.*, a gets $a + b$
$\lvert x \rvert$	Absolute value of x
$\lceil x \rceil$	Ceil value of x

Matrix Related

A, B, C	Matrices
b, c	Vectors
M, N, K	Dimensions of matrices and vectors, *ex.*, $A(M, K)$, $c(N)$
i, j, k	Matrix and vectors indices (for M, N and K, respectively), *ex.*, $A_{i,k}$, c_j
nnz	Number of non-zero elements; can be annotated, *ex.*, nnz_A or $nnz(A)$
$\overline{nnz_r}$, $\overline{nnz_c}$	Average number of non-zero elements per row or column, respectively; can be annotated, *ex.*, $\overline{nnz_r}(A)$
d	Density, the ratio of non-zero values; can be annotated, *ex.*, d_A or $d(A)$
d_{IB}, d_{OB}	Densities of in- and out-of-band regions of a matrix
w_B	*Band-width* of a matrix, that is the width of the diagonal band
w_{hB}	Half-*band-width* of a matrix, such that $w_B = 2 \times w_{hB} - 1$
d_{cd}	Average density of non-empty lines of a cache, or vertical-stripes of a matrix, ranging between $\frac{1}{l}$ and 1

Sparse Format Related

ptr, idx, val	Storages tables used for CSR and CSC formats; can be annotated, *ex.*, $A.idx$, $A.ptr$, $A.val$
pos	Index for the ptr table

Cache Related

S, W, l	Cache characteristics, *e.g.*, the number of *sets*, the number of *ways* and the number of elements per cache-line, respectively
s	Index for a *set* of the cache, ranging between 0 and $S-1$; can be annotated, *ex.*, s_{region} for a *set* in a cache region
s_C	Size of the cache in number of elements, such that $s_C = S \times W \times l$; can be annotated, *ex.*, $s_C(GRC)$
s_{DRC}, s_w, s_l	Size in bytes of the *DRC*, a word and a cache-line, respectively (used once in Section 6.2.3)
L_1 to L_4	Indices for cache-lines (used once in Section 4.4.1)

Analytical Model Related (Chapter 4)

$normMT$	Memory traffic normalized over nnz
M_l	The dimension M divided by the cache-line size, such that $M_l = \lceil \frac{M}{l} \rceil$
κ	Index of the M_l dimension, from the current iteration
$DCDT$	Disjoint Cache Density Table, the cache representation, of size M_l; can be indexed, *ex.*, $DCDT_\kappa$
MDT	Matrix Density Table, the matrix representation, of size (M_l, M); can be indexed, *ex.*, $MDT_{\kappa,j}$
d_{lM}	Disjoint-density of a vertical-stripe, equal to 0 or ranging between $\frac{1}{l}$ and 1; can be annotated, *ex.*, $d_{lM}(\kappa)$
$nnzcl$	Number of valid (non-empty, *e.g.*, non-zero) cache-lines; can be indexed, *ex.*, $nnzcl_\kappa$
$miss(\kappa)$	A miss occurs at the current iteration
$evict(\kappa)$	An eviction occurs at the current iteration
$nbEvict$	Total number of evictions
$nbFlush$	Total number of cache-lines flushed

Others

KiB, MiB, GiB	Kibibyte (1024^1), Mebibyte (1024^2) and Gibibyte (1024^3), respectively
25/50/25%	Regions Configuration for SpDCache, *ex.*, 25% of total size for the *GRC*, 50% for the *DRC* and 25% for the *SRT*

GLOSSARY

band Refers to an region of a matrix that is concentrated around the diagonal. 3, 9, 11, 12, 19, 57, 60, 61, 62, 64, 66, 70, 71, 72, 73, 74, 75, 76, 77, 78, 89, 90, 100, 108, 109, 110, 111, 138, 142

banded Refers to a matrix structure where the non-zero elements are concentrated around the diagonal. 3, 18, 19, 20, 30, 32, 52, 53, 57, 60, 64, 65, 66, 67, 70, 72, 73, 75, 77, 78, 89, 92, 100, 101, 108, 113, 114, 142

band-width With this typography, refers to the size of the band of a matrix (w_B), not to be confused with memory 'bandwidth'. 9, 19, 66, 70, 71, 72, 73, 75, 76, 90, 100, 106, 109, 110, 111, 112, 138

common rank Refers to a rank that is shared between two tensors. 21, 22, 27, 29

dense format Is the uncompressed format where all values of a matrix (zeros and non-zeroes) are stored directly in memory. 19, 26, 28

eviction Is a cache event where a valid cache entry is flushed and freed. 10, 41, 50, 51, 52, 53, 54, 55, 56, 57, 59, 60, 61, 62, 63, 64, 66, 67, 69, 70, 71, 72, 73, 74, 75, 76, 78, 81, 83, 84, 86, 87, 88, 96, 97, 99, 101, 102, 107, 108, 109, 111, 142

fiber Is the generic term to describe a one-dimensional stream of data or metadata. It corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata. 25, 29, 128

gather Refers to reading data from memory. We specifically use this term to categorize data transfers that are irregular. 3, 4, 27, 29, 32, 34, 35, 36, 37, 43, 46, 48, 49, 50, 51, 116, 117, 118, 125, 131

high-level Refers to an implementation, modeling approach, or language that is easily understood by the user, in general subject to many simplifications. 15, 16, 23, 32, 53, 78, 80, 127, 130

hit Is a cache event where a given address matches one of the valid cache entries. 41, 53, 54, 56, 58, 59, 60, 70, 75, 81, 83, 84, 86, 87, 88, 91, 95, 96, 99, 100, 101, 102, 107, 108, 109, 111, 123

ineffectual operation Is an operation that leads to a neutral result, 0 for an addition or 1 for a multiplication. 25, 38, 40

intersection Refers to the process of finding non-zero partial sums, when computing a matrix multiplication. Typically this involves finding the overlapping non-zero values in the input matrices. 21, 22, 24, 25, 26, 27, 29, 30, 32, 34, 38, 39, 40, 43, 49, 50, 132, 133, 136, 145

irregular access See "random" access. 11, 17, 18, 32, 53, 57, 66, 94, 108

irregular data Refers to data that has low spatial and temporal locality, thus being more likely to generate irregular accesses. 15, 32, 54, 133

metadata Is data that provides information about how to interpret the main data, such as index arrays of sparse formats. 11, 12, 19, 20, 25, 27, 38, 39, 129

miss Is a cache event where a given address does not match any of the valid cache entries. 10, 26, 30, 41, 53, 54, 55, 56, 58, 59, 60, 69, 70, 81, 83, 86, 87, 88, 91, 107, 108, 123, 130, 133

partial sum Refers to the intermediary result before updating the output, when computing a matrix multiplication. 11, 12, 22, 25, 27, 34, 39, 130, 131, 132, 133, 134, 135, 136

'perfectly' banded Refers to a matrix structure where all the non-zero elements are fully concentrated within a delimitated area around the diagonal (the band). 19, 66, 68, 70, 71, 72, 73, 77, 100, 101

pipeline Is a sequence of processing stages where instructions flow through each stage in order. 81, 82, 83, 84, 85, 86, 87, 88, 102, 103, 105, 107, 123, 131, 143

"random" access Is a memory access that occurs on an arbitrary or unpredictable memory address (not necessarily conscutive in memory). 11, 12, 18, 48

"random" update Is a "random" access where the access is a read, modify and write on a given address. It is similar to a "random" *reduction*. 26

rank Refers to a dimension of a tensor, such as a vector or a matrix. 11, 19, 21, 22, 24, 25, 29

reduction Refers to the process of updating an entry. Typically, in this thesis, a *reduction* consists of adding a partial sum to an output vector: $c_i \mathrel{+}= partial\_sum$. 12, 15, 16, 22, 27, 28, 29, 30, 31, 32, 34, 40, 41, 42, 43, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 64, 74, 80, 81, 82, 84, 85, 88, 91, 97, 100, 101, 102, 103, 106, 107, 108, 109, 110, 111, 113, 127, 129, 131, 142, 145

reduction cache Is a data-cache with hardware support for *reduction* operations. 3, 41, 54, 55, 56, 57, 60, 61, 62, 63, 64, 65, 66, 67, 69, 70, 71, 72, 73, 74, 77, 78, 82, 83, 85, 88, 102, 113

scatter Refers to updating data in memory. We specifically use this term to categorize data transfers that are irregular. 3, 4, 27, 29, 32, 34, 35, 36, 37, 41, 42, 43, 46, 48, 49, 50, 51, 116, 117, 118, 119, 121, 125, 131

sequential accesses Are a group of memory accesses (from a data array, for example) that occur on conscutive memory addresses, in order through time. 19, 53

set Refers to a fixed number of cache-lines, a subdivision related to *set*-associative caches. 10, 12, 53, 56, 57, 61, 63, 64, 66, 67, 68, 69, 72, 74, 76, 77, 78, 81, 83, 84, 89, 101, 106, 131

sparse format Is a compression format for sparse matrices, which avoids storing zeros by using metadata. Common sparse formats include COO, CSR and CSC. 12, 19, 20, 21, 23, 25, 27, 32, 44, 50, 55, 127, 129, 135, 136

tag Refers to cache metadata that allows identification of cache-line memory addresses. 53, 61

tile Refers to a sub-region of a *tiling* process. In the simplest case, this could correspond to a rectangular region in a matrix. 24, 25, 30, 37, 38, 39, 40, 130, 132, 135, 136

tiling Is a method consisting in splitting data into several independent sub-regions to be processed separately. 12, 24, 25, 27, 29, 30, 32, 38, 39, 40, 45, 49, 118, 133, 134, 135, 136, 142, 145

way Refers to the number of cache-lines that a *set* contains. 10, 53, 56, 61, 66, 76

ACRONYMS

AI Artificial Intelligence. 108

ASIC Application-Specific Integrated Circuit. 34, 46, 127

BaCSC *Banded Compressed Sparse Column*. 20, 32, 89, 90, 92, 94, 95, 96, 97, 98, 99, 110, 143, 146

BCSR *Blocked Compressed Sparse Row*. 20

CAM Content Addressable Memory. 40, 84, 86, 87, 88, 119, 130, 133

CMO Cache Management Operation. 81, 82, 84

CNN *Convolutional Neural Network*. 136

COO *COOrdinate*. 12, 19, 20, 28, 32, 40, 45, 115, 142

CSC *Compressed Sparse Column*. 10, 12, 20, 25, 26, 27, 28, 29, 32, 37, 46, 49, 55, 66, 89, 90, 91, 94, 96, 97, 98, 106, 107, 109, 111, 115, 127, 134, 142, 145

CSF *Compressed Sparse Fiber*. 132

CSR *Compressed Sparse Row*. 10, 12, 20, 25, 26, 27, 28, 29, 32, 37, 38, 40, 46, 49, 89, 115, 127, 132, 134, 142, 145

DIA *DIAgonal*. 20

DMA Direct Memory Access. 134

DNN *Deep Neural Network*. 127, 131, 133, 136

DRAM Dynamic Random-Access Memory. 46, 49, 129, 130, 133, 135

DRC *Dense Region Cache*. 10, 57, 58, 59, 60, 61, 62, 63, 64, 70, 71, 75, 77, 79, 80, 83, 84, 85, 86, 87, 88, 89, 90, 93, 94, 95, 96, 99, 101, 102, 106, 107, 108, 109, 110, 111, 112, 119, 120, 121, 122, 123, 124, 142

ELL *ELLPACK*. 20

FIFO First-In First-Outs Memory. 131, 133

FPGA Field-Programmable Gate Array. 34, 43, 44, 46, 127, 130, 131, 132, 134, 135, 136

FPU Floating-Point Unit. 105, 132

GC Generic Cache. 60, 61, 62, 63, 82, 88, 89, 90, 91, 94, 95, 96, 97, 98, 99, 101, 106, 120, 123, 143

GNN *Graph Neural Network*. 132, 135, 136, 137

GRC *Generic Region Cache*. 10, 57, 58, 60, 62, 63, 64, 66, 67, 79, 80, 83, 85, 88, 89, 101, 102, 107, 108, 109, 119, 120, 121, 122, 123, 124, 142

`Gust` *Gustavson*. 3, 4, 21, 22, 23, 24, 27, 29, 30, 31, 32, 34, 35, 36, 37, 40, 41, 44, 47, 48, 49, 50, 51, 108, 113, 116, 117, 118, 124, 125, 129, 130, 131, 132, 133, 134, 135, 143, 144, 145

HBM High Bandwidth Memory. 44, 130, 131, 134, 135, 136

HYB *HYBbrid*. 20

IBR *In-Band Region*. 57, 58, 60, 64, 85, 86, 87, 88, 90, 107, 121, 122, 143

`IP` *Inner-Product*. 3, 4, 21, 22, 23, 24, 25, 26, 27, 29, 30, 31, 32, 34, 35, 36, 37, 38, 39, 40, 44, 48, 49, 50, 51, 68, 116, 117, 118, 125, 130, 131, 133, 134, 144, 145

L1 First-Level Cache. 109, 110, 111, 135

L2 Second-Level Cache. 109, 111, 135

LLC Last-Level Cache. 109, 110, 111, 125

LRU *Least Recently Used*. 53, 61, 66, 76, 77, 119, 122

MM Matrix-Matrix. 21, 22, 23, 24, 29, 32, 33, 34, 35, 36, 37, 43, 44, 47, 48, 49, 50, 51, 113, 116, 117, 118, 125, 133, 136, 145

MV Matrix-Vector. 3, 21, 22, 23, 24, 29, 32, 33, 34, 35, 36, 37, 43, 44, 49, 50, 51, 113, 116, 117, 127, 136, 145

nop *no-operation*. 81

OBR *Out-of-Band Region*. 58, 59, 60, 64, 85, 87, 88, 90, 107, 121, 122, 143

`OP` *Outer-Product*. 3, 4, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 34, 35, 36, 37, 38, 41, 42, 44, 48, 49, 50, 51, 52, 53, 55, 56, 57, 60, 62, 64, 65, 66, 67, 68, 69, 70, 72, 77, 78, 89, 90, 91, 92, 94, 97, 106, 107, 108, 113, 116, 117, 118, 119, 121, 122, 123, 124, 125, 129, 130, 131, 132, 133, 134, 135, 136, 141, 142, 143, 144, 145, 146

PE Processing Element. 37, 38, 39, 40, 41, 127, 129, 130, 131, 132, 133, 134, 135, 136

PO *partial output*. 22, 23, 24, 26, 27, 29, 32, 37, 38, 40, 41, 44, 48, 63, 110, 129, 132, 135

PT Processing Tile. 37, 38

RAM Random-Access Memory. 81, 82, 83, 84, 86, 87, 88

RED *Reduction*. 54, 55, 56, 58, 60, 62, 64, 81, 83, 84, 85, 86, 87, 88, 90, 91, 96, 99, 101, 102, 106, 107, 108, 146

RedC Reduction Cache. 60, 61, 62, 63, 64, 82, 88, 89, 90, 91, 93, 94, 95, 96, 97, 98, 99, 100, 101, 106, 143

REG *Region*. 84, 85, 90, 107

ReRAM Resistive Random-Access Memory. 127

RMW *Read-Modify-Write*. 58, 59, 60, 61, 62, 74, 75, 84, 88, 105, 109, 110, 119

RMWQ *Read-Modify-Write Queue*. 58, 59, 84, 88

RTL Register Transfer Language. 3, 4, 16, 46, 47, 64, 80, 81, 82, 84, 88, 89, 94, 95, 96, 99, 100, 101, 102, 103, 104, 105, 106, 111, 122, 123, 124, 125, 138, 139, 140, 141, 143, 144

SIMD Single-Instruction Multiple-Data. 132

SpDC SpDCache. 3, 4, 10, 34, 45, 47, 53, 57, 58, 59, 60, 61, 62, 63, 64, 66, 67, 70, 71, 74, 77, 78, 79, 80, 81, 82, 83, 84, 85, 88, 89, 90, 91, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 116, 117, 119, 120, 121, 122, 123, 124, 125, 141, 142, 143

SpGEMM General Sparse Matrix-Matrix multiplication. 21, 116, 128, 129, 130, 131, 132, 133, 134, 136

SpGEMV General Sparse Matrix-Vector multiplication. 21

SPMD Single-Program Multiple-Data. 37, 38

SpMM Sparse Matrix-Dense Matrix multiplication. 21, 108, 116, 128, 132, 133, 134, 135, 137

SpMSpM Sparse Matrix-Sparse Matrix multiplication. 3, 21, 23, 25, 26, 27, 28, 29, 30, 31, 32, 37, 38, 40, 41, 104, 108, 109, 111, 114, 116, 117, 118, 120, 124, 125, 131, 133, 135, 136, 143, 144, 145

SpMSpV Sparse Matrix-Sparse Vector multiplication. 21, 108, 109, 116, 133, 134, 143

SpMV Sparse Matrix-Vector multiplication. 21, 24, 31, 32, 37, 41, 42, 51, 52, 53, 55, 56, 57, 60, 62, 64, 65, 66, 67, 68, 70, 72, 77, 78, 90, 91, 92, 94, 97, 104, 105, 106, 107, 108, 111, 113, 116, 117, 118, 119, 121, 122, 123, 124, 125, 127, 128, 130, 133, 134, 136, 141, 142, 145, 146

SRAM Static Random-Access Memory. 127, 133, 135

SRT *Sparse Region Table*. 10, 57, 58, 59, 60, 61, 62, 63, 64, 77, 79, 80, 83, 84, 85, 86, 87, 88, 89, 93, 94, 99, 101, 102, 107, 109, 119, 120, 121, 122, 123, 124, 142

INTRODUCTION

THE rapid growth of data-intensive applications has made sparse and irregular data structures increasingly prevalent in scientific computing, machine learning, and graph analytics. However, efficiently processing sparse matrices remains a fundamental challenge due to their inherent irregularity, which leads to unpredictable memory access patterns, limited opportunities for data reuse, and suboptimal utilization of hardware resources. Existing hardware accelerators have demonstrated various solutions, yet they often rely on restrictive assumptions, exhibit limited scalability, or fail to fully exploit the memory bandwidth.

This thesis addresses the following central question:

PROBLEM STATEMENT

How can hardware and architectural mechanisms be designed to efficiently support computation with sparse data, while ensuring scalability, adaptability, and high performance across diverse workloads?

To tackle this problem, we investigate the limitations of existing algorithms and accelerators, propose a novel cache architecture with support for read-accumulate-write operations, tailored to sparse data. We then evaluate the behaviour of this specialized cache through modeling, simulation, and hardware prototyping.

The manuscript is structured as follows. We begin with an in-depth discussion of irregular data, with a particular emphasis on sparse matrices (Chapter 1). This includes a study of fundamental sparse matrix multiplication kernels and their associated algorithms, highlighting the main hardware challenges that arise when dealing with sparse data. A high-level analysis of algorithms for sparse data computing is provided, with a focus on basic storage schemes and opportunities for data reuse. Building on this foundation, we review the state-of-the-art in hardware accelerators designed for efficient sparse matrix computing (Chapter 2). This review identifies common characteristics, proposes a classification of existing accelerators and design tendencies, offers a comparative analysis, and identifies the major tendencies across designs. This analysis guided our subsequent architectural choices.

Following this survey, we introduce a novel cache architecture specifically designed to address the challenge of read-accumulate-write operations on sparse data, called "random" *reductions* (Chapter 3). The proposed cache integrates *reduction* support and organizes its storage into multiple regions optimized for denser and sparser data regions. A Python model

was developed to simulate memory transactions in this cache and to compare its behavior with that of a generic cache. To further investigate its performance, we complement this analysis with a probabilistic model, that enables rapid simulation and design parameters exploration depending on matrix structure (Chapter 4).

At the micro-architectural level, we detail the virtual partitioning of cache memory into multiple configurable regions, which ensures both flexibility and adaptability to diverse workloads (Chapter 5). The proposed cache is then evaluated through RTL simulation, which reveals two distinct classes of matrices that exhibit different performance profiles. Finally, the manuscript outlines potential future enhancements to improve both performance and scalability, including a multicore extension of the cache architecture and optimizations targeting better usability (Chapter 6).

Contributions of the Thesis

The main contributions of this thesis can be summarized as follows:

1. **Analysis of sparse data computing** (Chapter 1)
   - Study of fundamental sparse matrix multiplication kernels and algorithms.
   - Identification of the hardware challenges associated with sparse data processing.
   - High-level analysis of algorithms for sparse data computing, with emphasis on storage schemes and data reuse opportunities.
2. **Survey and classification of state-of-the-art accelerators** (Chapter 2)
   - Comprehensive review of hardware accelerators for sparse matrix computing.
   - Identification of common architectural features and design tendencies.
   - Comparative analysis to explain design choices under varying constraints.
3. **Design of a cache architecture for "random" *reductions*** (Chapter 3)
   - Proposal of a cache with native *reduction* support, capable of handling both dense and sparse data via multiple cache regions.
   - Development of a Python-based simulation model to analyze memory transactions and benchmark against a generic cache.
4. **Probabilistic modeling and exploration of cache behavior** (Chapter 4)
   - Implementation of a lightweight C-based probabilistic model for rapid simulation.
   - Exploration of cache dimensioning and behavior across diverse workloads.
5. **Micro-architecture and evaluation through RTL simulation** (Chapter 5)
   - Introduction of a memory partitioning scheme enabling multiple configurable regions.
   - Assessment of the proposed cache architecture using RTL-level simulation.
   - Identification of two groups of sparse matrices leading to different performance profiles.
6. **Future directions for performance and scalability** (Chapter 6)
   - Extension of the cache architecture to multicore systems for improved scalability.
   - Optimization techniques to enhance usability and practical deployment.

CHAPTER

BACKGROUND ON SPARSE MATRIX COMPUTING

1

Contents

1.1 Introduction

MODERN computing systems are designed to deliver high performance by exploiting memory hierarchies, parallelism, and increasingly sophisticated hardware features. However, the efficiency of these systems is strongly tied to the regularity of memory access patterns. Applications with predictable, sequential data access can fully benefit from cache locality, prefetching mechanisms, and vectorization. In contrast, applications characterized by irregular accesses, where memory locations are not known in advance or follow non-contiguous patterns, are not able to achieve peak performance.

In domains such as graph analytics, sparse linear algebra, unstructured mesh computations, and database queries, the memory access patterns are often irregular. In these applications, pointer chasing, indirect indexing, and dynamically changing data structures disrupt spatial and temporal locality. As a result, the hardware struggles to hide the latency of memory accesses, leading to poor cache utilization and underutilization of compute ressources.

The impact of irregular accesses extends beyond raw performance. They also complicate algorithm design, hinder scalability on many-core architectures, and increase energy consumption due to repeated off-chip memory requests. Understanding and addressing these challenges is therefore a central concern in both computer design and application optimization. This chapter provides the necessary background for analyzing irregular accesses, highlighting their sources, effects, and the architectural considerations they expose, focusing on the case of indirect indexing in sparse matrix computation. The content of this chapter is partially based on the content from Papers I and II.

DEFINITION

Irregular accesses, or **"random" accesses**, are sequences of accesses where the sequence can not be easily predicted.

1.2 Sparse Matrices

SPARSE matrices are two dimensional arrays filled with many zeros and stored in compressed formats in memory. These sparse matrices are commonly used in diverse fields such as linear algebra, where sparse matrix multiplication is used to solve linear systems of equations for scientific purposes or simulation, as well as graph analytics where the vertices of a graph are represented as a sparse matrix [17], or finally transformers or other similar AI features where weights and activations can be represented and processed as sparse matrices [31, 36]. In these contexts, matrices are large, with dimensions up to billions of elements, and highly sparse, with non-zero densities down to 10^{-7} [67].

We note matrices with capital letters A, B, C, conversely to vectors a, b, c. Their dimension are denoted using M, N and K. For example, we can define a square matrix A of dimension M, or a matrix B of dimension (K, N). We note nnz the number of non-zero elements of a matrix, and d the associated density. With the example of a matrix $A(M, N)$:

$$d_A = \frac{nnz_A}{M \times N}$$

DEFINITION

A **sparse matrix** is a matrix filled with many zeros.

NOTE

In general, we consider a matrix to be highly sparse when its number of non-zero elements has the same order of magnitude than its dimension, $nnz \sim M$. But there is no intrinsic definition of the frontier between a sparse and a dense matrix. In this manuscript, we consider a matrix as sparse when it needs to be compressed in memory (see Section 1.3).

1.2.1 Banded Matrices

Matrices used in scientific computation originate from physical modelling where a system of partial differential equations is discretized, resulting in a large sparse matrix. In these systems, the forces between elements decreases quickly with their distance, creating banded matrices. Moreover, numerical techniques exist to transform matrices into banded matrices [79, 83, 117].

A banded matrix is a matrix which has a dense region around its main diagonal and which is sparser away from the diagonal. To give a few visual examples, Figure 1.1 shows the structure plots for four typical matrices found in the SuiteSparse Matrix Collection [67], having a high-density band along the diagonal. In Figure 1.2, we illustrate a banded, square matrix of dimension M. We define w_B as the width of the banded region, or *band-width*, as shown in *red* in the figure. Sometimes it is easier to refer to the half-*band-width*, w_{hB}, which respects the relation $w_B = 2 \times w_{hB} - 1$. The average density in the band and out of the band is defined as d_{IB} and d_{OB}, respectively. As in Figure 1.1d, in a 'perfectly' banded matrix, there are no elements outside the band, thus $d_{OB} = 0$.

(a) *roadNet-CA* **(b)** *scircuit* **(c)** *msc10848* **(d)** *raefsky3*

Figure 1.1 – Examples of Banded Matrix Structure

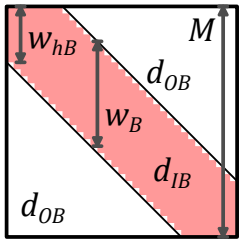


Figure 1.2 – Banded Matrix Definition

DEFINITION

A **banded matrix** is a sparse matrix which has a higher density of non-zero elements around the diagonal. A **'perfectly' banded matrix** concentrates all its non-zero elements within a delimitated region around the diagonal. This region is called the band.

NOTE

The SuiteSparse Matrix Collection [67] is a large, open source data base of sparse matrices from multiple real-world applications, many of which were provided by companies such as Boeing. It gathers around 3,000 matrices of various sizes, densities and structure, which are extensively used for benchmarking (see Chapter 2).

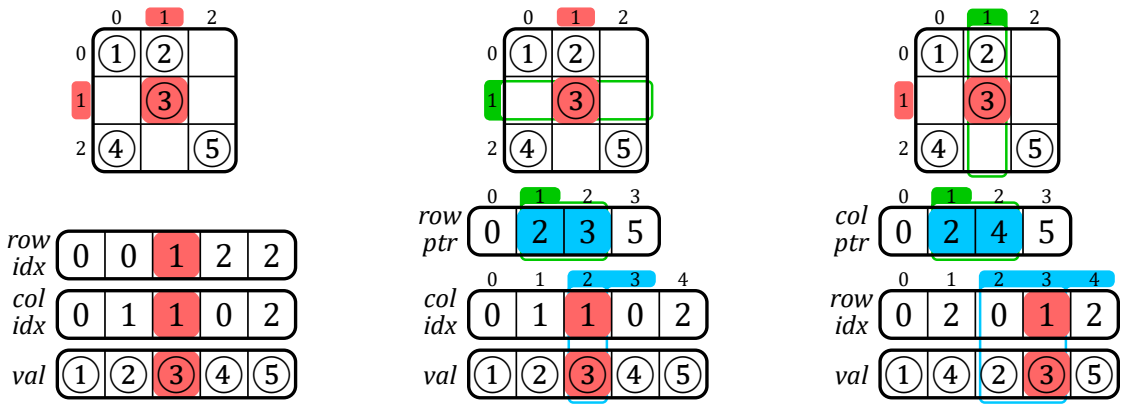
1.3 Sparse Formats

IN the absence of a compressed format, matrices are stored in fixed-size two-dimensional arrays. With this dense format, elements can be randomly accessed ($\mathcal{O}(1)$), and sequential accesses are highly efficient. The two dimensions are called ranks which correspond to the rows and columns. For matrices, there are thus two variants of the dense format, depending on which rank is stored sequentially in memory. In a *row-wise* (or *row-major*) storage, the elements within the rows are sequential, and conversely for *column-wise* (or *column-major*).

Sparse matrices contain a very large number of zeros, and to avoid storing these repeated zeros, there exist many compressed formats [11, 13, 21, 64, 99, 115], where certain formats are best-suited for matrices with a specific structure. The main principle of these sparse formats is to avoid storing the zero elements, but this requires additional metadata which gives the position of the non-zero elements.

For example, let us consider the commonly used *COOrdinate* (COO) format that is relatively simple. It consists of three tables of size nnz with only the non-zero elements (val table in

Figure 1.3a) and their row and column indices (row and col idx tables). The tuples do not necessarily need to be sorted, which makes it easy to append new entries. However, if the tuples are not sorted, this format is not well suited for sequential traversal or locating a specific matrix entry. This format can be sorted *row-wise* as in the example in Figure 1.3a or *column-wise* if needed, but in this case, it is less efficient than the two other widely used sparse formats *Compressed Sparse Row* (CSR) (Figure 1.3b) and its *column-major* counterpart *Compressed Sparse Column* (CSC) (Figure 1.3c), which are sorted by construction. With CSR, the val table of size nnz stores the non-zero elements sorted *row-wise* and the idx table of size nnz stores the corresponding column indices, similarly to COO. The third table ptr of size $M + 1$ stores the positions of the rows. For example, in Figure 1.3b, the second row of index 1 (*green*) has one non-zero element between positions 2 and 3 (*blue*, 3 excluded) of tables idx and val. The CSC format works similarly but in a *column-wise* manner: the val table of size nnz is sorted *column-wise* and the idx table of size nnz stores column indices. The ptr table of size $N + 1$ stores the positions of the columns. In the example in Figure 1.3c, the second column of index 1 (*green*) has non-zero elements between positions 2 and 4 (*blue*, 4 excluded) of tables idx and val. The ptr table allows CSR and CSC to have a lower memory footprint than COO.



(a) COO **(b)** CSR **(c)** CSC

Figure 1.3 – Example of a Sparse Matrix in COO, CSR and CSC Formats

Many others classes of sparse formats exists such as *Blocked Compressed Sparse Row* (BCSR), *DIAgonal* (DIA), *ELLPACK* (ELL) and *HYBbrid* (HYB), which all have their own advantages depending on the matrix structure or the hardware setup. In general, there are many derivates from the above main categories of sparse formats. Ultimately, we can create any format we want depending on the software and hardware constraints. Later in this manuscript, we propose our own custom sparse format called *Banded Compressed Sparse Column* (BaCSC) that is an extension of CSC for banded matrices.

DEFINITION

A **sparse format** is a memory representation of a sparse matrix which avoids storing zeros by using metadata.

NOTE

The most commonly used sparse formats are COO, CSR and CSC.

CONCLUSION

With the use of sparse formats, it is possible to greatly **decrease the memory footprint** of a sparse matrix while increasing the data dimensions. These formats **can be tailored** based on the needs of the software and hardware.

1.4 Algorithms for Dense and Sparse Matrix Multiplication

THE main matrix multiplication kernels that are of interest are Matrix-Vector (MV) and Matrix-Matrix (MM), which include SpMV and SpMSpV ($A \times b = c$), SpGEMV ($A \times b + d = c$), SpMM and SpMSpM ($A \times B = C$), and SpGEMM ($A \times B + D = C$), with 'Sp' for *sparse* and 'GE' for *general*. SpMV, as well as its variant SpGEMV, are by far the dominant operations in modern algebraic kernels (linear solvers, eigensolvers) which have been explicitly based on them [28, 41, 47, 60, 66, 94, 127]. Since these kernels are ubiquitous, their efficient implementation is primordial. In contrast, the SpMSpM multiplication is less frequent in standard algebraic kernels, but it is extensively used in algebraic graph manipulation [5, 16, 18, 26, 40, 62, 96], as well as in multigrid solvers [4, 15, 70, 98]. The growing importance of large graphs has motivated new research on optimizing SpMSpM applications.

In the following sections, we focus on SpMV and SpMSpM, with the notation $A(M,K) \times b(K) = c(M)$ and $A(M,K) \times B(K,N) = C(M,N)$, respectively. The rank K is the common rank between the two inputs A and B (or b). The existence of different ranks allows several algorithmic possibilities to compute matrix multiplication: the *Inner-Product* (`IP`) algorithm, the *Outer-Product* (`OP`) algorithm and *Gustavson*'s (`Gust`) algorithm [29, 45]. In all cases, the matrix A is traversed sequentially. The three algorithms, however, impose different access patterns for B (or b) and C (or c). Indeed, in many cases, they have to be accessed multiple times, as discussed in the following sections.

NOTE

In this manuscript, we focus on only two kernels, SpMV and SpMSpM. Note that, among MV kernels, SpMV is the most used and we specifically work on this kernel from Chapter 3. SpMSpM is also widely used and studying this MM kernel is of interest as the memory access patterns in both matrices are irregular.
The *general* ('GE') version of these kernels is not fundamentally different, as the majority of the compute operations are in the matrix multiplication, so going forward, we will ignore the 'GE' variants.

1.4.0.1 Inner-Product Algorithm

The *Inner-Product* algorithm [29] is an intuitive way to calculate a matrix multiplication because it follows the loop order of the mathematical formulas (1.1) and (1.2) for MV and MM, respectively. The common rank K is the innermost loop of the algorithm (see Algorithms 1.1 and 1.2, lines 2 and 3, respectively). The output C (or c) is always updated sequentially, but the B-matrix (or b-vector) will be read entirely M times. In the context of sparse matrices, the reads of B (or b) are conditioned by the non-zero elements of A, which requires indirect accesses. If B (or b) is also sparse, the algorithm needs to perform *intersections* between the non-zero elements in the rows of A and the columns of B (or in the b-vector) (see Section 1.5). In Algorithm 1.2, exchanging M and N (lines 1 and 2) simply causes the output to be updated *column-wise*.

$$\forall i \in [\![0, M-1]\!],\ c_i = \sum_{k=0}^{K-1} A_{i,k} \times b_k \tag{1.1}$$

$$\forall (i,j) \in [\![0, M-1]\!] \times [\![0, N-1]\!],\ C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \times B_{k,j} \tag{1.2}$$

Algorithm 1.1: Dense Matrix-Vector *Inner-Product* Algorithm

```
1 for i ∈ [0, M − 1] do // row first
2 |   for k ∈ [0, K − 1] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.2: Dense Matrix-Matrix *Inner-Product* Algorithm

```
1 for i ∈ [0, M − 1] do // A-row first
2 |   for j ∈ [0, N − 1] do
3 |   |   for k ∈ [0, K − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.2 Outer-Product Algorithm

The *Outer-Product* algorithm [29] has the opposite rank loop order than the *Inner-Product*. Indeed, Algorithms 1.3 and 1.4 show that the common rank K is the outermost loop (line 1). It changes nothing mathematically except that the B-matrix (or b-vector) is always read sequentially. In the context of sparse matrices, the output C (or c) is updated with indirect accesses. The algorithm needs to perform *reductions* of the partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), as we show in Section 1.5. To avoid irregularity when updating C (or c), the `OP` algorithm is often separated into two phases. First, during the *multiply* phase, we compute several *partial outputs* (*POs*) consisting of intermediate matrices (or vectors) of partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), then during the *merge* phase, we accumulate all the *POs* to form the final C-matrix (or c-vector). With this method, `OP` necessitates the generation of at most K *POs* of densities conditioned by the non-zeros elements of both inputs A and B (or b).

Algorithm 1.3: Dense Matrix-Vector *Outer-Product* Algorithm

```
1 for k ∈ [0, K − 1] do // column first
2 |   for i ∈ [0, M − 1] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.4: Dense Matrix-Matrix *Outer-Product* Algorithm

```
1 for k ∈ [0, K − 1] do // common rank first
2 |   for i ∈ [0, M − 1] do
3 |   |   for j ∈ [0, N − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.3 Gustavson's Algorithm

As the MM algorithms have three ranks instead of two for MV, there is a third loop ordering, known as *Gustavson*'s algorithm [45] where the common rank K is the middle-loop (see line 2 in Algorithm 1.5). With this approach, the output is computed row-by-row with *PO*-rows generated and accumulated for each output C-row. With this algorithm, updates of the C-rows are sequential, but elements within the rows are updated with indirect accesses. In the dense case, the B-matrix is entirely read M times and K *PO*-rows, of size N, are generated. In the context of sparse matrices, conversely to the C-rows, the selection of the rows in B is indirect, but the accesses are sequential within a row. Thus, the *intersection* search is only needed for the B-rows and not each element within rows. The *PO*-rows can be accumulated before computing the next C-row. Overall, this algorithm can be seen as a trade-off between

IP and **OP** for MM kernels, since the indirect accesses are shared between B and C. Again, for this algorithm, exchanging M and N causes the three matrices to be accessed *column-wise* instead of *row-wise*.

Algorithm 1.5: Dense Matrix-Matrix *Gustavson*'s Algorithm

```
1 for i ∈ [0, M − 1] do
2 |   for k ∈ [0, K − 1] do // common rank
3 |   |   for j ∈ [0, N − 1] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.4 Comparison of the Algorithms

In Table 1.1, we summarize the main features of **IP**, **OP** and **Gust** in terms of accesses to the matrix A, B (or b) and C (or c). From this high-level analysis, we can easily see that the data that is accessed with indirections shifts depending on the algorithm, from the input B (or b) to the output C (or c), with **Gust** being a trade-off between **IP** and **OP** for MM multiplication. To have a better understanding of the consequences on the actual number of memory accesses, we propose a more detailed analysis in Section 1.5.4, for the case of SpMSpM.

> **NOTE**
>
> Note that these algorithms are available in dense as well as in sparse MV and MM multiplications. In the sparse variants, the above algorithms (from 1.1 to 1.5) have the same behavior from a mathematical point-of-view, but need adjustments to traverse a sparse format instead of a dense matrix. Sparse versions of these algorithms are presented in Section 1.5.

Table 1.1 – Comparison of Matrix Multiplication Algorithms

	***Inner-Product* (IP)**[1]	***Outer-Product* (OP)**[1]	***Gustavson* (Gust)**[2]
A(M,K) **Accesses**	• *row-wise* • read N times[3,4] • sequential	• *column-wise* • read once • sequential	• *row-wise* • read once • sequential
B(K,N) **Accesses**	• *column-wise* • read M times[5] • not sequential	• *row-wise*[6] • read $M \times d(A)$ times[4,7] • sequential	• *row-wise* • read $M \times d(A)$ times[7] • sequential within rows
C(M,N) **Accesses**	• generated *row-wise*[6] • one sequential traversal	• generated without specific structure • K *POs* produced[8] (each *PO* has $\overline{nnz_c}(A) \times \overline{nnz_r}(B)$ non-zero elements[9])	• generated *row-wise* • $K \times d(A)$ *PO*-rows produced[10] for each row of A (M) ($\overline{nnz_r}(B)$ non-zero elements per *PO*-row)

[1] Considering $N = 1$ in the case of MV multiplication.
[2] Only for MM multiplication.
[3] Or less than N times if B has empty columns.
[4] Read once in the case of MV multiplication.
[5] With a sorted sparse format for B, else it would be read $\overline{nnz_r}(A) \times M = nnz(A)$ times.
[6] *Element-wise* in the case of MV multiplication (there is no row).
[7] Which is equal to $\overline{nnz_c}(A)$, the average number of non-zero elements in the columns of A.
[8] Or less than K times if A has empty columns or B has empty rows.
[9] $\overline{nnz_c}(A)$ in the case of MV multiplication.
[10] Which is equal to $\overline{nnz_r}(A)$, the average number of non-zero elements in the rows of A.

CONCLUSION

There are *three main algorithms* to compute sparse MV and MM kernels, ***Inner-Product*** **(IP)**, ***Outer-Product*** **(OP)** and ***Gustavson*** **(Gust)**. Each presents different characteristics, the main one being the **irregularity of the accesses**: on the **input** matrix B (or vector b) with **IP**; on the **output** matrix C (or vector c) with **OP**; on both with **Gust** but at **row scale**.

1.4.1 Tiling

The *tiling* process [43] consists in partitioning a matrix: a matrix may be divided into tiles, non-overlapping sub-matrices. In other words, this method consists of adding ranks to delimit the tiles, and thus adding loop iterations in the kernel algorithm.

Tiling methods make it possible to process smaller portions of the matrix and thus to adapt the working set to the size of the local memory [68, 122], particularly when the matrix has a structure with denser regions. *Tiling* can also increase parallelism opportunities. Software optimizations for sparse matrix computation, such as OSKI [116] and Sparse BLAS [28], are based on *tiling*. In previous sections, the study of the **IP**, **OP** and **Gust** algorithms highlighted the influence of rank ordering on the sequentiality and data movement, and thus, adding intermediate ranks with *tiling* breaks the regularity of pure **IP** or **OP** algorithms, requiring additional hardware support (see Section 1.5).

For example, let us consider the A-matrix in Figure 1.4 which is divided into four tiles (one level of *tiling*). For a SpMV, two new ranks appear for a total of four ranks, as shown in Algorithm 1.6: M_1 and K_1 allow tile selection and M_2 and K_2 allow data access within a tile. Thus, we could decide to apply an **OP** on tiles and conversely an **IP** within tiles, mixing-up the pros and cons of these two algorithms at different rank levels. In the case of the example in Figure 1.4, the upper-left A-tile (*red*) is evaluated first, sequentially updating the upper tile of the c-vector in an **IP** manner. The lower-left A-tile (*yellow*) is then computed the same way. At this point, *intersections* have been done with only the upper b-tile, and c has been updated sequentially (first *PO*). The two next A-tiles (*green*, then *blue*) will compute *intersections* with the lower b-tile and update the entire c-vector sequentially, showing the **OP** characteristic of having *POs* to *merge*, two in this case.

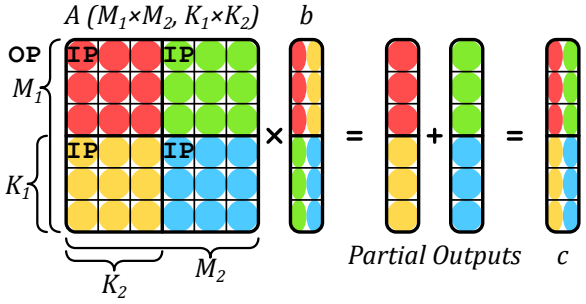


Figure 1.4 – An Example of *Tiling* on SpMV

Algorithm 1.6: Matrix-Vector Algorithm with Custom *Tiling* of Figure 1.4

```
1 for k1 ∈ [[0, K1 − 1]] do
2     for i1 ∈ [[0, M1 − 1]] do
3         for i2 ∈ [[0, M2 − 1]] do
4             for k2 ∈ [[0, K2 − 1]] do
5                 i ← i1 × M2 + i2
6                 k ← k1 × K2 + k2
7                 ci += Ai,k × bk
```

DEFINITION

Tiling is a method for partitioning matrices in non-overlapping sub-matrices, called **tiles**.

NOTE

Note that in case of *tiling*, the A-matrix is no longer read sequentially, which can be problematic if it is stored in a classical sparse format like CSR that is specifically made for *row-wise* accesses. Custom sparse formats adapted for *tiling* can be used [54], but this requires preprocessing to perform the conversion.

CONCLUSION

Tiling methods are widely used to **distribute workloads** across multicore systems, and to **locally reduce the data dimensions**. Complex *tiling* algorithms with many possible rank ordering **inherit the behaviors of `IP` and `OP`**, and can be described as a combinations of both at different tile levels.

1.5 Hardware Challenges for Sparse Matrix Computation

IN the previous section we presented a brief background on matrix multiplication without considering the underlying hardware. Although the various sparse formats reduce the memory footprint, they have the disadvantage that indirect accesses are required, which creates data-dependencies and reduces the effectiveness of caches. As we have seen, with all three algorithms, at least some of the accesses are irregular. The lack of spatial locality for these accesses reduces the benefit of having caches [84, 124]. In other words, a full cache-line is loaded, but only a few entries are read or modified. Moreover, the matrices of interest are extremely large and even the non-zero elements, and associated metadata, can not generally fit in the cache. Depending on the algorithm, successive accesses to the same element may be too far apart to benefit from temporal locality in the cache.

1.5.1 Inner-Product Algorithm

In an `IP` algorithm, the A-metadata needs to be read to indirectly access the B-matrix (or b-vector), which may itself be stored in a sparse format. In this case, the algorithm needs to check if the non-zero A-indices are present in B (or b). If an index appears in neither list, there is no need to perform the multiplication (ineffectual operation). More generally, this problem applies to any *fiber*(1) and is called the *intersection* search. As a minimum, this requires reading all the metadata for A and B (or b), and identifying those present in both.

In Algorithm 1.7, we present the pseudo-code for an *Inner-Product* (`IP`) SpMSpM with A and B stored in CSR and CSC formats, respectively. The outer-loop (M, line 1) iterates over the rows of A. The middle-loop (N, line 2) iterates over the columns of B. The inner-loop (line 4) iterates over the non-zero elements of the current row (i) of A, and for each index (k_A of position pos_A in the idx array of A, line 5), a search is performed in the metadata of B (line 6), to see if the corresponding index in B is present. Assuming it is ($isMatch$ is true, line 7), the partial sum is computed and then accumulated locally (acc, line 8). This example highlights the need to efficiently search for the *intersection* of indices in the metadata of the inputs.

Different algorithms can be used to address the *intersection* search, depending on whether the elements are sorted. When the elements are sorted, the *intersection* becomes a *merge* and generally requires at most $2 \times nnz - 1$ comparisons (in some cases the number of comparisons can be reduced [8]). In Algorithm 1.7, we use sorted sparse formats (CSR and CSC). Thus,

(1). This term is used in higher-dimensional tensor kernels [110].

the naïve *intersection* search proposed (line 10) does at most one traversal of each the current row of A (i) and the current column of B (j). Note that computing an *intersection* in software induces the use of conditional loops and branches, which can be costly for classical CPU branch predictors. In case the matrix B (or vector b) is stored in a dense format, the *intersection* search is simplified, since all indices are implicitly present, but the accesses to B (or b) remain indirect. As we see in Chapter 2, certain accelerators provide support for performing the *intersection* operation directly in hardware.

Algorithm 1.7: `IP` SpMSpM Performing *Intersection* Searches with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

1 **for** $i \in [\![0, M-1]\!]$ **do** // Iterations over the rows of A
2 **for** $j \in [\![0, N-1]\!]$ **do** // Iterations over the columns of B
3 $acc \leftarrow 0$; $pos_B \leftarrow B.ptr_j$;
4 **for** $pos_A \in [\![A.ptr_i, A.ptr_{i+1} - 1]\!]$ **do**
 // Iterations over the non-zero elements of the i-th row of A
5 $k_A \leftarrow A.idx_{pos_A}$;
 // Column-index of the (i, pos_A) non-zero element of A
6 $(isMatch, pos_B) \leftarrow IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr)$;
 // Return true and the matching pos_B if idx exists in the j-th column of B
7 **if** $isMatch$ **then** // If indices did match
8 $acc \leftarrow acc + A.val_{pos_A} \times B.val_{pos_B}$;
 // Local partial sum accumulation
9 $C_{i,j} \leftarrow acc$; // Write in memory of the resulting (i, j) element of C
10 **function** $IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr)$ **is**
  // Naive intersection search allowing the i-th row of A and the j-th column of B to be traversed only once at iteration (i, j)
11 **while** $B.idx_{pos_B} < k_A$ ***and*** $pos_B < B.ptr_{j+1}$ **do**
12 $pos_B \leftarrow pos_B + 1$;
13 **if** $B.idx_{pos_B} = k_A$ **then** // Returns true if indices from A and B match
14 **return** $(true, pos_B)$;
15 **return** $(false, pos_B)$;

DEFINITION

The ***intersection* search** consists in founding non-zero elements of two sparse arrays that have matching indices. This search is costly in software on classical CPUs, thus being key to improve performance of `IP` algorithm.

1.5.2 Outer-Product Algorithm

In an `OP` algorithm, the inputs A and B (or b) are accessed sequentially, but the updates of the output C (or c) require indirect accesses. The pattern of the accesses, while the output is generated, is irregular and determined by the structure of the inputs. Furthermore, if these "random" updates provoke cache misses where only one word within a cache-line is modified, the bandwidth to external memory is poorly utilized. The `OP` algorithm generates several *partial outputs* (*POs*) that must be accumulated to obtain the final output. We can identify two strategies: *immediate update* and *deferred update*. In the former, the final output

is immediately updated as each partial sum is generated. If the output is stored using a sparse format, either the index being updated does not yet exist, and the new partial sum must be inserted. Or, if it exists, a lookup must be done to locate it, the update performed and the result written back. The problem associated with combining the *POs* is called *reduction*.

In Algorithm 1.8, we present two pseudo-code for an *Outer-Product* (**OP**) SpMSpM using *immediate update* (from line 1) and *deferred update* (from line 7) with A and B stored in CSC and CSR formats, respectively. The outer-loop (K, line 1 or 7) iterates over the columns of A and the rows of B (common rank). The middle-loop (line 2 or 9) iterates over the non-zero elements of the current column (k) of A. The inner-loop (line 4 or 11) iterates over the non-zero elements of the current row (k) of B. Because we only iterate over the non-zero values, there is no need for an *intersection* search. The partial sum is then computed (line 6 or 15). Finally, in the *immediate update* version, the partial sum must be accumulated to C (line 6). These accesses are irregular, making it necessary to lookup the value in C, update it, and write it back. This is a *reduction* operation, performed on irregular data.

To avoid the complexities of these "random" *reductions* with *immediate updates*, the *deferred update* version postpones the accumulation and simply stores the partial sums in memory. One such technique is called *output batching* [65]. With this technique, arrays containing the indices and the partial sum are streamed sequentially to memory (PO_k, lines 13-15). Then, during a second pass (line 17), they are read back and the accumulations are performed, an approach which results in sequential memory accesses during both the first and second passes, with the downside that the volume of data transferred to memory is increased. As we see in Chapter 2, hardware accelerators may combine the *immediate* and *deferred update* techniques to optimize memory bandwidth.

DEFINITION

"Random" *reductions* occur when updating data from memory at irregular addresses, which happen on *immediate update* **OP**. With *deferred update* **OP**, the *reductions* are sequential but large sets of data need to be buffered in memory. Optimizing *reduction* operations is key to improving the performance on **OP** algorithm.

1.5.3 Summary

In Table 1.2, we summarize the advantages and hardware challenges for each algorithm, including **Gust** which presents a trade-off between **IP** and **OP**. The potential for parallelism and data-reuse varies across algorithms, **IP** being the most limited unless *tiling* methods are used. With **OP**, *POs* can easily be generated across a multicore system but at the cost of transferring a large volume of data to memory. With *immediate updates*, **OP** transfers less data but all accesses on the output are "random" *reductions*.

The issues presented with **IP** and **OP** can be generalized to two concepts: *gather* and *scatter*. The *gather* problem exists when the inputs are indirectly accessed, and need to be read from memory using metadata to present the processor with the correct terms for computing the partial sums. Typically, with **IP**, the *gather* problem is more difficult. The *scatter* problem exists when the order in which the output matrix/vector is generated is not sequential and is typically associated with the **OP** algorithm. In the case of *tiling* or with **Gust**, both *gather* and *scatter* must be performed creating a need for efficient *intersection* searches and random *reductions*.

Algorithm 1.8: OP SpMSpM Performing *Reduction* with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $PO_k(M, N)$ for all $k \in [\![0, K-1]\!]$ in COO: $PO_k.val, PO_k.row, PO_k.col$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

// *
// * * * 'Immediate updates' version *
1 **for** $k \in [\![0, K-1]\!]$ **do** // Iterations over the columns of A and the rows of B
2 **for** $pos_A \in [\![A.ptr_k, A.ptr_{k+1}]\!]$ **do**
  // Iterations over the non-zero elements of the k-th column of A
3 $i_A \leftarrow A.idx_{pos_A}$; // Row-index of the ($pos_A$, k) non-zero element of A
4 **for** $pos_B \in [\![B.ptr_k, B.ptr_{k+1}]\!]$ **do**
  // Iterations over the non-zero elements of the k-th row of B
5 $j_B \leftarrow B.idx_{pos_B}$;
  // Column-index of the (k, pos_B) non-zero element of B
6 $C_{i_A,j_B} \leftarrow C_{i_A,j_B} + A.val_{pos_A} \times B.val_{pos_B}$;
  // Reduction (to memory) of the partial sum into the (i_A, j_B) element of C
    (immediate update)
// *
// * * * 'Deferred updates' version *
// Multiply phase
7 **for** $k \in [\![0, K-1]\!]$ **do** // Iterations over the columns of A and the rows of B
8 $pos_{PO} \leftarrow 0$;
9 **for** $pos_A \in [\![A.ptr_k, A.ptr_{k+1}]\!]$ **do**
  // Iterations over the non-zero elements of the k-th column of A
10 $i_A \leftarrow A.idx_{pos_A}$; // Row-index of the ($pos_A$, k) non-zero element of A
11 **for** $pos_B \in [\![B.ptr_k, B.ptr_{k+1}]\!]$ **do**
  // Iterations over the non-zero elements of the k-th row of B
12 $j_B \leftarrow B.idx_{pos_B}$;
  // Column-index of the (k, pos_B) non-zero element of B
13 $PO_k.row_{pos_{PO}} \leftarrow i_A$;
14 $PO_k.col_{pos_{PO}} \leftarrow j_B$;
15 $PO_k.val_{pos_{PO}} \leftarrow A.val_{pos_A} \times B.val_{pos_B}$;
  // The k-th partial sum of the (i_A, j_B) element of C is stored in memory in the
    k-th partial output
16 $pos_{PO} \leftarrow pos_{PO} + 1$;
  // Each partial output is generated sorted, row-wise
// Merge phase
17 **for** $k \in [\![0, K-2]\!]$ **do** // Naive serial merge of the K partial outputs
18 $PO_{k+1} \leftarrow 2DListsMerge(PO_k, PO_{k+1})$;
// PO_{K-1} is C stored in COO format
19 **for** $pos \in [\![0, length(PO_{K-1}) - 1]\!]$ **do**
  // Update C (dense) from the fully merged partial outputs
20 $i \leftarrow PO_{K-1}.row_{pos}$;
21 $j \leftarrow PO_{K-1}.col_{pos}$;
22 $C_{i,j} \leftarrow PO_{K-1}.val_{pos}$; // Write in memory of the (i, j) element of C

Table 1.2 – Comparison of Advantages and Challenges with Matrix Multiplication Algorithms

		Inner-Product **(IP)**	***Outer-Product*** **(OP)**	***Gustavson*** **(Gust)**
Access Count	**MV**	***A*:** once ***b*:** $nnz(A)$ ($\sim M\times$) ***c*:** once	***A*:** once ***b*:** once ***c*:** $nnz(A)$ ($\sim K\times$)	*N/A*
	MM	***A*:** $N\times$ ***B*:** $M\times$ ***C*:** once	***A*:** once ***B*:** $M \times d(A)\times$ ***C*:** $nnz(A) \times \overline{nnz_r}(B)$ ($\sim K\times$)	***A*:** once ***B*:** $M \times d(A)\times$ ***C*:** $nnz(A) \times \overline{nnz_r}(B)$
Sequential Accesses		Accesses to A and C	Accesses to A and B	Accesses to A and C Accesses to B-rows
Input Format		Accesses to A and B benefit from alternate formats (CSR and CSC)		Allows consistent format for A and B
Parallelism Opportunity		Possible with *tiling*	Generation of *POs* over common rank of A and B	Reading A-rows and updating C-rows
Reuse Opportunity		Potentially B, but limited by its size	An A-column or B-row while computing *POs*	A set of B-rows, based on non-zero elements of A-rows
Challenges		*Intersection* search between non-zero elements in A and B	Storage for *POs* or in-place *merge* and *reduce*	*Merge* and *reduce* of *PO*-rows

CONCLUSION

There are **two main challenges** to address when computing sparse matrices. When reading irregular data from memory, addressing the ***gather*** problem is key. It consists of processing efficient ***intersection* searches**. When updating irregular data from memory, addressing the ***scatter*** problem is key. It consists of **managing *POs*** in memory or processing efficient **"random" *reductions***. These two challenges are tied to `IP` an `OP`, respectively, and are present in any higher rank algorithms for sparse matrix computing.

1.5.4 SpMSpM Algorithm Analysis

To better understand how the algorithms impact the number and types of memory accesses, we developed, and presented in Figure 1.5, a model based on memory accesses counts for SpMSpM. We identify five memory access patterns: (1) those where a matrix is read only once, thus there is no opportunity for reuse, (2) where the accesses are sequential, but a *fiber* is read multiple times, (3) where *fibers* are read consecutively, but the accesses within a *fiber* are "random", (4) where the *fibers* are accessed "randomly", but within a *fiber* the accesses are sequential, and (5) where the matrix elements are accessed multiple times, and with a "random" access pattern.

For each of the three algorithms, we counted the number of read and write accesses, based on the density of the input matrices, and the local memory storage available ("0D", "1D" and "2D"), as presented in Table 1.3. We refer to a system with no local storage or cache as '0 Dimensional' ("0D"), a system which has local storage that is sufficient to hold a single row or column as '1 Dimensional' ("1D"), and a system which can cache an entire matrix as '2 Dimensional' ("2D"). In Figure 1.5, we plotted the number of memory accesses for SpMSpM, in a logarithmic scale, based on a square matrix with $M = 10,000$. Indeed, the number of non-zero elements in C depends on the structure of the input matrices and in this model, we have assumed the non-zero entries in the input matrices are uniformly, randomly distributed.

This corresponds to a *worst case* for sparse matrices (no spatial and temporal locality). If A or B are banded matrices, for example, there would be fewer non-zero elements in C. In each graph, we have separated the accesses into A (*bottom*), B (*middle*) and C (*top*). The color code shows the access pattern, based on the five categories described above.

Table 1.3 – Memory Access Count for Each Matrices of **IP**, **OP** and **Gust** SpMSpM

	***Inner-Product* (IP)**	***Outer-Product* (OP)**	***Gustavson* (Gust)**
$\mathbf{A}(M,K)$	**0D**: $N \times nnz(A)$ **1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$
$\mathbf{B}(K,N)$	**0D-1D**: $M \times nnz(B)$ **2D**: $nnz(B)$	**0D**: $M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(B)$	**0D-1D**: $M \times d(A) \times nnz(B)$ **2D**: $nnz(B)$
$\mathbf{C}(M,N)$	**0D-1D-2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D-1D**: $2 \times M \times d(A) \times nnz(B)$ **2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D**: $2 \times M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(C)$ (max: $M \times N$)[1]

[1] We can not easily quantify $nnz(C)$ knowing $nnz(A)$ and $nnz(B)$. We thus use a statistic approximation.

In the top-row of Figure 1.5 (graphs ①, ② and ③), we start by considering a naïve system which has no local storage ("0D"), thus all data comes from main memory. From the graphs in this row, we clearly see that **IP** requires more accesses at low density, as the A-matrix must be read N times (graph ①). Whereas for **OP** and **Gust**, the A-matrix is read only once (thus the *grey* color). For **OP** and **Gust** the total number of accesses is the same, however, with **OP** (graph ②), the accesses to C are "random" (*red*). With **Gust** (graph ③), the accesses to B are sequential within the rows (*orange*) and the C-rows are generated sequentially (*yellow*), thus **Gust** avoids having any fully "random" accesses (*red*).

Since main memory accesses are the principal bottleneck, systems integrate local storage (such as buffers or caches). Thus, we consider the case of a system with sufficient local memory to store one full row, per matrix (equivalent to few sparse rows, "1D"). We assume this local storage is perfect (no misses) and we re-calculate the number of remaining main memory accesses, as shown in the middle-row of Figure 1.5 (graphs ④, ⑤ and ⑥). The matrices are assumed to be initially stored in main memory, thus they must be accessed at least once. As can be seen, for **IP** (graph ④), the number of A-accesses is reduced, because rows are reused, thus the total number of A-accesses becomes equal to that in **OP** and **Gust**. For **OP** (graph ⑤), the single row buffer greatly reduces the number of B-accesses, however, it does nothing for C (despite the appearance, due to the logarithmic scale). For **Gust** (graph ⑥), the single row buffer is sufficient to cache the *reduction* operations in C, and thus, it now has fewer overall accesses than **OP**. It is also interesting to note that **OP** and **Gust** scale better than **IP** with lower densities, as seen by the shallower slope.

Finally, we have considered the extreme case where an system has a buffer of sufficient size to store an entire matrix ("2D"), which is shown in the bottom-row of Figure 1.5 (graphs ⑦, ⑧ and ⑨). Although this case is not practical for large matrices, through the proper use of *tiling* to locally reduce the dimensions of a sub-matrix, this case can be achieved at the level of a tile. In this case, the total number of external accesses is equal for the three algorithms, as each matrix is accessed only once from main memory. For **IP** (graph ⑦), it is interesting to note that such a large buffer is needed to locally address the *intersection* search in B. Similarly for **OP** (graph ⑧), a large storage is required to address the *reductions* in C. For **Gust**, in fact, it is not necessary to buffer the entire B-matrix. Indeed, buffering a few rows (corresponding to

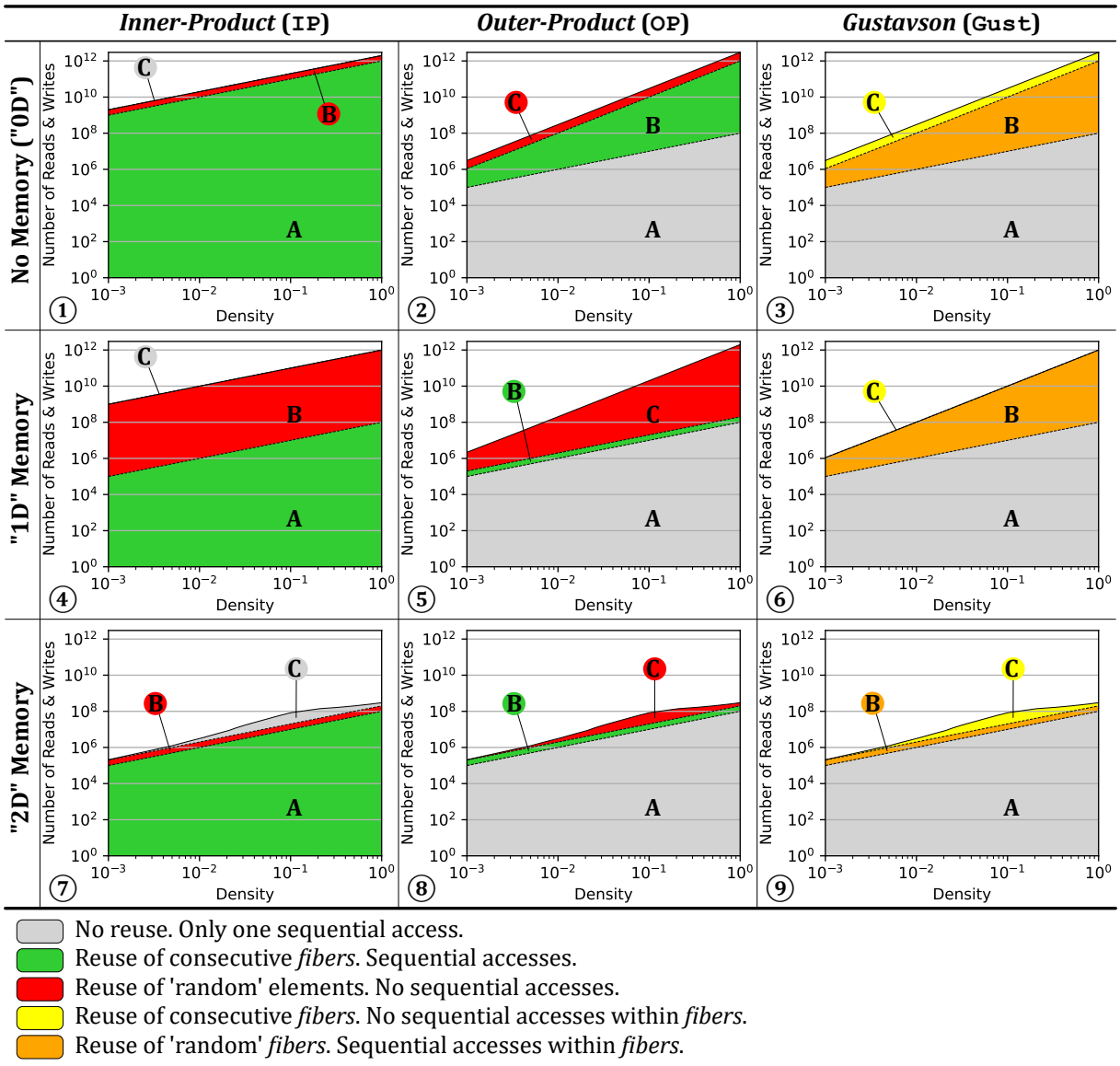


Figure 1.5 – Model of Memory Access Counts Depending on Algorithm and Local Memory Size

$nnz_r(A)$) is sufficient to achieve the number of accesses shown in graph ⑨. This corresponds to an intermediate design point (multiple row buffers for B) not explicitly shown in the figure (between graphs ⑥ and ⑨).

When viewed overall, Figure 1.5 highlights the benefits of **Gust**. We clearly see that **Gust** does not require any fully "random" accesses (*red*). With a single row buffer, it is possible to address the *reductions* in C. And, with a limited number of row buffers, the number of external memory accesses for B can be reduced. Based on this analysis, it is clear that, among the three base algorithms, **Gust** is the best candidate for hardware implementation of SpMSpM.

> **NOTE**
>
> For SpMV, such an analysis does not show significant differences between **IP** and **OP**, as both algorithms require similar number of memory accesses. However, **OP** shows more potential for parallelism and data-reuse as seen in the previous section (1.5.3), thus being preferred

for SpMV hardware accelerators.

CONCLUSION

Based on a high-level analysis of the memory accesses on SpMSpM, ***Gustavson*'s algorithm** appears as the **best trade-off** to minimize memory traffic with reasonable amount of local storage. The second best option is `OP`, which requires fewer memory accesses than `IP` when the matrices have a low density.

1.6 Conclusion

IN this chapter, we established the foundation for understanding the relationship between applications working with irregular data and the underlying challenges in hardware. Irregular accesses are common in sparse computing and cause reduced cache efficiency, limited parallelism, and higher memory latency exposure. We analysed the most representative algorithms for sparse matrix multiplication kernels, and identified the operation that provoke irregular accesses. *Intersections* and *reductions* on irregular data with classical systems are costly, thus they represent the central challenge for efficient computation on sparse data. To improve the performance of these operations, specialized hardware has been proposed as will be seen in the next chapter.

TAKEAWAYS

Sparse matrices: large, highly sparse, structured (such as banded matrices)

Sparse formats: diverse, reduce memory footprint
- COO, the simplest
- CSR and CSC, the mostly used
- BaCSC, our own sparse format for banded matrices

Sparse kernels: sparse MV and MM multiplications (such as SpMV and SpMSpM), generate indirect accesses

Algorithms: *Inner-Product* (`IP`), *Outer-Product* (`OP`) and *Gustavson* (`Gust`), extended with *tiling* methods
- Best potential to reduce memory transfers: `Gust`, followed by `OP` at low density

Problem: irregular accesses, generate high memory transaction latencies

Challenges:
- *gather*: *intersection* searches
- *scatter*: "random" *reductions*, or storing and *merging partial outputs* (*POs*)

CHAPTER

STATE-OF-THE-ART OF HARDWARE ACCELERATORS FOR SPARSE DATA

2

Contents

2.1 Introduction

THE challenges of efficiently processing sparse matrices have been recognized since the 1960s, when it became clear that using dense array representations for sparse datasets was highly inefficient [99]. The seminal work of OSKI [116] in 2003 formalized alternative sparse representations and identified numerous optimization opportunities, paving the way for specialized software libraries for sparse algebra [123]. However, even with these advancements, single-core software implementations often achieve only around 10% of peak performance [37], highlighting the need for more efficient hardware solutions. Sparse matrix kernels, particularly sparse Matrix-Vector (MV) and sparse Matrix-Matrix (MM) multiplication, pose distinct computational challenges:

- Reading of sparse input data, which may require efficient handling of irregular memory patterns.

- *Reduction* operations on intermediate results, where partial sums must be accumulated and managed.
- Formatting and writing back results to memory.

As seen in the previous chapter, the choice of algorithm, *e.g.*, *Inner-Product* (**IP**), *Outer-Product* (**OP**), or *Gustavson*'s (**Gust**), directly influences how these challenges are addressed. For example, with **IP** algorithms, the key challenge is *intersection* searches (*gather* problem), while **OP** algorithms must deal with irregular updates (*scatter* problem). *Gustavson*'s algorithm offers a trade-off, balancing input reuse and output generation but still requires careful optimization. On conventional CPUs, memory throughput is the primary bottleneck, leading to optimizations such as row reordering, blocking, and loop transformations to improve data locality. For GPUs, the challenge shifts to thread assignment, warp balancing, and efficient use of shared memory [75]. Sparse MM introduces additional complexity due to dynamic *reductions* and irregular output generation [38], which dominate execution time and memory traffic on both CPUs and GPUs [24, 89, 130]. Hardware accelerators address these issues by optimizing the memory hierarchy, ensuring input data availability near compute units and offloading accumulation/*reduction*/*merge* operations from processors. This chapter reviews many state-of-the-art hardware accelerators for sparse matrix computations, comparing their architectures, algorithms, and performance. Our analysis identifies common architectural trends, evaluates design trade-offs, and provides insights for future accelerator development. The goal is to offer a structured comparison that helps designers select or refine architectures based on specific workload requirements.

2.2 Hardware Accelerators for Sparse Matrices and Vectors

IN this section, we describe the recent (2015-2025) hardware accelerators for sparse MM and MV, targeting mainly ASIC-type implementation. We note there are many FPGA-based accelerators, however, to limit the scope we do not focus on these and only include those which can be classified along the ASIC-type accelerators. We try to use consistent notation, for example in the color code in the architecture figures. Particularly, compute units are represented in *red*, control units in *orange*, data transfer units in *yellow*, caches in *blue*, and memories in *green*. We include our work, the SpDCache accelerator [II], which we describe in subsequent chapters, as we published initial results in 2024. Before going into detailed overviews of specific accelerators, in Table 2.1 we present a criteria-based classification of the designs which were studied.

A first criterion to classify the accelerators is between *standalone* accelerators that address the full kernel without the assist of a processor (*ex.*, OuterSPACE [91]) and ones that address only a part of the kernel and need a processor to manage the computation (*ex.*, PHI [86]). Most of the *standalone* accelerators are directly connected to the main memory and internally manage their local memories. Accelerators can also intervene at different levels of the memory hierarchy, giving a second classification criterion. Certain accelerators intervene near the processors (*ex.*, ASA [132]), near the cache (*ex.*, PHI), or near the main memory (*ex.*, SPiDRE [10]).

Each accelerator is optimized for one or several specific algorithms (**IP**, **OP** or **Gust**), our third criterion. For example, some accelerators (*ex.*, PHI) address the *reduction* problem and are adapted to **OP** algorithms but *reductions* are also required with **Gust** and highly-tiled algorithms. ExTensor [48] is a general purpose accelerator that is demonstrated for a highly-tiled **OP** but also makes major contributions for **IP**.

From a memory perspective, the two main problems are *gather* and *scatter*, our fourth criterion. Certain accelerators offer solutions for one, the other or both. For example, some accelerators (*ex.*, OuterSPACE) are specialized for an `OP` algorithm but optimize both *gather* and *scatter*. Finally, accelerators may be optimized for sparse MM, MV or both, our final criterion.

After this first comparison, in Sections 2.2.1 to 2.2.4 we present, in detail, three *standalone* accelerators (*e.g.*, OuterSPACE, ExTensor and Gamma) and one *processor-assist* accelerator (*e.g.*, PHI), which cover several types of architectures and algorithms. In Section 2.2.5, we summarize the other accelerators from Table 2.1.

Table 2.1 – Algorithmic and Architectural Classification of Existing Accelerators

		Autonomy		**Position in Memory Hierarchy**			**Most Adapted Algorithm(s)**			**Hardware Support for**		**Sparse Kernels Addressed**	
Name	**Year**	***Standalone***	***Processor-Assist***	***Near Processors***	***Near Caches***	***Near Main Memory***	***Inner-Product***	***Outer-Product***	***Gustavson***	***Gather***	***Scatter***	***MV***	***MM***
Coup [134]	2015		✓		✓			✓			✓	✓	✶[3]
EIE [46]	2016	✓				✶[1]		✓		✓	✓	✓	
OuterSPACE [91]	2018	✓				✶[1]		✓		✓	✓	✓	✓
GraphR [107]	2018	✓				✶[1]		✓		✓	✓	✓	
SDE [51]	2019	✓				✶[1]	✓		✶[3]	✓	✶[2]	✓	✶[3]
ExTensor [48]	2019	✓				✶[1]	✓	✶[2]		✓	✶[2]	✶[3]	✓
PHI [86]	2019		✓		✓			✓	✶[3]		✓	✓	✶[3]
SPiDRE [10]	2019		✓			✓	✓			✓		✓	✶[3]
ITS [100]	2019	✓				✶[1]		✓		✓	✓	✓	
SPU [22]	2019	✓				✶[1]	✓	✓		✓	✓	✓	
Tensaurus [110]	2020	✓				✶[1]	✶[3]	✶[2]		✓	✓	✓	✓
MatRaptor [109]	2020	✓				✶[1]			✓	✓	✓		✓
SpArch [137]	2020	✓				✶[1]		✓		✓	✓		✓
Alrescha [3]	2020	✓				✶[1]	✓			✓	✓	✓	
SIGMA [95]	2020	✓				✶[1]	✓			✓	✓		✓
SpaceA [126]	2021	✓				✓	✓			✓	✓	✓	

✶[1] *Most standalone accelerators connect directly to main memory.*

✶[2] *Not main contribution but addressed in the article.*

✶[3] *Could be adapted but not discussed in the article.*

✶[4] *New dataflow based on one or several algorithms.*

Continued on next page

Table 2.1 – Algorithmic and Architectural Classification of Existing Accelerators (Continued)

Name	Year	Autonomy		Position in Memory Hierarchy			Most Adapted Algorithm(s)			Hardware Support for		Sparse Kernels Addressed	
		Standalone	*Processor-Assist*	*Near Processors*	*Near Caches*	*Near Main Memory*	*Inner-Product*	*Outer-Product*	*Gustavson*	*Gather*	*Scatter*	*MV*	*MM*
Gamma [133]	2021	✓				⋆[1]			✓	✓	✓		✓
SPAGHETTI [50]	2021	✓				⋆[1]		✓		✓	✓		✓
InnerSP [7]	2021	✓				⋆[1]			✓	✓	✓		✓
CoSPARSE [32]	2021	✓				⋆[1]	✓	✓		✓	✓	✓	
SortCache [108]	2021		✓		✓			✓	⋆[3]		✓	⋆[3]	✓
Sextans [106]	2022	✓				⋆[1]		✓		✓	✓		✓
ASA [132]	2022		✓	✓				⋆[3]	✓		✓	⋆[3]	✓
Spada [74]	2023	✓				⋆[1]			⋆[4]	✓	✓		✓
Flexagon [85]	2023	✓				⋆[1]	✓	✓	✓	✓	✓		✓
MOSCON [35]	2023	✓				⋆[1]		✓		✓	✓		✓
GSCSp [72]	2023	✓				⋆[1]			✓	✓	✓		✓
SDMA [39]	2023	✓				⋆[1]			⋆[4]	✓	✓		✓
SSSR [103]	2023		✓	✓			✓			✓		✓	⋆[3]
Funnel [82]	2024	✓				⋆[1]	✓		✓	✓	✓		✓
GAS [136]	2024	✓				✓		✓			✓	✓	✓
ACES [78]	2024	✓				⋆[1]			✓	✓	✓		✓
FullSparse [129]	2024	✓				⋆[1]		✓		✓	✓		✓
Sparm [80]	2024	✓				⋆[1]	✓	✓	✓	✓	✓		✓
SpDCache [II]	2024		✓		✓			✓	⋆[3]		✓	✓	⋆[3]
SPADES [53]	2024	✓				⋆[1]	⋆[3]	⋆[3]	⋆[3]	✓		✓	⋆[3]
DySpMM [118]	2024	✓				⋆[1]	✓			✓	✓		✓
Vesper [61]	2024	✓				⋆[1]	✓	✓	✓	✓	✓	✓	✓
Mentor [77]	2024	✓				⋆[1]			✓	✓	✓		✓
IOPS [111]	2025	✓				⋆[1]	✓	✓		✓	✓		✓

⋆[1] *Most standalone accelerators connect directly to main memory.*

⋆[2] *Not main contribution but addressed in the article.*

⋆[3] *Could be adapted but not discussed in the article.*

⋆[4] *New dataflow based on one or several algorithms.*

Continued on next page

Table 2.1 – Algorithmic and Architectural Classification of Existing Accelerators (Continued)

Name	Year	Autonomy		Position in Memory Hierarchy			Most Adapted Algorithm(s)			Hardware Support for		Sparse Kernels Addressed	
		Standalone	*Processor-Assist*	*Near Processors*	*Near Caches*	*Near Main Memory*	*Inner-Product*	*Outer-Product*	*Gustavson*	*Gather*	*Scatter*	*MV*	*MM*
Leda [128]	2025	✓				⋆[1]		✓		✓	✓		✓
DMSA [88]	2025	✓				⋆[1]			⋆[4]	✓	✓		✓
C^{3}ache [73]	2025		✓		✓			⋆[4]	⋆[4]	✓	✓		✓
ScanNow [81]	2025	✓				⋆[1]			⋆[4]	✓	✓		✓

⋆[1] *Most standalone accelerators connect directly to main memory.*
⋆[2] *Not main contribution but addressed in the article.*
⋆[3] *Could be adapted but not discussed in the article.*
⋆[4] *New dataflow based on one or several algorithms.*

CONCLUSION

Hardware accelerators for sparse matrix operations are categorized by their level of **autonomy** (*standalone*, *processor-assist*), **memory hierarchy position** (near-core, near-cache, near-main memory), supported **algorithms** (`IP`, `OP` and `Gust`), supported **memory flows** (*gather* and *scatter*), and **kernels** (sparse MV and MM). While ***standalone* accelerators dominate** the landscape with diverse architectural approaches focusing on specific applications, recent trends show **increased focus on algorithm-adaptive designs and *processor-assist* variants** for generic problem classes.

2.2.1 OuterSPACE

OuterSPACE [91] is an `OP` accelerator with a focus on SpMSpM kernels, although it can also handle SpMV. It is a highly parallelized architecture which employs Single-Program Multiple-Data (SPMD)-style Processing Elements (PEs) that fetch data from main memory through a two-level highly-banked cache hierarchy as shown in Figure 2.1. The input matrices A and B are stored in CSC and CSR formats, respectively, to match the read access patterns of the `OP` algorithm. The *partial outputs* (*POs*) and final output matrix C are stored in CSC format.

The computation is temporally divided into the *multiply* and *merge* phases, as shown in Figure 2.2. During the first phase, inputs are divided into tiles of several rows of A and columns of B and distributed through the Processing Tiles (PTs) by a *control unit*. Within each PT, each PE processes multiplications of one element of the k^{th} column of A with all elements of the k^{th} row of B and an L0 cache in the PT facilitates the reuse of the B-rows between PEs. This phase generates a *PO* per PT, corresponding to the tile of rows of A and columns of B that it was assigned.

Then, during the *merge* phase, the *POs* are combined to generate the final matrix C. Each PT locally sorts the *POs*, and then the PTs work together to merge them. This algorithm,

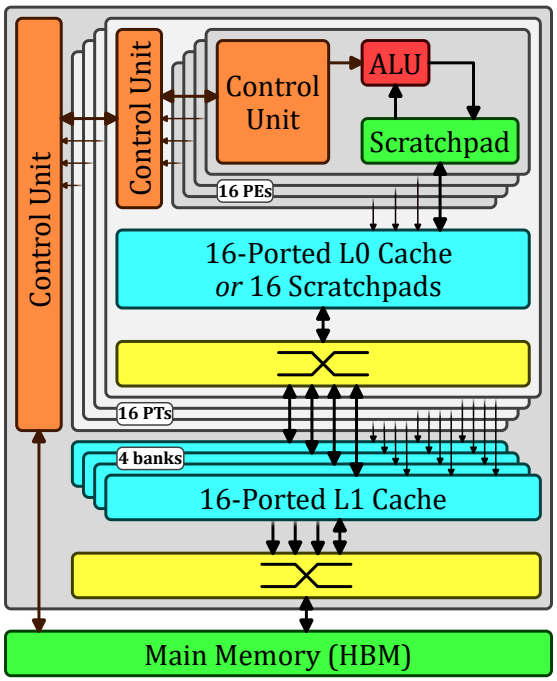


Figure 2.1 – Architecture of OuterSPACE

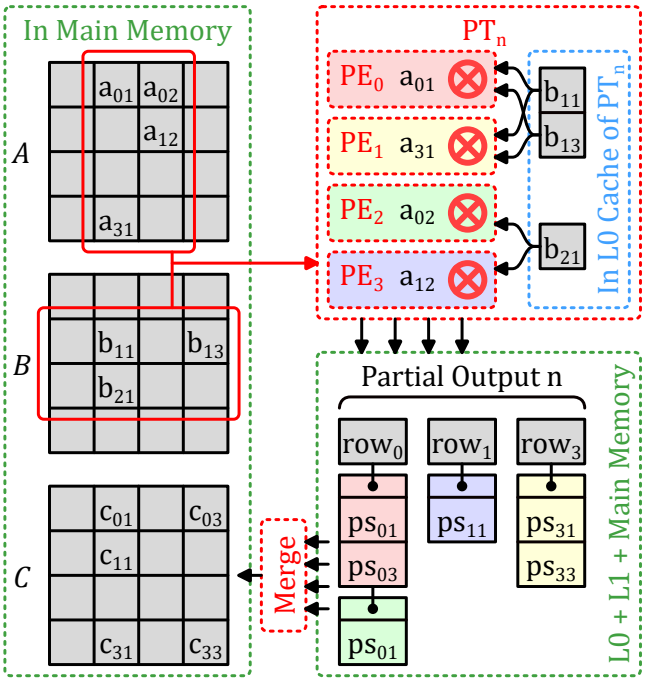


Figure 2.2 – OuterSPACE Dataflow

which is a tiled, *deferred updates* version of **OP**, was chosen to maximize data locality and parallelism, although it is less efficient. This *merge* phase only uses half of the PEs so the others are disabled (clock-gated) and the caches are reconfigured in a scratchpad mode to save energy.

Note, if non-zero elements in the *POs* exceeds the L0 cache capacity, it overflows to the last-level cache, which results in reduced performance. OuterSPACE performs best when the $nnz_c(A)$ and $nnz_r(B)$ are both uniform which ensures a uniform distribution of work across the large number of PTs and PEs.

CONCLUSION

OuterSPACE is a highly parallelized, **OP**-based accelerator for SpMSpM, leveraging a two-level cache hierarchy and SPMD-style PEs to optimize data locality and parallelism through tiled, *deferred update* computation.

2.2.2 ExTensor

The authors created ExTensor [48] as a general purpose sparse tensor algebra accelerator (including SpMSpM). It is an **IP** accelerator but due to its support for three levels of fixed *tiling* of the inputs, it optimizes the *intersection* search between non-zero elements or tiles. By doing the *intersections* ahead of time, at different levels of the memory hierarchy (which correspond to the tile levels), ExTensor is able to efficiently eliminate ineffectual operations, thus addressing the main issue in **IP** or complex *tiling* algorithms. Note that the input matrices would typically be stored in a complex sparse format derived from CSR, corresponding to three levels of *tiling*. Thus, at a given tile level, each tile is represented by coordinates (*coord*), the CSR metadata stored in idx and ptr, with the tile data itself compressed using CSR (next tile level). ExTensor is designed with a flexible architecture composed of different bricks, as

shown in Figure 2.3. To aid in the understanding, the interactions between the bricks are described using the semantics of function calls.

- The *Scanner* is a storage unit that delivers a stream of coordinates in increasing order, fetching them from memory. The coordinates are delivered through the *Iterate()* operation call.
- The *Intersect* is a hardware block that co-iterates over several *Scanners* in parallel and compares the coordinate streams. This unit performs the *intersection* search required by `IP` type algorithms, delivering a single stream with the matching coordinates, which are the coordinates of non-zero partial sums. To optimise the *intersection* step, it is possible to skip a range of coordinates in a stream depending on the current coordinate of another stream. Thus, the *Intersect* sends skip messages with *SkipTo()* operations to the other *Scanners*.
- The *Coordinator* is a hardware unit that includes *Scanners* and *Intersect* units, and manages the input and output data depending on the current processed tile, as shown in Figure 2.3. Data and metadata are respectively stored in storage units and *Scanners*, while a *Sequencer* manages the *Iterate()* calls so that the *Intersect* brick delivers the stream of effective coordinates. In addition, in the *Tile Coord* brick, the offsets of the tile are added back, so the output of the *Coordinator* is in absolute coordinates.

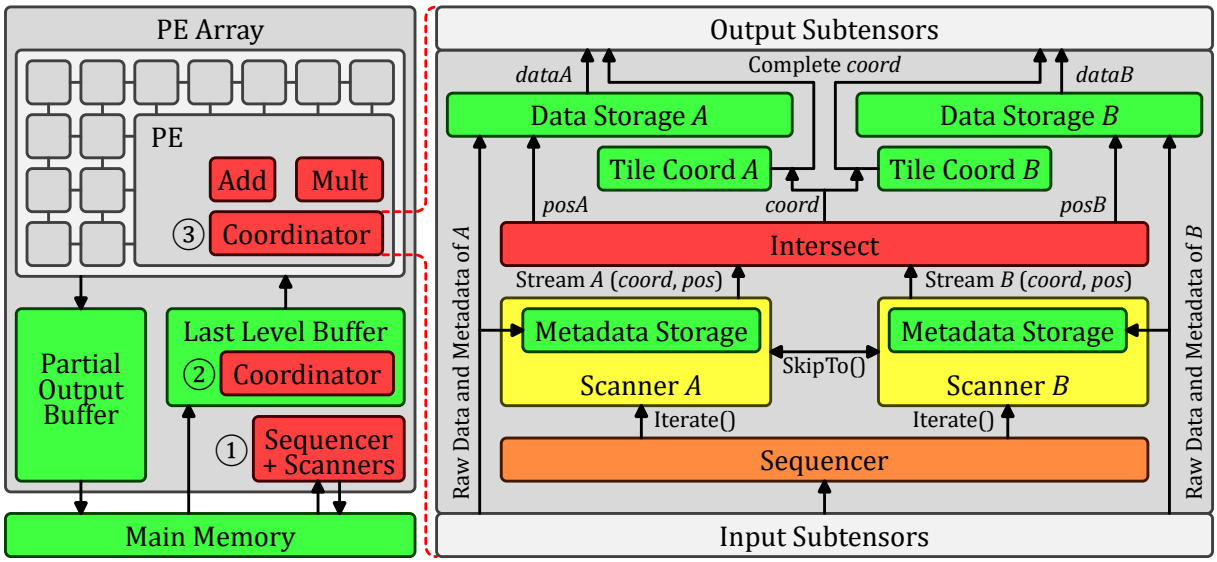


Figure 2.3 – Architecture of ExTensor

These bricks can be placed at different positions in a memory hierarchy, depending on the *tiling* and selected hardware. Thus, the *Coordinator* is able to intersect at coarse-granularity with output data being metadata of the next tile, skipping an entire tile when appropriate, with low computation cost, and also at fine-granularity when output data are the actual non-zero elements that need to be multiplied.

ExTensor is designed with three levels of *tiling* such that each have either input or output sequentiality. First, a *Sequencer* plus *Scanners* block ① is placed close to the main memory to determine which tiles to send to the next storage, the *Last Level Buffer* (*LLB*). Then, for each tile, a *Coordinator* ② in the *LLB* intersects the next level tiles, which are sent to several PEs that will process in parallel. Finally, in each PE, a *Coordinator* ③ intersects the non-zero elements of the computed tile and sends them to the arithmetic unit of the PE for multiplication.

In addition to these bricks which are the main contributions, ExTensor still needs to perform the *merge* phase, including the *reduction* of the *POs* which are stored in COO format. The *Partial Output Buffer* (*POB*), is managed by tiles, and the content of the tile is stored on-chip using a Content Addressable Memory (CAM) for quick access, enabling *reductions* to be performed. On overflow, the tiles are flushed to main memory. To benefit from ExTensor's support for *tiling*, the input matrices need preprocessing, making it difficult to directly compare performance with other accelerators.

CONCLUSION

ExTensor is a versatile, `IP`-based sparse tensor algebra accelerator that optimizes *intersection* searches through a three-level *tiling* hierarchy, enabling efficient elimination of ineffectual operations and supporting both coarse and fine-grain parallelism.

2.2.3 Gamma

Gamma [133] is a dedicated accelerator for SpMSpM kernels, which uses the `Gust` algorithm. The two input matrices are both stored in CSR format and the output is generated in the same format, providing consistency.

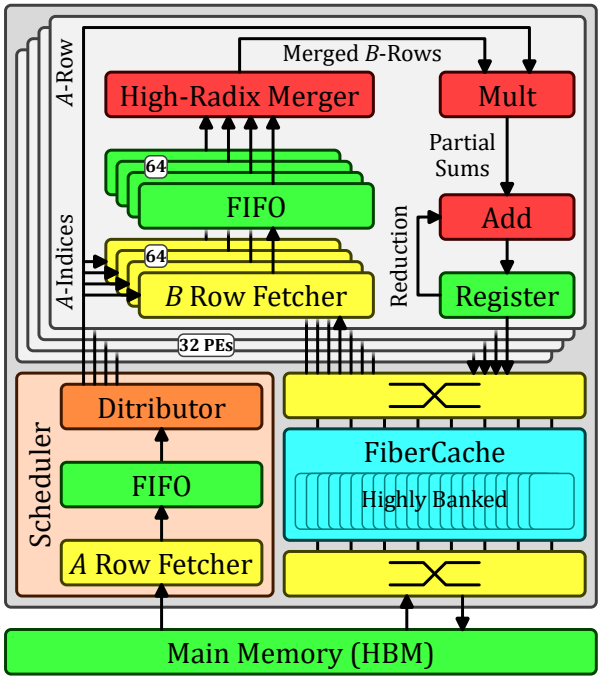


Figure 2.4 – Architecture of Gamma

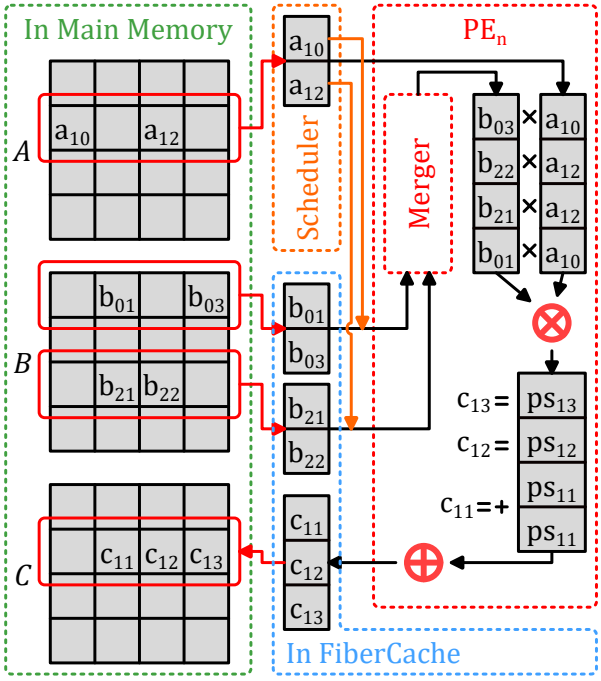


Figure 2.5 – Gamma Dataflow

The Gamma architecture, presented in Figure 2.4, is composed of a fetcher for the rows of A that contains a *Distributor* to distribute them to 32 PEs. Each PE processes an entire A-row and computes the corresponding C-row, as shown in Figure 2.5. The PE fetches the required rows of B and merges and sorts them in a 64-wide *merge* block, before multiplying with the elements of A. As a result, the output C-row is generated in order. The *reduction* step is, thus, highly simplified, because any duplicate indices are adjacent. Normally, the C-row can be written back to main memory. However, if the row of A contains more than 64 non-zero elements, then it must be processed with multiple passes. After each pass, the partial C-row

is stored temporarily. Finally, the temporary C-rows are read-back and merged by the PEs. Of course, the rows of B must be accessed repeatedly by the different PEs, and Gamma proposes *FiberCache* which is a highly banked cache which fetches the B-rows in advance, based on the column index table of A. It keeps track of which rows of B are most needed by the PEs, and when an eviction is necessary, the victim is selected to be the row which is least needed.

Gamma encounters some difficulties when the matrix A has rows that are too dense, requiring multiple iterations through the PEs, which creates a high-load on the cache. The authors propose a software preprocessing technique which splits dense rows of A and rearranges them to make better use of the *FiberCache*.

CONCLUSION

Gamma is a *Gustavson*-based SpMSpM accelerator which is highly cited in the literature that leverages a 32-PE architecture with a *FiberCache* to optimize row-wise processing and reduce memory traffic, while simplifying *reductions* through ordered output generation.

2.2.4 PHI

The authors of PHI [86] focus on graph algorithms, and include the SpMV kernel as it is core in graph processing. They note that many large graph algorithms favour pull implementations (each node reads inputs from its neighbours) compared to push implementations (each node propagates results to its neighbours), since in a traditional memory system, push implementations require read-modify-writes which require accessing complete cache-lines, even though a small unit of data is modified. The push implementation corresponds to the **OP** algorithm, which suffers from poor memory performance with traditional caches.

PHI is a modified cache which is optimized to efficiently process commutative[(1)] *scatter* updates, also called *reductions*. It stores, and coalesces values to update, in the cache, as shown in Figures 2.6 and 2.7. This type of cache is called a *reduction cache*. PHI works in two phases, managing to take advantage of the two version of the **OP** algorithm: *immediate updates* and *deferred updates*. Three new features are introduced, compared to a standard cache:

- *In-cache reduction storage and processing*. In the first phase, when an update for a line not in the cache is received (cache-miss), the cache does not fetch the data-line from memory, but rather marks a free cache-line as being in *buffer mode* (*B*), and simply locally stores the update. If there are subsequent updates that hit the cache-line, they are stored, or combined, using the in-cache commutative operator, as shown in the case 1 of Figure 2.7 (top). When a cache-line is evicted, if it contains multiple updates, PHI assumes that the coalescing has been effective, and it updates the main memory via a line-wise read-accumulate-write operation. This is the normal mode of operation of a traditional *reduction cache*, which processes *immediate updates*.
- *Output Batching*. In the first phase, if a cache-line is evicted, but contains few updates (managed with a configurable threshold), PHI batches these updates as *(address, value)* tuples, allocating a new cache-line in *update batching mode* (*UB*) to assemble the batch, as shown in the case 2 of Figure 2.7 (bottom). Groups of updates are organized by *bins* corresponding to a small memory region. The batches are streamed to main memory, assembling the *partial outputs* (*POs*) typical of the *deferred updates* version of the **OP**

(1). As PHI focuses on graph algorithms, the *reduction* operation could be any commutative operator including logical operations (OR/AND) as well as addition.

algorithm. Later, in the second phase, the *deferred updates* are read back and processed. The reading of the batches is efficient, as they are contiguous (see the OP algorithm in Section 1.4).

- *Reduced Cache Synchronization Traffic.* When scaling PHI to multi-core systems, private caches act as local buffers, coalescing the updates and reducing coherency protocol traffic.

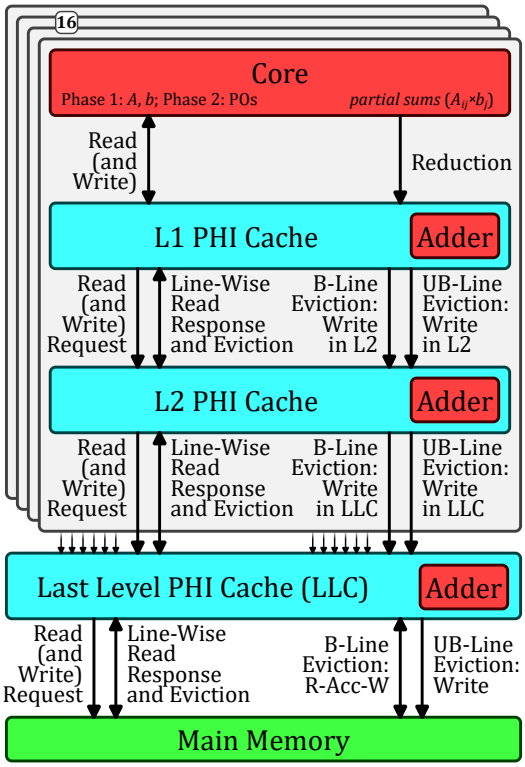


Figure 2.6 – Architecture of PHI

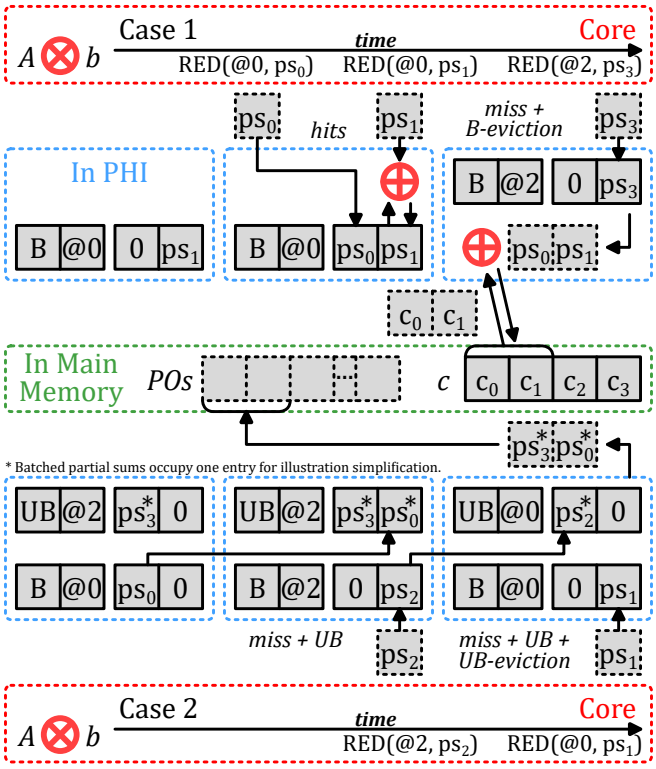


Figure 2.7 – PHI Dataflow

This combination of in-cache coalescing and update buffering enables PHI to take advantage of locality when possible (like Coup [134]), but with the addition of *UB*, PHI improves memory utilization, even when the data being updated is too large to fit in the cache. For SpMV, the authors report that much of the benefit comes from the *reduction* in memory synchronization traffic for multi-core systems.

CONCLUSION

PHI is a *reduction*-aware cache architecture that optimizes commutative *scatter* updates by combining in-cache coalescing, output batching, and reduced synchronization traffic, effectively improving memory utilization for both *immediate* and *deferred update* OP SpMV kernel.

2.2.5 Other Accelerators

In the previous sections, we introduced four sparse accelerators, each implementing different matrix multiplication algorithms and architectures. This section explores additional accelerators to showcase the wide range of architectural approaches available. We focus on their primary applications and design features, emphasizing their advantages and limitations.

In Appendix A, the reader can find a short summary of over 40 different sparse accelerators that were studied, some of which were also presented in Paper I.

Coup [134], PHI [86], SortCache [108], ASA [132], SpDCache [II] and C^{3}ache [73] are *processor-assist* accelerators that modify an existing cache to support near-memory *reduction* operations with in-place accumulation. By focusing solely on the *scatter* operation, they achieve performance improvements while maintaining scalability, versatility, and low area overhead. However, their performance relies on shifting computation toward the memory hierarchy, which can create bottlenecks. Among these, SortCache incorporates a large merger, which introduces scalability challenges, as large mergers are difficult to scale efficiently. In contrast, SortCache offers a solution to completely reorganize data storage and processing within caches, while Coup, PHI, and SpDCache are lightweight solutions that retain more of the original cache functionality. ASA follows a similar approach but adds a dedicated cache near the core, parallel to the memory hierarchy. While most of these accelerators integrate into classical memory hierarchies, C^{3}ache is specifically designed for GPGPU (General-Purpose GPU) integration.

ExTensor [48], SPiDRE [10] and SSSR [103] are *processor-assist* accelerators that specifically target the *gather* operation. SSSR and ExTensor utilize efficient streamers to process *intersections* between input data, while SPiDRE employs a near-main memory converter to fetch and reorganize data in advance. These solutions address *gather* operations with simple, lightweight architectures suitable for general purpose computing. SSSR is the least resource-intensive and highly flexible but requires in-core integration (developed for RISC-V). SPiDRE, on the other hand, demands more area and may dominate main memory traffic. ExTensor, while offering general purpose streamers, is optimized for building *standalone*, highly tiled accelerators.

Alongside *processor-assist* accelerators, a wide variety of *standalone* accelerators have been proposed. These achieve achieve high performance by providing a customized memory hierarchy which can feed parallel compute units and merge results back to memory. However, they often come with high area overhead and face significant scalability challenges. Most *standalone* accelerators rely on large mergers or lookup tables to efficiently handle *gather* and *scatter* operations. Additionally, many are tailored to specific applications or algorithms.

EIE [46], SDMA [39], Funnel [82], FullSparse [129], CANDLES [44] (2), MaxK-GNN [93] (2), SCNN [92] (2), SparTA [138] (2) and SparTen [42] (2) are specialized for neural network applications, focusing on MM kernels. They employ various techniques to exploit sparsity in weights and activations. Funnel, in particular, uses a prediction-based mechanism, while FullSparse dynamically switches between dense and sparse modes depending on the data.

GraphR [107], SpaceA [126] and GAS [136] leverage in-memory computing, demonstrating significant potential for data reuse and reducing memory traffic. However, analog in-memory computing approaches are limited to low-precision data and are generally not suitable for high-performance computing. GraphR and SpaceA specialize in graph applications, focusing on MV kernels, while GAS is designed for general purpose use.

SDE [51], SPAGHETTI [50], Sextans [106], MOSCON [35], GSCSp [72], DySpMM [118], AiSpGEMM [113] (2), GraphLily [52] (2), HiSparse [27] (2), HitGraph [139] (2), Leda [128] (2) and ThunderGP [20] (2) are highly optimized for FPGA implementations across various applications, all centered on MM kernels. Some, such as HiSparse, are specifically optimized for

(2). Not listed in Table 2.1. We explore these accelerators in our study but our classification does not fit their architecture.

FPGAs equipped with High Bandwidth Memory (HBM). Notably, HitGraph, GraphLily, and ThunderGP are specialized for graph applications.

Tensaurus [110], Block-SpMM [90][(2)], DTC-SpMM [30][(2)], Flash-LLM [125][(2)], HARP [63][(2)], TC-GNN [120][(2)] and vectorSparse [37][(2)] are designed for tensor kernels (MV, MM, and higher dimensions). Most are general purpose but rely on highly tiled architectures that require dedicated sparse formats. HARP, however, aims to eliminate this constraint.

Beyond ITS [100], which is specialized for graph applications, other accelerators, such as OuterSPACE [91], SPU [22], MatRaptor [109], SpArch [137], Alrescha [3], SIGMA [95], Gamma [133], InnerSP [7], Mentor [77], ScanNow [81] and RTSpMSpM [135][(2)], are not application-specific but focus primarily on MM kernels, with many employing the *Gustavson* (**`Gust`**) algorithm. SPU adapts to both dense and sparse data using two modes, similar to FullSparse. Alrescha offers a configurable design for general purpose applications but focuses on the *Inner-Product* (**`IP`**) algorithm, much like SIGMA. SpArch is tailored for the *Outer-Product* (**`OP`**) algorithm, aiming to reduce the number of *partial outputs* (*POs*), while InnerSP, Mentor, ScanNow, MatRaptor, and RTSpMSpM concentrate on **`Gust`**, with design goals akin to Gamma.

CoSPARSE [32], Spada [74], Flexagon [85], ACES [78], Sparm [80], Vesper [61], IOPS [111] and DMSA [88] represent some of the latest accelerators, all emphasizing adaptive dataflows that combine **`IP`**, **`OP`**, and **`Gust`** algorithms. They feature switching modes or innovative dataflows inspired by the strengths and weaknesses of these three approaches. Notably, CoSPARSE and Spada employ hardware/software co-optimization strategies.

Finally, SPADES [53] stands out as the only accelerator in this list that does not rely on sparse formats. Instead, it uses GZIP compression, with dedicated hardware for efficient, on-the-fly decompression during sparse computing.

CONCLUSION

This overview demonstrates that sparse matrix accelerators have evolved significantly from **2015 to 2025**, with diverse architectural approaches addressing different algorithmic paradigms, memory hierarchies, and sparsity patterns to maximize performance and energy efficiency when handling increasingly irregular and diverse sparse workloads in machine learning, graph processing, and scientific computing.

2.3 Comparison of Existing Accelerators

IN this section, we analyze and compare the accelerators described in Section 2.2. To maintain clarity and avoid information overload, we focus on a representative subset of these accelerators for the tables and analysis in Section 2.3.2, though our observations apply more broadly. Table 2.2 lists this subset chronologically with the corresponding research institutions.

Several trends emerge from this overview. The United States (US) and China lead in accelerator development, with China showing particularly strong activity since 2023. Most accelerators originate from universities and institutes, with MIT (US) notably contributing to ExTensor, PHI, SpArch, and Gamma, while ICT (China) contributed to GAS, ACES, and Mentor. Industry participation is also evident: NVIDIA was involved in five accelerators (*e.g.*, EIE, ExTensor, SpArch, Flexagon, and Vesper), while companies such as Axelera, Intel, Meta, Microsoft, SiFive, and Zettaflops contributed to individual projects. The research is geograph-

ically distributed across Canada, China, France, Hong Kong, India, Iran, Italy, Japan, Singapore, South Korea, Spain, Switzerland, the United Kingdom, and the United States.

Table 2.2 – Chronological Summary of a Representative Sparse Accelerator Subset

Name	Year	Institute / University
OuterSPACE [91]	2018	Univ. of Michigan (US) / ASU (US)
ExTensor [48]	2019	Univ. of Illinois (US) / NVIDIA (US) / MIT (US)
PHI [86]	2019	CSAIL, MIT (US) / CMU (US)
SPiDRE [10]	2019	UPC, BSC (Spain) / Arm Research
ITS [100]	2019	CMU (US)
SPU [22]	2019	Univ. of California (US)
SpaceA [126]	2021	UCSB (US)
Gamma [133]	2021	CSAIL, MIT (US)
SortCache [108]	2021	Georgia Tech (US) / Zettaflops (US) / Sandia Labs (US)
ASA [132]	2022	Lehigh Univ. (US) / LBNL (US)
Spada [74]	2023	THU (China) / Northwestern Univ. (US) / DAMO Academy (China) / SQI (China)
Flexagon [85]	2023	Univ. of Murcia (Spain) / Georgia Tech (US) / NVIDIA (US)
SSSR [103]	2023	ETH Zurich (Switzerland) / SiFive (US) / Axelera (France) / Bologna Univ. (Italy)
GAS [136]	2024	ICT (China) / UCAS (China)
ACES [78]	2024	Illinois Tech (US) / ICT (China)
SpDCache [II]	2024	CEA (France), TIMA (France), UGA (France)
IOPS [111]	2025	SUSTech (China)
DMSA [88]	2025	Tokyo Tech (Japan)

2.3.1 Benchmarks, Evaluation and Performance

Up to this point, we have discussed the architecture of various accelerators, without reporting the performance they achieve. Most of the matrices used for benchmarking come from the SuiteSparse Matrix Collection [67] that contains a large number of matrices from diverse, real applications. Domain specific matrix libraries are also used [6, 69, 97, 105], as well as some synthetic matrix generators [1, 12]. In Table 2.3, we note the number of matrices used for evaluation across our subset of accelerators. Most evaluations employ around ten or several tens of matrices, with few exceeding one hundred (none in the table). Given the significant performance variation across different matrices, evaluating with a larger matrix set provides more reliable performance estimates. Consequently, we evaluate SpDCache using a comprehensive set of matrices (detailed in Chapters 3 and 5). Matrix densities are typically below 10^{-2} and can be as low as 10^{-7}. Some accelerators, such as ExTensor, Tensaurus, SortCache, and Flexagon, also include denser matrices in their benchmarks. All studies evaluate large matrices with several million non-zeros, which is critical since these exceed typical cache sizes. However, performance results often depend heavily on the matrices chosen.

Matrices are typically stored in straightforward formats such as COO (row-major sorted) as found in the SuiteSparse Matrix Collection, necessitating preprocessing for compatibility with certain accelerators. For instance, some accelerators, such as EIE, ExTensor, Tensaurus, MatRaptor, Alrescha, SpDCache and C^{3}ache, require conversion to their custom formats.

Other *standalone* accelerators utilize unique internal formats with *tiling* methods but perform the conversion in hardware (*e.g.*, InnerSP, Spada, Flexagon, IOPS, DMSA). Accelerators like Coup, PHI, SortCache, and ASA do not require preprocessing as they focus solely on *scatter*. *Gather* focused accelerators such as SPiDRE and SSSR do not require preprocessing, as they directly stream matrices from standard sparse formats. CoSPARSE is notably the only accelerator proposing to maintain two matrix copies in main memory (one in CSR and one in CSC) to enable joint software/hardware optimization. Regardless, fair benchmarking must consider the costs associated with specialized preprocessing if this is required.

Table 2.3 – Qualitative Analysis and Evaluation of Sparse Accelerators Subset

Name	Functional Evaluation (Tool / Evaluation Specifics / Host*)	Number of Matrices	Local Memory Size
OuterSPACE [91]	*gem5* / Model only key elements; Skip memory allocation; Ignore start-up time / ∅	20	528 KiB
ExTensor [48]	Custom Python simulator / Only the *Coordinator* is evaluated; Remainder modeled analytically / ∅	14	∼38 MiB
PHI [86]	*ZSim* / Entirely evaluated / 16-cores system with 3 level cache	6	32.3 MiB
SPiDRE [10]	*gem5* / *Unknown* / Single ARM core	*unknown*	∅ (main memory size)
ITS [100]	∅ / Evaluated on ASIC and FPGA / ∅	31	∼64 MiB
SPU [22]	*gem5* / SPU part implemented in Chisel; Control core simulated / ∅	24	∼2.5 MiB
SpaceA [126]	Custom simulator / Simulation tracking values stored in 3D-DRAM / ∅	15	4 GiB (main memory size)
Gamma [133]	Custom simulator / Entirely evaluated / ∅	37	3 MiB
SortCache [108]	Custom simulator / Entirely evaluated / Single threaded, in-order core with 3 level cache	11	16.3 MiB
ASA [132]	Modified *ZSim* / Entirely evaluated; One ASA per core / 8-cores with 3 level cache	9	8× 8 KiB
Spada [74]	Custom Rust simulator / Entirely simulated; RTL of key components / ∅	27	1.5 MiB
Flexagon [85]	Custom simulator based on STONNE / Entirely evaluated / ∅	9	1.3 MiB
SSSR [103]	∅ / Entirely evaluated through RTL implementation / RISC-V Snitch System	*unknown*	128 KiB
GAS [136]	HSPICE / Entirely simulated; Models for FeFETs and MOSFETs / 96 cores CPU, NVIDIA GPU	20	∼40 MiB
ACES [78]	Custom simulator / RTL of main components / ∅	18	2 MiB
SpDCache [II]	Custom Python simulator / Entirely evaluated; Not cycle-accurate / Single-core system with one level cache	48	32 KiB
IOPS [111]	∅ / Entirely evaluated through RTL implementation / ∅	10	∼77 KiB
DMSA [88]	∅ / Evaluated on FPGA / ∅	10	∼22 MiB

** Mainly for processor-assist accelerators.*

Not all accelerators have reached the same level of design maturity, and in Table 2.3, we have shown how each design of our subset of accelerators was analyzed or simulated. Most designs report that they have been simulated at a cycle-accurate level. Most of the accelerators use custom simulators or tools such as *gem5* [76], *ZSim* [101] and STONNE [87], or HSPICE (Synopsys®) for in-memory computing, but unfortunately there is no common approach, making it difficult to fairly compare the reported performance. RTL descriptions

are often only generated for key parts of the design, except for a few that actually simulate the RTL for evaluation (*e.g.*, EIE, ITS, SSSR, IOPS, DMSA[3]). To estimate power and area, authors used tools such as CACTI [9], McPAT [71] and NVSim [25]. We have not attempted to compare power/energy as the data in the literature is heterogeneous and can not be fairly compared. In the last column of the table, we show the size of the local memory storage used in simulation by our subset of accelerators. Except for SPiDRE and SpaceA which work directly in main memory, the local memories are in the range of 32 KiB to 64 MiB, corresponding to 4k to 8.4M entries (if an entry is 8B). Thus, in the model presented in Section 1.5.4 of the previous chapter, most of the accelerators are situated in the middle-row of Figure 1.5, with a local memory corresponding to a few matrix rows. Indeed, some accelerators have so much storage, they are closer to the bottom-row of Figure 1.5, where the full matrix could be held entirely in local memory.

The accelerators' performance is reported as a speedup versus a baseline, which may differ according to their hardware and software platforms: typically the Intel MKL library [55], the Armadillo library [102], SparseBLAS [28] for CPUs, or GridGraph [140], Gunrock [119] and GraphMat [112] specifically for graphs on CPUs, or cuSPARSE [90] and CUSP [23] for GPUs. Since different baselines with different numbers of threads and cores are used, these speedups need to be interpreted with care. The speedups are in general higher for the *standalone* accelerators as they provide optimized hardware for the entire kernel, while *processor-assist* accelerators have less impressive speedups. Although many *standalone* sparse accelerators have been proposed by the academic community, it is not clear how such an accelerator could become an economically viable product. However, light-weight *processor-assists* could be integrated into general purpose processors or caches to improve their performance, while remaining general purpose.

The only accelerators that have a comparable baseline are some of the *standalone* ones addressing sparse MM kernels. Some of them compare themselves to OuterSPACE with the same benchmark matrices, so we can sort them by their speedup: SPAGHETTI (13×), Gamma (6.6×), InnerSP (4.6×), SpArch (4.0×), SIGMA (3.0×), IOPS (2.9×), MatRaptor (1.8×), Alrescha (1.7×), MOSCON (1.1×), OuterSPACE (1×). Some accelerators compare themselves against other accelerators beyond OuterSPACE, making direct speedup comparisons problematic. An extended speedup ranking derived from indirect comparisons to OuterSPACE should be interpreted cautiously: GAS (26×), SPAGHETTI (13×), Sparm (7.9×), Gamma (6.6×), DMSA (6.2×), InnerSP (4.6×), SpArch (4.0×), SIGMA (3.0×), IOPS (2.9×), GSCSp (1.8×), MatRaptor (1.8×), Alrescha (1.7×), MOSCON (1.1×), OuterSPACE (1×). Additionally, Spada, Flexagon, ACES, and Mentor can be indirectly compared to OuterSPACE through intermediate accelerators, though the resulting speedups lack accuracy due to propagated uncertainties: Spada (5.8× -110×), Flexagon (6.8× -14×), ACES (12× -240×), Mentor (8.6× -25×), and ScanNow (20× -440×). This performance inconsistency stems from the absence of a common baseline and variations in benchmark matrices across studies. Averaging the estimates for the three most recent accelerators yields: ScanNow (140×), ACES (77×), Spada (43×), GAS (26×), Mentor (14×), SPAGHETTI (13×), Flexagon (9.8×), Sparm (7.9×), Gamma (6.6×), DMSA (6.2×), InnerSP (4.6×), SpArch (4.0×), SIGMA (3.0×), IOPS (2.9×), GSCSp (1.8×), MatRaptor (1.8×), Alrescha (1.7×), MOSCON (1.1×), OuterSPACE (1×). These results warrant careful interpretation. Even so, we observe that eleven of these nineteen *standalone* accelerators employ the **`Gust`** algorithm (*e.g.*, ScanNow, ACES, Spada,

(3). SpDCache from Paper II has no RTL evaluation. We provide a full RTL evaluation in Chapter 5.

Mentor, Flexagon, Sparm, Gamma, DMSA, InnerSP, GSCSp and MatRaptor), compared to seven using **OP** (*e.g.*, GAS, SPAGHETTI, Sparm, SpArch, IOPS, MOSCON and OuterSPACE) and four using **IP** (*e.g.*, Sparm, SIGMA, IOPS and Alrescha). This comparison suggests that **Gust** is the most suitable algorithm for high-performance hardware accelerators, with **OP** as the second choice.

These measurements are consistent with our model presented in Section 1.5.4 (Figure 1.5). OuterSPACE, the baseline, corresponds to graph ⑤. MatRaptor, the first **Gust** accelerator, was able to achieve higher performance by reducing the number of "random" accesses, and creating better regularity (*red* to *yellow*), as seen in graph ⑥. SpArch and GAS are **OP** accelerators with a larger local memory, which is used to address the *red* region in graph ⑤. The A-matrix condensing method of MatRaptor, allows to reduce the number of *partial outputs* (*POs*), and approach the performance achievable in graph ⑧, although, *in fine*, it is impossible to store all the *POs* in local memory. GAS leverages in-memory computing techniques to mitigate the limitations of the **OP** approach, thereby approaching the performance levels achievable in graph ⑧. **Gust** accelerators correspond to optimizations of the algorithm approach in graph ⑥, to approach the performance in graph ⑨. This is mainly achieved by custom local memories and caches for the B-matrix to address the *orange* region, given that it is only necessary to store a limited number of B-rows, and to get ahead-of-time information from the reading of the matrix A. **IP** accelerators face inherent challenges in achieving the performance levels of other approaches, a limitation also predicted by the model in Figure 1.5. Nevertheless, several recent accelerators (*e.g.*, CoSPARSE, Flexagon, Sparm, Vesper and IOPS) have demonstrated that **IP** delivers competitive performance on certain matrices, comparable to **OP** and **Gust** approaches. To capitalize on this observation, these designs support multiple or all three algorithms, enabling adaptive algorithm selection. While this flexibility for *standalone* accelerators increases hardware complexity, it yields substantial performance gains and drives the field toward adaptive dataflow architectures. These findings are consistent with the model presented in Section 1.5.4 and demonstrate that it provides valuable insights into local memory requirements for sparse MM accelerators.

CONCLUSION

This comparative analysis of state-of-the-art sparse matrix accelerators reveals that ***Gustavson*'s algorithm is the most effective for high-performance** when coupled with *standalone* hardware implementations, **followed by *Outer-Product*** which is used especially in *processor-assist* designs.

2.3.2 Analysis for Designers

There is no single "best overall" accelerator. A designer must consider the required performance, the available area and the class of problems to be addressed. In Figure 2.8 we present a flow diagram which can guide a designer towards an architecture based on their requirements. Starting from the top-right of the figure, the designer must first decide between a *specialized accelerator* for a specific kernel or a *general purpose accelerator*. The former is preferable if performance must be maximized, requiring a *standalone* accelerator addressing both *gather* and *scatter* (*e.g.*, ITS, SpaceA, Gamma, Spada, Flexagon, ACES, IOPS and DMSA, from the subset of accelerators).

A *general purpose accelerator* can either take the form of a highly configurable *standalone* block (*e.g.*, OuterSPACE, ExTensor, SPU and GAS) or a *processor-assist* that boosts perfor-

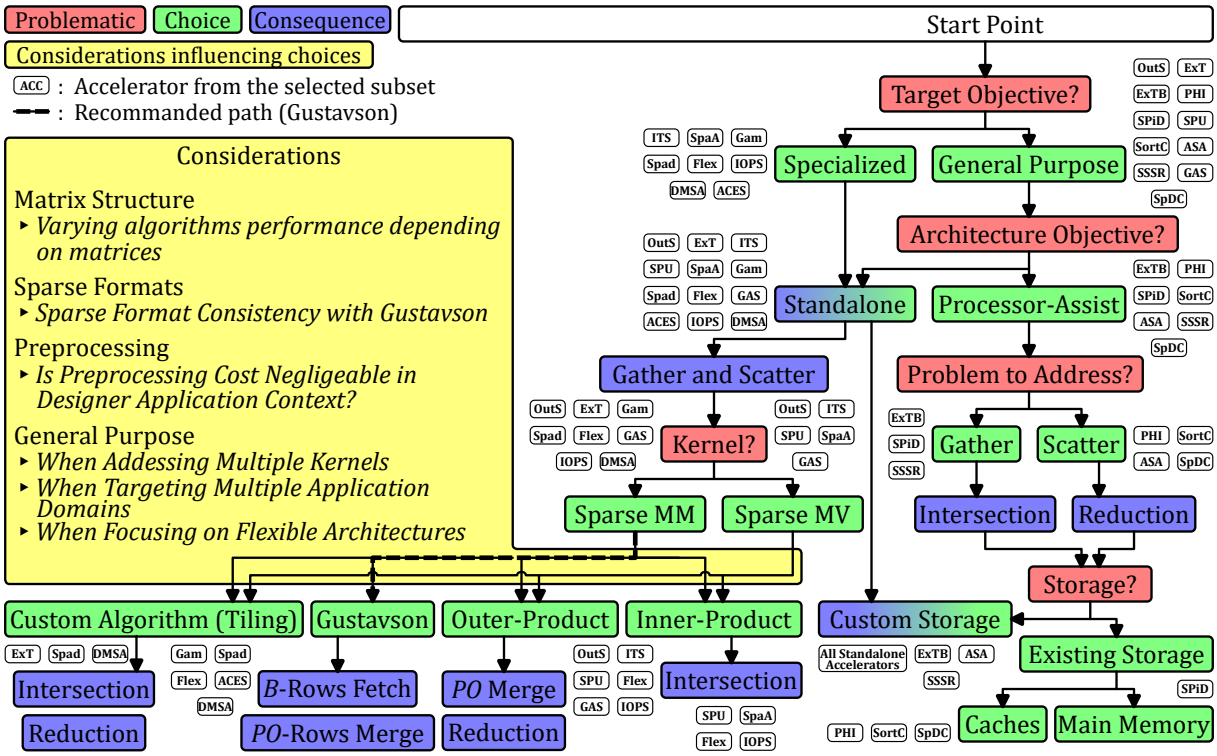


Figure 2.8 – Flow Diagram for Architecture Selection

mance for a specific task (*e.g.*, ExTensor bricks, PHI, SPiDRE, SortCache, ASA, SSSR and SpDCache). With a *processor-assist* (rightmost branch in the figure), the processor still performs important parts of the kernel, typically the multiplications, but off-loads the *gather* or *scatter* tasks to the accelerator, as these are the tasks which are the bottleneck with a traditional cache hierarchy. For example, for an **IP** algorithm, the difficult part of the *gather* task is performing the *intersections* (*e.g.*, the ExTensor bricks, SPiDRE and SSSR). Conversely, with **OP** or **Gust**, the challenge with the *scatter* is to efficiently perform *reductions* (*e.g.*, PHI, SortCache, ASA and SpDCache). Finally, the designer must decide whether the storage is done by reusing existing memory resources (*e.g.*, PHI, SPiDRE, SortCache and SpDCache) or through custom caches and memories.

Going back to the case of a *standalone* accelerator, the algorithms differ according to the supported kernels. For sparse MV, most of the accelerators studied here have opted for an **OP** algorithm. For sparse MM, the designers of nine of the eleven highest performing accelerators have chosen the **Gust** algorithm, as this algorithm provides a good trade-off, simplifying the *gather* by reusing input B-rows and the row-by-row generation of the output, simplifying the *scatter* aspect. The fact that the outputs are generated row-by-row, facilitates parallel implementations. **Gust** works well with CSR or CSC, providing consistent input and output formats. Beyond the basic algorithm, *tiling* approaches or new dataflows mixing algorithms can be explored (*e.g.*, ExTensor, SPU, Spada, Flexagon, IOPS and DMSA).

Orthogonal to previous choices, other design considerations are important. First, the starting point needs to be defined: some accelerators require software to perform *tiling* (*e.g.*, ExTensor), to align data with DRAM-blocks (*e.g.*, SpaceA), or to rearrange the rows to improve performance (*e.g.*, Gamma). The cost of such preprocessing can be high, and is usually only justified if the same matrix is reused multiple times.

CONCLUSION

This analysis provides a structured decision-making framework for designers, emphasizing that the choice between specialized (*standalone*) and general purpose (*processor-assist*) accelerators **depends on performance needs, area constraints, and problem-specific requirements**, with algorithm selection (*e.g.*, *Gustavson* for sparse MM, *Outer-Product* for sparse MV) and memory optimization strategies playing pivotal roles in tailoring solutions to diverse sparse computing challenges.

2.4 Conclusion

THE rapid expansion of data-intensive applications in computational physics, machine learning, and graph analytics has driven the development of numerous hardware accelerators over the past ten years, each aiming to surpass the performance limitations of traditional CPUs and GPUs. This chapter presented a comprehensive review of the most significant designs, offering a structured comparison and highlighting the core hardware challenges associated with sparse matrix multiplication kernels.

Our analysis reveals that the primary challenge in sparse matrix computations lies in efficiently managing memory access patterns, particularly the dereferencing required for sparse formats. Depending on the chosen algorithm, *Inner-Product* (`IP`), *Outer-Product* (`OP`), or *Gustavson*'s (`Gust`), the main bottleneck shifts between *gather* and *intersection* operations (for `IP`) or *scatter* and *reduction* operations (for `OP`). *Gustavson*'s algorithm emerges as a balanced solution, as it reduces the number of B-rows that need to be accessed while generating outputs row-by-row, simplifying both *gather* and *scatter* processes. The most advanced *Gustavson*-based accelerators further optimize performance through intelligent cache eviction strategies, where victims are selected based on forward-looking analysis of A-indices.

While *standalone* accelerators dominate the landscape, *processor-assist* designs, such as those that modify cache behavior or leverage existing memory resources, also play a crucial role, particularly in systems where flexibility and lower area overhead are prioritized. In contrast, high-performance *standalone* accelerators often feature dedicated, massively parallel *reduction*/merger units to maximize parallelism and minimize memory bottlenecks.

The diversity of existing designs underscores that there is no single "best" sparse accelerator. Instead, the optimal architecture depends on the specific problem class, matrix structure, and performance requirements. Future advancements will likely focus on tailoring accelerators to the unique characteristics of different matrix structures, ensuring that caches, memory hierarchies, and computational units are optimized for the workload at hand.

This chapter not only summarizes the current state of sparse matrix acceleration but also provides a roadmap for future research, emphasizing the need for architectures that are both highly efficient and adaptable to the diverse and evolving landscape of sparse data processing.

TAKEAWAYS

Classification of Sparse Matrix Accelerators: Hardware accelerators for sparse matrix computations can be categorized by:

- **Autonomy:** *standalone* versus *processor-assist*
- **Position in memory hierarchy:** near-processor, near-cache, or near-main memory

- **Algorithm compatibility:** *Inner-Product* (`IP`), *Outer-Product* (`OP`), or *Gustavson* (`Gust`)
- **Supported Memory Flows:** *gather*/*scatter* hardware support
- **Targeted kernels:** sparse Matrix-Vector (MV), sparse Matrix-Matrix (MM), or both

Key Findings:

- **Algorithm Selection:** *Gustavson*'s algorithm offers the best trade-off for hardware implementation, balancing input reuse and output generation, while the *Outer-Product* algorithm remains a strong alternative, particularly for parallelism.
- **Memory Optimization:** Efficient use of local memory, whether through custom caches or smart eviction policies, is critical to reduce off-chip traffic.
- **Adaptability:** The most successful accelerators dynamically adjust their dataflows and memory strategies to match the sparsity patterns of the input matrices, highlighting the importance of adaptability.
- **Scalability and Parallelism:** High-performance designs leverage parallel *reduction* and merger units and optimized memory hierarchies.

Future Research Directions: Next-generation accelerators will increasingly feature **adaptive mechanisms**, **multi-core architectures**, and **specialized memory systems** tailored to diverse sparse matrix structures and evolving application requirements.

SpDCache: Our work focuses on near-cache, *processor-assist* accelerators that computes *scatter*-focused *Outer-Product* SpMV kernels.

CHAPTER

ARCHITECTURAL PRESENTATION OF SPDCACHE 3

Contents

3.1 Introduction

IN the previous chapters, we analyzed the causes and effects of irregular memory accesses in sparse computations and identified that the *Outer-Product* (**OP**) algorithm of Sparse Matrix-Vector multiplication (SpMV) ($A \times b = c$) has potential for high performance and optimization. The **OP** algorithm generates irregular updates on the output vector c, called "random" *reductions*, as each non-zero element of the input matrix A produces updates to distinct, and often distant, location in the output vector c. These "random" *reductions* render stress the memory system.

Conventional cache hierarchies are designed for regular, predictable access streams that exploit spatial and temporal locality. In contrast, **OP** SpMV exhibits poor locality, leading to low cache utilization, frequent evictions, and thus requiring more memory bandwidth for the output vector c.

Previous works, such as Coup [134], PHI [86], SortCache [108], and ASA [132], have proposed *reduction*-aware cache mechanisms to alleviate irregularity by improving accumulation and storage efficiency. However, these solutions do not exploit the structural properties of the processed matrices, and therefore miss optimization opportunities when the matrix has some regularity. This observation motivates our focus on banded sparse matrices, which combine

overall sparsity with a dense region near the diagonal. Banded matrices are very common and serve as an initial example to study how dense and sparse regions can be leveraged to enhance performance.

This chapter begins by reviewing the main concepts of generic data-caches, introducing certain terminology and mechanisms which are required to discuss cache behavior under irregular access conditions. It then presents a *reduction*-aware cache architecture, designed to efficiently support "random" *reductions* for the `OP` SpMV kernel. The proposed design, SpDCache (presented in Paper II), extends traditional caching models with a region-based organization and native *reduction* support. Finally, we evaluate its performance through high-level modeling on banded and non-banded matrices, highlighting the influence of data regularity on cache efficiency.

3.2 Generic Data-Caches

MODERN processors and accelerators integrate data-caches to bridge the latency and bandwidth gap between computation units and main memory [49]. A cache is a small, fast memory that stores recently accessed data anticipating that the data will be reused in the near future. Its efficiency relies on two main forms of locality:

- temporal locality, where recently accessed data is likely to be reused within a short interval of time, and
- spatial locality, where data stored near recently accessed addresses is likely to be used next.

By exploiting these properties, caches reduce the average access latency and increase the overall bandwidth efficiency.

A cache is composed of several cache-lines, each grouping contiguous memory words. The cache-line is the minimum granularity of data movement between memory levels. When an access occurs, the cache controller searches for the requested address among the currently stored cache-lines. If the address is found, a hit occurs and the data is served directly to the core. Otherwise, a miss occurs, triggering a memory fetch from a lower cache level or from main memory. If necessary, an existing cache-line is evicted to make room for the new data. Each cache-line is associated with a *tag* that records its address and additional bits to keep track of its state.

Several mapping policies exist. In a direct-mapped cache, each memory block maps to a single cache-line. This approach is simple to implement but results in conflicts and poor utilization. A n-*way set*-associative cache allows each block to be placed in one of n possible cache-lines within a *set*, improving hit rate with only a moderate additional hardware cost. A fully-associative cache removes these constraints, as any block may occupy any cache-line, but requires more complex *tag* searches. In all cases, when a line must be evicted and multiple candidates exist, a replacement policy such as *Least Recently Used* (*LRU*) or *Random* decides which entry is evicted.

Caches have different policies when writes occur. With a *write-through* policy, every store operation updates both the cache and main memory, maintaining consistency but increasing traffic. A *write-back* policy delays the update until eviction, reducing memory bandwidth at the cost of additional state bits (dirty flags) to maintain consistency.

For workloads dominated by sequential accesses, these mechanisms and policies are highly effective. However, irregular accesses tend to degrade cache efficiency by increasing

miss rates and provoking frequent evictions.

Modern processors implement hierarchical cache organizations to balance latency, capacity, and throughput. A typical CPU integrates small L1 caches close to each core, larger L2 caches shared by a group of cores, and a high-capacity L3 cache shared at the chip level. Accelerators and domain-specific architectures often employ local data storage or scratchpad memories serving the same purpose, providing low-latency access to frequently reused data (see Chapter 2).

The generic cache assumes regular read and write operations with predictable memory access patterns. Yet workloads operating on sparse data violate this assumption. In particular, "random" *reductions* often have poor performance because each *reduction* triggers a read and a write of a full cache-line, to simply update one word. Furthermore, the lack of spatial locality means each *reduction* can trigger an eviction, in the worst case.

In the following sections, we introduce cache architectures with native *reduction* support, specifically designed to handle irregular data. These architectures extend the classical cache model with *reduction*-aware mechanisms, improving bandwidth efficiency under irregular access patterns.

CONCLUSION

Generic data-caches are effective when access patterns exhibit spatial and temporal locality. This is not the case for sparse data and their performance is particularly poor for "random" *reduction* operations. This limitation motivates the introduction of ***reduction*-aware cache architectures** which are optimized for such "random" *reductions*.

3.3 Reduction Cache

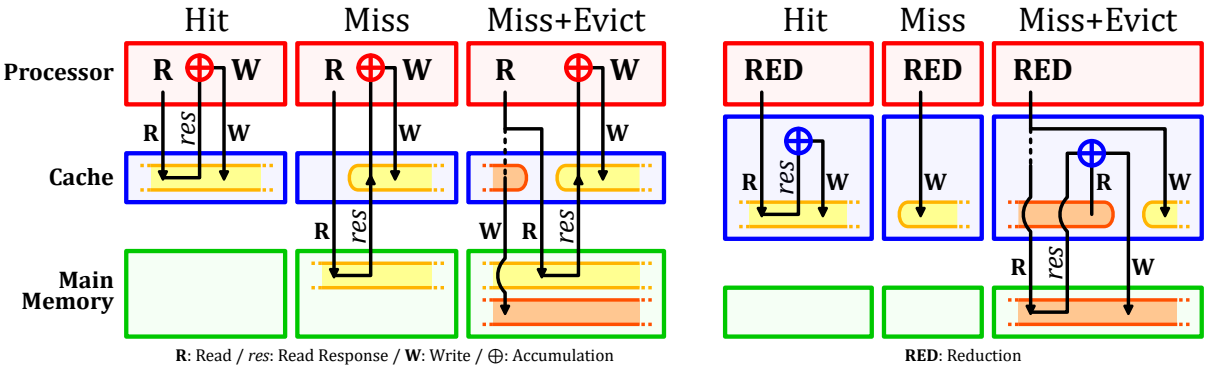
PERFORMING a *reduction* in hardware requires a read from memory, followed by an accumulation and a write back. This operation is denoted $RED(key, value)$ [108] with the *key* being the position in the output vector c.

In Figure 3.1a, we show three cases that occur in a generic data-cache when a *reduction* is performed. If the vector entry (c_{key}) hits in the cache, the update is performed by the processor (in *red*) with no external memory access (on the left in the figure). When there is a miss, the operation is blocked until the read from main memory completes (in the middle of the figure). Potentially, the miss will also trigger an eviction, which exacerbates the blocking condition, because a cache-line must first be written back, before the read from main memory can be performed (on the right in the figure). In scientific applications, the matrices are extremely large and only a small portion of the vector c can reside in the cache, thus the *miss+evict* case arises frequently.

A *reduction cache*, such as [108] or [86], is a data-cache which accepts *reduction* instructions (*RED*) from the processor and performs accumulations locally (near-memory). In the case of a hit, see Figure 3.1b, the *reduction* takes place in the cache (in *blue*, on the left in the figure). For a miss, the cache allocates a new line with zeros, and simply inserts the value (in the middle of the figure). In this case, the cache-line is inconsistent with main memory. Only when the line needs to be evicted, is a read performed (on the right in the figure). In the cache, the values in the line are accumulated with the main memory data, and the result is written back.

As seen in the figure, for hit and *miss+evict* cases, the number of memory transactions

is the same as a generic cache. However, a *reduction cache* reduces the number of main memory accesses in the case of a miss, by postponing the main memory update until eviction. Instead of having the classic ascendant policy where data is stored in the cache to be pulled-up again by the processor latter-on, the *reduction cache* has a descendant policy: data is written/updated in the cache to be pushed-down to memory latter-on. Instead of two memory transactions that can stall the processor, the *reduction cache* only needs a single "fire-and-forget" instruction, meaning it can be issued without waiting for an answer, thus avoiding processor stalls. At the end of the algorithm, the *reduction cache* must be flushed.



(a) Generic Cache (Write-Back) **(b)** Reduction Cache

Figure 3.1 – Read (R) and Write (W) Transactions of a *Reduction* Operation in Generic and *Reduction* Caches

DEFINITION

A ***reduction cache*** is a data-cache containing an arithmetic unit and a dedicated support for *reduction* operations. One implementation is to provide the processor with a *RED* instruction which tells the cache to perform the *reduction*. For the processor, this instruction is "fire-and-forget".

CONCLUSION

To improve the processing efficiency of *reduction* operations, a ***reduction cache*** can be used to **avoid unnecessary memory transactions and stalls**, by offloading the accumulation near-memory, and updating the main memory only at eviction.

3.4 Outer-Product SpMV with a Reduction Cache

A widely used sparse format to compress the non-zero elements in a matrix A is CSC [99], where A is a sparse matrix of dimensions (M, N) with a number of non-zero elements nnz. As seen in Chapter 1, it consists of three tables (val, idx, ptr) used to store non-zero values, row indices and column positions. When performing an `OP` SpMV ($A \times b = c$) with this format (Algorithm 3.1), we see that a *RED* is needed to accumulate the output vector c (line 6). Moreover, A is read sequentially while c is updated "randomly". In fact, the *key* is determined via an indirection with an access to idx.

In memory, A is stored as three dense tables (CSC), but conceptually, as we perform the `OP` multiplication, the matrix is traversed from top to bottom, then left to right. A row in the matrix corresponds to a *key*, or equivalently, a position in the output vector c. In Figure 3.2,

Algorithm 3.1: *Outer-Product* SpMV – Single Threaded

```
1 for n ∈ [[0, N − 1]] do // Loop over the columns of A
2     for pos ∈ [[ptr_n, ptr_{n+1} − 1]] do
3         key ← idx_pos; // Row index
4         val ← val_pos × b_n; // val = A_key,n × b_n
5         RED(key, val); // c_key += val
```

we show an example of a sparse matrix with a few non-zero values (① - ⑥). As we traverse the matrix, we first perform a *reduction* for the value on row key_{10}, then on row key_{20} and then key_{30}. Each of these results in a *RED* on c. As we continue, we then hit another value ⑤ on the row key_{10} which must be *reduced* with an existing entry ① (as shown on the left of Figure 3.2, *Conceptual View*).

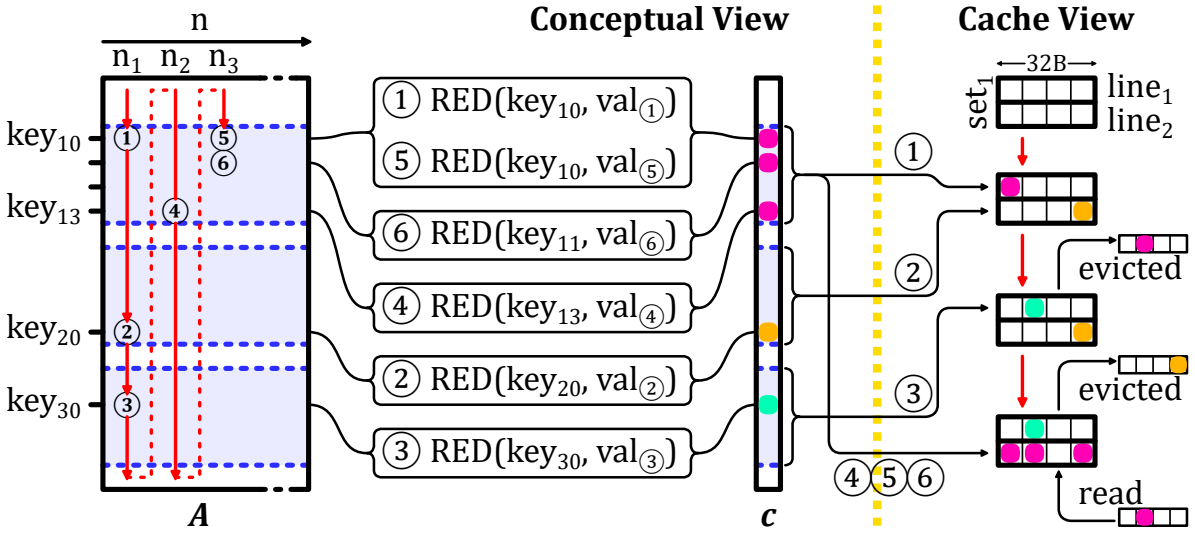


Figure 3.2 – Traversal of Non-Zero Elements of Input Matrix for `OP` SpMV (left) and Impact on Cache (right)

Let us consider how c is stored in a two-*way* *set*-associative cache with 32B lines. We assume that all the *keys* map to set_1. When ① is updated, there is a miss and $line_1$ is allocated for key_{10}. Similarly with ②, $line_2$ is allocated for key_{20}. When key_{30} comes along, an eviction must be performed. We assume $line_1$, holding key_{10}, is evicted. We then come to ④ which has key_{13}, adjacent in memory to key_{10}, and thus stored in the same line. This line must be brought back to process ④, ⑤ and ⑥.

This example illustrates how a conventional cache can be inefficient. First, we note the line holding key_{10} was evicted, although it was frequently accessed after ①. This eviction was triggered by key_{30}, which was only accessed once. Furthermore, for key_{20} and key_{30}, a full line was loaded, only to modify a single entry. A "smarter" cache would have maintained the line with key_{10} to avoid an unnecessary main memory access. Furthermore, this "smarter" cache would not have allocated full lines for key_{20} and key_{30} to only modify a single word.

The globally low local and temporal localities prevent caches, and even *reduction caches*, to take advantage of any reuse opportunity. However, sparse matrices have various types of structure which may have reuse potential in some parts of the matrix. In the example in Figure 3.2, the first rows (with key_{10} to key_{13}) are denser than the rest of the matrix. A "smarter" cache would prefer to focus on these rows, keeping the corresponding cache-line

and accumulating every elements before a final eviction. On the contrary, the sparser rows of the matrix would not be kept by the "smarter" cache, thus avoiding thrashing data from denser regions.

The matrix structure is key to designing an efficient *reduction cache* for `OP` SpMV. Going forward, we focus on the case of banded matrices, a widely used type of sparse matrix, which have a denser region around the diagonal. For a *reduction cache* to work efficiently with such matrices, the dense band should be prioritized, and the isolated elements out of the band should be processed separately to avoid these sparse elements disturbing the storage of the dense data. This is the main idea of SpDCache (Paper II), which is a *reduction cache* with dedicated memory regions for the in-band and out-of-band of a matrix. We describe its behavior in the following sections.

CONCLUSION

When computing an `OP` **SpMV kernel** on a very **large matrix**, a **conventional *reduction cache* does not provide enormous benefit** because the cache is not large enough to reuse data from one column iteration to the next.

We propose **SpDCache**, a *reduction cache* with dedicated memory regions, **leveraging matrix structure** with dense and sparse regions which correspond to the structure of **banded matrices**.

3.5 Architecture of SpDCache

We design a specialized data-cache architecture, called SpDCache, that exploits the two-region structure of banded matrices to reduce memory traffic during *reductions* operations on the output vector c. To better understand SpDCache behavior, our study focuses on a single-level cache hierarchy with a single core executing SpMV kernels on banded matrix case. However, SpDCache is designed to extend to multilevel memory hierarchies and multi-core architectures (see Chapter 6), and to adapt to other region configurations. As an isolated data-cache, SpDCache must satisfy three key requirements:

- **Generic cache functionality:** SpDCache operates as a standard cache for load and store instructions from the processor, enabling sequential reads of matrix A and vector b to leverage spatial locality.
- **Reduction capability:** SpDCache functions as a *reduction cache* to minimize inherent memory traffic costs of *reduction* operations on output vector c.
- **Region-wise reduction:** Dedicated cache regions for sparse and dense matrix regions mitigate the irregular access patterns of *reductions*.

As shown in Figure 3.3, SpDCache is divided into three memory regions. The first, *Generic Region Cache* (*GRC*), is for all memory transactions unrelated to the output vector c, including reads of A and b. It is a classic, *set*-associative cache and is required for the processor to operate normally, but is not the focus in this chapter. The other two regions, called *Dense Region Cache* (*DRC*) and *Sparse Region Table* (*SRT*), are able to perform *reductions* on c. The *DRC* works as a line-wise, *set*-associative *reduction cache* to hold high density data in the *In-Band Region* of the matrix (*IBR*, *orange*). The difference with the *GRC* is that it is able to perform *reductions* and it is dedicated to the *banded* region, so that spurious accesses outside of the band do not provoke undesired evictions. Finally, the *SRT* works as an element-wise,

fully-associative table whose role is to handle sparse regions, such as the *Out-of-Band Region* in the matrix (*OBR*, *blue*). This region is well-suited for handling isolated data and reducing memory traffic, with further analysis of this design choice presented in the next chapter.

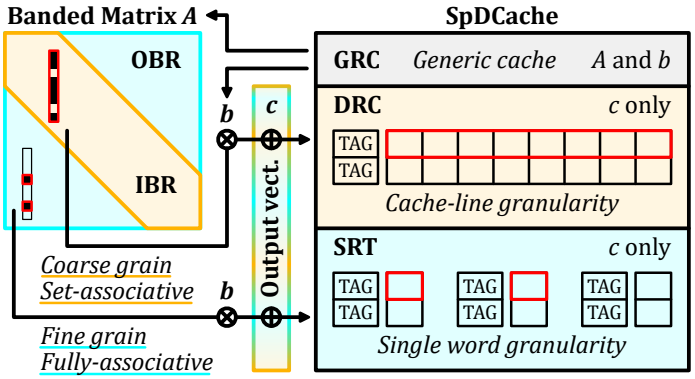


Figure 3.3 – Data-Cache Partitioning into Regions (*GRC*, *DRC*, *SRT*)

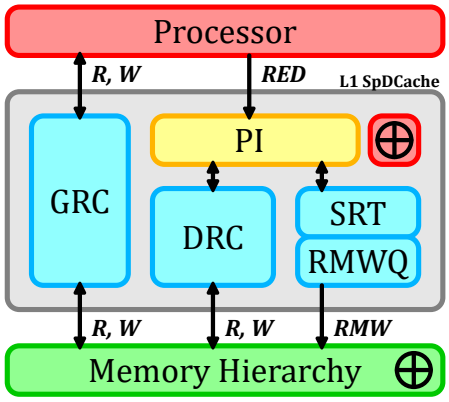


Figure 3.4 – Simplified View of the Memory Hierarchy with SpDCache

Figure 3.4 shows a simplified view of a memory hierarchy with SpDCache. The *Processor Interface* (*PI*) takes *reduction* instructions, *RED(key, value, region)*, from the processor. The *RED* instruction contains a 'region' field which indicates whether the value is expected to be in a dense region (*DRC*) or a sparse region (*SRT*). As we will see, the hardware may override the software selection to avoid duplication. The *DRC* communicates with the memory hierarchy using read and write transactions over entire cache-lines, to propagate the local *updates* to the downstream memory. As *SRT updates* are for isolated elements, it does not make sense to bring a full line into the L1 to modify a single word. The *SRT* thus propagates *updates* by issuing atomic, element-wise *Read-Modify-Write* transactions (*RMWs*), which must be processed remotely and close to the main memory. This is a stringent requirement for the memory hierarchy, but it is realistic. Indeed, existing protocols such as AXI-4 [2] already support atomic transactions for certain operations (logical, integer add, min, max) and these could be extended to include floating point addition. Moreover, this requirement becomes more manageable when extending SpDCache to multilevel cache hierarchies (see Chapter 6). The *RMWs* from the *SRT* are temporarily buffered in a small queue (*RMWQ*), to avoid congestion.

SpDCache is driven by three key design principles. First, we ensure a given *key* is in only one region at a time, to avoid coherency problems. Second, the *DRC* is given priority over the *SRT*, to enable coalescing. Third, we avoid bringing a full line from memory to the L1 to simply update one or a few entries.

In Figure 3.5, we describe the operation of SpDCache, via a series of scenarios, each shown in a different color. When a new *reduction RED(key, value, region)* is received from the processor, we test for a hit in the *DRC*, regardless of the *region* argument. If it hits, the value (case `0b` – *turquoise*) can be accumulated in the corresponding cache-line of the *DRC*. We give this priority to the *DRC* as part of the mechanism to avoid duplication.

If there is a miss in the *DRC*, then the target region is determined based on the *region* argument of the *RED* transaction. If it is *IBR*, then it is necessarily a miss case, and the value (`02` – *orange*) must be inserted into a free cache-line of the *DRC*. When we allocate this line, we ensure that it does not create a duplication of existing entries in the *SRT*. If there is

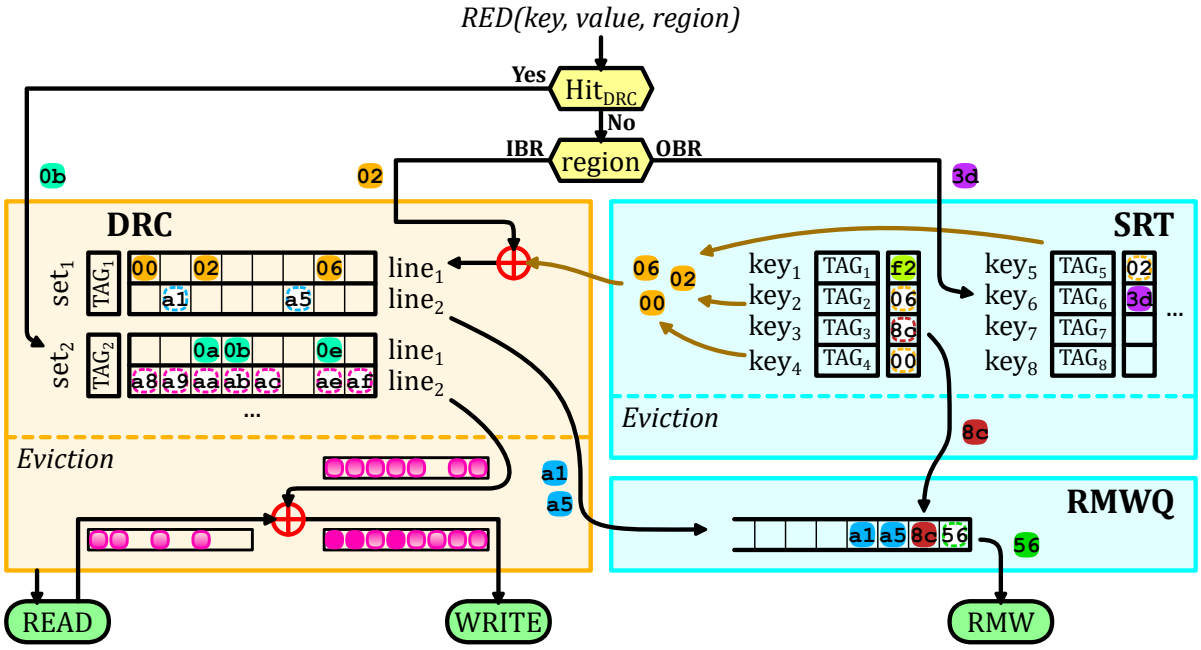


Figure 3.5 – Internal Operation of SpDCache with Examples

duplication, as in our example, these entries (`00`, `02`, `06`) are deleted from the *SRT* and merged into the newly allocated cache-line ($set_1, line_1$).

If the target region is the *OBR*, then the *reduction* (`3d` – *purple*) is processed in the *SRT*, either allocating a new entry (miss case) or accumulating in cache with an existing entry (hit case).

Now, we describe the eviction scenarios. In the *DRC*, the line to evict is selected ($set_2, line_2$ – *pink*), a read to main memory is performed, the values from the evicted cache-line are accumulated and a write to main memory is issued. If the evicted cache-line only has a few non-zero elements and the *RMWQ* has enough free entries, we enter into a special case. To avoid bringing a full line from main memory, the non-zero values (`a1` and `a5` – *blue*) are converted to atomic *RMWs* and pushed onto the *RMWQ*. The decision to convert evicted cache-lines to atomic *RMWs* is based on a configurable density threshold. PHI [86], an accelerator similar to SpDCache, used a threshold of one non-zero word per eight words in a cache-line. Given our queuing mechanism to handle congestion, we use a more aggressive threshold of three non-zero words per cache-line. This threshold is chosen empirically, but its specific value has minimal impact, as our evaluation shows that very few elements are transferred through this mechanism.

The width of the *band* (w_B), must be selected so that one column of the *band* remains blocked in the *DRC*, otherwise, this region of the cache is always thrashing. This is our approach to selecting w_B, which we detail in the next chapter. Note that this approach selects w_B based on the *DRC* size, independently of the matrix and its structure.

In the *SRT*, an eviction simply consists of moving the element from the table to the *RMWQ* (`8c` – *brown*). If the *RMWQ* is not full, *SRT* evictions are issued in advance when the *SRT* table is nearly full, using another configurable threshold set to the *SRT* size minus half the size of the *RMWQ* in our experiments. The *RMWQ* issues atomic *RMWs* when it is not empty (`56` – *dark green*).

NOTE

We make the assumption that memory regions in the *GRC* and the *DRC*/*SRT* are disjoint. Currently, we do not manage coherency between the two, although this could be a future extension.

NOTE

SpDCache is not limited to banded matrices. Any sparse matrix with a denser region can benefit from this architecture, by adapting the definition of the *band* accordingly, and making sure that the blocking effect in the *DRC* is preserved. The selection of where the dense region is located is done in software via the region argument of the *RED* instruction.

CONCLUSION

SpDCache is a data-cache splits in **three regions**:

- *GRC*, a **generic** cache region which handles the reads of A and b and any other memory accesses performed by the processor,
- *DRC*, a *reduction cache* region dedicated to *reductions* on c when computing the dense region of the matrix, which is the **band** (*IBR*) in the cases of banded matrices,
- *SRT*, a *reduction cache* region dedicated to *reductions* on c when computing the sparse regions of the matrix, which correspond to the **out-of-band** elements (*OBR*) for a banded matrix.

The two last regions use **memory transactions adapted to the region density** (line-wise or element-wise) to avoid transferring unnecessary data.

3.6 High Level Evaluation of SpDCache

To validate the SpDCache architecture, we simulate its behavior to assess memory traffic gains and explore various parameters. Combined with the theoretical analysis presented in the next chapter, this approach enables us to validate both the principle and performance of SpDCache. We modelled the behavior of SpDCache based on the flowchart in Figure 3.5, using a transaction level model in Python, for a single-core, single-thread system executing an `OP` SpMV kernel (Algorithm 3.1). We do not model timing, as the order of the memory accesses determines the behavior of the cache (hits, misses, evictions), which depends only on the SpMV algorithm. The model checks the final numerical result and reports the main memory traffic associated with the *updates* to the output vector (*Output Memory Traffic*), where we count the bytes transferred (64B, 8 words of 8B, for each read and write, and 8B per atomic *RMW*, with our configuration). For each write transaction, it tracks how many words in the line were actually used (*Evicted Cache Line Utilization*) and it tracks how many words in the cache held useful data (*Cache Memory Utilization*). We consider a word in the cache to be useful if it is updated or inserted by the processor. Our goal is to validate the principle of SpDCache using real application matrices from the SuiteSparse Matrix Collection [67].

As baselines, we implemented a fully generic cache (GC) that handles reads and writes for both inputs (A, b) and output (c), following the data movements shown in Figure 3.1a, and a simple *reduction cache* (RedC) that processes *RED* transactions in a dedicated region using the flow shown in Figure 3.1b. In the RedC, 25% of the memory is allocated for the inputs (A and b) and the other 75% for *REDs* on c. In SpDCache (SpDC), 25% is allocated for the *GRC*, 50% for the *DRC* and 25% for the *SRT*. In the next section, we present more results with varying

region sizes, confirming that this allocation is indeed a good compromise. All three caches use a *Least Recently Used* (*LRU*) eviction policy, except the *SRT* region of SpDCache which uses a *random* policy. All three caches have a total data storage of 32 KiB, not including the storage. The *set*-associative regions have 4 *ways* and 64B cache-lines (8 *double* words per line).

We simulated 48 matrices that have been used by the state-of-the-art accelerators [7, 48, 91, 108, 109, 132, 133], which are listed in Appendix B. We excluded matrices whose output vector c fits entirely in the cache (fewer than 4096 elements), since they do not stress the cache. In Table 3.1, we report the results for all three caches, along with statistical indicators. Most, but not all, of the matrices have a dense band and even some matrices without an obvious band benefit from the blocking effect in the *DRC*.

Table 3.1 – Decrease in Memory Traffic and Improvement in Cache Utilization from Python Simulation

	Output Memory Traffic			**Evicted Cache Line Utilization**			**Cache Memory Utilization (%)**		
	RedC	**SpDC**		**GC**	**RedC**	**SpDC**	**GC**	**RedC**	**SpDC**
	%/GC	*%/GC*	*% RMW*	*Avg*	*Avg*	*Avg*	*Avg*	*Avg*	*Avg*
Geometric Mean	**89.2**	**24.5** [a]	40.5	3.60	3.56	7.21 [b]	36.4	45.8	75.2 [c]
Standard Deviation	29.8	20.9	33.8	2.45	2.77	2.08	10.9	32.3	18.6
Minimum	28.5	7.00	0.00	1.06	1.00	4.36	16.6	11.9	30.0
Maximum	172	98.4	100	8.00	8.00	8.00	49.6	97.5	98.9
First Quartile	76.0	11.4	35.9	1.76	1.73	6.03	28.4	22.0	65.5
Third Quartile	115	46.6	91.7	6.60	7.65	8.00	48.6	88.8	93.9
Outperform GC	45.8	100	Ø	Ø	45.8	91.7	Ø	64.6	100
Outperform RedC	Ø	93.8	Ø	Ø	Ø	85.4	Ø	Ø	77.1

[a] *Decrease by 4.1× compared to GC, and 3.6× compared to RedC.*
[b] *Increase by 2.0× compared to GC, and 2.0× compared to RedC.*
[c] *Increase by 2.1× compared to GC, and 1.6× compared to RedC.*

As expected, the simple *reduction cache* (RedC) decreases the main memory traffic to 89.2% of the GC baseline (10.8% decrease). SpDCache has significantly better performance than RedC, reducing the traffic to 24.5% (75.5% decrease). SpDCache thus achieves a 4.1× decrease in memory traffic compared to GC, and a 3.6× decrease compared to RedC, on average over the set of matrices. SpDCache outperforms GC on all matrices, and RedC on 93.8% of them.

The benefits of SpDCache can be partly explained by the improved utilization of the cache-lines. On average, a cache-line evicted from SpDCache has 7.21 modified words, compared to 3.56 and 3.60 with the baselines. This improved utilization is the result of blocking the dense region in the *DRC*, where multiple *reductions* have the time to accumulate in the cache before being evicted. The *SRT* also contributes to this effect, as it captures irregular accesses outside the band, which would otherwise evict useful data from the *DRC*. The overall cache memory utilization is also significantly improved, with 75.2% of the cache holding useful data in SpDCache, compared to 45.8% and 36.4% for RedC and GC, respectively.

We need to consider an additional metric, the percentage of traffic sent as atomic *RMW* transactions (% *RMW*). Indeed, if this value is too high, it means we have simply shifted the workload to the lower level caches, which inherently have limited compute capability. On

average, 40.5% of the memory traffic of SpDCache is composed of atomic *RMWs*, which is we consider acceptable given the significant overall reduction in memory traffic. However, this ratio varies widely between matrices. There are some matrices where most of the traffic is composed of atomic *RMWs*, meaning that most of their non-zero values are outside the band. For such matrices, which are basically not banded, SpDCache still greatly decreases the memory traffic, but better performance could be achieved by choosing a matrix-fitting region partitioning strategy. For example, the matrix *cit-Patents* has no elements in the band, and all non-zero values are scattered under the bottom part of the matrix (an unusual extreme case). While having a memory traffic decreased to 7.2% compared to GC, it could benefit from a better use of the *DRC* by choosing a horizontal band covering the non-zero elements, with correct sizing to leverage the blocking effect. On the other hand, some matrices have very low *RMW* usage, indicating that most non-zero values are within the band and efficiently handled by the *DRC*. In this case, 25% of the available memory was unused. The *SRT* size could be adjusted to gain performance.

CONCLUSION

SpDCache achieves a 4.1× **decrease in memory traffic** on the output vector c compared to a generic cache, over a set of 48 matrices. It also **outperforms a simple *reduction cache*** by 3.6× , showing the importance of having separate dedicated cache regions for the in- and out-of-band of a matrix.

3.7 State-of-the-Art Comparison and Region Sizing

To compare SpDCache with the state-of-the-art, we simulated PHI [86], the existing solution that is closest to our proposal, using the same methodology as in the previous section. PHI is a *reduction*-aware cache that uses a single region to process *RED* transactions (see Section 2.2.4 from previous chapter). In PHI, the cache is not separated into regions, however, it treats load/store and *RED* operations differently. It also handles isolated elements, but do not consider the matrix structure. Unlike SpDCache, PHI relies on the *deferred update* version of the `OP` SpMV algorithm (see Algorithm 1.8). We thus adapted our simulation to use this algorithm for PHI. To ensure a fair comparison, we allocated the same amount of memory to PHI as for SpDCache and the baselines (32 KiB). In Table 3.2, we report the results for PHI alongside our previous results (columns n° 1, 2, 5 and 6).

As expected, PHI (n° 5) achieves a significant reduction in memory traffic compared to the GC (n° 1), a 50.4% reduction on average. This represents a 2.0× decrease compared to the GC, which is consistent with the results reported in the original PHI paper [86], and a 1.8× decrease compared to the RedC (n° 2). However, SpDCache (n° 6) still outperforms PHI, cutting memory traffic by 2.1× . The improved performance of SpDCache can again be explained by the better utilization of the cache-lines, with 7.21 useful words per evicted line, compared to 5.00 for PHI. The overall cache memory utilization is also significantly better with SpDCache, with 75.2% of the cache holding useful data, compared to only 30.6% for PHI.

As explained in Chapter 2, PHI mixes the 'immediate updates' and 'deferred updates' strategies for `OP` SpMV. In the first phase of the algorithm, it processes *RED* transactions as 'immediate updates', and PHI handles them with a mechanism similar to a classical *reduction cache*. The main differences are that the *GRC* and *DRC* are merged, which can cause unnecessary evictions of useful data, and more importantly, PHI uses a batching mechanism at eviction to

Table 3.2 – All Results from Python Simulation

GMeans (48 mat.)		**GC**	**RedC**			**PHI**	**SpDC**					
sets	*GRC*	***128***	***32***	*8*	*4*	***128***	***32***	*8*	*8*	*4*	*4*	*4*
	DRC	***Ø***	***96***	*120*	*124*		***64***	*104*	*112*	*92*	*116*	*122*
	SRT	***Ø***	***Ø***	*Ø*	*Ø*		***32***	*16*	*8*	*32*	*8*	*2*
Output Memory Traffic (% over GC)		100 ■1.0	89.2 ▼1.1	81.6 ▼1.2	80.3 ▼1.3	50.4 ▼2.0	24.5 ▼4.1	24.3 ▼4.1	25.5 ▼3.9	23.2 ▼4.3	25.3 ▼4.0	32.0 ▼3.1
Ratio of RMWs in Memory Traffic (%)		Ø	Ø	Ø	Ø	Ø	40.5	35.7	36.5	36.5	36.2	41.1
Total Memory Traffic (% over GC)		100 ■1.0	93.9 ▼1.1	91.9 ▼1.1	93.2 ▼1.1	98.8 ■1.0	56.1 ▼1.8	56.6 ▼1.8	56.9 ▼1.8	57.7 ▼1.7	58.2 ▼1.7	59.2 ▼1.7
Evicted Cache Line Utilization (max: 8)		3.60 ■1.0	3.56 ■1.0	3.66 ■1.0	3.67 ■1.0	5.00 ▲1.4	7.21 ▲2.0	7.21 ▲2.0	7.23 ▲2.0	7.22 ▲2.0	7.22 ▲2.0	7.25 ▲2.0
Cache Memory Utilization (%)		36.4 ■1.0	45.8 ▲1.3	47.0 ▲1.3	47.1 ▲1.3	30.6 ▼0.8	75.2 ▲2.1	68.3 ▲1.9	65.0 ▲1.8	72.5 ▲2.0	65.1 ▲1.8	60.9 ▲1.7
Column nº		*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*

'defer' some updates to the second phase. When a cache-line is evicted and contains fewer non-zero entries than a fixed threshold, PHI batches the entries in *partial outputs* (*POs*) defined by the software and stored in memory. These *POs* are processed in the second phase of the algorithm, as in the 'deferred updates' variant. If the threshold is set too high, PHI batches many entries and increases second-phase memory traffic. If it is set too low, PHI batches few entries, reducing batching effectiveness and increasing the number of 'immediate updates', making PHI behavior closer to a *reduction cache*. PHI sets this threshold to one non-zero word per eight words in a cache-line, meaning that for each batched evicted line, PHI reduces bytes transferred by 4× (4 $(word, address)$ tuples fitting in a single 64B cache-line, instead of 4 full 64B lines with 7 unused words each). However, we observed that, on the geometric mean over the 48 matrices, only 4.2% of PHI's output memory traffic comes from these batched updates (maximum 28.6%), indicating that PHI's overall behavior remains close to a *reduction cache*. In contrast, SpDCache addresses the drawbacks of a *reduction cache* by using dedicated regions for denser and sparser accesses, avoiding unnecessary evictions, without incurring the additional second-phase traffic introduced by PHI's batching mechanism.

We also explored different region sizing for SpDCache, varying the sizes of the *GRC*, *DRC* and *SRT* while keeping the total cache size constant at 32 KiB. The results are also reported in Table 3.2 (columns nº 7 to 11). Reducing the size of the *GRC* from 8 KiB (32 *sets*) to 2 KiB (8 *sets*) and 1 KiB (4 *sets*) has little impact on performance, as the *GRC* is mainly used for temporary storage of input vector elements. However, reducing the *GRC* under 1 KiB (4 *sets*) increases the total memory traffic, as more frequent evictions occur before the data is used, an effect already noticeable when going from 2 KiB to 1 KiB. With constant *GRC* size, when reducing the size of the *SRT*, which means allocating more space to the *DRC*, we observe a slight increase in memory traffic, as the *SRT* becomes less effective at capturing irregular accesses. Thus, the *SRT* sizing appears to be more critical than the *DRC* sizing. We will explore the reasons behind this effect in the next chapter. Overall, fine tuning the region sizes can lead to small performance improvements, but the original sizing of 8 KiB for the *GRC*, 16 KiB for the *DRC* and 8 KiB for the *SRT* appears to be a good compromise for the set of matrices we evaluated.

We also explored different sizing strategies for the RedC baseline (columns n° 3 and 4), varying the size of its single *reduction* region while keeping the total cache size constant at 32 KiB. Allocating more than 75% of the cache (96 *sets*) to store *RED* transactions slightly improves performance, as it increases the likelihood of capturing multiple updates to the same output element before eviction, without overly compromising the storage of input elements.

CONCLUSION

SpDCache outperforms PHI, a state-of-the-art *reduction* cache, achieving a 2.1× decrease in memory traffic, due to the dedicated regions which improve the effectiveness of the cache in dense regions, and the use of external atomic accesses to reduce the cost of isolated updates. Additionally, we show that the initial region sizing of SpDCache (25/50/25%) provides good performance with only minor variations observed when adjusting the sizes of the *GRC*, *DRC*, and *SRT*.

3.8 Conclusion

We have presented the concept of SpDCache, a *reduction cache* that has optimized storage and compute techniques for the sparse and dense regions of banded matrices. This approach does not reduce the number of compute operations, however, it significantly decreases memory traffic (4.1×). These benefits can be attributed to three techniques: (1) blocking the dense part of the band in the *DRC* to take benefit of the existing spatial locality, (2) using fine-grained storage in the *SRT* to adapt to isolated elements, and (3) off-loading the *reductions* for isolated elements to the downstream caches to prevent local evictions and reduce traffic.

Exploiting the coarse-grain structure of matrices is key to designing hardware optimized for sparse matrix kernels. In the next chapter, we present a more theoretical analysis of the behavior of SpDCache before moving onto an actual RTL implementation in Chapter 5.

TAKEAWAYS

Reduction cache:

- Provides hardware support for performing ***reductions***: $RED(key, value)$.
- The accumulations are done **near memory**, in the cache.
- **Reduces memory traffic** for kernels with a large number of *reduction* updates.

SpDCache:

- Considers that the matrix has a **dense region** and a less dense, **sparse region**. For banded matrices, these correspond to the in-band (*IBR*) and the out-of-band (*OBR*).
- Divides the L1 data-cache into **three regions**:
  - ***GRC***, generic region cache for regular memory accesses (such as reading A and b);
  - ***DRC***, dense region cache for *reductions* in the dense region of the matrix;
  - ***SRT***, sparse region table for *reductions* in the sparse region of the matrix.
- Uses memory transactions adapted to the region density: **cache-line-wise versus element-wise** for the *DRC* and the *SRT*, respectively.
- For a set of 48 matrices, results in **4.1× less memory traffic** for `OP` SpMV kernel.

CHAPTER

ANALYSIS OF REDUCTION CACHE BEHAVIOR AND PARAMETER EXPLORATION

4

Contents

4.1 Introduction

SPARSE matrix processing exposes processor systems to irregular memory access patterns, which degrade cache efficiency and increase memory traffic. The magnitude of these effects on memory system performance remains challenging to predict due to the diversity of data patterns. In this chapter, we develop a theoretical model to analyze and optimize the behavior of a *reduction cache* when executing an *Outer-Product* (`OP`) SpMV kernel.

We propose a hybrid probabilistic/fast simulation model that predicts the memory transactions of a *reduction cache* when executing an `OP` SpMV kernel ($A \times b = c$). This model allows us to quickly explore the impact of different cache configurations on memory traffic, using banded matrices as a representative case study. By isolating the sequential reads of the input data and analyzing the density distribution of non-zero elements, we identify key design principles for optimizing cache performance.

The chapter demonstrates the importance of separating the in-band and out-of-band regions of the matrix into dedicated cache regions. This separation allows the cache to exploit the structural regularity of banded matrices while efficiently handling irregular accesses in sparser regions. The theoretical analysis validates the architectural choices of SpDCache that was presented in previous chapter and provides insights for sizing and configuring cache regions to minimize memory traffic.

In our analysis of *reduction caches*, we use a *set*-associative configuration, with 8 byte elements which we consider as the basic memory unit. s_C is the size of the cache, in elements. The cache is divided into *sets*, *ways* and cache-lines, of size S, W and l, respectively. Thus, $s_C = S \times W \times l$ elements. We recall that a banded square matrix of dimension M is defined by its *band-width* w_B, or its half-*band-width* w_{hB}, such that $w_B = 2w_{hB} - 1$. The in- and out-of-band densities are named d_{IB} and d_{OB}, respectively. When d_{OB} is zero, the matrix is said to be 'perfectly' banded.

4.2 Isolating the Input Reads in a Generic Cache

WHEN computing an **OP** SpMV kernel, all the irregular accesses come from updating the output vector c. Indeed, the input matrix A and vector b are read sequentially, and can thus be efficiently cached by a generic cache that naturally exploits spatial locality.

With the matrix A stored in CSC format, each iteration of the **OP** algorithm necessitates reading four tables: $A.ptr$, $A.idx$, $A.val$ and b. In the same iteration, a fifth table, c, is updated "randomly". If we consider cache-lines of size l elements, then each cache-line read of the $A.ptr$ table brings l consecutive entries, which are pointers to l consecutive columns of the matrix. Similarly, each cache-line read of the b table brings l consecutive entries, which will be multiplied by the non-zero elements of l consecutive columns of the matrix. These reads are needed in the outer-loop of the **OP** algorithm. In the inner-loop, each cache-line read of the $A.idx$ and $A.val$ tables brings l consecutive non-zero elements of the matrix to process, which will generate l "random" updates to the output vector c, thus l different cache-lines of c could be accessed in the worst case.

To avoid unnecessary evictions, at each iteration, the generic cache should, obviously, keep $A.ptr$, $A.idx$, $A.val$ and b cache-lines up until they are no longer needed. That means l iterations for $A.idx$ and $A.val$, and around $l \times nnz_c$ iterations for $A.ptr$ and b. However, the cache must also keep the cache-lines of c that are being updated, which, over the course of $l \times nnz_c$ iterations, can necessitate up to the same number of different cache-lines in the worst case, on unpredictable addresses. Thus, it would be difficult for a generic cache to keep all the useful data until it is no longer needed. $A.ptr$ and b cache-lines, which are accessed once every nnz_c iterations, could easily be evicted by a *LRU* policy which is unaware of their future use. By isolating the input reads in a dedicated cache, as proposed with SpDCache and the *reduction cache* baseline in the previous chapter, we greatly increase the chances that all the cache-lines of A and b remain in the cache until they are no longer needed. With a dedicated cache region for the inputs, the *GRC* introduced in previous chapter, only the four sequentially read tables need to be considered. These tables are read only once, and isolating them in the *GRC* decreases the risk of undesired evictions.

With a *LRU* policy, cache-lines holding $A.ptr$ and b are more at risk of being evicted than $A.idx$ and $A.val$ because they are accessed less frequently, only once every $\sim \frac{nnz_c}{l}$ times per column. $A.idx$ and $A.val$ are accessed every iteration of the inner-loop. To avoid premature

evictions of $A.ptr$ and b, the *GRC* must be sized to handle the $\frac{nnz_c}{l}$ cache-line reads which occur while computing a column. In Equation 4.1, we show the minimum size of the *GRC* in elements needed to avoid unnecessary evictions during the traversal of the columns, thus optimizing memory traffic.

$$s_{C_{min}}(GRC) = 2 \times l + 2 \times nnz_c \tag{4.1}$$

This relationship is not exact, as it does not consider address alignment effects nor temporal variability in the downstream memory, but it gives a good approximation of the minimum size needed for the *GRC* to avoid unnecessary evictions. It explains why, in the previous chapter, the total memory traffic slightly drops when the *GRC* size is decreased. If we return to Table 3.2, in the previous chapter, in the columns for SpDC, we now understand why the total memory traffic is higher when the *GRC* size is reduced from 32 to 4 *sets*.

Separating the inputs in a dedicated cache region also has the advantage of allowing prefetching techniques [104] to be used efficiently, as prefetched cache-lines will not be undesirably evicted before use. For the `OP` SpMV kernel, prefetching does not reduce the memory traffic, but it does improve performance by masking the main memory latency.

CONCLUSION

To avoid unnecessary evictions, it is advantageous to **separate** the sequential reads of the inputs A and b from the "random" updates of the output c, into **dedicated cache regions**. A generic cache of reasonable size can **mask the latency** of the main memory when reading A and b, and ensure this data is **read exactly once**.

4.3 Modeling a Reduction Cache

To go beyond simply observing the behavior of a specific accelerator processing a given matrix, a model of the matrix and the cache is required. We present a hybrid probabilistic/fast simulation model for a *reduction cache*, processing `OP` SpMV, which predicts the volume of memory traffic issued by the L1 cache, starting from a matrix density distribution and a cache configuration.

The model represents the non-zero density both in the matrix and in the cache, with cache-line granularity. Using these coarse grain densities, a fast simulation of the `OP` SpMV is performed. The state of the cache can be predicted at each step of the simulation. By counting the evictions, we evaluate the output memory traffic, the memory trafic of the output vector c. We can thus predict the total number of memory accesses for different types of matrices and quickly explore different *reduction cache* configurations.

We do not consider the sequential accesses to the input matrix A and vector b in our model. Indeed, the matrix tables val, idx and ptr, and the vector b are each read once, giving a total memory traffic of $2 \times nnz + (M + 1) + M$ elements. We showed in the previous section that a simple generic cache of reasonable size is sufficient to minimize the associated memory traffic.

Our model uses the cache configuration parameters S, W and l. Although our model is not limited to banded matrices, they are the focus of this work. The matrix density distribution thus takes on two different values, either d_{IB} in the band, or d_{OB} outside of the band.

We normalize the memory traffic by dividing the number of elements that are read or written by the number of non-zero elements (nnz) in the matrix, so we can compare performance

between matrices of different sizes and densities. Indeed, we can interpret the normalized memory traffic, $normMT$, as the number of elements transferred from/to memory per non-zero matrix element. If the memory traffic remains constant while the matrix density increases, we expect $normMT$ to decrease.

4.3.1 Operation of the Analytical Model

Matrix Representation The square matrix is of dimension M, as represented in Figure 4.1. Its rows and columns are indexed by $0 \leqslant i, j \leqslant M-1 \in \mathbb{N}$. Each column is split into vertical-strips of size l, where l is the cache-line size. Indeed a vertical-strip is a portion of a column j of size l, from row-indices i to $i+l-1$, as illustrated by the *red* rectangle within the matrix in the figure. Within a column, vertical-strips are indexed by $k \in [\![0, M_l - 1]\!]$ [(1)], with $M_l = \lceil \frac{M}{l} \rceil$, and $k = \lceil \frac{i}{l} \rceil$.

The matrix is thus represented as a static, dense two dimensional table, MDT (*Matrix Density Table*), of size (M_l, M) that contains the density in $[0,1]$ of each vertical-strip. We assume that the distribution of the non-zero elements within a vertical-strip is uniform, as assumed in related work [114] where the authors studied a directly-mapped generic cache for an *Inner-Product* SpMV kernel, only for the case of 'perfectly' banded matrices.

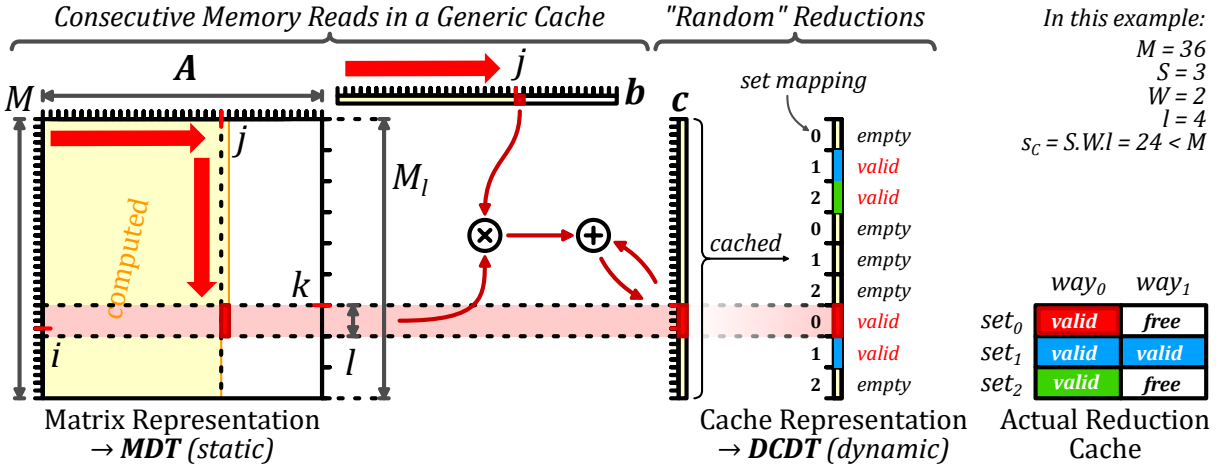


Figure 4.1 – Input Matrix and Cache Representation of the Model

Cache Representation Still in Figure 4.1, the cache is represented as a single column, which maps directly to the output vector c. This "column-representation" of the cache is partitioned into cache-lines, corresponding directly to the vertical-strips of a given column of the matrix, as shown by the horizontal *light red* rectangle. A cache-line $k \in [\![0, M_l - 1]\!]$ is stored in a unique *set* $s \in [\![0, S-1]\!]$ of the cache such that $s = k\%S$. Unlike the matrix representation which is static, the cache representation is continuously updated as the model evaluates **OP** SpMV.

Of course, the real cache-size ($S \times W \times l$) is not necessarily equal to the output vector size (M). Thus, at a given time, the number of valid (non-empty) cache-lines contained in the cache representation must not exceed the actual cache-size. Specifically, we ensure the number of valid cache-lines in a *set* of the cache representation does not exceed the number of ways W.

(1). We use $[\![a, b]\!]$ for the integer intervals, and $[a, b]$ for real intervals.

In the example in Figure 4.1, the actual cache is holding four valid cache-lines, which are the only valid cache-lines present in the cache representation.

To count the number of valid cache-lines, we further distinguish them from empty ones. For this reason, we set the cache-line densities to be either zero (the cache-line is empty), or a density is in the range $\left[\frac{1}{l}, 1\right]$ (the cache-line is valid). A valid cache-line has a density between $\frac{1}{l}$ and 1, corresponding to the probabilistic number of non-zero elements.

The state of the cache is thus represented with the dynamic table $DCDT$ (*Disjoint Cache Density Table*), of dimension M_l that contains the disjoint-density in $\{0\} \cup \left[\frac{1}{l}, 1\right]$ of each cache-line, at a given iteration. Initially, all cache-lines are set to zero density, since the cache is considered empty.

Iterative Process In the fast simulation phase, the analytical model [33] iterates over the vertical-strips in the matrix, following the dense traversal of the `OP` algorithm, as described in Chapter 1 and shown with *red* arrows in Figure 4.1. In a given iteration (column j, vertical-strip κ), we use the matrix density of the current vertical-strip, $MDT_{\kappa,j}$, and the current state of $DCDT$ to determine if an eviction occurs, and update it for the next iteration, if necessary.

Disjoint-Density of the Current Vertical-Strip The density of the current vertical-strip in the matrix has to be disjointed at each iteration, to determine whether the strip is empty or not, as done for the cache-lines. We define the vertical-strip disjoint-density as $d_{lM}(\kappa) \in \{0\} \cup [\frac{1}{l}, 1]$.

- When $MDT_{\kappa,j} = 0$, the vertical-strip in the matrix is empty, thus, $d_{lM}(\kappa) = 0$.
- When $0 < MDT_{\kappa,j} < \frac{1}{l}$, the vertical-strip can have either 0 or 1 non-zero elements. To resolve this, we maintain a running counter of the accumulated $MDT_{\kappa,j}$ values, and when this counter exceeds $\frac{1}{l}$, the current vertical-strip is considered non-empty, so we set $d_{lM}(\kappa) = \frac{1}{l}$. If the counter is below this threshold, $d_{lM}(\kappa) = 0$.
- When $MDT_{\kappa,j} \geqslant \frac{1}{l}$, the vertical-strip has one or several non-zero elements. Thus, $d_{lM}(\kappa) = MDT_{\kappa,j}$.

With this definition, we can distinguish empty ($d_{lM}(\kappa) = 0$) and non-empty ($d_{lM}(\kappa) \neq 0$) vertical-strips.

Eviction Processing With the above definitions, we can determine if a miss occurs at the current iteration. A miss occurs when the cache-line κ is not valid and the vertical-strip is not empty. Recall that in a *reduction cache*, a miss without eviction results in the allocation of a free cache-line, initially filled with zeros, and produces no memory traffic. We define $miss(\kappa) = (DCDT_\kappa = 0)\ and\ (d_{lM}(\kappa) \neq 0)$, a boolean indicating whether a miss occurs in the current iteration.

When allocating a free cache-line, it is necessary to ensure that the number of valid cache-lines in the *set* $s = \kappa \% S \in [\![0, S-1]\!]$ does not exceed W. The new allocation may thus trigger an eviction. Stated formally, $nnzcl_\kappa$ is the count of the cache-lines k from the *set* s that verify $DCDT_k \neq 0$. We define $evict(\kappa) = (miss(\kappa))\ and\ (nnzcl_\kappa = W)$, a boolean indicating whether an eviction occurs in the current iteration.

In the case of an eviction, for the probabilistic model, we use a *random* eviction policy to select a valid cache-line in the *set* s (when $DCDT_k \neq 0$). The index of the evicted cache-line is $e \in [\![0, M_l - 1]\!]$.

Cache Update The last step is to update the $DCDT$ before proceeding to the next iteration. (1) If a cache-line is evicted, we must account for the memory traffic and reset the victim line e; (2) The density of the current cache-line κ may increase if there is a hit or if it is newly allocated:

1. In the case of an eviction ($evict(\kappa)$ is true), we reset the victim cache-line (e) by setting $DCDT_e = 0$.
2. We update the density of the cache-line κ. In the case of a miss ($DCDT_\kappa = 0$), the cache-line density is set to the disjoint-density of the matrix vertical-strip κ. In the case of a hit, the cache-line and the vertical-strip densities are merged:
   $DCDT_\kappa \leftarrow DCDT_\kappa + (1 - DCDT_\kappa) \times d_{lM}(\kappa)$
   The updated cache-line density is either 0 or greater than $\frac{1}{l}$, thus respecting the constraint that the cache-line is either empty or valid.

Final Iteration Starting with an initially empty cache, we run all $M \times M_l$ iterations and count the total number of evictions ($nbEvict$). After the last iteration, we account for the memory transactions required to flush the cache ($nbFlush$), writing the valid cache-lines to memory, by counting lines where $DCDT_k \neq 0$, for $k \in [\![0, M_l - 1]\!]$. $nbFlush$ will never exceed $S \times W$.

In a *reduction cache*, evictions and flushes result in a line-wise memory read, a modification, and a line-wise memory write. The update memory traffic is $2 \times l \times (nbEvict + nbFlush)$ elements, and $normMT$ is this value divided by nnz.

CONCLUSION

We have presented a **hybrid probabilistic/fast simulation analytical model** for a *reduction cache*, processing `OP` SpMV, which predicts the volume of memory traffic issued by the L1 cache, starting from a **matrix density distribution** and a **cache configuration**. The model represents the non-zero density both in the matrix and in the cache, with **cache-line granularity**. Using these coarse grain densities, a **fast simulation** of the `OP` SpMV is performed. In this way, we can quickly explore different *reduction cache* configurations to evaluate their impact on memory traffic.

4.4 Analysis of Reduction Cache Behavior

We now study the *reduction cache* behavior when computing `OP` SpMV, starting with the simple case of 'perfectly' banded matrices with varying *band-widths*. We then use the analytical model to analyse the *reduction cache* behavior, varying the out-of-band density and the cache-size. We show the benefit of storing the in- and out-of-band of the matrix in separate regions of the cache, to minimize memory traffic.

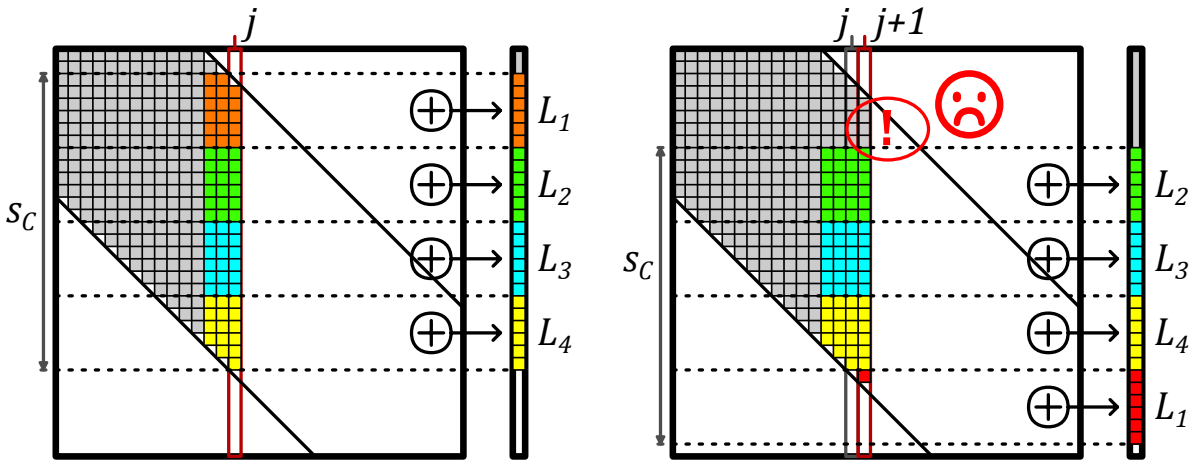
4.4.1 Behavior of a Cache for a Perfectly-Banded Matrix

In the previous chapter, we presented SpDCache and a *reduction cache* baseline with a specific region (*DRC*) dedicated to storing the banded part of the matrix. We defined the *bandwidth* of the matrix based on the *DRC* size. In this section, we explain and justify this choice by analyzing the behavior of a *reduction cache* when processing an `OP` SpMV kernel with a 'perfectly' banded matrix.

We show that the *reduction cache* must be 'properly' dimensioned with respect to the *band-width* of a 'perfectly' banded matrix, in order to avoid unnecessary evictions. Indeed, if the cache is too small compared to the band, evictions occur before the column iteration is over, leading to poor data-reuse for the next iteration. To achieve high data-reuse, we show that the cache-size (s_C) and the *band-width* (w_B) must respect the relationship given in Equation 4.2.

$$s_C \geqslant w_B + l - 1 \tag{4.2}$$

We consider two examples with a cache having four cache-lines (L_1 to L_4) with $l = 6$. In Figure 4.2, we illustrate the case when the cache is too small ($s_C < w_B + 5$). In Figure 4.2a, we see the state of the cache after iterating up to column j. Then, in Figure 4.2b, we see the state after processing the next column, $j + 1$. The cache-line L_1 (*orange*) is evicted, and allocated to new elements at the bottom of the band, below the *yellow* area. In this case, there are still elements of the band above the *green* area, whose corresponding cache-line has, unfortunately, been evicted. Thus, on iteration $j+2$, cache-line L_1 must be fetched again and this pattern continues creating *thrashing*.



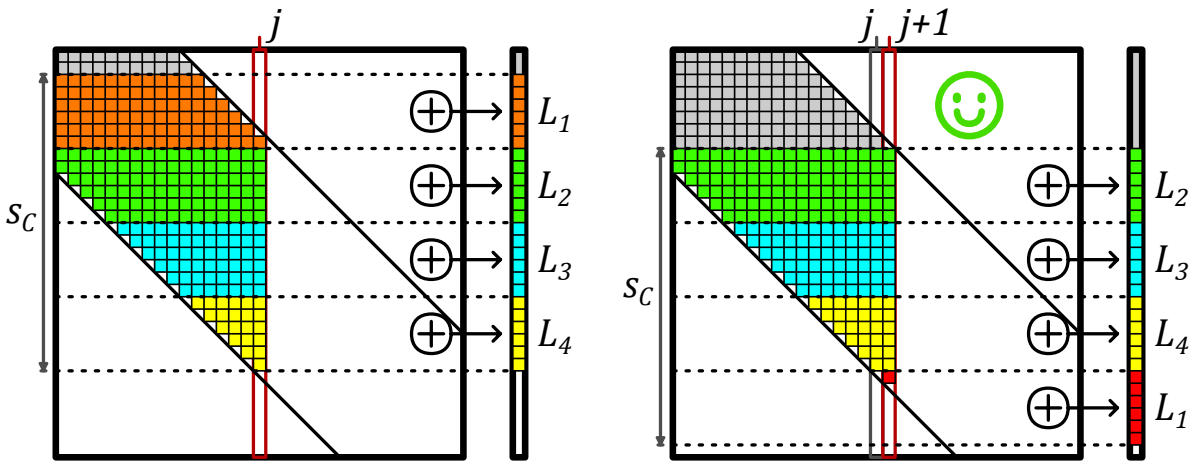
(a) Column iteration j **(b)** Column iteration $j + 1$

Figure 4.2 – Example of the Eviction Rotation in a Perfect Band when the Cache is Undersized ($s_C < w_B + l - 1$): $s_C = 24$, $w_B = 23$ and $l = 6$

In Figure 4.3, we illustrate the case when the cache is 'properly' sized ($s_C \geqslant w_B + 5$). As in the previous example, Figures 4.3a and 4.3b show the state of the cache at iterations j and $j + 1$, respectively. This time, the cache-line L_1 is evicted only when all elements of the band in the *orange* region have been processed. Indeed, the eviction of cache-line L_1 occurs just in time so that it can be allocated to the new region at the bottom of the band, below the *yellow* area. In this way, when processing columns, the cache-lines do a rotation where the top one is evicted and allocated at the bottom, with one eviction occurring after the processing of every l columns.

If the cache-size is larger than required by Equation 4.2, the rotations and the eviction-rate are the same. To produce this perfect cache rotation, we assume an LRU eviction policy and dense vertical-strips, although, even with a *random* policy, the general pattern still holds.

This analysis shows that, with a "properly-sized" *reduction cache*, a 'perfectly' banded matrix provokes no unnecessary evictions and the cache-lines are fully utilized. With a given cache-size, Equation 4.2 defines the maximum *band-width* that can be efficiently stored in



(a) Column iteration j **(b)** Column iteration $j+1$

Figure 4.3 – Example of the Eviction Rotation in a Perfect Band when the Cache is 'Properly' Sized ($s_C = w_B + l - 1$): $s_C = 24$, $w_B = 19$ and $l = 6$

the *reduction cache* or in the *DRC* of SpDCache. For example, with a cache-size of $s_C = 2048$ elements (16 KiB) and a cache-line size of $l = 8$ elements, the maximum *band-width* is $w_B = 2041$ elements (which means a half-band of $w_{hB} = 1020$ elements).

Large, 'perfectly' banded matrices are rare in practice, especially with such narrow -width. In the next sections, we extend this analysis to banded matrices, studying the impact of out-of-band non-zero elements on the *reduction cache* behavior.

CONCLUSION

A *reduction cache* can achieve an efficient **rotating pattern** when processing a 'perfectly' banded matrix, provided that the cache-size and *band-width* respect the **relationship given in Equation 4.2**: $s_C \geqslant w_B + l - 1$. This relationship defines the **maximum *band-width*** that can be efficiently stored in the *reduction cache* for a given cache-size.

4.4.2 Impact of Out-Of-Band Density on Memory Traffic

We now apply the analytical model of Section 4.3.1 to study the performance of a *reduction cache* for banded matrices, processing an `OP` SpMV kernel ($A \times b = c$). We consider banded matrices of dimension $M = 1000$, with a half-band of size $w_{hB} = 125$ and an in-band density $d_{IB} = 1$. We analyse a 4-way *set*-associative cache ($W = 4$) with a cache-line size of $l = 8$ elements, with a *random* eviction policy. The size of the cache varies from $S = 1$ to $S = 32$ *sets*. When the cache-size reaches $S = 32$, the cache can hold the full output vector: $S \times W \times l \geqslant M$. We developed a C program which implements the hybrid probabilistic-fast simulation model and in Figure 4.4, we plot the normalized memory traffic, $normMT$, for the output vector c, depending on the out-of-band density d_{OB}, for varying cache-sizes.

On the right of the graph, we notice that $normMT$ reaches a peak when the out-of-band density equals $\frac{1}{l} = 0.125$. Indeed, when there is at least one non-zero element per cache-line, the number of main memory accesses approaches that required for a dense matrix. As the density increases further, no additional memory accesses are required, but each line becomes denser, thus $normMT$ drops, as seen in the graph in Figure 4.4.

For the plot corresponding to $S = 32$, the cache is large enough to store the entire output

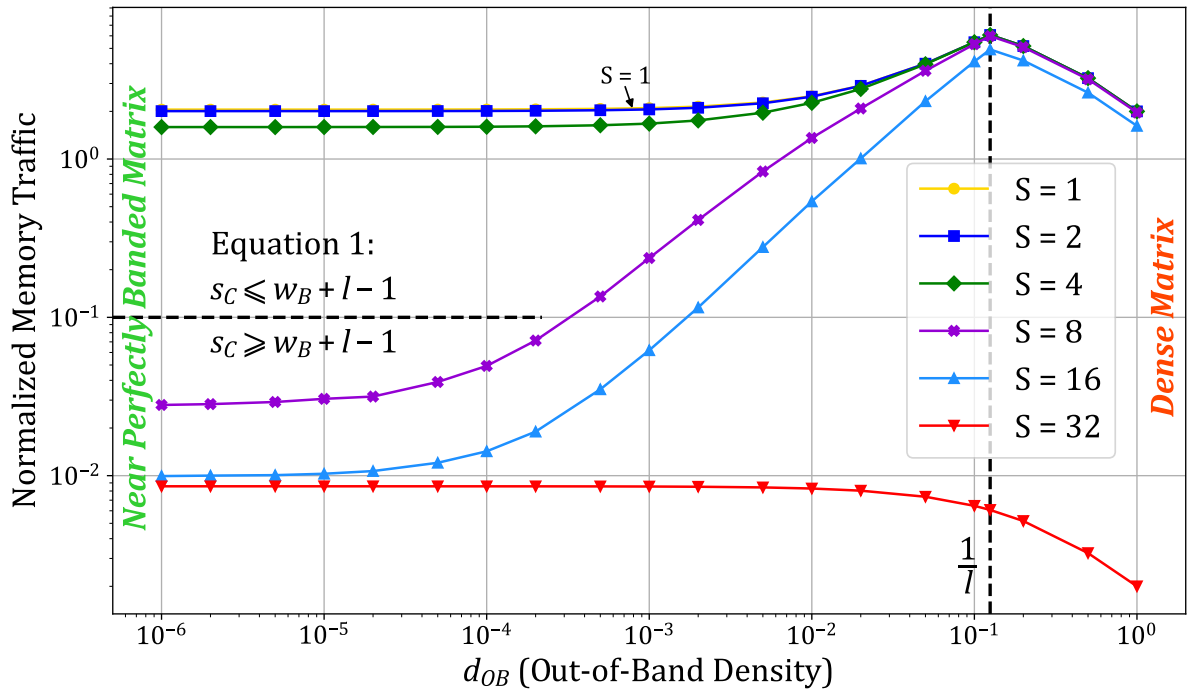


Figure 4.4 – Normalized Output Memory Traffic Depending on the Out-of-Band Density for Several Cache-Sizes

vector c, thus, there are no evictions and $normMT$ is constant, corresponding to the flush traffic at the end. This plot gives the lower bound of memory traffic for the output vector c.

The first takeaway from this graph is that variations in d_{OB} heavily impact $normMT$ (note the logarithmic scale). Isolated non-zero elements outside the band trigger misses and evictions of cache-lines corresponding to the banded region. Furthermore, when a cache-line is allocated for an isolated out-of-band element, most of the cache-line remains empty. In this case, instead of the efficient rotating pattern described in Section 4.4.1, the cache is *thrashing* with low utilisation. Regardless of the cache-size, the lower the out-of-band density, the lower $normMT$, as seen by the plots that tend downward toward the left of the graph. Avoiding the perturbations induced by the out-of-band elements would enable the cache to achieve minimal memory traffic, as with a 'perfectly' banded matrix.

Not surprisingly, at the left of the graph, $normMT$ varies depending on the cache-size. This is due to the relative size of the band and the cache. Indeed, if the matrix *band-width* is large compared to the cache-size, processing one column of the band will generate many evictions. To avoid evictions in the band, the relationship given by Equation 4.2 in Section 4.4.1 must be fulfilled. Indeed, for this example, for $S \geqslant 8$, this relationship holds and the corresponding plots have significantly lower $normMT$, up to two orders of magnitude lower. Conversely, the first three plots ($S = 1$ to 4) do not respect this condition, explaining their high $normMT$. Thus, the second takeaway from this graph is that, given a cache memory size, there is an ideal *band-width* that generates minimal $normMT$. Increasing the cache-size beyond this threshold only produces a minor reduction in memory traffic, between $\sim 9 \times 10^{-3}$ and $\sim 2 \times 10^{-2}$, since at the beginning of matrix traversal, the first eviction is deferred before reaching the steady state rotating pattern, as discuss in Section 4.4.1.

These observations explains the need of dividing the *reduction cache* into two regions. One

which is optimized for storing the band and the other which handles all the other elements. The in-band region of the cache can thus operate with maximal efficiency corresponding to the bottom-left of Figure 4.4. In subsequent sections, we analyse the out-of-band traffic.

CONCLUSION

The out-of-band density heavily impacts the normalized memory traffic, $normMT$, as the isolated elements trigger unnecessary evictions, preventing the efficient rotating pattern shown in the previous section. Thus, to minimize memory traffic, it is important to **separate** the band and out-of-band regions of the matrix into **dedicated regions** of the *reduction cache*.

4.4.3 Design Analysis for Out-of-Band Region

For the type of large matrices under study, there are usually too many non-zero elements in the out-of-band region for them to be cached from one column to the next, thus making it difficult to achieve a high level of reuse from column to column. Therefore, the size of the cache for the out-of-band is not as important as the choice of associativity and data granularity. To justify this affirmation, we use the analytical model from Section 4.3.1 to analyse the performance of the *reduction cache* when processing only the non-zero elements outside of the band. As we have already discussed the handling of the in-band region, we now consider only the out-of-band elements, which is equivalent to setting the in-band density to $d_{IB} = 0$. Using the same matrix parameters as in the previous section, we explore different choices of cache-line size ($l \in \{1, 8\}$ elements), associativity (*set*-associative or fully-associative), memory transaction type (*R/W* or *RMW*), and total cache-size ($s_C \in \{128, 256\}$ elements). Because the out-of-band data is highly sparse, we want to explore having an element-wise cache ($l = 1$) instead of the typical line-wise cache ($l = 8$). As this cache region can be relatively small, a fully-associative configuration ($S = 1$, $l = 1$) is reasonable in terms of hardware implementation, and this full associativity enables more efficient storage of the data elements. Indeed, when dealing with sparse elements, it is wasteful to bring a full line into the cache to only modify one element and then evict it, which we denote as *R/W*. Thus, we consider offloading the *reduction* to a higher level cache, thus reducing unnecessary data transfers. We assume the higher level cache can support this element-wise operation which we denote as *RMW*, following the design choice of SpDCache in Chapter 3. We recall that protocols such as AXI [2] support atomic, element-wise transactions (AMOs), which could easily be extended for *reduction* operations.

To analyse the impact of the cache-size, we consider two different sizes (*solid* and *dashed*). For a given cache-size, we adjust the number of ways (W), based on the associativity, to keep the cache-size constant. In Figure 4.5, we plot $normMT$ for the output vector c, depending on d_{OB}, for varying cache configurations.

Let us start by considering a classic *set*-associative cache with a line-wise memory interface (*solid blue*, *dot*), where we see a dependence of $normMT$ on d_{OB} on the right of the graph ($d_{OB} \geqslant \frac{1}{l}$). Indeed, we observe the same behavior as in Section 4.4.2, where the cache behavior approaches that seen for a dense matrix.

Looking at the plot for the fully-associative cache with a line-wise memory interface (*solid green*, *square*), we see that the normalized traffic is not impacted by the density of the out-of-band, as expected. With the fully-associative cache, $normMT$ is constant near 16, as, on average, there are two line-wise transactions per non-zero element processed, one read to fill the line, one write to evict it.

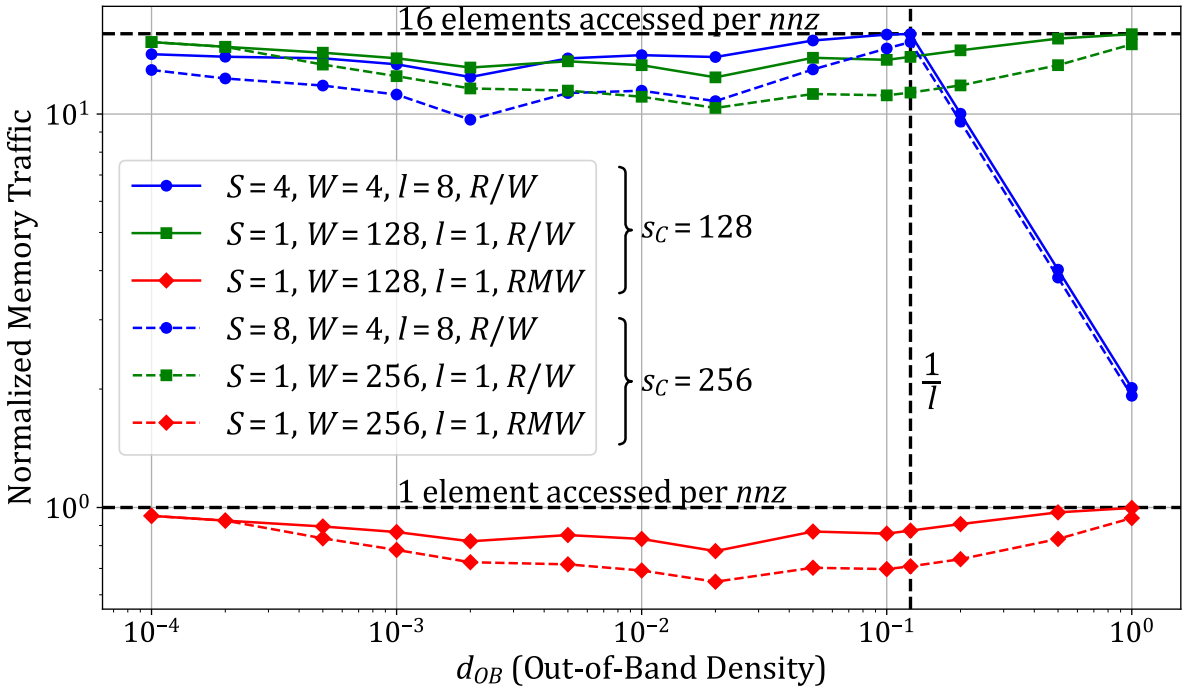


Figure 4.5 – Normalized Output Memory Traffic of the Out-of-Band, Depending on the Density for Several Cache-Sizes

The *red* plots (*diamond*) correspond to a fully-associative cache with an element-wise memory interface, which can offload element-wise *RMW* transactions. We see that this approach results in over an order magnitude reduction in $normMT$. First, we avoid transferring full cache-lines which are mostly empty, and we only send the *RMW* in one direction ("fire-and-forget"). Indeed, $normMT$ is now reduced to below one transaction per non-zero element processed. The fact that this value is below one shows that this cache region does hit and reduce memory traffic. Increasing the cache-size from 128 (*solid*) to 256 (*dashed*) elements reduces $normMT$ by at most 10^{-1}, the hit-rate being slightly increased as expected.

CONCLUSION

For the out-of-band region of the cache, the most important characteristics are: (1) a **fully-associative** element-wise data-storage to avoid virtually empty cache-lines, and (2) an **element-wise memory interface** to reduce memory traffic. With these two characteristics, the normalized memory traffic can be reduced to below one transaction per non-zero element processed.

4.4.4 Impact of In-Band Density

Up to now in this chapter, we considered a dense band ($d_{IB} = 1$) to show the importance of having a rotating eviction pattern to decrease memory traffic. However, matrices from real applications do not have fully dense bands. Indeed, from our set of 48 matrices used in the previous chapter, considering a *band-width* of $w_B = 2041$ that fits into a *DRC* of 16 KiB ($s_C = 2048$ elements), the average in-band density is $d_{IB} = 2.11 \times 10^{-3}$, ranging from 2.00×10^{-8} and 5.44×10^{-2}, compared to an average out-of-band density $d_{OB} = 1.28 \times 10^{-4}$, ranging from 0.00 and 1.04×10^{-2}. Considering that a part of the matrices of our set are not considered

as banded, this order of magnitude difference between d_{IB} and d_{OB} on average across all 48 matrices shows that, even in matrices that do not appear to be banded, it is reasonable to assume that the region along the diagonal is much denser (10×) than the region away from the diagonal.

Since the in-band density is lower than 1 in practice, we use the model to analyse the impact on normalized memory traffic of d_{IB} variations. We use the same parameters as in the previous sections, a matrix of dimension $M = 1000$ and a cache with four *ways* ($W = 4$) and cache-lines of size $l = 8$. We do not consider the out-of-band ($d_{OB} = 0$) since it is processed in a separate region. We focus on the in-band region for several cache configurations, making sure that the *band-width* for each *set* size follows the relationship in Equation 4.2. In Figure 4.6, we plot $normMT$ for the output vector c, depending on d_{IB}, for varying cache configurations.

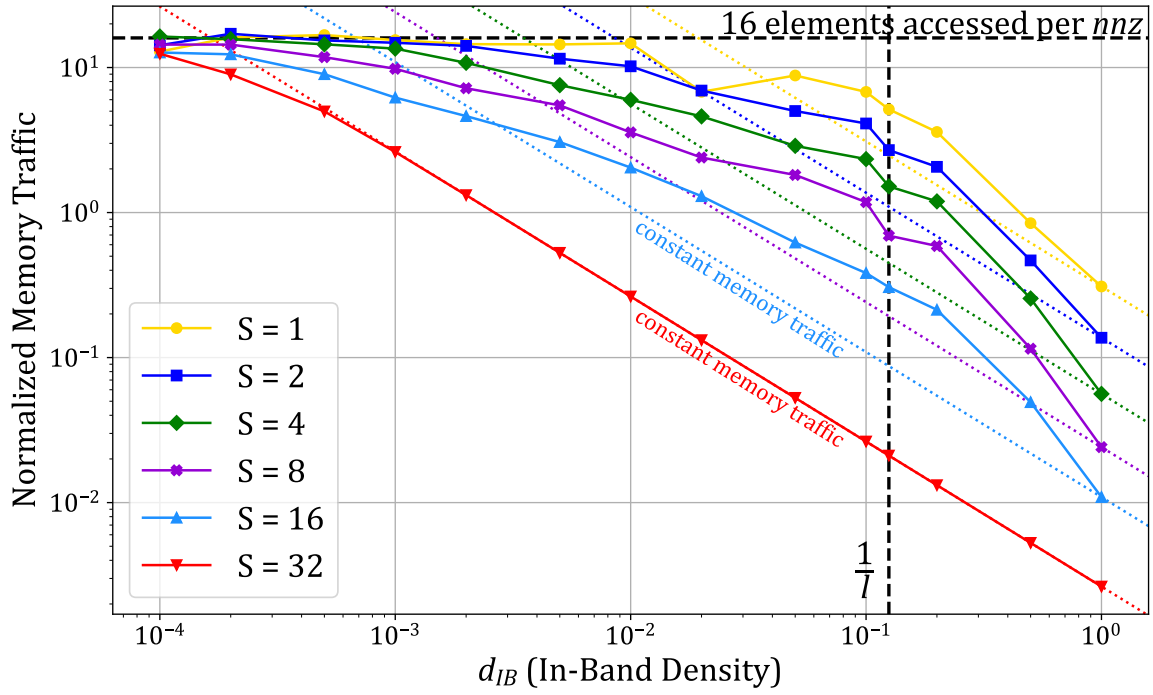


Figure 4.6 – Normalized Output Memory Traffic of the In-Band, Depending on the Density for Several Cache-Sizes

We observe that $normMT$ increases as d_{IB} decreases, generally following a constant negative slope (*dashed*) on the right side of the graph for each cache size. Thus, across a broad range of in-band densities, the raw memory traffic remains relatively constant. As noted in Section 4.4.2, the $S = 32$ curve indicates the lower bound of memory traffic since the entire matrix can fit within the cache. In this scenario, there are no evictions in the band until the density drops sufficiently low ($< 10^{-3}$), resulting in isolated non-zero elements. For other cache sizes, variations around the constant raw memory traffic are observed. With an ideal eviction policy, allowing the rotating pattern to function as intended (refer to Section 4.4.1), we would expect all curves to align with their respective *dashed* lines, indicating constant raw memory traffic. However, the model employs a *random* policy, which disrupts the rotating pattern and accounts for the observed deviations. An *LRU* policy would likely yield improved results, aligning with the constant line for densities above $1/l$ and deviating for lower densities. The optimal policy would prioritize evicting lower addresses (the top of the band), thereby

maintaining the rotating pattern.

For low values of d_{IB} (left side of the graph), $normMT$ plateaus at 16 elements transferred per non-zero element for all cache sizes. With very low density the non-zeros are isolated and provide no reuse: each non-zero triggers a full cache-line read and a full-line write-back. With $l = 8$ this corresponds to 16 elements transferred per non-zero.

It is therefore important to select the densest region of the matrix to be processed in a dedicated cache region such as the *DRC* of SpDCache. However, the selected region needs not be highly dense: for a 'perfectly' banded matrix with $M = 1000$, columns become nearly empty for $d_{IB} \lesssim 10^{-3}$, yet the raw memory traffic remains nearly constant in the range 10^{-2} to 10^{-3} depending on cache size. Because real, larger matrices often have a mean d_{IB} around 10^{-3}, a *DRC* sized to capture this denser region still provides good memory-traffic performance.

NOTE

In the SpDCache architecture, the *DRC* employs an *LRU* or pseudo-*LRU* eviction policy instead of an ideal custom policy. Despite this, we show that these simple, well-known policies still deliver good performance.

CONCLUSION

The **in-band density** has only a **minimal impact on memory traffic**, with real matrices typically exhibiting in-band densities around 10^{-3}.

4.4.5 Resulting Approach and Discussion

In the previous sections, we analyzed the behavior of *reduction caches* executing an `OP` SpMV kernel with banded matrices. We evaluated the expected memory traffic using an analytical model, separately considering the in- and out-of-band regions. We identified the cache characteristics required for both regions, which are quite different. We thus validate the proposition to divide the available cache memory into two dedicated regions: one for the in-band, and one for the out-of-band; the *DRC* and *SRT* of SpDCache, respectively.

To obtain the efficient rotating pattern for the in-band region, which is the key to massively reducing memory traffic, Equation 4.2 must be respected. In practice, this defines the size of the band that can be allocated to the in-band cache region. As we have seen, this cache region can operate efficiently with a *set*-associative line-wise configuration, on a wide range of in-band densities.

For the out-of-band region, a fully-associative element-wise cache with an element-wise memory interface can significantly reduce the memory traffic.

When sizing the two regions, once Equation 4.2 is satisfied, enlarging the in-band region yields only a marginal reduction in memory traffic (on the order of 10^{-2}). By contrast, a moderate increase of the out-of-band region produces a larger reduction (on the order of 10^{-1}). Therefore, the out-of-band region should be made as large as practical, subject to the hardware limits imposed by full associativity. This correlates with the results presented in Chapter 3, where we showed better performance with SpDCache configurations having a larger *SRT* (Table 3.2).

Although this analysis is sufficient to guide the design of SpDCache for `OP` SpMV with banded matrices, it is important to note that the analytical model that we have presented is based on the assumption of a uniform distribution of the non-zero elements. In practice, this is not always the case, and this non-uniformity can impact the cache behavior, as will be seen

later in Chapter 5. Indeed, if the non-zero elements are clumped, the cache-lines tend to be better filled, resulting in lower memory traffic. In other words, our uniform assumption leads to a pessimistic prediction of memory traffic.

CONCLUSION

To minimize memory traffic when processing banded matrices with a *reduction cache*, it is important to **divide** the cache into two dedicated regions: (1) an **in-band region** dedicated to a band in the matrix whose size respects the relationship in Equation 4.2, and configured as a ***set*-associative line-wise** cache, and (2) an **out-of-band region** configured as a **fully-associative element-wise** cache with an **element-wise memory interface**. The **out-of-band region** should be made as large as practical, as it has a greater impact on memory traffic than enlarging the in-band region.

4.5 Conclusion

THIS chapter presented a theoretical model to analyze the behavior of *reduction caches* when processing sparse matrices, specifically focusing on the `OP` SpMV kernel. A C program implementing a hybrid probabilistic/fast simulation model was developed, and we demonstrated how the structure of banded matrices can be leveraged to optimize cache performance.

Our analysis highlighted the importance of separating the in-band and out-of-band regions of the matrix into dedicated cache regions. For the in-band region, a *set*-associative, line-wise cache configuration ensures efficient data reuse by maintaining a rotating eviction pattern, minimizing unnecessary memory transactions. For the out-of-band region, a fully-associative, element-wise cache with an element-wise memory interface significantly reduces memory traffic by avoiding the transfer of mostly empty cache-lines.

The theoretical insights gained from this model validate the architectural choices of SpDCache, particularly the use of distinct regions for denser and sparser data. These findings provide a robust foundation for designing and sizing cache regions to minimize memory traffic, ultimately improving the efficiency of sparse matrix computations. In the next chapter, we will delve into the microarchitectural details of SpDCache, translating these theoretical insights into practical hardware design.

TAKEAWAYS

Simulating *reduction caches* and SpDCache

To study the concepts presented in this thesis, three models were developed with varying levels of abstraction. Starting from the most abstract model:

- **High-level probabilistic/fast model in C** (4.3):
  - fast simulation ($\sim 10^5$ nnz/s), using generic matrix density distributions, easy parameter exploration, deep understanding of cache behaviors, but,
  - very high-level (some approximations and hypothesis), limited on eviction policies.
- **Transaction level model in Python** (3.6):
  - faster than a full RTL simulation ($\sim 10^3$ nnz/s), accurate performance on memory traffic, functional testing, but,
  - slower than the probabilistic model, demands real matrices as input, two factors that alleviate the parameter exploration capabilities, no latency performance com-

pared to RTL simulation.

- **RTL implementation** (next chapter):
  - accurate results in both memory traffic and latencies, hardware testing, but,
  - slow simulation (~10 nnz/s), demands real matrices in input, time-consuming implementation.

Validation and sizing of SpDCache three regions

- ***GRC***: can take advantage of prefetching methods, size depends on nnz_c.
- ***DRC***: must respect the relationship $s_C \geqslant w_B + l - 1$ (Equation 4.2).
- ***SRT***: internal storage and memory interface benefit from element-wise granularity, benefits from a larger size but must achieve a compromise with hardware cost.

To simplify address decoding, the initial region sizing for SpDCache was chosen somewhat arbitrarily to be **25/50/25%** for the *GRC*, *DRC* and *SRT*, respectively. Our analysis later confirmed that this partitioning is actually close to optimal for memory traffic performance.

CHAPTER

MICROARCHITECTURE AND EVALUATION

5

Contents

5.1 Introduction

IN the previous chapters, we established the theoretical foundations and high-level design principles for SpDCache, a specialized cache architecture tailored to optimize memory transactions for sparse matrix computations. This chapter bridges theory and practice by presenting the microarchitectural implementation of SpDCache and evaluating its performance through Register Transfer Language (RTL) simulations.

We begin by detailing the microarchitecture of SpDCache, focusing on its three distinct regions: the *Generic Region Cache* (*GRC*), the *Dense Region Cache* (*DRC*), and the *Sparse Region Table* (*SRT*). Each region is designed to handle specific types of memory accesses, sequential reads, dense *reductions*, and sparse *reductions*, respectively, thereby addressing the irregular

memory access patterns inherent in sparse matrix kernels. The RTL implementation of SpDCache is described, highlighting the hardware components and pipeline stages that enable efficient data processing and *reduction* operations.

Following the microarchitectural overview, we present the evaluation methodology and the results obtained from RTL simulations. These simulations provide a detailed assessment of SpDCache's performance, including improvements in memory traffic and latency. We analyze the behavior of SpDCache processing a diverse set of sparse matrices, comparing its performance against baseline cache architectures and validating the theoretical insights gained from earlier chapters.

By translating the theoretical model into a practical hardware design, this chapter demonstrates the effectiveness of SpDCache in minimizing memory traffic and improving computational efficiency for sparse matrix kernels. The results underscore the importance of region-based caching and *reduction* support, paving the way for further optimizations and scalability enhancements discussed in the following chapter.

5.2 SpDCache Microarchitecture

SPDCACHE is implemented as an L1 data cache for a CVA6 RISC-V core, building upon an existing open-source high-performance *set*-associative data cache known as HPDcache [34], fully implemented in RTL with SystemVerilog. In contrast to previous RISC-V data caches, HPDcache introduces new features, including buffering techniques for memory writes, *set*-associative storage to manage read miss requests, and dynamic configurability of each cache-line to operate in either write-back or write-through modes. It is designed for high performance, utilizing a three-stage pipeline to handle multiple instructions while minimizing stalls through out-of-order cache execution. The CPU can issue multiple types of read and write requests to the cache including atomic operations (AMOs), cache management operations (CMOs) as well as the new *reduction* operation (*RED*) introduced by SpDCache. In this way, we can say that HPDCache and SpDCache executes "instructions" which correspond to these different types of operations.

An overview of SpDCache's microarchitecture is shown in Figure 5.1, where *black* and *blue* components are directly reused from HPDcache. The first stage (*st0*) decodes instructions and issues reads to the cache directory and data RAMs (*Dir* and *Data*). These reads require one cycle to respond. The directory holds the state of the selected cache-line which is required in the second stage (*st1*). The response from the directory at *st1* indicates whether a hit or miss occurred and if an eviction is necessary. The instruction proceeds to the final stage (*st2*) only if a miss or eviction must be processed. At *st0*, the instruction is not yet consumed. Depending on the response and the availability of cache resources at *st1*, a *no-operation* (*nop*) may be inserted in the pipeline, and the instruction may be placed in a *replay table*. The instruction is then replayed with the highest priority once its dependencies have been met. If the instruction does not need to be replayed at *st1*, it is consumed and processed. This pipeline serves as the foundation for implementing SpDCache's *reduction* operations.

The *cache controller* (*cache ctrl*, top of the figure) coordinates several auxiliary units. The *miss handler*, is allocated requests from *st2*, tracks outstanding memory read requests and notifies the *cache controller* with a *refill* request when responses arrive. An optional *write buffer* gathers recent writes to memory, allowing coalescing of transactions that target the same cache-line. The *flush controller* (*flush ctrl*), also invoked by *st2*, reads cache-line data

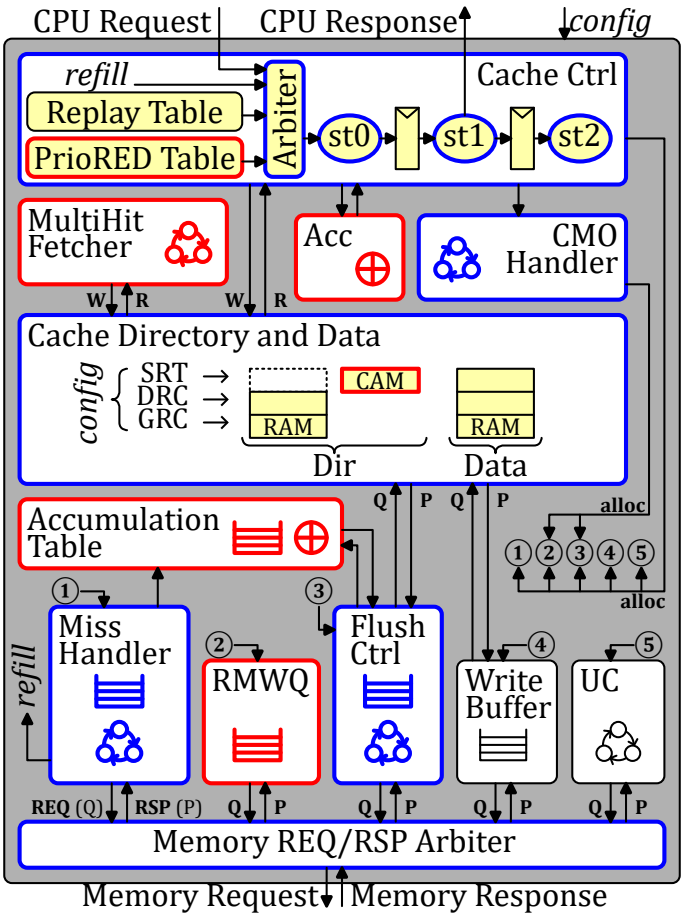


Figure 5.1 – SpDCache Microarchitecture, based on HPDcache

from the *Data* RAMs and issues memory writes. It temporarily stalls the pipeline to prevent RAM conflicts. HPDcache further provides an *UC* unit to handle uncached transactions. Finally, the *CMO handler* performs internal cache operations (*e.g.*, , invalidations and flushes). When active it takes control of the cache and can invoke the *flush controller* if necessary. Most of these units are reused and extended to implement SpDCache.

HPDcache provides extensive static and dynamic configurability, making it a flexible foundation for further extensions. The changes to SpDCache were done in a way such that, by simply changing SystemVerilog parameters, the same RTL code can implement (1) the original generic cache (GC) as the first baseline, (2) a standard *reduction cache* (RedC) as the second baseline, or (3) SpDCache (SpDC), the solution presented in the previous chapters.

NOTE

Statistics on the development effort and timeline for the complete SpDCache implementation are provided in Appendix C.

CONCLUSION

HPDcache is an **open-source**, high-performance cache which we used as the **starting point** for our work. We extended it to support *reduction* operations, **operating configurably** as either a simple *reduction cache* (RedC) or as the full SpDCache outlined in the previous chapters.

5.2.1 RTL Implementation of SpDCache

We configure the original HPDcache as a 4-way *set*-associative cache with 128 *sets* and 64-byte cache-lines, resulting in a total cache size of 32 KiB. We extend the three-stage pipeline to handle the new *RED* instruction ($RED(key, value)$). Figure 5.1 provides an overview of SpDCache's microarchitecture, highlighting new blocks in *red*, modified blocks in *blue*, and untouched components in *black*. The overall architecture aligns with the design presented in Chapter 3, adapted here for hardware implementation. Let us briefly describe the operation of the *DRC* under three scenarios: hits, misses with available cache-lines, and misses requiring eviction.

In the case of a hit, the value of the *RED* instruction is accumulated with its corresponding value in the cache ($data_{key} \mathrel{+}= value$). At *st1*, the cache value, read in *st0* from *Data*, is accumulated with the *RED* value using a dedicated accumulator (*Acc*). The result is written back to *Data* at *st2*.

In the case of a miss, if there is an available cache-line, the value of the *RED* instruction is simply stored in this free entry of the cache. A free cache-line is selected at *st1*, and at *st2*, the *RED* value is written to *Data* along with zeros to reset the selected cache-line. A write to *Dir* is also performed to change the state of the cache-line from free to valid.

If a miss occurs but no cache-line is free, an eviction must be processed before handling the *RED* instruction. The *RED* instruction is placed in the *PrioRED table* to be replayed with higher priority than the *replay table*. Meanwhile, at *st2*, entries are allocated in the *miss handler*, *flush controller* and *accumulation table* to process the eviction. The eviction is a "fire-and-forget" operation, meaning the *RED* instruction only needs to wait for the entry in *Data* to be freed, not for the entire eviction to finish. The eviction process begins by fetching and resetting the chosen 64B cache-line from *Data*, which is done in one cycle by the *flush controller*. Simultaneously, the *miss handler* issues a read to the main memory. The *accumulation table* waits and temporarily stores both the old cache-line from the *flush controller* and the 64B response data from the *miss handler*. It then accumulates the cache-line with the data, word by word, and sends the 64B result back to the main memory using the existing write interface.

With the changes described above, we have transformed HPDcache into a *reduction cache*. To achieve multiple cache regions, we virtually separate the cache RAMs. To maintain HPDcache's *set*-associative operation, we divide the cache memory along its *sets*. Thus, each region of the cache has the same structure as the overall cache but is smaller in size: each *set* contains a fixed number (the number of ways) of 64B cache-lines, as depicted in Figure 5.2. Each region can thus map any possible address. To ensure that every address matches a unique *set* within each region, we allocate a power-of-two number of consecutive *sets* to a region. Considering the total number of *sets* S_T and our region R mapping on $S_R < S_T$ of them, starting at the *set* s_{offset}, if an address matches one of the *sets* of this region, no action is required. However, if the address matches a *set* s_{addr} outside the region, we need to map it to a unique *set* in the region such that $s_{region} = (s_{addr} \% S_R) + s_{offset}$, as illustrated in Figure 5.2. The three regions *GRC*, *DRC*, and *SRT* (*green*, *orange* and *blue*, respectively, in the example) are each defined by their number of *sets* S_R and their first *set*, s_{offset}. SpDCache can thus be reconfigured to operate as a generic cache, a traditional *reduction cache* or a region-based *reduction cache* based on the needs of the application (*config* in Figure 5.1).

The *SRT* region requires additional adjustments to function as an element-wise fully-

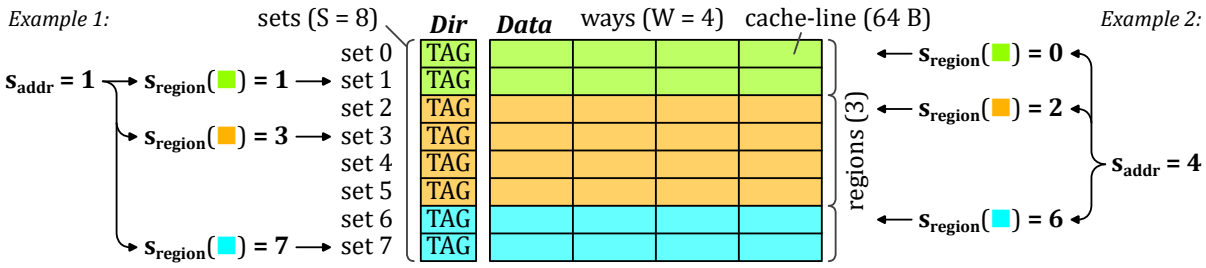


Figure 5.2 – Set Redirection Depending on the Region

associative cache. This region operates as if all the *sets* were combined into a single *set*, containing numerous elements. Each element requires its own directory entry. Therefore, we replace the *Dir* RAMs of this region with a CAM (*Content-Addressable Memory*) containing the addresses of each valid element in the *SRT* region. With the CAM, we can check for a hit across the entire region and retrieve the matching value stored in the *Data* RAMs. We also added a queue (*RMWQ*) to buffer the outstanding element-wise *Read-Modify-Write* (*RMW*) memory transactions.

To avoid address duplication between regions, the *MultiHit fetcher* in Figure 5.1 manages coherency between the *DRC* and *SRT*, following the rules defined in Section 3.5. Since software selects the region for each operation, it is possible that addresses that were allocated in the *SRT* need to be move to the *DRC*. The *MultiHit fetcher* performs two tasks. First, given an address, it probes the CAM for entries whose cache-line address matches. If one or more matches are found, it records their positions locally and notifies the *cache controller*. Second, when activated, the *MultiHit fetcher* gathers the corresponding elements from the *Data* RAMs of the *SRT* and assembles them into a single 64B line that will be written into the *DRC*.

Contrary to the description in Chapter 3, we do not transfer isolated elements from the *DRC* to the *RMWQ* on eviction. In our Python simulations we observed that only a negligible fraction of elements were transferred (mean 0.3%, range 0 to 5% across 48 matrices), which demonstrates the effectiveness of the *DRC* blocking behavior. To simplify the RTL implementation, this transfer is therefore omitted.

CONCLUSION

The SpDCache implementation introduces two primary extensions. First, it integrates **support for *reduction* instructions (*REDs*)** into the cache pipeline while preserving the generic cache behavior. This entails mechanisms for accumulating entire cache-lines and for handling element-wise atomic *RMW* transactions. Second, it implements a **virtual partitioning of the cache RAMs**: the memory is logically split and a *set*-redirection scheme is applied to dynamically create multiple regions.

5.2.2 Software Interface

We have added several custom instructions to the CVA6 RISC-V core [121]. The primary addition is the *reduction* instruction (*RED*), which takes an address and a value as arguments and expects no response. This instruction is accompanied by a separate region instruction (*REG*) that specifies in which region subsequent *reductions* should be processed. We have also added an instruction to flush the *reduction* regions (*DRC* and *SRT*) of the cache, which is necessary at the end of the algorithm. It is performed in the cache by the *CMO handler*. Finally,

we have included an instruction to enable and disable the *reduction* mode of SpDCache.

We demonstrate a simple example in Source Code 5.1 that accumulates multiple values at a single address. Before executing *RED* transactions, the *reduction* engine must be enabled (line 3). The cache transitions from a fully generic state to SpDCache with multiple regions using a hardware-defined static configuration. This example assumes a configuration with three regions: *GRC*, *DRC*, and *SRT*. For a simpler *reduction cache* configuration with only two regions (*GRC* and *DRC*), the *REG* instruction would be unnecessary. Additionally, the instruction can accept a dynamic configuration argument specifying word granularity and operation type. A future optimization would allow passing the entire configuration as a parameter, enabling dynamic configuration of SpDCache regions at runtime. Next, at line 8, we select the *In-Band Region* (*IBR*) region using the *REG* instruction. The subsequent two *RED* instructions are then executed and stored in the *DRC*. At line 13, we switch to the *Out-of-Band Region* (*OBR*) region, causing the following *RED* instruction to be processed by the *SRT*. Once all *RED* operations complete, SpDCache is flushed at line 17 before disabling the reduction engine, reverting to standard cache operation.

Source Code 5.1 – A Simple Example of *RED* Operations Usage

```
1  double data = .0; /* Initialize a data with 0 */
2
3  redEngineOn(config); /* Start the RED engine */
4  /* In this example, SpDCache is configured into three regions */
5  /* We can pass on a dynamic configuration to set the word granularity
6     and the type of operation */
7
8  REG(IBR); /* Following reductions are processed by the DRC */
9
10 RED(&data, 1.); /* data += 1 */
11 RED(&data, 2.); /* data += 2 */
12
13 REG(OBR); /* Following reductions are processed by the SRT */
14
15 RED(&data, 3.); /* data += 3 */
16
17 redFlush(); /* The DRC and SRT are flushed to memory */
18
19 redEngineOff(); /* Stop the RED engine */
20 /* SpDCache is now back to one fully generic cache region */
21
22 printf("%lf", data); /* Should print 6 */
```

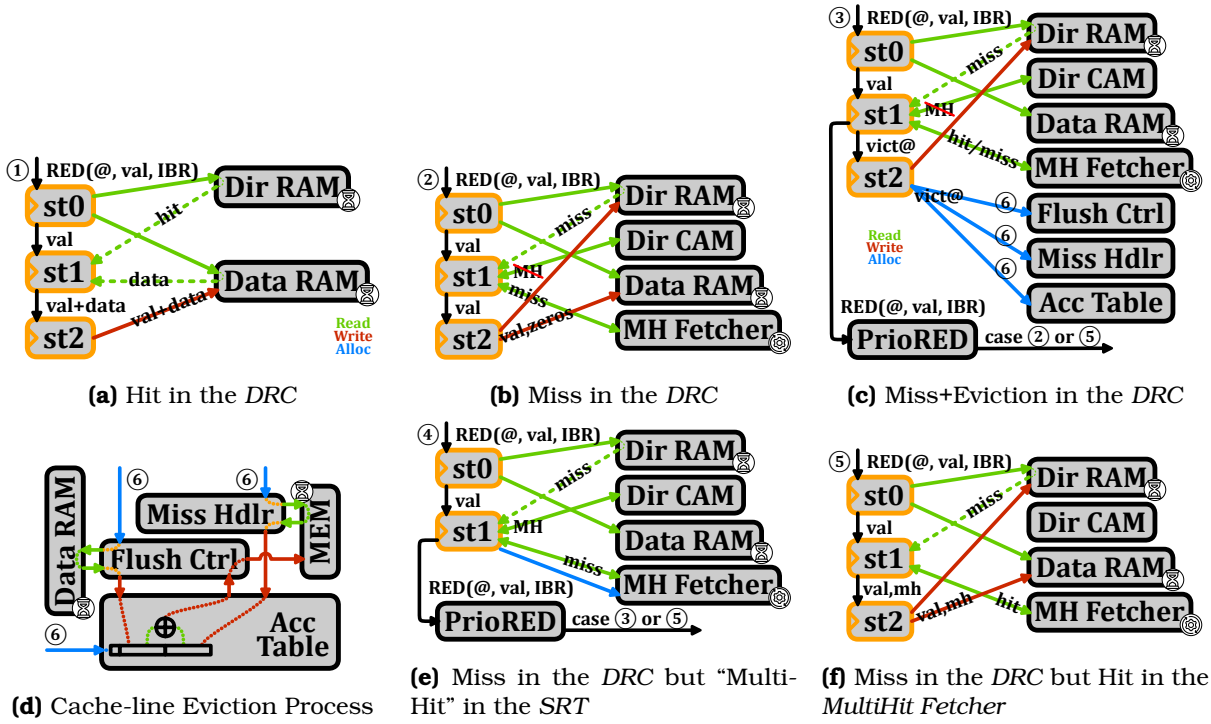
CONCLUSION

SpDCache provides a simple and flexible software interface through a minimal set of custom RISC-V instructions. The ability to dynamically enable and disable *reduction* mode allows the cache to seamlessly transition between generic and specialized operation, adapting to application requirements.

5.2.3 Detailed Operation of the SpDCache Pipeline

In this section, we detail the SpDCache pipeline architecture, which extends the HPDcache original pipeline by adding new behavior for *reduction* operations while preserving existing functionality. Figures 5.3 and 5.4 illustrate the various scenarios that occur when processing *RED* instructions, depending on the selected region (*IBR* or *OBR*, respectively). We represent the three pipeline stages (*st0*, *st1*, and *st2*) and their interactions with the auxiliary units

involved in processing *RED* instructions. The *green*, *red*, and *blue* arrows represent read and write operations, and unit allocations, respectively. The *Dir* and *Data* RAM units, marked with an hourglass, require one full cycle to respond.



(a) Hit in the *DRC*

(b) Miss in the *DRC*

(c) Miss+Eviction in the *DRC*

(d) Cache-line Eviction Process

(e) Miss in the *DRC* but "Multi-Hit" in the *SRT*

(f) Miss in the *DRC* but Hit in the *MultiHit Fetcher*

Figure 5.3 – Pipeline Operations when the Region Argument is *IBR*

We now describe the scenarios when the region argument is *IBR*. In Figure 5.3a, the pipeline issues reads to the *Dir* and *Data* RAMs at *st0*. Responses arrive one cycle later at *st1*, indicating whether a hit or miss occurs in the *DRC*. This initial pipeline stage is identical across all scenarios. In case ①, a hit is detected, so the data received at *st1* is accumulated with the *RED* value and forwarded to *st2* for write-back to the *Data* RAM.

When a miss occurs, multiple scenarios are possible (Figures 5.3b to 5.3f), as an eviction may be required and the *MultiHit Fetcher* may be active. In case ②, upon detecting a miss at *st1*, the pipeline checks the *Dir* CAM of the *SRT* for a "multi-hit". As described in Chapter 3, when a miss occurs in the *DRC*, we gather elements with the same cache-line address from the *SRT* to prevent duplication. This check completes within the cycle and notifies the *MultiHit Fetcher*. If a "multi-hit" is found, the *MultiHit Fetcher* allocates an entry and takes control of the pipeline to read and invalidate matching entries from the *Data* RAM, storing them locally for later use. If no "multi-hit" occurs (case ②), we handle a simple miss by writing the *RED* value to a free cache-line in the *Data* RAM at *st2*.

In case ④, a "multi-hit" is detected and the *MultiHit Fetcher* is allocated. The *RED* instruction is placed in the *PrioRED Table* for immediate replay once the pipeline becomes available, proceeding to case ③ or ⑤ depending on whether an eviction is needed.

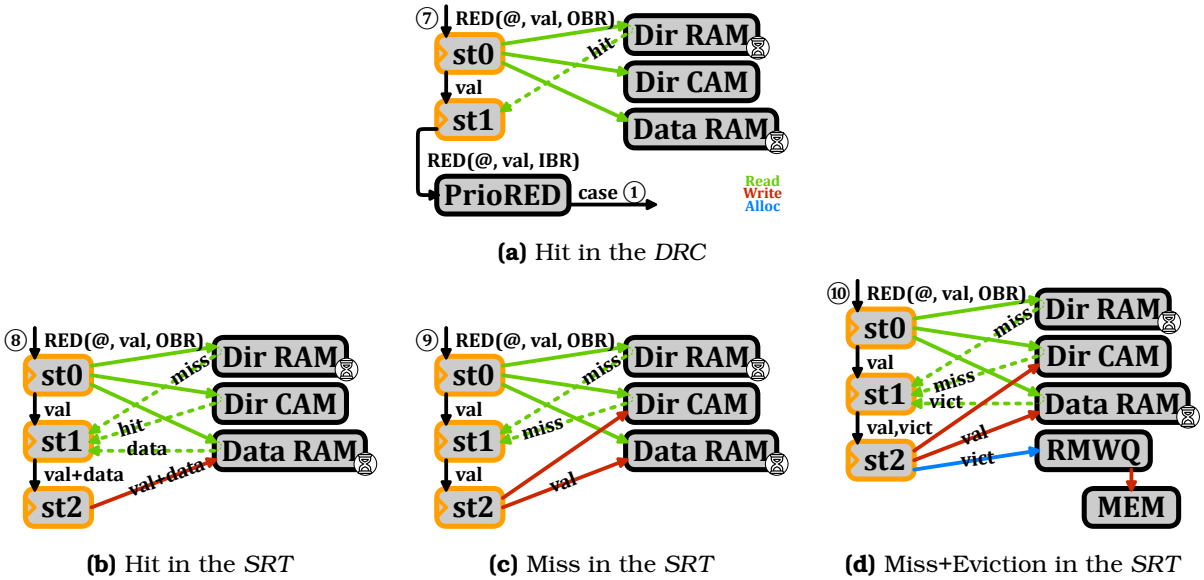
In case ⑤, which occurs only upon replay, the *RED* instruction misses in the *DRC* but hits in the *MultiHit Fetcher*, which has cached the gathered data from the previous case ④. The *RED* value is then accumulated with the *MultiHit Fetcher* data and written to the *Data* RAM at *st2*.

Case ③ handles the eviction scenario, which applies whether or not a "multi-hit" replay

is involved. When an eviction is required, a victim cache-line is selected at *st1* and must be evicted before inserting the new value. The eviction process begins at *st2*, with the *Miss Handler*, *Flush Controller*, and *Accumulation Table* allocated for the victim address. Concurrently, the *RED* instruction is placed in the *PrioRED Table* for immediate replay. Once the victim cache-line is freed, the instruction is replayed as case ② or ⑤, depending on whether data exists for the same cache-line address in the *MultiHit Fetcher*. Note that when a "multi-hit" occurs (case ④), the *MultiHit Fetcher* retains the gathered data until reaching case ⑤. Subsequently, since the data is no longer present in the *Dir* CAM and *Data* RAM, no "multi-hit" will occur for that address on replay.

Figure 5.3d depicts the eviction process, which executes in parallel with the pipeline. Upon receiving the victim address in case ③, the *Flush Controller* immediately reads the *Data* RAM and invalidates the entry in a single cycle. This allows the pipeline to replay the *RED* instruction without waiting for the entire eviction to complete. The *Flush Controller* then forwards the victim data to the *Accumulation Table* for temporary storage. Meanwhile, the *Miss Handler* issues a memory read, which may take on the order of a hundred cycles. Upon receiving the data from memory, the *Miss Handler* sends it to the *Accumulation Table*. Once the *Accumulation Table* has both data lines, it performs word-by-word accumulation and returns the result to the *Flush Controller*, which writes it back to memory to complete the eviction.
The following sequences enumerate all possible cases when the region argument is *IBR*:

- hit in the *DRC*: ①;
- miss in the *DRC*: ②;
- miss+eviction in the *DRC*: ③ + ⑥ → ②;
- miss in the *DRC* with "multi-hit" in the *SRT*: ④ → ⑤;
- miss+eviction in the *DRC* with "multi-hit" in the *SRT*: ④ → ③ + ⑥ → ⑤.



(a) Hit in the *DRC*

(b) Hit in the *SRT*

(c) Miss in the *SRT*

(d) Miss+Eviction in the *SRT*

Figure 5.4 – Pipeline Operations when the Region Argument is *OBR*

We now describe the cases when the region argument is *OBR*, as illustrated in Figures 5.4a to 5.4d. In all cases, we check for hits in both the *DRC* and *SRT* at *st0*. If a hit occurs in the *DRC*, case ⑦, the *RED* instruction is processed in the *DRC* regardless of the region argument. It is then sent to the *PrioRED Table* for replay as a *DRC* hit, corresponding to case ①.

If no hit is found in the *DRC*, we check the *Dir* CAM for hits in the *SRT*. In case of a hit, ⑧, the data is accumulated with the *RED* value and written back to the *Data* RAM. In case of a miss, ⑨, the data is inserted into the *Data* RAM. For an eviction, ⑩, the victim data is read directly between *st0* and *st1*, since the *Dir* CAM responds within one cycle and can initiate the victim read before *st1*. Thus, the *RED* value is inserted into the *Data* RAM at *st2*, while the victim data is sent to the *RMWQ*. The *RMWQ* independently issues atomic *RMW* transactions to memory as entries become available.
The following sequences enumerate all possible cases when the region argument is *OBR*:

- hit in the *DRC*, ⑦ → ①;
- hit in the *SRT*, ⑧;
- miss in the *SRT*, ⑨;
- miss+eviction in the *SRT*, ⑩.

NOTE

This implementation leverages most of the original pipeline capabilities while keeping modifications to a minimum. We do not claim this is the optimal design for SpDCache. Further optimizations and architectural refinements are likely possible.

CONCLUSION

In this section we explained the exact steps that a *reduction* instruction takes when it targets either the *IBR* or *OBR*. In both cases we need to handle the cases of hits, misses and evictions while **preventing duplications**.

5.3 Evaluation

In this section, we aim to evaluate the performance of various cache configurations and their impact on memory traffic and latency during RTL simulations. We will first outline the methodology used for these evaluations, followed by a detailed analysis of the results obtained from the simulations, ending with comparisons with the previous models.

5.3.1 Methodology for RTL Simulations

The starting point is a CVA6 [131] RISC-V core with HPDcache [34] as an L1 data-cache with size 32 KiB ($S = 128$, $W = 4$, $l = 8$ elements). This serves as our baseline for performance comparison which we refer to as the Generic Cache (GC). Next we evaluate a *reduction cache* which we derived from the HPDcache and configured to have two regions: a generic region with 16 KiB, and a *reduction* region with 16 KiB whose behavior is the same as the *DRC* region of SpDCache. We refer to this design as RedC. Finally, we evaluate SpDCache (SpDC) with a reference configuration of: *GRC* = 8 KiB, *DRC* = 16 KiB, *SRT* = 8 KiB, which corresponds to the 25/50/25% configuration selected in Chapter 3. The configurations are summarized in Table 5.1.

All our simulations are performed using Siemens Questasim™ 2023.3 and the memory accesses outside the L1 incur a fixed latency of 64 clock cycles. We do not aim to evaluate a full system in this work and thus use a single-core, single-cache-level environment. The CVA6 core is single-issue and can process at most one instruction per cycle. The fixed memory latency of 64 cycles provides a reasonable approximation of the latency incurred by

Table 5.1 – Region Sizing of the Cache Configurations for Evaluation

Configurations	*GRC*	*DRC*	*SRT*
GC	100% ($S = 128$)	0% ($S = 0$)	0% ($S = 0$)
RedC	50% ($S = 64$)	50% ($S = 64$)	0% ($S = 0$)
SpDC	25% ($S = 32$)	50% ($S = 64$)	25% ($S = 32$)

requests passing through the other memory levels, and a deterministic memory model makes the results easily reproducible.

Currently, our SystemVerilog implementation only supports 64-bit integer additions, precluding floating-point accumulations. Extending SpDCache to support floating-point operations remains future work. We employ 64-bit integers throughout and assume this is representative because sparse kernels are memory-bound, and 64-bit integers have the same data size as FP64 doubles.

A second implementation constraint is that region sizes for *GRC*, *DRC*, and *SRT* must be powers of two in terms of *sets*, unlike the analytical and Python models described in Chapters 3 and 4. This limitation motivates our choice of a 50/50/0% configuration for RedC, which we expect to slightly underperform compared to a configuration which allocates more space to the *DRC*. Future optimizations of the implementation could relax this constraint.

CONCLUSION

The RTL simulation evaluation was performed using three cache configurations, GC, RedC, and SpDCache, using a single-core CVA6 processor with a single cache level.

5.3.2 Sparse Formats Used for the RTL Simulations

The CSR/CSC formats are widely used in high-performance computing. As we are using an `OP` algorithm, we use the CSC format for the majority of our simulations. However, in our evaluation with banded matrices, we also simulated SpDCache with a modified CSC format tailored for banded matrices which we call BaCSC. Indeed, since SpDCache must know where the banded region lies, and to avoid calculating it using precious processor instructions, this information can be placed in memory with the matrix. As shown in Figure 5.5, compared to a CSC, we added extra information in the ptr table to identify the beginning and end of the band. Indeed, three pointers, instead of one, are now needed per column, as shown with the example of column 2 in *yellow*. The column is split in one in-band region in the middle (IB, from ptr 7 to 9) and two out-of-band regions around (OB, from ptr 6 to 7, and 9 to 10). The ptr table size thus becomes $3 \times M + 1$. With this format, we can avoid the computational cost of identifying in which region non-zero elements are, while traversing the matrix, with the drawback of having to preprocess the matrix to store this additional information.

CONCLUSION

We employ the CSC format for most simulations and additionally evaluate SpDCache using BaCSC, a specialized format for SpDCache which specifies the location of the banded region of the matrix.

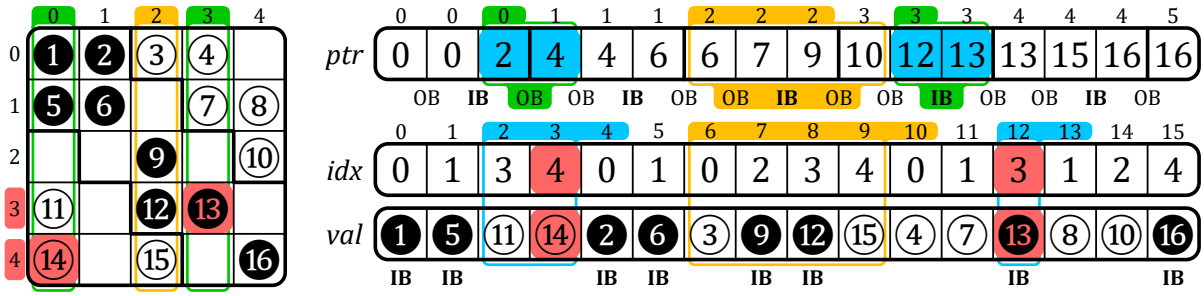


Figure 5.5 – Example of a Sparse Matrix in BaCSC Format

5.3.3 Software Implementation of Outer-Product SpMV Kernel

The code running on the CVA6 is written in generic C, compiled with *gcc* 11.2.0 and -O3 optimization. The *RED*-related instructions used in the `OP` SpMV kernel are manually inserted using *asm* directives, as illustrated in Source Code 5.2.

The code shown is for SpDCache. Variants for other designs differ as follows: RedC omits region selection (line 15) and invokes *REG* once before the loops with *IBR* as the argument; GC requires no *REG* instruction and replaces *RED* with standard accumulation: $c[idx] \mathrel{+}= val$.

For SpDCache, it is necessary to know the target region for each *RED* instruction based on the size of the band (line 15). The half-*band-width* w_{hB} is derived from Chapter 4 (Equation 4.2) to ensure the in-band region fits within the *DRC* independent of matrix structure. In our reference configuration ($s_C(DRC) = 2048$, $l = 8$), $w_{hB} = 1021$. Region selection follows Equation 5.1 via an inline function, avoiding conditional branches within the inner-loop.

$$\forall (i,j) \in [\![0, M-1]\!] \times [\![0, N-1]\!],\ region = \begin{cases} IBR & \text{if } |i-j| < w_{hB} \\ OBR & \text{else} \end{cases} \tag{5.1}$$

On-the-fly region computation introduces overhead in the critical inner-loop. To mitigate this, we propose an alternative implementation (Source Code 5.3) that precomputes regions and stores them using the BaCSC format (see Section 5.3.2). Each column iteration begins with lookups in the outer-loop (line 5) to retrieve pointers for three sections: top out-of-band (*OB1*), in-band (*IB*), and bottom out-of-band (*OB2*). The column is then processed through three separate inner-loops (lines 13, 21, 29) without region computation overhead.

Each approach trades off one limitation for another. SpDCache(CSC) uses simple CSC format with minimal preprocessing and reduced memory overhead, but incurs additional inner-loop instructions and thus latency per iteration. SpDCache(BaCSC) reduces instructions and latency at the cost of matrix preprocessing into BaCSC format and an increased memory footprint for the matrix. Both variants are evaluated in the results section.

These software approaches avoid hardware modifications, providing flexibility. However, hardware-computed regions with CSC format would eliminate the noted drawbacks. A detailed breakdown of each design's instruction characteristics:

- GC: 10 inner-loop instructions; 3 instructions per *RED* with critical load dependency.
- RedC: 8 inner-loop instructions; 1 "fire-and-forget" *RED* instruction; 1 *REG* setup per entire computation.
- SpDCache(CSC): 14 inner-loop instructions; 1 "fire-and-forget" *RED* instruction; region computation within inner-loop.
- SpDCache(BaCSC): 8 inner-loop instructions; 1 "fire-and-forget" *RED* instruction; outer-

Source Code 5.2 – C Code for the `OP` SpMV with *RED* Operations

```
/* Data:
 *   A(M, N), stored in CSC: A.ptr, A.idx, A.val
 *   b(N), dense:            b
 *   c(M), dense:            c
 * */

redEngineOn(config);

int currCol = A.ptr[0];
for (int j = 0; j < N; j++) { /* Column first */
    int nextCol = A.ptr[j+1];
    int bj      = b[j];

    for (int pos = currCol; pos < nextCol; pos++) {
        int idx = A.idx[pos];
        int val = A.val[pos] * bj;

        /* Without RED engine: c[idx] += val; */
        /* With RED engine: */
        int *addr    = c + idx;
        region_t reg = regionSelect();

        asm volatile ( /* RISC-V ISA: Type R */ /* REG(reg); */
            "reg x0, x0, %[rs2]"
            :
            : [rs2] "r" ((int) reg)
        );
        asm volatile ( /* RISC-V ISA: Type S */ /* RED(addr, val); */
            "red.w %[rs2], 0(%[rs1])"
            :
            : [rs2] "r" (addr), [rs2] "r" (val)
        );
    }

    currCol = nextCol;
}

redFlush();
redEngineOff();
```

loop overhead.

To prevent software from limiting performance, we unroll inner-loops by a factor of 8 (not shown in Source Codes 5.2 and 5.3). Each unrolled iteration executes 8 logical iterations with optimized instruction scheduling. With cache-line size $l = 8$, this unrolling hides read-miss latency through hits on consecutive addresses. "Random" *reduction* accesses do not benefit from this latency hiding. Consequently, GC experiences potential latency from both *reductions* reads and writes, while RedC and SpDCache benefit from "fire-and-forget" semantics and consecutive *reduction* scheduling.

SpDCache(CSC) is constrained to 4× unrolling due to an insufficient number of register in the CVA6 CPU. This limitation, addressable with a higher-register CPU, highlights another advantage of hardware based region computation.

CONCLUSION

We utilize four distinct software implementations of the `OP` SpMV, each customized for our four simulations and highly unrolled to minimize performance limitations due to loop overheads.

Source Code 5.3 – Optimized C Code for the `OP` SpMV when Using BaCSC

```
/* Data:
 *   A(M, N), stored in BaCSC: A.ptr, A.idx, A.val
 *   b(N), dense:              b
 *   c(M), dense:              c
 * */

redEngineOn(config);

/* A is stored in BaCSC format with pointers delimiting the band sections */
int currCol_OB1     = A.ptr[0];   /* begining of the column - first OB section*/
for (int j = 0, int k = 0; j < N; j++, k += 3) {
    int currCol_IB  = A.ptr[k+1]; /* begining of the band   - IB section */
    int currCol_OB2 = A.ptr[k+2]; /* end of the band        - second OB section */
    int nextCol     = A.ptr[k+3]; /* end of the column      - next column */
    int bj          = b[j];

    /* 1st step: top OB section of the column */
    REG(OBR);
    for (int pos = currCol_OB1; pos < currCol_IB; pos++) {
        int idx = A.idx[pos];
        int val = A.val[pos] * bj;
        RED(c + idx, val);
    }

    /* 2nd step: IB section of the column */
    REG(IBR);
    for (int pos = currCol_IB; pos < currCol_OB2; pos++) {
        int idx = A.idx[pos];
        int val = A.val[pos] * bj;
        RED(c + idx, val);
    }

    /* 3rd step: bottom OB section of the column */
    REG(OBR);
    for (int pos = currCol_OB2; pos < nextCol; pos++) {
        int idx = A.idx[pos];
        int val = A.val[pos] * bj;
        RED(c + idx, val);
    }

    currCol_OB1 = nextCol; /* begining of the next column */
}

redFlush();
redEngineOff();
```

5.3.4 Matrix Selection for the RTL Evaluation

We have selected an extended set of 62 square matrices from the SuiteSparse Matrix Collection [67] which includes virtually all the matrices used by recent hardware accelerators [91, 108, 126, 132, 133] (full list in Appendix B). We verified that this set is diverse containing banded and non-banded sparse matrices with various sizes ($M \in [1.2 \times 10^2, 2.0 \times 10^6]$), numbers of non-zero elements ($nnz \in [2.2 \times 10^3, 1.4 \times 10^7]$) and a wide range of densities ($d \in [1.4 \times 10^{-6}, 7.8 \times 10^{-1}]$).

Based on our experimental results presented in the following section, we categorize these matrices into two groups according to their performance characteristics. To facilitate this classification, we introduce a new metric, d_{cd}, defined as the average density of cache-line-sized clusters within matrix columns. Specifically, for a cache with line size l, this metric quantifies the average density of non-empty cache-lines (or vertical-stripes as referenced in Chapter 4), ranging between $\frac{1}{l}$ and 1.

This metric provides a fine grain view of matrix density, which aligns more closely with cache characteristics. When d_{cd} approaches $\frac{1}{l}$ for a given matrix, it indicates that most non-zero elements are separated by at least $l-1$ zeros. Consequently, the cache-lines are often nearly empty, which may render the line-wise *DRC* of a RedC inefficient, while the element-wise *SRT* of SpDCache becomes more beneficial. Conversely, as d_{cd} increases, the non-zero elements of the matrix cluster more closely, enhancing the efficiency of the *DRC*.

In our experiments with $l = 8$, we establish two groups of matrices based on a d_{cd} threshold of 20%. This threshold corresponds to either one to two non-zero elements per eight. At this point, this threshold may appear somewhat arbitrary and it, of course, depends on the processor and caches. As we presents the results, this choice, for our CVA6 core, will become clearer.

Using the threshold based on d_{cd}, we categorize 21 matrices in group 1 ($d_{cd} \leqslant 20\%$) and 41 in group 2 ($d_{cd} > 20\%$). We have selected 13 well-known matrices from both groups that are frequently used for performance evaluation, as illustrated in Table 5.2. The selected matrices represent a range of overall matrix densities (d) and d_{cd} values, as depicted by the *red* and *purple* points in the scatter plot in Figure 5.6. In the following evaluation, we report the performance results of the 13 matrices from group 1 and 2, the geometric means for both groups (covering 21 matrices in group 1 and 41 in group 2), and the geometric mean for the full set of 62 matrices.

Table 5.2 – Characteristics of Selected Group 1 and 2 Matrices

ID	Matrix	M	nnz	Density	d_{cd}
Group 1					
11	poisson3Da	13.5 k	352 k	1.93 e-3	14%
12	cit-HepTh	27.8 k	353 k	4.57 e-4	13%
13	soc-Epinions1	75.8 k	508 k	8.84 e-5	16%
14	sme3Db	29.1 k	2.08 M	2.46 e-3	15%
15	Stanford	282 k	2.31 M	2.91 e-5	12%
16	web-Google	916 k	5.11 M	6.08 e-6	13%
Group 2					
21	msc10848	10.8 k	1.23 M	1.05 e-2	73%
22	opt1	15.4 k	1.93 M	8.09 e-3	86%
23	bcsstk32	44.6 k	2.01 M	1.01 e-3	75%
24	xenon2	15.7 k	2.87 M	1.56 e-4	40%
25	consph	83.3 k	6.01 M	8.66 e-4	57%
26	x104	108 k	10.0 M	8.66 e-4	78%
27	pwtk	218 k	11.6 M	2.45 e-4	75%

CONCLUSION

Benchmarking was done using 62 diverse matrices from the SuiteSparse Matrix Collection which we divided into two groups based on their local density (d_{cd} metric).

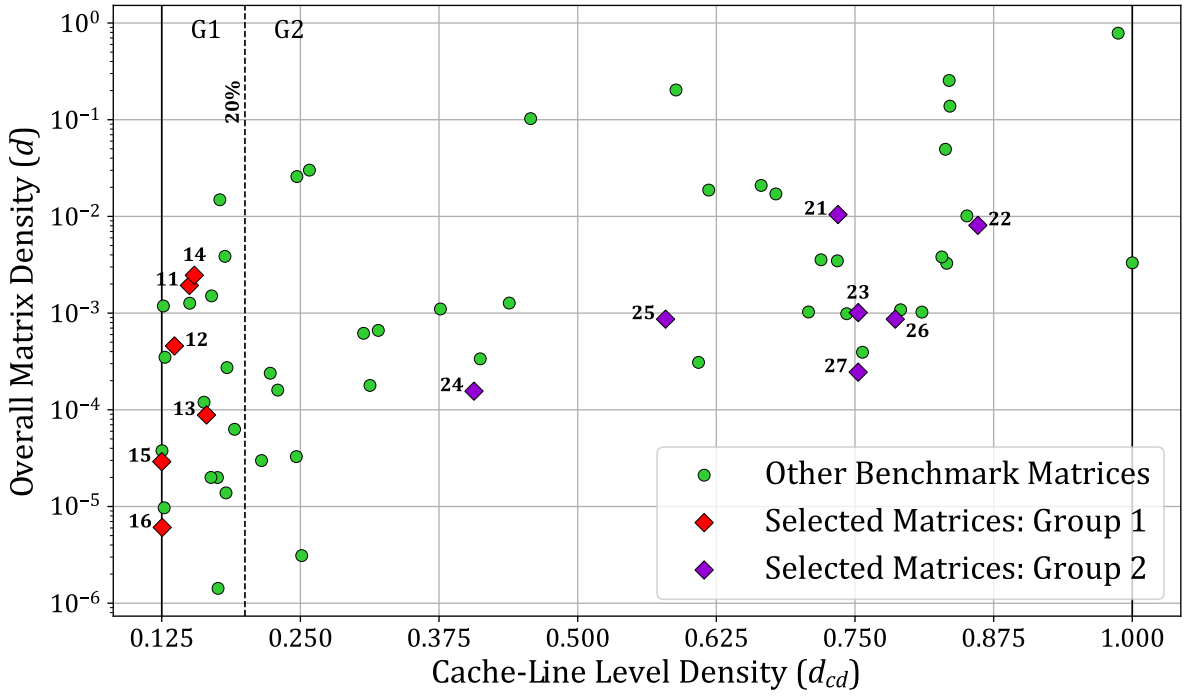


Figure 5.6 – Density Characteristics of Benchmark Matrices: d_{cd} vs Density (d)

5.3.5 Results Analysis

In this section, we present and analyze performance results in two complementary ways. First, in Sections 5.3.5.1 and 5.3.5.2, we report memory traffic and speedup for RedC and SpDCache normalized relative to the baseline GC. Second, in Section 5.3.5.3, we report memory traffic and speedup for GC, RedC, and SpDCache normalized per matrix non-zero elements (nnz). This two-part analysis provides complete understanding of performance behavior across the two matrix groups defined in the previous section.

5.3.5.1 Memory Traffic Performance

For all benchmark matrices, we simulated the RTL model executing the full **OP** SpMV and measured output memory traffic and input memory traffic. In Figures 5.7 and 5.8, we show the output memory traffic and the total memory traffic (output + input), respectively, for RedC, SpDCache with CSC, and SpDCache using the BaCSC format, normalized compared to the output traffic when simulating with GC.

As shown in Figure 5.7, looking at the geometric mean on all 62 matrices, our main result is that SpDCache achieves a 3.9× output memory traffic reduction compared to GC, which is 2.8× better than RedC (1.4×). While SpDCache performs consistently for both groups of matrices, RedC lacks hardware adaptation for irregular access patterns, making its performance heavily group-dependent. For low d_{cd} (sparse cache-lines, group 1), the line-wise cache proves ineffective. Conversely to RedC, SpDCache maintains dense data in the *DRC* across columns, enabling the rotating pattern from Chapter 4. Outside the *DRC*, the element-wise *SRT* organization efficiently avoids transferring mostly empty cache-lines.

For group 1 matrices, RedC traffic exceeds GC because GC's cache is 2× larger than RedC's

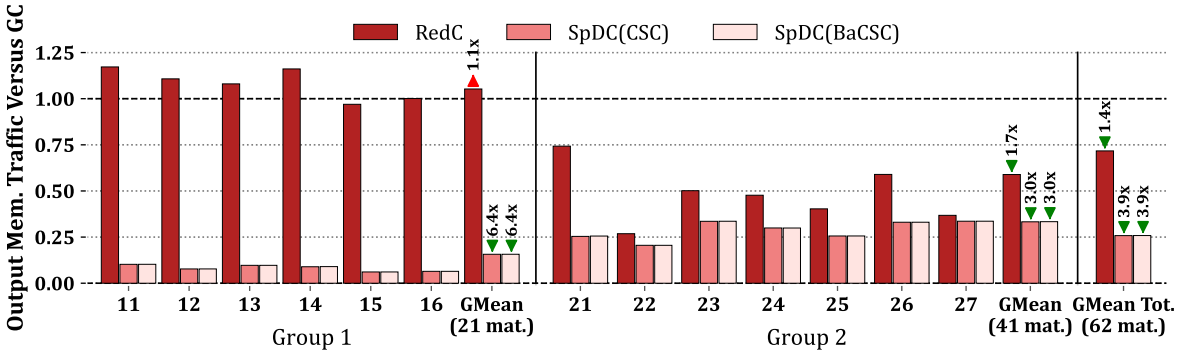


Figure 5.7 – Output Memory Traffic Normalized Compared to GC, Measured from RTL Simulations

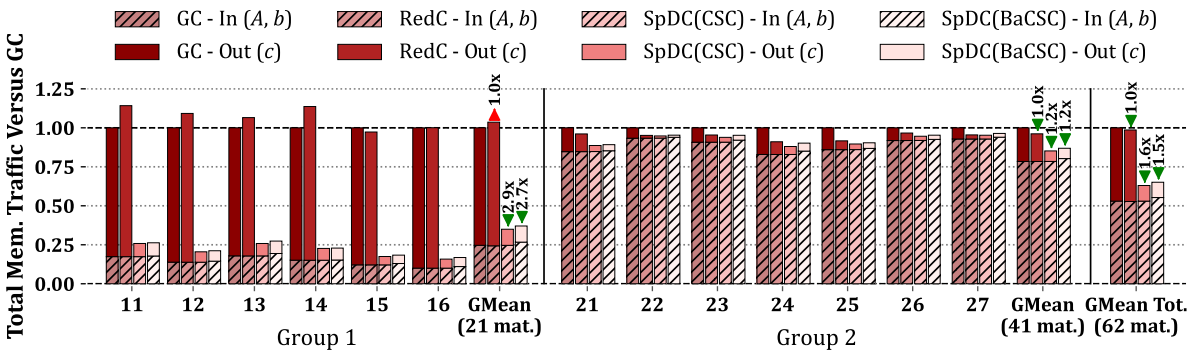


Figure 5.8 – Total Memory Traffic for Input Matrix A, Input Vector b and Output Vector c, Normalized Compared to GC, Measured from RTL Simulations

DRC, reducing performance [1]. For group 2 matrices, higher cache-line density compensates, allowing RedC to leverage near-memory accumulation through increased hit-rates.

In Figure 5.8 we show the total memory traffic broken down by input data reads (A and b, *hatched*) and output vector updates (c, *plain*). As the two SpDCache variants show no difference in output traffic, the total memory traffic shows that the increased matrix storage, which incurs additional memory reads, for SpDCache with BaCSC has an overhead below 10% (1.6× versus 1.5× when looking at the geometric mean on all 62 matrices). Otherwise, input memory traffic (A and b, *hatched*) is constant on every configurations and both groups of matrices [2].

CONCLUSION

SpDCache achieves 3.9× output memory traffic reduction versus GC, substantially outperforming RedC's 1.4× reduction. Group 1 matrices (low d_{cd}) benefit most from traffic reduction due to sparse cache-lines, while group 2 (high d_{cd}) shows more modest gains as denser cache-lines are inherently more efficient. The total memory traffic overhead for the BaCSC format is under 10%.

(1). Recall that for the benchmarking, the total cache size was kept constant at 32 KiB. The RedC has a 16 KiB generic region and a 16 KiB *DRC*, whereas the GC can flexibly use the full 32 KiB.
(2). *Hatched* bars seem to vary between matrix groups. It is a visual effect of the normalization versus GC.

5.3.5.2 Latency Performance

We now analyze total compute time, recalling that all main memory reads incur a fixed 64 cycle latency. Figure 5.9 presents the speedup relative to GC.

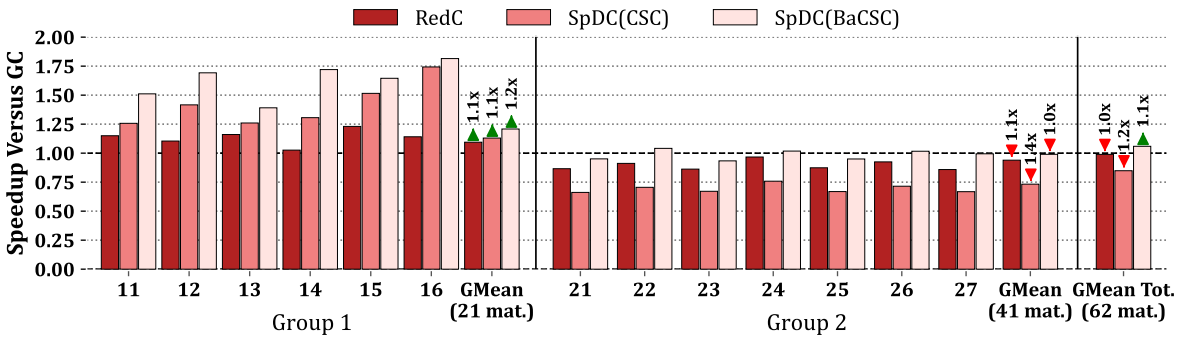


Figure 5.9 – Speedup Compared to GC, Measured from RTL Simulations

Group 1 matrices show speedup across all cache configurations, whereas group 2 shows slowdown. Overall, looking at the geometric mean on all 62 matrices, RedC achieves no speedup over GC, SpDCache with CSC incurs 1.2× slowdown, and SpDCache with BaCSC achieves 1.1× speedup.

For group 1 matrices, RedC obtains a small speedup (1.1×) because *RED* instructions are "fire-and-forget", replacing multiple read-accumulate-write operations. However, blocking occurs when the *accumulation table* processes prior evictions limiting the benefit. Group 1's sparse cache-lines trigger frequent eviction bursts, dominating the performance and making the system highly memory-bound. Thus, SpDCache's substantial memory traffic reduction translates directly to speedup by eliminating costly line-wise evictions. SpDCache with BaCSC performs best, with a 1.2× speedup, by eliminating region calculation overhead in the critical loop.

For group 2 matrices, our single-core RISC-V processor becomes compute-bound. Higher cache-line density means evictions are typically followed by non-blocking hits. In the extreme case, 7 hits following an eviction require 56-98 instructions (8-14 instructions per iteration, see Section 5.3.3) before the next potential eviction, sufficient to hide the 64 cycle memory latency.

Consequently, SpDCache with BaCSC surpasses GC performance. RedC underperforms by 1.1× due to its 2× smaller *DRC* versus GC. With a larger *DRC*, RedC would yield comparable performance. SpDCache with CSC underperforms by 1.4× because, in compute-bound regimes, reducing inner-loop instruction count is critical for performance.

CONCLUSION

For group 1 matrices (low d_{cd}), **SpDCache(BaCSC) performs best**, with a 1.2× speedup, by eliminating costly evictions and region computation overhead. For group 2 matrices (high d_{cd}), the workload becomes compute-bound but SpDCache(BaCSC) still matches the GC performance. Overall, SpDCache(BaCSC) achieves a 1.1× average speedup, while the other configurations underperform on average, with **gains concentrated in memory-bound scenarios**.

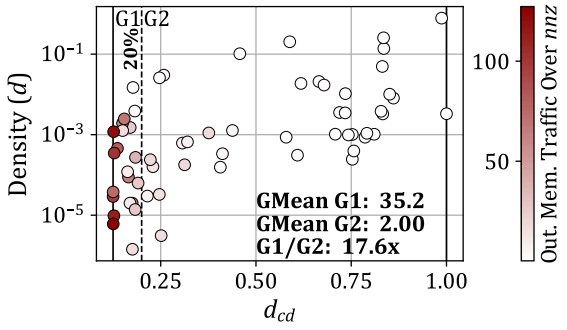
5.3.5.3 Further Discussion Regarding the Two Performance Groups

In the two previous sections, we showed performance normalized to GC, highlighting how SpDCache affects baseline performance, assuming it was used as an alternative cache. Here, we further analyze performance by normalizing against the number of non-zero elements (nnz), yielding average raw output memory traffic and latency per **OP** SpMV iteration, or equivalently per *reduction* processed. This reveals raw performance variations across matrix groups and configurations (GC, RedC, SpDCache(CSC), and SpDCache(BaCSC)). We can thus analyze performance behaviors across the two matrix groups without the normalization bias introduced by comparing against GC. Figure 5.10 (eight graphs: four rows, two columns) organizes results as follows: each row of graphs represents a configuration; columns show output memory traffic and latency normalized by nnz. Within each graph, dots represent the 62 matrices scattered by density (d) and d_{cd}, and *white*-to-*red* coloring indicates performance data (output traffic or latency) which values are visible on the right-side bars. For both output traffic and latency metrics, for which lower values are better, white dots indicates matrices achieving better performance (*white* and *red* corresponds to low and high values, respectively). Moreover, each graph is annotated with the geometric mean performance over the group 1 matrices (G1, before the *dashed*, vertical threshold at 20%), the geometric mean performance over the group 2 matrices (G2, after the *dashed*, vertical threshold at 20%), and the performance of group 1 compared to group 2 (G1/G2, higher it is, better performance group 2 has compared to group 1). Note that the performance scale is not the same across graphs (varying minimums and maximums on the right-side bars), as we no more directly compare configurations between each other (previous sections), but solely compare performance between matrices for each configuration.

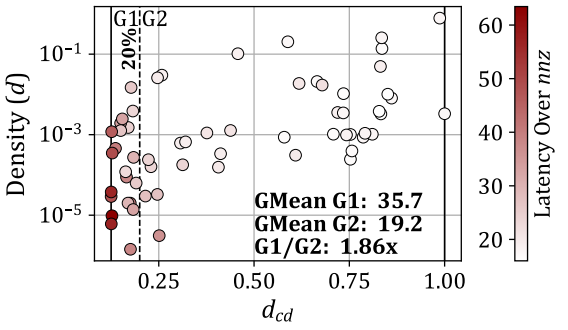
We first notice that GC (graphs 5.10a and 5.10b) shows group-dependent variation. Indeed, the GC output memory traffic exhibits 17.6× higher traffic on group 1 than group 2 (G1/G2). This normalization bias relative to GC becomes evident when examining raw per-nnz metrics, explaining why SpDCache demonstrates greater relative benefit to group 1 matrices in previous comparisons against GC (Figure 5.7).

Input memory traffic exhibits the same bias across groups. Notably, GC input memory traffic normalized per nnz is only 1.1× higher for group 1 compared to group 2. Consequently, when examining the total memory traffic in Figure 5.8 from Section 5.3.5.1, the input traffic component (*hatched* bars) remains nearly constant between groups. Output traffic (*plain* bars) thus dominates GC performance in group 1, representing 75% of the total, whereas it accounts for less than 25% in group 2. This observation supports that the output traffic reduction achieved by SpDCache provides substantially greater benefit to group 1 matrices, where output traffic is the primary bottleneck, compared to group 2 matrices, which already exhibit low output traffic under GC.

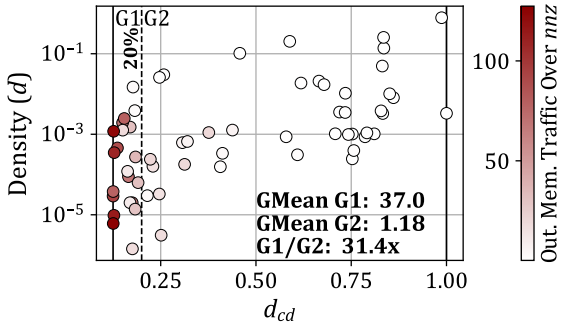
Going back on the graphs, we then observe that group 2 consistently outperforms group 1 across all configurations and metrics (all eight graphs), *e.g.*, the *red* dots are nearly all group 2 matrices, on the left side. Group 1 exhibits from 8.27× to 31.4× higher memory traffic per nnz (G1/G2s of first column of graphs) and from 1.20× to 1.86× higher latency per nnz (G1/G2s of second column of graphs) than group 2. When d_{cd} is low (left of each graph), cache-lines remain nearly empty, increasing eviction frequency. Additionally, low d_{cd} causes consecutive **OP** SpMV iterations to generate eviction bursts. Consequently, when d_{cd} is low: (1) eviction count increases and (2) evictions cluster temporally. The first effect raises output vector c traffic, potentially triggering critical memory reads that affect latency. The second exacerbates la-



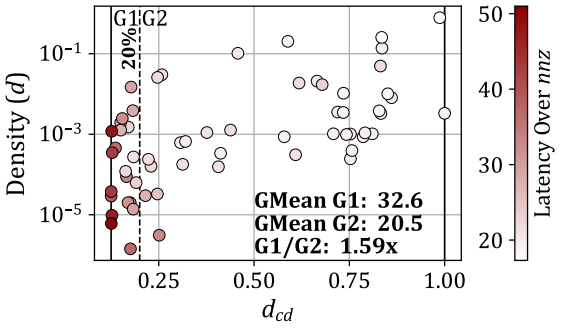
(a) GC Output Memory Traffic Over nnz



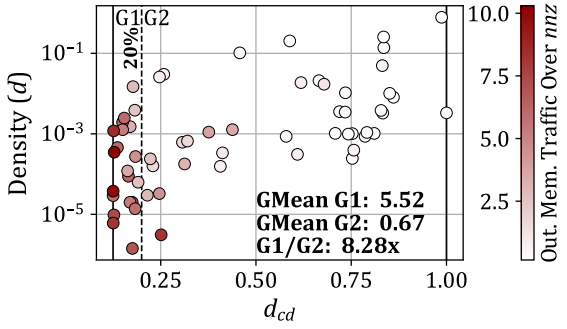
(b) GC Speedup Over nnz



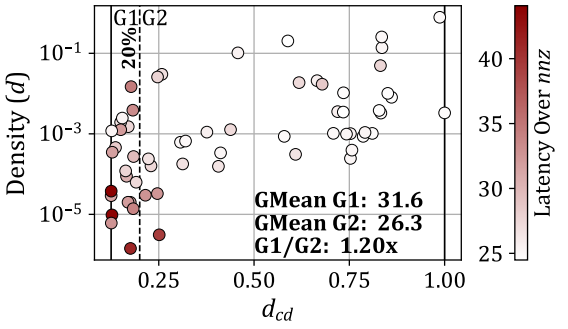
(c) RedC Output Memory Traffic Over nnz



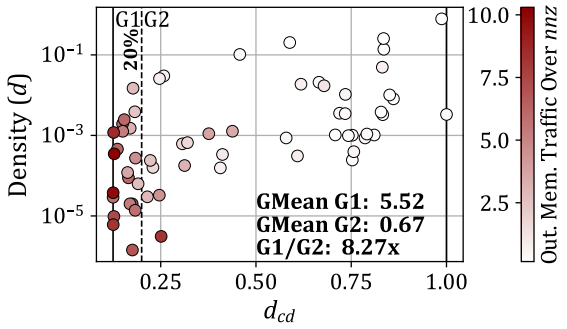
(d) RedC Speedup Over nnz



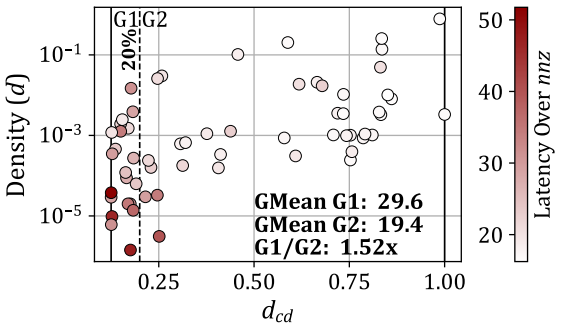
(e) SpDC(CSC) Output Memory Traffic Over nnz



(f) SpDC(CSC) Speedup Over nnz



(g) SpDC(BaCSC) Output Memory Traffic Over nnz



(h) SpDC(BaCSC) Speedup Over nnz

Figure 5.10 – Output Memory Traffic and Speedup Performance, Normalized Over nnz, and Visualized Across Benchmark Matrix Densities and d_{cd}

tency: when consecutive iterations process evictions, the intervening ∼10 instructions cannot hide our fixed, 64 cycle read latency (insufficient loop unrolling). As visible in graph 5.10b, worst-case group 1 matrices average 64 cycles latency on GC (maximum *red* value on the right-side bar of the graph). Repeatedly waiting for evictions causes processor stalls when d_{cd} is low, making the performance highly memory-bound.

As d_{cd} increases (right of each graph), consecutive iterations exhibit cache-line hits. Eviction frequency and burst clustering both decrease. Between successive evictions, hit iterations execute sufficient instructions to overlap with prior read latency. Once the eviction latency falls below the loop iteration time, the workload becomes compute-bound.

Group 1 benefits significantly from memory traffic reduction, while group 2 is constrained by the latency of the inner-loop in software implementation. Previous sections demonstrated this while taking GC as the performance reference. While GC and RedC remain bottlenecked, SpDCache substantially reduces costly evictions and avoids bursting by accumulating with high hit-rates in the *DRC* or evicting via low-cost, element-wise transactions within the *SRT*. Consequently, SpDCache delivers significant latency reduction for the matrices in group 1 which are memory-bound. For the compute-bound matrices in group 2, traffic reduction yields modest latency improvements. Here, the BaCSC format reduces the number of instructions in the inner-loop by avoiding the region calculation.

For GC, loop unrolling allows the *miss handler* to queue up to eight concurrent eviction reads through its multiple entries. However, this proves insufficient for group 1 matrices, as the unrolled inner-loop contains too many instructions to hide the memory latency. Additionally, the *miss handler*'s parallelism cannot be fully exploited if the *accumulation table* lacks sufficient entries to process the corresponding *RED* evictions. Increasing the *accumulation table* to eight entries similarly yields negligible speedup (<1%, not shown in any figure) for RedC and SpDCache, since the bottleneck remains the critical path length of the inner-loop instructions.

CONCLUSION

Overall, **group 2 always outperforms group 1** when compared based on raw memory traffic and latency per nnz. Group 1 matrices (low d_{cd}) are **memory-bound** due to sparse cache-lines causing frequent eviction bursts, while group 2 matrices (high d_{cd}) are **compute-bound** with dense cache-lines hiding eviction latency. Consequently, SpDCache achieves speedup in group 1 through memory traffic reduction, but only matches baseline performance in group 2 where the inner-loop latency becomes the bottleneck.

5.3.5.4 Area Overhead

Both HPDcache and SpDCache systems were synthesized in GF22FDX technology (22 nm) using Synopsis DC™. The original area was 0.99 mm^2 and the SpDCache version was 1.3 mm^2, an increase of 31%. These results are preliminary since the SpDCache RTL implementation has not undergone synthesis optimization. This shows that SpDCache could reasonably be integrated into a processor, providing a large performance improvement (3.9× in memory traffic), with an area overhead that is much lower than predicted by Pollack's law [14] on a single core ($< 3.9^2 = 1500\%$). For context, similar state-of-the-art accelerators scaled to 22 nm technology for a single cache exhibit the following area overheads: PHI requires 9%, SortCache requires 172%, ASA requires <1% and C^{3}ache requires 9%. In comparison, *standalone* accelerators have area overheads about one order of magnitude higher at least.

5.3.6 Comparison with the Analytical Model

To validate consistency between our models and implementations, we compare the normalized output memory traffic produced by three sources: the analytical model of Chapter 4, the Python model of Chapter 3, and the RTL implementation presented in this chapter. For a fair comparison, the caches for RedC and SpDCache are configured to 32 KiB total (see Table 5.1): 50/50/0% for RedC and 25/50/25% for SpDCache. The analytical model accepts global parameters and density distributions rather than actual matrices, so we synthesize banded matrices for the Python and RTL simulations that match those parameters. All synthetic matrices use $M = 10^4$, half-*band-width* $w_{hB} = 1021$, in-band density $d_{IB} = 10^{-2}$, and out-of-band density d_{OB} varying from 10^{-7} to 10^{-2}. Note that at $d_{OB} \simeq 8 \times 10^{-3}$, approximately one non-zero element exists per column in the out-of-band region, whereas at $d_{OB} \simeq 8 \times 10^{-7}$, approximately one non-zero element exists in the entire out-of-band region of the full matrix, resulting in a nearly 'perfectly' banded matrix.

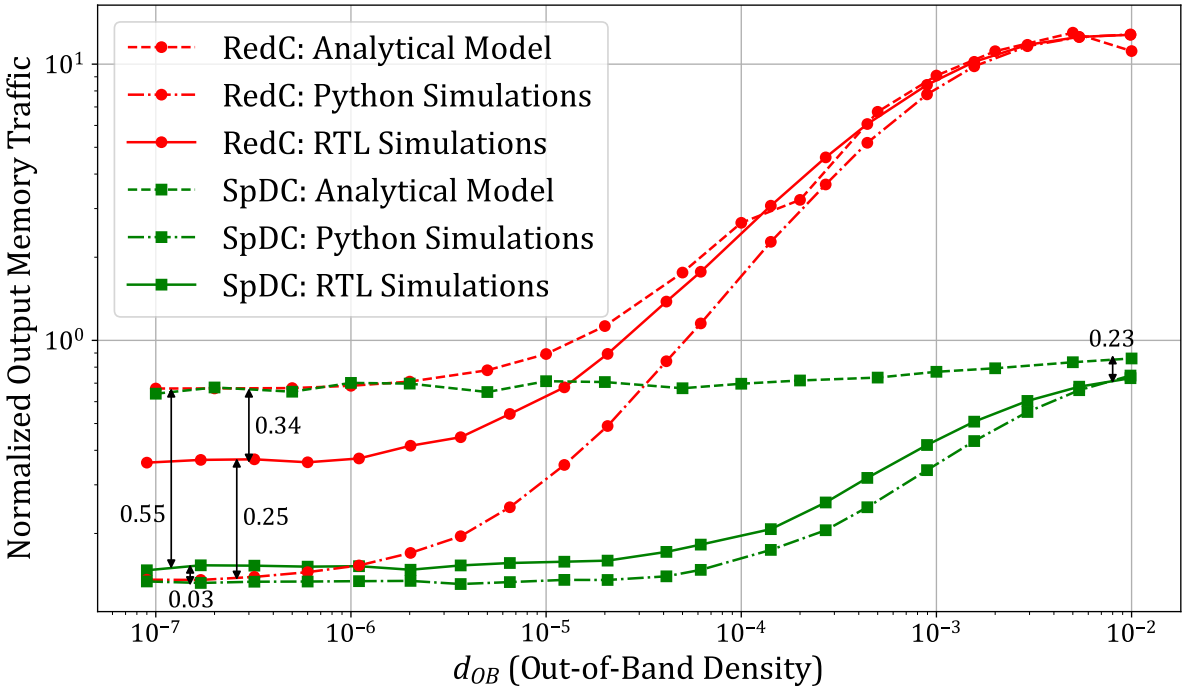


Figure 5.11 – Normalized Output Memory Traffic Depending on the Out-of-Band Density for the RedC and SpDCache Configurations, Through Different Simulation Models

Figure 5.11 shows the results: SpDCache (*green*) outperforms RedC (*red*) in normalized output memory traffic ($normMT$). Two observations are noteworthy. First, the three approaches agree most closely when d_{OB} is high. Indeed, at $d_{OB} = 10^{-2}$ there is no measurable difference for RedC, while SpDCache exhibits a 0.23 difference in $normMT$. At the smallest d_{OB} the discrepancies reach 0.58 (0.55 + 0.03) element transferred in memory per *reduction*. Overall trends hold across models and deviations remain small.

Second, the analytical model consistently overestimates traffic (except on two points), the Python model consistently underestimates it, and the RTL results fall between the two. Indeed, the analytical model operates on matrix distributions rather than actual generated matrices, using a uniform distribution in this case. At very low densities ($\leqslant 10^{-2}$), the analytical model exhibits consistently poor cache hit-rates. In contrast, the Python and RTL models

generate matrices with the same uniform distribution, but their randomly fixed non-zero elements can still produce some cache hits. Furthermore, matrices generated with uniform distributions exhibit low d_{cd} values (between 13% and 14%, see Appendix B), placing them in a regime where evictions occur in bursts. Under these burst-heavy conditions, the RTL implementation cannot effectively leverage its out-of-order execution capabilities (see next section).

CONCLUSION

Across synthetic banded matrices, all three simulation models (analytical, Python, RTL) demonstrate consistent memory traffic results, with only minor differences as matrices approach the 'perfectly' banded regime, without affecting the conclusions.

5.3.7 Comparison with the Python Model

To further validate our implementation over real, not necessarily banded, matrices, we compare the RTL results against the Python simulations from Chapter 3. Table 5.3 presents the Python results for the RTL cache configurations shown in Table 5.1. We include the RedC 50/50/0% configuration for comparison, though it is suboptimal according to the Python analysis. Table 5.4 reports RTL results with geometric mean over the same 48 matrices used in Python simulations, comparing output and total memory traffic.

Table 5.3 – Results of Main Cache Configurations from Python Simulation

GMeans (48 mat.)		**GC**	**RedC**	**SpDC**
sets	*GRC*	*128*	*64*	*32*
	DRC	*Ø*	*64*	*64*
	SRT	*Ø*	*Ø*	*32*
Output Memory Traffic (% over GC)		100 ■1.0	103 ■1.0	24.5 ▼4.1
Total Memory Traffic (% over GC)		100 ■1.0	98.5 ■1.0	56.1 ▼1.8

Table 5.4 – RTL Results Comparison with Python Simulations

GMeans (48 mat.)		**GC**	**RedC**	**SpDC**
sets	*GRC*	*128*	*64*	*32*
	DRC	*Ø*	*64*	*64*
	SRT	*Ø*	*Ø*	*32*
Output Memory Traffic (% over GC)		100 ■1.0	75.5 ▼1.3	20.0 ▼5.0
→ **RTL/Python** (%)		116 ▲0.9	85.0 ▼1.2	94.6 ▼1.1
Total Memory Traffic (% over GC)		100 ■1.0	99.3 ■1.0	54.7 ▼1.8
→ **RTL/Python** (%)		75.5 ▼1.3	76.1 ▼1.3	73.6 ▼1.4

The total memory traffic for RedC and SpDCache relative to GC is nearly equivalent across both simulations. However, output memory traffic shows notable differences: RTL achieves better traffic reduction than Python, 1.3× for RedC (vs. 1.0× in Python) and 5.0× for SpDCache (vs. 4.1× in Python). Comparing raw traffic values between RTL and Python simulations across all configurations, we observe approximately 25% lower total memory traffic in RTL. Output memory traffic varies by ±15%, with RTL showing lower results for both RedC and SpDCache. Indeed, the RTL model manages *reductions* out-of-order using the *replay table*, whereas the Python model simulates them sequentially without overlap. For instance, in the Python model, a *RED* triggering an eviction completes entirely before processing the next *RED*. Conversely, in the RTL model, a *RED* triggering an eviction can be queued in the *replay table* if the *miss handler* or *accumulation table* reach capacity, allowing the cache to accept subsequent *RED* operations. If a later *RED* hits the cache-line that would have been

evicted by an earlier operation, an unnecessary eviction is avoided, an event that would have been counted in the Python model. This out-of-order execution capability, inherited from HPDcache, thus contributes to reduced memory traffic in the RTL implementation.

CONCLUSION

Over a set of real application matrices, the RTL implementation corroborates the Python simulations, exhibiting better memory traffic performance due to its out-of-order execution.

5.4 Conclusion

IN this chapter, we presented the microarchitectural implementation of SpDCache and evaluated its performance through RTL simulations. By translating the theoretical insights from previous chapters into an actual hardware design, we demonstrated the effectiveness of SpDCache in addressing the challenges of irregular memory access patterns in sparse matrix computations.

Our RTL implementation of SpDCache, with its three dedicated regions, *GRC*, *DRC*, and *SRT*, successfully optimizes memory transactions for both dense and sparse regions of matrices. The results from RTL simulations highlight significant reductions in memory traffic and improved cache utilization, validating the architectural choices and theoretical models developed earlier. Specifically, SpDCache achieves a substantial decrease in memory traffic compared to generic and *reduction caches*, showcasing its ability to leverage the coarse-grain structure of sparse matrices.

Our findings also show the importance of fine-grain matrix structure. The RTL simulations revealed two distinct groups of matrices, split based on the d_{cd} metric, which measures the density of non-zero elements within cache-line-sized portions of the matrices. This distinction highlights how the fine-grain distribution of non-zero elements within matrices impacts performance. By effectively handling both coarse-grain and fine-grain structure, SpDCache demonstrates its adaptability across a diverse set of sparse matrices.

As a contribution, this chapter detailed the RTL implementation of SpDCache, including the hardware components and pipeline stages that enable efficient data processing and *reduction* operations. Through RTL simulations, we assessed SpDCache's performance, demonstrating its superiority in reducing memory traffic and improving cache utilization. The RTL results validated the theoretical models and insights, confirming the benefits of region-based caching and *reduction* support. Additionally, we identified and analyzed the impact of fine-grain matrix structure on performance, revealing two distinct groups of matrices based on the d_{cd} metric.

However, several limitations must be acknowledged. The current RTL implementation is a first iteration and could be further optimized, as some features are still missing or not fully refined. The use of element-wise, atomic transactions for offloading sparse data carries the risk of overwhelming the rest of the memory hierarchy if it is not designed to handle such transactions efficiently. Finally, the evaluation was conducted on a single-core, single-cache level system, which helps us easily understand the cache behavior but limits raw performance compared to more complex, multi-level, or multi-core architectures, which are standard in high-performance computing systems. However, we believe that with a more powerful processor (such as multiple-issue), the pressure on the memory system would be higher and the threshold between the two groups of matrices would be shifted to the right, with more

matrices benefiting from SpDCache.

Overall, this chapter highlights the critical role of both coarse-grain and fine-grain matrix structure in optimizing sparse matrix computations. The contributions of SpDCache not only advance the state-of-the-art in cache architectures but also provide a robust foundation for future enhancements in scalability and bandwidth efficiency, as discussed in the following chapter. The ability to adapt to varying matrix structure ensures that SpDCache remains effective across a wide range of applications and workloads.

TAKEAWAYS

Contributions:

- **Microarchitectural Design:** We detailed the RTL implementation of SpDCache, including the hardware components and pipeline stages that enable efficient data processing and *reduction* operations.
- **Performance Evaluation:** Through RTL simulations, we assessed SpDCache's performance, demonstrating its ability to reduce memory traffic and improve cache utilization.
- **Validation of Theoretical Models:** The RTL results validated the theoretical models and insights, confirming the benefits of region-based caching and *reduction* support.
- **Fine-Grain Structure Analysis:** We identified and analyzed the impact of fine-grain matrix structure on performance.

Limitations:

- **RTL Implementation Optimization:** The current RTL implementation is a first iteration and could be further optimized, as some performance features are missing.
- **Offloading Risk:** The use of element-wise, atomic transactions for offloading sparse data carries the risk of overwhelming the downstream memory hierarchy.
- **Single-Core, Single-Cache Level System:** The evaluation was conducted on a single-core, single-cache level system, which is ideal for detailed analysis but limits performance compared to more recent architectures with multi-level caches and multi-core processors with multi-issue cores.

CHAPTER

6 SpDCache Optimizations

Contents

6.1 Introduction

The increasing complexity and scale of sparse matrix computations demand not only efficient single-core architectures but also scalable solutions that can leverage multi-core and multi-level memory hierarchies. While the previous chapters have focused on the design, theoretical analysis, and microarchitectural implementation of SpDCache, this chapter explores opportunities to further enhance its performance and scalability.

In Chapter 5, we demonstrated the effectiveness of SpDCache in reducing memory traffic and improving cache utilization for Sparse Matrix-Vector multiplication (SpMV). However, as applications grow in size and complexity, the need for scalable architectures that can efficiently distribute workloads across multiple cores and memory levels becomes critical. This chapter addresses these challenges by proposing optimizations and extensions to SpDCache that target both single-core and multi-core systems.

We begin by discussing RTL-level optimizations for SpDCache, focusing on fine-tuning the hardware implementation to maximize performance and resource utilization. Next, we explore the application of SpDCache to Sparse Matrix-Sparse Matrix multiplication (SpMSpM),

extending its benefits beyond SpMV and demonstrating it can be adapted to a more complex kernel.

Finally, we investigate the integration of SpDCache into multi-level, multi-core architectures. This includes designing a scalable version of SpDCache that can operate efficiently across multiple cores, as well as optimizing memory bandwidth usage to ensure high throughput in distributed environments. By addressing these aspects, we aim to provide a comprehensive roadmap for scaling SpDCache for high-performance sparse computation.

Through these optimizations and extensions, this chapter shows that the concepts of SpDCache can be generalized and scaled to enhance performance of sparse matrix processing in real-world high-performance computing systems.

6.2 RTL Optimizations for SpDCache

THE RTL implementation of SpDCache presented in Chapter 5 was essential to validate our performance models and to analyze performance based on matrix structure. However, as already noted several times in the previous chapter, the RTL implementation is missing some features and could be optimized for better practicality and performance. In this section, we detail the major features and optimizations that are still to be implemented.

6.2.1 Floating-Point Support Implementation

The key focus of this thesis has been the structure on the non-zero elements in the matrix, however, the data-type of the matrices plays an important role in their storage footprint and the arithmetic complexity. Indeed, in scientific computing, matrices are primarily stored in double precision (64-bits) floating-point. In other applications such as DNNs or graph analytics, lower precision formats (*ex.*, binary, 16-bits floating-point, 8-bits integer) are frequently used. To simplify the design complexity of SpDCache, we chose to work with 64-bits integer format. This results in the same data-traffic as double floating-point, but simplifies the design, as the arithmetic operations (accumulations) can be done in a single cycle. To deploy SpDCache for high-performance computing applications, it must be extended to support the double data-type.

To support the accumulation of double format data between *st1* and *st2* in the pipeline (Section 5.2.3), we could either deploy a single-cycle high-performance FPU with the area trade-offs, or extend the pipeline to accommodate two-cycles, for example, floating-point accumulation. The latter approach better suits our *accumulation table*, which does not stall the pipeline and can parallelize element-wise accumulations across stored 64-byte lines. Additionally, full floating-point support requires extending the AXI port to handle element-wise *Read-Modify-Write* (*RMW*) operations beyond its native integer capabilities [2].

Floating-point support is critical for SpDCache deployment. Our expectation is that the associated RTL changes, while time-consuming, should not present fundamental technical obstacles.

CONCLUSION

Floating-point support is achievable through pipeline extension and requires extending the AXI port for element-wise operations, presenting no fundamental implementation issues.

6.2.2 Region-Sizes Configuration

One strength of SpDCache's RTL implementation is its highly configurable design, inherited from the HPDcache implementation used as a foundation. This allowed us to develop a single RTL codebase that could be compiled and simulated for multiple designs, GC, RedC, and SpDC, each with different region size configurations. However, these configurations are static, requiring RTL recompilation for each design variant.

Currently, SpDC's *reduction* mode can be dynamically enabled or disabled via an instruction issued before and after the SpMV kernel. This switches between a generic cache and a hard-coded RTL configuration. To enhance software flexibility, SpDC should support dynamic configuration settings through the initial configuration instruction.

A fundamental limitation is that region sizes are restricted to powers of two, which severely constrains configuration options and led to the 25/50/25% configuration used in the previous chapter. This restriction stems from address decoding that uses bit shifts to extract the *set* index, an approach inherited from HPDcache. Supporting arbitrary region sizes would require integer division and modulo operations, which could introduce longer, critical paths. However, our Python simulations in Chapter 3 demonstrate that precise region sizing is unnecessary for achieving good performance, making this optimization less important.

A practical compromise would be to provide one or two predefined configurations for RedC and SpDC, allowing dynamic selection by software while maintaining a simple user interface.

CONCLUSION

Dynamic region-size configuration through software, while desirable, has a high hardware complexity without significant performance gains. A practical approach is to provide **predefined configurations for dynamic selection**, balancing flexibility with implementation simplicity.

6.2.3 Hardware Calculation of Region and Band-Width

SpDCache evaluation shows that depending on matrix characteristics, the `OP` SpMV kernel can be compute-bound, making region calculation a critical software bottleneck. We addressed this by using a custom sparse format that converts latency costs into memory traffic costs. To eliminate any latency or memory traffic overhead, we could compute the region argument of the *RED* instruction directly in hardware. This approach sacrifices software flexibility but enables CSC format compatibility and simplifies the `OP` SpMV kernel to the version shown in Algorithm 5.2. The required SpDCache modifications are:

- Configuration: The initial instruction enabling SpDCache's *reduction* mode must accept a configuration argument specifying the word size in bytes (s_w), the output vector address (c), and the *DRC* size in bytes (s_{DRC}). The hardware could fetch the last parameter independently. When this instruction is issued, prior to each SpMV, the half-*band-width* is computed and retained until reconfigured, where s_l denotes cache-line size in bytes:

$$w_{hB} = \frac{s_{DRC} - s_l}{2 \times s_w} + 1$$

  The half-*band-width* could also be computed in software, and given as argument to the configuration.
- Column indexing: Each outer-loop iteration requires issuing a new instruction contain-

ing the current column index $j \in [\![0; M-1]\!]$, denoted $CCOL(j)$. This value persists until the next $CCOL$ instruction arrives, consuming one cycle per outer-loop iteration. Note that the *REG* instruction becomes unnecessary.

- Region determination: For each $RED(addr, val)$ instruction, SpDCache computes the current row $i \in [\![0; M-1]\!]$ from the address: $i = \frac{addr - c}{s_w}$, then determines the region as *IBR* if $|i - j| \leqslant w_{hB}$, otherwise *OBR*.

These modifications impose minimal hardware overhead, with per-*reduction* computations executed at *st0* in the SpDCache pipeline. This feature becomes essential when extending SpDCache to multi-level, multi-core architectures (Section 6.4).

CONCLUSION

Hardware-based region computation eliminates unnecessary instructions in the software's inner-loop, enabling CSC format compatibility and simplifying the SpMV kernel with minimal hardware overhead.

6.2.4 Complete Inter-Region Cache Coherency

When executing an `OP` SpMV kernel with SpDCache, we assume that the processor issues *RED* instructions strictly on addresses of the output vector c, and that loads and stores are issued only on addresses not in the output vector c. SpDCache's current implementation provides no hardware safeguards if this assumption is violated. While we provide local coherency between the two *reduction* regions (*DRC* and *SRT*) to prevent c address duplication, we lack coherency with the *GRC*. A simple detection system could verify that *reductions* are issued exclusively on addresses within the output vector's range, while other instructions operate outside this range. PHI [86] demonstrates that full coherency is achievable, as required in their design, proving that *reductions* and generic instructions can safely coexist within the same cache hierarchy. In SpDCache's case, implementing global inter-region coherency would require the following modifications:

- Configuration: When enabling *reduction* mode, the configuration parameter must specify the address range for *RED* instructions (*e.g.*, , output vector c and its extent).
- Region Assignment: To prevent address duplication between *reduction* regions and the *GRC*, loads and stores on c addresses target *reduction* regions, while *REDs* on non-c addresses target the *GRC*. The following cases apply only when *reduction* mode is enabled.
- Load on c address:
  - Misses in *DRC* and *SRT*: Issue memory read and return data to processor, bypassing cache.
  - Hit in *DRC* or *SRT*: Issue memory read, accumulate with cached data, and return result to processor.

  We do not update *reduction* regions on load to avoid complexity. Updating would require writing zeros to memory to prevent double accumulation upon eviction or flush.
- Store on c address:
  - Misses in *DRC* and *SRT*: Issue memory write with store value, bypassing cache.
  - Hit in *DRC* or *SRT*: Overwrite cached data with store value and issue memory write of a zero to maintain consistency.

  Cache policy (write-through and write-back) can influence this design choice. Crucially, data must not exist in both memory and cache simultaneously.
- *RED* on non-c address:

- Miss in *GRC*: Issue memory read, accumulate with *RED* value, insert into *GRC* (with eviction if needed), and mark dirty (write-back) or write to memory (write-through).
- Hit in *GRC*: Accumulate cached data with *RED* value, update *GRC* accordingly, and mark dirty (write-back) or write to memory (write-through).

It is to be expected that incorrect region usage increases memory traffic and latency, degrading performance. Implementing a coherency or detection scheme would ensure deterministic behavior, a clear benefit for SpDCache deployment.

CONCLUSION

Complete inter-region cache coherency prevents address duplication across *reduction* and generic regions, ensuring deterministic behavior at minimal hardware cost through address-range configuration and region-specific load/store/reduction handling.

6.3 SpDCache on Matrix-Matrix Multiplication Kernels

To demonstrate SpDCache's applicability beyond the SpMV kernel, we present its use for Sparse Matrix-Sparse Matrix multiplication (SpMSpM, $A \times B = C$), a widely used kernel essential for graph analytics and AI applications.

As discussed in Chapter 1, *Gustavson*'s algorithm (`Gust`) offers the best performance potential for SpMSpM. However, it requires handling irregular accesses to both input matrix B and output matrix C. Since SpDCache targets *reductions* on the output, additional hardware for fetching B is necessary to achieve peak performance, as demonstrated by the Gamma accelerator [133].

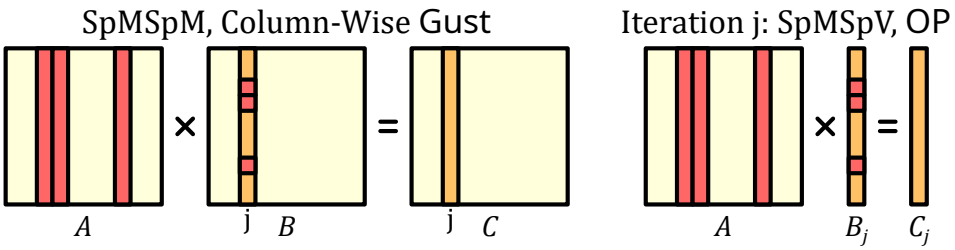


Figure 6.1 – *Gustavson* SpMSpM Kernel with an *Outer-Product* SpMSpV Iteration

A SpMSpM kernel can be decomposed into a series of SpMSpV (Sparse Matrix-Sparse Vector multiplication) operations. As shown in Figure 6.1, each column C_j is computed from column B_j and matrix A, yielding M independent matrix-vector multiplications: $A \times B_j = C_j$. SpDCache readily extends to this scenario.

Since matrix B is sparse, so are its columns. When computing the *Outer-Product* SpMSpV, only columns of A corresponding to non-zero entries in B_j contribute to the result. While SpMV (dense vector) processes the entire matrix structure of A, SpMSpV accesses only a subset of columns, which are not necessarily consecutive. As the band in A is disrupted, SpDCache's hit rate in the *DRC* decreases with sparse vectors, but retains its benefits. When B is banded, this limitation is alleviated as columns of A are consecutive in a subset. This issue vanishes entirely for Sparse Matrix-Dense Matrix multiplication (SpMM).

Explicit flushes are unnecessary between successive SpMSpV operations except the last one, since each iteration updates a distinct address range in C. The next iteration implicitly flushes previous values through replacements and evictions.

This approach suits multicore architectures, where multiple SpMSpV iterations can execute in parallel. The primary hardware challenge is managing shared access to matrix A across cores.

CONCLUSION

SpDCache effectively extends to SpMSpM kernels through decomposition into **SpMSpV operations**, maintaining performance benefits while introducing manageable trade-offs in cache hit rates for sparse vectors. This demonstrates SpDCache's **versatility** across different sparse linear algebra kernels.

6.4 SpDCache on Multi-Level, Multi-Core System

THROUGHOUT this manuscript, we studied SpDCache in isolation to clearly understand its performance characteristics across different matrices. While we validated SpDCache's individual performance and provided thorough analysis, we must now address how to scale SpDCache to a multi-level, multi-core system. This scaling is non-trivial and requires careful consideration. In this section, we describe a complete memory hierarchy for SpDCache, though we do not evaluate it as the full system remains unimplemented.

As a concrete example, we consider a simple three-level, two-core system where each core has two private consecutive caches (L1 and L2) and shares a last-level cache (LLC). As shown in Figure 6.2, each core computes half of the matrix in parallel, divided vertically. This approach favors the CSC format, which stores matrices column-wise, minimizing required software changes. Consequently, each core reads a different portion of matrix A and vector b, while both cores perform updates to the entire output vector c.

Two key requirements emerge: first, all core updates must eventually be merged into main memory; second, the *reduction* process must remain "fire-and-forget" across cache levels to prevent stalling the entire memory hierarchy through inter-level dependencies.

Our proposed architecture replaces each cache level with adapted SpDCache versions, where the *DRC band-width* scales with cache size. As shown in the figure, the memory hierarchy separates into two parts: *GRCs* function as a conventional memory hierarchy with coherency management, while *DRCs* and *SRTs* manage *reductions* independently. *DRC band-width* grows from L1 to LLC, with the LLC defining the matrix band, portions of which cascade to L1 and L2. The out-of-band region is fixed by the LLC and remains constant across levels.

When L1 receives a RED instruction from its core, the SpDCache procedure routes it to the *DRC* or *SRT* (checking the *DRC* first, then the region argument, with *SRT*-to-*DRC* transfers preventing duplication, as seen Chapter 3). If the *reduction* targets in-band data not in L1's sub-band, it forwards to L2. The L1 accumulator processes hits but not *DRC* evictions. Upon *DRC* eviction in L1, the cache-line transfers to L2's *DRC* with a line-wise RED instruction. Upon *SRT* eviction, the element transfers to L2 with an element-wise RED instruction.

L2 behaves similarly to L1 but receives both line-wise and element-wise RED instructions and maintains a wider band. L1 *DRC* evictions thus populate L2's *DRC* alongside forwarded in-band elements, with further forwarding to the LLC if elements exceed L2's band.

The LLC merges core contributions by receiving element-wise and line-wise RED instructions, queuing them in arrival order. It then operates as an isolated SpDCache connected to main memory, managing *DRC* evictions through line-wise reads and writes, and *SRT* evictions through element-wise *RMW* transactions.

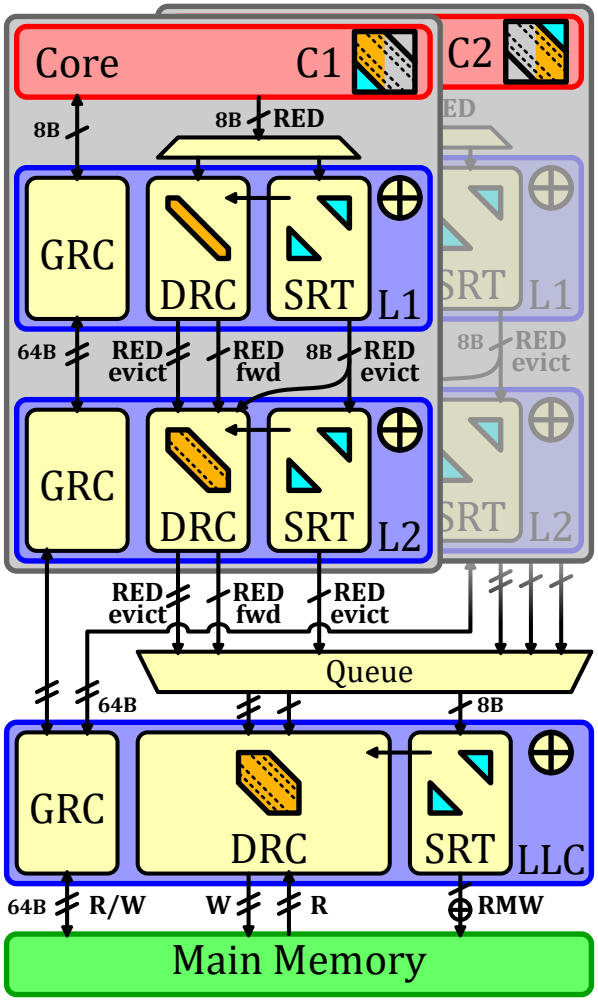


Figure 6.2 – An Example of SpDCache Usage on a Three-Level, Two-Core System

RMW transactions can be processed at the LLC level. Since *RMWs* between L1 and the LLC are replaced by element-wise *reductions* with equivalent memory traffic, implementing *RMWs* between the LLC and main memory has minimal impact and can be realized as standard read-accumulate-write operations.

This architecture demonstrates how SpDCache must adapt to its position in the memory hierarchy. With this design, the *reduction* path requires no coherency management, provided software only issues *reduction* on the output vector c. Vertical matrix partitioning enables each core to generate *partial outputs* (*POs*), which the LLC merges in real-time.

This system requires detecting four matrix regions rather than two, from L1's smallest band to the LLC's largest band, plus out-of-band. Software-based region calculation would significantly increase per-iteration latency, and extending the BaCSC format risks substantial memory overhead. Therefore, hardware-based region calculation (Section 6.2) is necessary for scalability.

Note that LLC *DRC band-width* calculation differs slightly from other levels: two matrix band portions are simultaneously computed by both cores. The LLC must allocate sufficient memory space for both portions to prevent one core's *reductions* from evicting the other's. Thus, LLC *DRC band-width* should be calculated as: *DRC* size divided by the number of cores.

This system carries congestion risk when merging LLC requests, as many element-wise and line-wise *reductions* compete for the accumulator. Further investigation with proper evaluation is warranted.

Alternative configurations merit exploration: for example, SpDCache at L1 with a single band and *reduction*-only caches at L2 and LLC would be simpler but sacrifices the benefit of region separation to minimize unnecessary evictions. Similarly, adjusting band sizing policies involves trade-offs: sizing L1 bands identically across L2 and LLC simplifies region calculation and eliminates RED forwarding but underutilizes L2 and LLC *DRC* space; conversely, sizing by LLC band would cause unwanted L1 and L2 evictions.

Our example vertically partitions the matrix into two parts for simplicity and CSC alignment. However, alternatives exist: for instance, odd and even columns could be assigned to separate cores, remaining CSC-compliant while allowing both cores to do *reductions* on the same addresses at the same time and thus using the entire LLC *DRC* size for the *band-width*. The drawback is reduced L1 and L2 hit rates, concentrating most hits (and accumulations) in the LLC, potentially causing congestion. Additionally, dynamic partitioning strategies could be explored through a dedicated scheduler, distributing columns to available cores for example, enabling comprehensive load-balancing analysis for multi-core SpDCache implementations. Determining the optimal approach requires empirical evaluation. Many other matrix partitioning and system configurations are conceivable and warrant investigation, but remain beyond this manuscript's scope.

Note that computing multiple SpMVs simultaneously is feasible. For instance, each core could execute a distinct SpMV operation on the same matrix A but with different input vectors b. The LLC *DRC band-width* must then be divided by the number of SpMVs to prevent overlap in the allocated space for each output vector c.

CONCLUSION

The proposed multi-level, multi-core architecture for SpDCache **effectively addresses the challenges of scaling across cache levels and cores** while optimizing memory usage. Future work could focus on evaluations of various configurations to validate performance.

6.5 Conclusion

THIS chapter explored optimizations and extensions to SpDCache, focusing on enhancing its performance and scalability for sparse matrix computations. We refined the RTL-level implementation to maximize efficiency and easy-of-use, ensuring SpDCache remains robust and adaptable to modern computational demands.

We extended SpDCache to matrix-matrix multiplication kernels, focusing on SpMSpM, demonstrating its versatility beyond SpMV. This extension highlights its adaptability to complex operations, reinforcing its potential as an important component for diverse sparse matrix computations.

A key focus was integrating SpDCache into multi-level, multi-core architectures, addressing the need for scalable, distributed processing. By demonstrating potential extension to parallel systems, SpDCache becomes a viable solution for large-scale sparse kernels.

While this chapter presents conceptual insights and proposed optimizations, no evaluation has been conducted yet. These proposals position SpDCache as a versatile and scalable solution for modern sparse matrix processing systems.

TAKEAWAYS

SpDCache Optimizations:

- **Floating-Point Support:** Handle floating-point arithmetic within SpDCache;
- **Region-Size Configuration:** Methods to dynamically adjust region sizes in SpDCache;
- **Region Calculation in Hardware:** Hardware-based calculations for *DRC band-width*;
- **Complete Inter-Region Cache Coherency:** Comprehensive cache coherency protocol for SpDCache, ensuring data integrity and preventing software issues.

Using SpDCache for Matrix-Matrix Kernels:

- SpDCache effectively **extends to matrix-matrix kernels** through decomposition into matrix-vector operations;
- This demonstrates SpDCache's versatility across different sparse linear algebra kernels.

Multi-Level, Multi-Core SpDCache Architecture:

- SpDCache can be **scaled to multi-level, multi-core systems** by slightly adapting its architecture to each cache level and core;
- Vertical matrix partitioning enables efficient parallel processing across cores;
- The proposed architecture effectively addresses the challenges of scaling across cache levels and cores while optimizing memory usage.

CONCLUSION

IN this thesis, we addressed the central question of **how hardware and architectural mechanisms can be designed to efficiently support computation with sparse data, while ensuring scalability, adaptability, and high performance across diverse workloads**. Our approach focused on the challenges posed by irregular memory access patterns in sparse matrix computations, particularly in the context of the ***Outer-Product* (OP) Sparse Matrix-Vector multiplication (SpMV)** kernel. We identified that the OP algorithm, while promising for parallelism and optimization with MV kernels, generates "random" *reductions* (irregular updates on the output vector) that stress traditional memory systems. To mitigate these challenges, we proposed **SpDCache**, a novel **region-based *reduction cache*** architecture designed to optimize memory transactions for sparse data by leveraging the structural properties of banded matrices.

Through this work, we developed a **lightweight, flexible solution** that can be integrated into any kernel requiring efficient "random" *reductions*, recognizing that caches are a fundamental component of memory hierarchies in CPUs, GPUs, and accelerators. Our design prioritizes **scalability** and the required storage in SpDCache (32 KiB) is much smaller than most accelerators (Chapter 2). Additionally, we addressed the critical influence of **matrix structure** on performance, conducting a detailed analysis of cache behavior specifically for banded matrices. By adapting our architecture to better handle dense and sparse regions, we provided insights into how matrix structure can inform the design of efficient architectures, with **SpDCache** serving as a practical example. This work also demonstrated how **simple yet targeted modifications** to cache design can yield significant performance improvements for *reduction*-based kernels.

While our approach has limitations, such as the absence of a full evaluation on multi-level, multi-core systems and the early-stage refinement of SpDCache, we intentionally focused on a **simpler and explainable design** rather than raw performance. This emphasis on intrinsic understanding, rather than merely achieving impressive results, offers a more meaningful response to the central question and gives greater understanding of the execution patterns, some that is often overlooked in existing accelerator research.

KEY RESULTS AND INSIGHTS

Algorithmic Choice: *Outer-Product* and *Gustavson*'s algorithms are the most effective for hardware acceleration, with ***Gustavson* offering the best trade-off** for sparse Matrix-Matrix kernels on *standalone* accelerators. Recent advances in **hybrid algorithmic approaches**

that adaptively mix sparse matrix algorithms are emerging as **high-performance solutions**, suggesting a promising direction for future accelerator designs.

Cache Optimization: SpDCache's **region-based partitioning** and **reduction support** significantly reduce memory traffic, particularly for matrices with **structured sparsity** (*e.g.*, banded matrices), while maintaining **low implementation overhead**.

Fine-Grain Structure Matters: The d_{cd} **metric** revealed two distinct matrix groups based on their local cache-line densities, highlighting the need for **adaptive cache configurations** tailored to matrix structure.

Scalability and Usability: SpDCache's **multi-core extensions** and **optimizations for SpM-SpM** demonstrate its potential for **scalable, high-performance sparse matrix processing**, while maintaining **versatility and ease of use**.

FINAL THOUGHTS

This thesis contributes a **reduction-aware cache architecture** that bridges the gap between theoretical insights and practical hardware implementations for sparse matrix computing. By addressing the challenges of **irregular memory accesses** and **efficient data reuse**, SpDCache provides a **scalable, adaptable solution** for modern computational demands. As sparse data continues to grow in importance across scientific, machine learning, and graph analytics applications, architectures like SpDCache will play a critical role in enabling **high-performance, energy-efficient processing** of these challenging workloads.

SYNTHÈSE EN FRANÇAIS

Sommaire

Ce chapitre présente une synthèse en français des contributions du manuscrit, où chaque section résume les idées clés, les contributions majeures et les résultats marquants des Chapitres 1 à 6. L'objectif n'est pas de remplacer le contenu détaillé des chapitres précédents, mais d'en extraire une vision synthétique des concepts fondamentaux, des avancées proposées et des conclusions significatives.

1 Synthèse du Chapitre 1 : Contexte sur le calcul des matrices creuses

Le Chapitre 1 pose les bases théoriques et pratiques pour comprendre les défis liés au traitement des matrices creuses, omniprésentes dans des domaines tels que le calcul scientifique, l'analyse de graphes et l'apprentissage automatique. Ces matrices, caractérisées par une majorité d'éléments nuls, nécessitent des formats de stockage compressés (comme COO, CSR ou CSC, voir la Figure 1.3) pour réduire leur empreinte mémoire et optimiser leur manipulation. Cependant, leur structure irrégulière engendre des accès mémoire imprévisibles, limitant l'efficacité des caches traditionnels et compliquant la parallélisation des calculs.

Algorithmes de multiplication de matrices creuses

Trois algorithmes principaux sont analysés pour les opérations de multiplication matrice-vecteur MV (SpMV, SpMSpV ; $A \times b = c$) et matrice-matrice MM (SpMM, SpMSpM, SpGEMM ; $A \times B = C$) :

- *Inner-Product* (**IP**) (voir Algorithmes 1.1 et 1.2) : Intuitif mais souffrant de recherches d'intersection coûteuses entre les éléments non nuls des matrices/vecteurs d'entrée, lorsque les données sont stockées dans des formats compressés.
- *Outer-Product* (**OP**) (voir Algorithmes 1.3 et 1.4) : Génère des réductions aléatoires sur le vecteur ou la matrice de sortie, stressant le système mémoire en raison d'accès irréguliers et de mises à jour de sommes partielles fréquentes.
- *Gustavson* (**Gust**) (voir Algorithme 1.5) : Un compromis entre **IP** et **OP**, uniquement pour les opérations MM, optimisant la réutilisation des données d'entrée (lignes de B) et la génération séquentielle des résultats (lignes de C), ce qui en fait le candidat le plus adapté pour une implémentation matérielle efficace.

Une analyse comparative (Tableau 1.2) met en évidence les forces et faiblesses de chaque algorithme en termes d'accès mémoire, de parallélisme et de réutilisation des données. Par exemple, pour des opérations MM, **Gust** évite les accès aléatoires sur une matrice entière et réduit significativement le trafic mémoire avec une mémoire locale limitée, contrairement à **IP** ou **OP**.

Défis matériels et solutions

Les accès irréguliers (*gather* pour **IP**, *scatter* pour **OP**) constituent le principal goulot d'étranglement, nécessitant des solutions matérielles dédiées :

- Problème de *gather* : Les recherches d'intersection entre indices non nuls (ex. : entre les lignes de A et les colonnes de B (ou b) en **IP**) sont coûteuses en logiciel. Des accélérateurs matériels (comme ExTensor [48], Chapitre 2) proposent des unités spécialisées pour ces opérations.
- Problème de *scatter* : Les réductions aléatoires en **OP** (mises à jour irrégulières de la matrice C ou du vecteur c) saturent la bande passante mémoire. Des caches de réduction (comme PHI [86] ou notre travail, SpDCache [II]) optimisent ces opérations en rassemblant et fusionnant les mises à jour autant que possible.

Analyse des algorithmes pour SpMSpM

Une modélisation des accès mémoire pour le noyaux de calcul SpMSpM (Figure 1.5) montre que :

- Sans mémoire locale ("0D"), **Gust** surpasse **IP** et **OP** en évitant les accès aléatoires.
- Avec une mémoire locale limitée ("1D", ex. : une ligne de matrice), **Gust** et **OP** réduisent davantage le trafic mémoire que **IP**, grâce à une meilleure réutilisation des données.
- Avec une mémoire locale étendue ("2D"), les trois algorithmes deviennent équivalents, mais **Gust** reste le plus efficace en pratique car il nécessite moins de mémoire pour atteindre ce niveau de performance.

CONCLUSION

Ce chapitre souligne que les matrices creuses structurées (comme les matrices bandées) et les algorithmes adaptés (notamment **Gust** pour les opérations MM) offrent des opportunités d'optimisation matérielle. Les défis clés (recherches d'intersection et réductions aléatoires) nécessitent des architectures spécialisées, comme celles explorées dans les Chapitres 3 à 6, où SpDCache est proposé comme solution innovante pour les caches de réduction adaptés aux matrices bandées.

2 Synthèse du Chapitre 2 : État-de-l'art des accélérateurs matériels pour le calcul des matrices creuses

Le Chapitre 2 propose une analyse comparative des accélérateurs matériels dédiés au traitement des matrices creuses [I]. Ces matrices, caractérisées par leur basse densité et leurs accès mémoire irréguliers, posent des défis majeurs aux architectures traditionnelles (CPU/GPU), limitant leur efficacité à environ 10% des performances maximales théoriques. Les accélérateurs matériels visent à surmonter ces limites en optimisant les hiérarchies mémoire, les algorithmes de multiplication (*Inner-Product*, *Outer-Product*, *Gustavson*) et les opérations de *gather-scatter*.

Classification et tendances architecturales

Les accélérateurs (listés en Table 2.1) sont classés selon cinq critères principaux :

1. Autonomie : complètement autonomes, c.à.d. qui exécute entièrement une opération et ne communique qu'avec la mémoire principale (ex. : OuterSPACE [91], Gamma [133]), ou assistés par processeur (ex. : PHI, SpDCache).
2. Position dans la hiérarchie mémoire : proches du processeur (ex. : ASA [132]), des caches (ex. : PHI), ou de la mémoire principale (ex. : SPiDRE [10]).
3. Algorithmes supportés :
   - *Gustavson* (**Gust**) : Préféré pour les noyaux MM (ex. : Gamma, MatRaptor [109]) grâce à son équilibre entre réutilisation des données d'entrée (B) et génération séquentielle des résultats (C).
   - *Outer-Product* (**OP**) : Adapté aux noyaux SpMV (ex. : OuterSPACE, GAS [136]) pour son potentiel de parallélisme, malgré des réductions aléatoires coûteuses.
   - *Inner-Product* (**IP**) : Moins performant seul, mais combiné à d'autres algorithmes dans des architectures adaptatives (ex. : Flexagon [85], IOPS [111]).
4. Opérations matérielles : Support pour le *gather* (recherche d'intersections, ex. : ExTensor) et/ou le *scatter* (réductions aléatoires, ex. : PHI, SortCache [108]).
5. Noyaux ciblés : MV ou MM (en croissance, notamment pour les graphes et l'IA).

Accélérateurs emblématiques

Quelques accélérateurs représentatifs, en guise d'example :

- OuterSPACE (2018, Section 2.2.1) : Architecture **OP** hautement parallélisée avec hiérarchie de caches à deux niveaux, optimisée pour SpMSpM. Utilise des *processing tiles* ("éléments de calcul") pour diviser le calcul en phases de multiplications et de fusion des résultats partiels.

- ExTensor (2019, Section 2.2.2) : Accélérateur **IP** générique pour l'algèbre tensorielle, avec un système de *tiling* (découpage en "tuiles") à trois niveaux pour éliminer les opérations inefficaces (qui retourne zéro) via des intersections matérielles.
- Gamma (2021, Section 2.2.3) : Spécialisé en SpMSpM avec l'algorithme **Gust**, utilisant un cache spécialisé (*FiberCache*) pour précharger les lignes de B et simplifier les réductions.
- PHI (2019, Section 2.2.4) : Cache de réduction pour les mises à jour commutatives (**OP**), combinant accumulation en-cache et *batching* ("regroupement par lots") du vecteur de sortie pour réduire le trafic mémoire.

Comparaison et performances

Les accélérateurs sont évalués sur des matrices issues de la *SuiteSparse Matrix Collection* [67] (densités < 10^{-2}, jusqu'à 10^{-7}). Les vitesses d'exécution sont rapportées par rapport à des bibliothèques logicielles (comme MKL [55] et cuSPARSE [90]) ou à des accélérateurs de référence comme OuterSPACE. Les résultats montrent que :

- *Gustavson* domine pour les opérations MM (ex. : ScanNow [81], ACES [78], Spada [74]).
- **OP** reste compétitif pour le SpMV (ex. : GAS, SSSR [103]), surtout dans les architectures assistées par processeur.
- Les solutions hybrides (**IP+OP+Gust**, ex. : Flexagon, Vesper [61]) gagnent en popularité pour leur adaptabilité.

Les principales limitations sont :

- La prétraitement des matrices (ex. : conversion en formats propriétaires pour ExTensor ou MatRaptor).
- L'hétérogénéité des benchmarks, rendant les comparaisons directes difficiles.
- La complexité matérielle et la spécificité des accélérateurs autonomes, souvent compensée par des gains de performance.

Perspectives pour les concepteurs

Le choix d'un accélérateur dépend des contraintes applicatives (voir Figure 2.8) :

1. Besoin de performance maximale : Privilégier les architectures autonomes (ex. : Gamma, Spada) avec support **Gust** ou **OP**, et des mémoires locales optimisées.
2. Flexibilité et faible empreinte : Opter pour des architectures assistées par processeurs (ex. : PHI, SortCache) ciblant le *scatter* ou le *gather*.
3. Adaptabilité algorithmique : Les designs récents (ex. : Flexagon, Vesper) intègrent plusieurs algorithmes pour s'adapter dynamiquement aux matrices.

CONCLUSION

Ce chapitre souligne que l'algorithme de *Gustavson* est le plus efficace pour les accélérateurs autonomes, suivi de **OP** pour les solutions assistées par processeur, tandis que l'**IP** reste pertinent dans des contextes spécifiques. Les défis qui ont dirigés les développement de la thèse incluent :

- L'optimisation des hiérarchies mémoire pour réduire le trafic mémoire.
- Le développement d'architectures adaptatives, capables de s'adapter aux structures des matrices.

- L'intégration multi-cœur pour améliorer la scalabilité.

Contribution de la thèse : Le travail se concentre sur les caches de réduction (SpDCache), une approche légère et scalable pour les noyaux **OP** SpMV, complétant l'état-de-l'art par une analyse fine des structures matricielles (bandes et densités locales).

3 Synthèse du Chapitre 3 : SpDCache, un cache de réduction basé sur les régions pour les noyaux de matrices creuses

Le Chapitre 3 introduit SpDCache, une architecture de cache innovante conçue pour optimiser les opérations de réduction aléatoire (*scatter*) dans le cadre de la multiplication matrice creuse-vecteur (SpMV, $A \times b = c$) utilisant l'algorithme *Outer-Product* (**OP**). L'objectif principal est de réduire le trafic mémoire et d'améliorer l'efficacité des caches traditionnels, souvent limités par les accès irréguliers générés par les matrices creuses.

Contexte et motivations

Les caches génériques (ex. : caches *set-associatifs* avec politique *LRU*) sont optimisés pour des accès séquentiels et locaux, mais peinent à gérer les mises à jour aléatoires typiques des algorithmes **OP**. Ces dernières provoquent des évincements fréquentes de lignes de cache très peu modifiées, saturant la bande passante mémoire et dégradant les performances. SpDCache propose une solution structurelle en exploitant les propriétés des matrices bandées, où les éléments non nuls sont concentrés autour de la diagonale.

Architecture de SpDCache

SpDCache repose sur un partitionnement de la mémoire cache en trois régions distinctes (voir Figure 3.3), chacune optimisée pour un type d'accès spécifique :

- *GRC* (*Generic Region Cache*) : Gère les accès séquentiels (ex. : lecture des entrées de la matrice A ou du vecteur b), similaire à un cache générique.
- *DRC* (*Dense Region Cache*) : Optimisée pour les zones denses de la matrix A, qui génèrent des réductions fréquentes sur le vecteur de sortie c. Cette région utilise un mécanisme de *coalescence* ("rassembler et fusionner") pour fusionner les mises à jour avant leur écriture en mémoire principale, réduisant ainsi le trafic.
- *SRT* (*Sparse Region Table*) : Table associative (CAM) pour les zones creuses de A, qui génèrent des réductions rares et isolées sur le vecteur de sortie c. Elle stocke uniquement les éléments non nuls au lieu de ligne de cache, évitant les lectures/écritures inutiles. Elle utilise des transactions mémoires adaptées, qui ne transfèrent qu'un élément à la fois en mémoire (*RMW*).

SpDCache intègre également des mécanismes pour éviter les duplications d'adresse entre les régions *DRC* et *SRT* (voir Figure 3.5).

Fonctionnement et avantages

- Réduction du trafic mémoire : En séparant les régions denses et creuses, SpDCache minimise les évincements coûteux et les lectures/écritures redondantes. Par exemple,

les mises à jour dans la *DRC* sont rassemblées et fusionnées avant d'être propagées, tandis que la *SRT* évite de charger des lignes de cache entière pour des éléments non nuls isolés.
- Flexibilité et adaptabilité : Le partitionnement est configurable (ex. : 25/50/25% pour *GRC*/*DRC*/*SRT*), permettant une étude du dimensionnement des régions pour optimiser les performance en fonction des matrices.
- Compatibilité avec les caches existants : SpDCache peut être intégré comme remplaçant d'un ou plusieurs caches dans les hiérarchies mémoire traditionnelles (ex. : entre le cache L1 et la mémoire principale), sans modifier significativement l'architecture sous-jacente.

Évaluation préliminaire

Une modélisation haut niveau (Python) compare SpDCache à un cache générique (GC) sur 48 matrices issues d'applications réelles (ex. : *roadNet-CA* de la SuiteSparse Matrix Collection). Les résultats montrent :
- Une réduction significative du trafic mémoire (4.1× par rapport à GC, voir Table 3.1), notamment grâce à la *coalescence* des réductions dans la *DRC*.
- Une meilleure utilisation des lignes de cache dans les régions denses, où les accès sont concentrés.
- Une légère sensibilité à la configuration des régions (voir Table 3.2) : Un dimensionnement optimal dépend de la structure de la matrice, mais en moyenne, une configuration simple (25/50/25%) suffit.

Comparaison avec l'état-de-l'art

SpDCache se distingue des autres accélérateurs matériels par :
- Son approche légère et générique : Contrairement aux solutions dédiées (ex. : ASIC pour SpMSpM), SpDCache cible spécifiquement le problème des réductions aléatoires, sans imposer de modifications majeures au reste du système.
- Son focus sur les matrices bandées : En exploitant leur structure, SpDCache optimise les accès mémoire sans nécessiter de prétraitement coûteux (ex. : réorganisation des données).

CONCLUSION

SpDCache représente une avancée significative dans la conception de caches adaptés aux données creuses, en combinant analyse structurelle (matrices bandées) et optimisations matérielles (*coalescence*, partitionnement). Ses contributions clés incluent :
- Une architecture modulaire intégrant des régions dédiées aux accès denses/creux.
- Une réduction démontrée du trafic mémoire via des mécanismes de *coalescence* et de stockage associatif.
- Une approche scalable compatible avec les hiérarchies mémoire existantes et les systèmes multi-cœurs.

Ce chapitre pose les bases pour les analyses approfondies (Chapitre 4) et les implémentations microarchitecturales (Chapitre 5), tout en ouvrant la voie à des optimisations multi-cœurs (Chapitre 6).

4 Synthèse du Chapitre 4 : Analyse théorique et modélisation des caches de réduction pour les noyaux SpMV en *Outer-Product*

Le Chapitre 4 développe un cadre théorique pour analyser et optimiser le comportement des caches de réduction lors de l'exécution de noyaux de calculs SpMV basés sur l'algorithme *Outer-Product* (`OP`). L'objectif est de comprendre comment la structure des matrices creuses, en particulier les matrices bandées, influence l'efficacité des caches, et d'en dériver des principes de conception pour minimiser le trafic mémoire, justifiant la conception de špdc.

Contexte et motivations

Le Chapitre 3 a introduit SpDCache, une architecture de cache partitionnée en régions dédiées aux données denses et creuses. Ce chapitre approfondit cette approche en proposant un modèle hybride probabiliste et analytique pour simuler le comportement d'un cache de réduction lors d'une opération SpMV en `OP`. L'analyse se concentre sur l'exploitation de la structure des matrices bandées, où les éléments non nuls sont concentrés autour de la diagonale, séparant ainsi la matrice en deux zones :

- Région en-bande (*IBR*) : Zone dense près de la diagonale, où les réductions sont fréquentes.
- Région hors-bande (*OBR*) : Zone creuse, où les accès sont plus rares et isolés.

Modélisation du cache de réduction

1. Isolation des lectures séquentielles

Le modèle commence par isoler les lectures séquentielles (ex. : accès aux matrice/vecteur d'entrées A et b), qui sont gérées efficacement par un cache générique (ex. : *GRC*). Ces accès ne posent pas de problème majeur en `OP` et sont donc exclus de l'analyse approfondie.

2. Modèle probabiliste pour les réductions

Pour les écritures irrégulières (*scatter*) sur le vecteur de sortie c, le chapitre propose :

- Une représentation tabulaire de la matrice et du cache, où chaque entrée décrit la densité des lignes de cache ou de bandes verticales de la matrice de même taille que les lignes de cache (voir Figure 4.1).
- Un algorithme itératif simulant le comportement du cache :
  - Évincements en-bande (*IBR*, correspond à la *DRC*) : Utilisation d'un cache *set-associatif* avec une politique d'évincement rotative, optimisée pour les données denses dans la bande de la matrice. Les lignes de cache sont réutilisées de manière cyclique, minimisant les transferts mémoire inutiles.
  - Évincements hors-bande (*OBR*, correspond à la *SRT*) : Cache *fully-associatif* avec une interface mémoire élément par élément, évitant de transférer des lignes de cache majoritairement vides.

3. Analyse des résultats

Le modèle met en évidence deux comportements distincts :

- Relation dimensionnelle critique (voir Figure 4.4) : La taille de la région en-bande (*IBR*) doit satisfaire une condition spécifique par rapport à celle de la *DRC* (Équation 4.2) pour permettre des évincements rotatifs, ce qui réduit fortement le trafic mémoire.
- Gestion de la région hors-bande (*OBR*, voir Figure 4.5) : Bien que cette région ait un impact majeur sur le trafic mémoire, son traitement reste efficace avec une *SRT* de taille modérée.

Validation et implications

Les observations théoriques confirment les choix de conception de SpDCache (Chapitre 3) :

- La séparation des régions *IBR*/*OBR* est essentielle pour adapter le cache à la densité des données.
- La *coalescence* des réductions dans la région dense (*DRC*) et le stockage associatif pour les données creuses (*SRT*) optimisent l'utilisation de la bande passante mémoire.

Par ailleurs, ce modèle permet une exploration rapide des configurations de cache (taille des régions, degré d'associativité, etc.) à partir de distributions de densités matricielles, sans nécessiter de simulations RTL coûteuses et chronophages.

CONCLUSION

Ce chapitre fournit une base théorique solide pour la conception de caches de réduction adaptés aux noyaux SpMV `OP`, en mettant l'accent sur :

- L'exploitation de la structure matricielle pour optimiser les politiques d'évincement.
- La séparation des régions du cache selon la densité des données, validant l'architecture de SpDCache.

Ces analyses préparent le terrain pour le Chapitre 5, qui traduira ces principes en une implémentation matérielle (RTL) de SpDCache, tout en identifiant des classes de matrices aux profils de performance distincts.

5 Synthèse du Chapitre 5 : Microarchitecture et évaluation de SpDCache

Le Chapitre 5 présente la microarchitecture détaillée de SpDCache. L'objectif est de valider, par des simulations RTL, les principes architecturaux introduits dans les Chapitres 3 et 4.

Microarchitecture de SpDCache

SpDCache repose sur un partitionnement virtuel de la mémoire cache en trois régions distinctes, suivant les spécifications des chapitres précédents : *GRC* (accès séquentiels, cache générique *set-associatif* avec *LRU*), *DRC* (zones denses, cache de réduction *set-associatif* avec *lru*), et *SRT* (accès isolés via table *fully-associative* stockant les éléments par mot). Ce partitionnement est dit virtuel, car il ne divise pas physiquement la mémoire (voir Figure 5.2) : chaque région utilise une partie des sets du cache, et un mécanisme de redirection dynamique des adresses vers les sets permet de basculer efficacement entre les régions. Cette approche offre une reconfigurabilité flexible, permettant au cache de passer dynamiquement d'un mode

générique (100/0/0%) à un mode réduction (ex. : 25/50/25%). Le fonctionnement de SpDCache s'articule autour d'un pipeline (chaîne d'exécution) en trois étapes (Figures 5.1, 5.3 et 5.4) :

1. Décodage de l'instruction : Le cache est interrogé pour déterminer s'il y a hit ou miss dans la région cible.
2. Traitement de l'instruction : Selon la réponse du cache, la réduction est exécutée localement via un accumulateur dédié.
3. Évincement : En cas d'évincement, un processus indépendant gère la communication avec la hiérarchie mémoire et effectue les accumulations nécessaires. Ce processus est découplé du pipeline principal, permettant un traitement des instructions suivantes en parallèle.

Évaluation et résultats

Configuration et méthodologie

- Outils : Simulations RTL avec des matrices issues de la SuiteSparse Matrix Collection [67], incluant des matrices bandées et non structurées.
- Métriques : Trafic mémoire et latences. Pour les matrices, une nouvelle métrique, d_{cd}, qui représente la densité moyenne de sections (de la taille d'une ligne de cache) non nul de la matrice (voir Figure 5.6).
- Paramètres : Cache de 32 KiB partitionné selon la configuration 25/50/25%, avec des lignes de cache de 64 octets et une associativité de 4 voies pour *GRC*/*DRC*.

Principaux résultats

1. Réduction du trafic mémoire (voir Figure 5.7) : SpDCache réduit le trafic du vecteur de sortie de 3.9× en moyenne par rapport à un cache générique (GC) pour toutes les matrices.
2. Réduction des latences d'exécution (voir Figure 5.9) : SpDCache réduit les latences de 1.1× en moyenne par rapport à un cache générique (GC) pour toutes les matrices.
3. Comportement par classe de matrices (voir Figure 5.10) :
   - Matrice à basse d_{cd} : SpDCache obtient les meilleurs performances (6.4× en trafic mémoire, speedups positifs) lorsque la densité à petit grain est faible. Dans ces conditions, la *SRT* traite un maximum d'éléments isolés, et la *DRC* densifie la bande.
   - Matrice à haute d_{cd} : SpDCache obtient des performances moins élevées (3.0× en trafic mémoire, speedups négatifs ou unitaires) lorsque la densité à petit grain est élevée. Dans ces conditions, les matrices profitent déjà de meilleurs performances avec la baseline GC qui sait tirer avantage de cette localité spatiale. Bien que la *DRC* fonctionne toujours de façon optimale, la *SRT* isole des éléments qui étaient groupés.

CONCLUSION

Ce chapitre valide expérimentalement les contributions des Chapitres 3 et 4, démontrant que :

- SpDCache améliore significativement l'efficacité mémoire pour les opérations SpMV **`OP`**, en exploitant la structure gros grain des matrices (bandes).

- L'architecture modulaire (*GRC*/*DRC*/*SRT*) permet une adaptabilité aux différents profils de matrices, avec des gains variables selon leur structure.
- Les simulations RTL confirment l'importance de la granularité petit grain (densité des lignes de cache).

6 Synthèse du Chapitre 6 : Optimisations et perspectives pour SpDCache

Le Chapitre 6 conclut la thèse en explorant les optimisations matérielles et les perspectives d'évolution de SpDCache. Il s'articule autour de quatre axes principaux : les optimisations au niveau RTL, l'extension aux noyaux SpMSpM, et l'intégration dans des systèmes multi-cœurs.

1. Optimisations RTL pour SpDCache

Quatre axes d'optimisation sont envisagés pour améliorer SpDCache :

- Support des nombres flottants : L'implémentation actuelle se limite aux entiers 64 bits, ce qui, bien que suffisant pour l'analyse des résultats, ne couvre pas les cas d'usage réels où les matrices utilisent majoritairement des données flottantes. L'ajout de ce support est donc essentiel pour un déploiement futur.
- Configuration dynamique des régions : Les mécanismes de configuration des régions (GRC, DRC, SRT) sont actuellement partiellement statiques. Une approche entièrement dynamique améliorerait la flexibilité du cache.
- Calcul matériel de la région cible : La détermination logicielle de la région cible des réductions pourrait être déléguée au matériel, réduisant significativement le nombre d'instructions exécutées par le processeur et donc les latences et optimisant les performances globales.
- Cohérence inter-régions : Contrairement aux *SRT* et *DRC*, la *GRC* autorise des duplications de données, ce qui pourrait complexifier la gestion de la cohérence. Une implémentation de la cohérence globale entre toutes les régions renforcerait la robustesse du système.

2. Extension aux noyaux SpMSpM

SpDCache a été initialement conçu pour les noyaux SpMV basés sur l'algorithme `OP`, mais son architecture modulaire permet une extension naturelle aux opérations SpMSpM. Pour ce faire, il est possible d'interpréter SpMSpM avec l'algorithme *Gustavson* comme une succession de sous-opérations SpMV `OP`, appliquées aux colonnes de la matrice de sortie (voir Figure 6.1).

3. Intégration multi-cœur et hiérarchie mémoire

Le passage à l'échelle de SpDCache dans les systèmes multi-cœurs et multi-niveaux de cache est naturelle (voir Figure 6.2) : il suffit de substituer les caches traditionnels de la hiérarchie mémoire par des instances de SpDCache. Les opérations de réduction se propagent alors de manière progressive depuis les cœurs vers la mémoire principale, sans risque de conflit de cohérence, car les données réduites ne sont jamais lues avant la finalisation de

l'opération. Au niveau du cache partagé de dernier niveau (LLC), les réductions en provenance des différents cœurs sont automatiquement rassemblées et fusionnées, assurant une consolidation efficace des résultats partiels.

CONCLUSION

Ce chapitre démontre que SpDCache, bien que conçu initialement pour SpMV **OP**, possède un potentiel significatif pour :

- Gagner en efficacité et performance grâce à quelques optimisations légères mais impactantes.
- Optimiser les noyaux de calcul de type MM.
- Passer à l'échelle vers des systèmes multi-cœurs.

En résumé, SpDCache offre une solution scalable et adaptative pour les défis des données creuses.

7 Conclusion

CETTE thèse aborde un enjeu majeur dans le domaine du traitement des données creuses, omniprésentes dans des applications exigeantes comme le calcul scientifique, l'analyse de graphes et l'apprentissage automatique. L'objectif principal était de répondre à la question suivante : comment concevoir des mécanismes matériels et architecturaux efficaces pour supporter ces calculs, tout en garantissant scalabilité, adaptabilité et haute performance ?

Pour y répondre, nous avons d'abord analysé les algorithmes fondamentaux de multiplication de matrices creuses, *Inner-Product* (**IP**), *Outer-Product* (**OP**) et *Gustavson* (**Gust**), et identifié leurs défis matériels respectifs, notamment les problèmes de *gather* (recherche d'intersections) et de *scatter* (réductions aléatoires). L'algorithme de *Gustavson* s'est révélé le plus prometteur pour une implémentation matérielle, grâce à ses compromis équilibrés entre réutilisation des données et génération séquentielle des résultats, tandis que l'**OP** est particulièrement adapté aux noyaux de calcul SpMV.

Notre contribution principale est SpDCache, une architecture de cache de réduction basée sur des régions, optimisant les transactions mémoire pour les données creuses en exploitant les propriétés structurelles des matrices bandées. SpDCache partitionne la mémoire cache en régions dédiées aux zones denses et creuses, réduisant significativement le trafic mémoire par rapport aux caches génériques. Les contributions clés incluent :

- Une modélisation architecturale via des modèles Python et probabilistes (en C) pour analyser les transactions mémoire et explorer le dimensionnement des caches.
- Une implémentation matérielle validée par des simulations RTL, mettant en lumière l'importance de la structure petit grain des matrices (comme la densité des lignes de cache).
- Des optimisations pour la scalabilité, incluant des extensions aux noyaux SpMSpM et aux architectures multi-cœurs.

Cette thèse fait progresser l'état-de-l'art des architectures de cache pour les données creuses, en mettant l'accent sur une compréhension intrinsèque des structures matricielles et une conception adaptative pour des solutions matérielles hautes performances. Les résultats montrent que SpDCache offre une réduction significative du trafic mémoire et une meilleure utilisation du cache, tout en restant flexible et adaptable à divers noyaux de calcul.

APPENDICES

Contents

A Extended List of State-of-the-Art Accelerator Summaries

THIS appendix chronologically summarizes all the accelerators listed in Table 2.1 (Chapter 2) to help readers understand how they operate.

Coup – 2015 Coup [134] (Commutative Updates Protocol) is a cache-coherence extension that reduces the cost of updates (*i.e.*, *reductions*) by allowing multiple private caches to simultaneously hold update-only permission for the same cache-line. It leverages the commutativity of operations like addition or bitwise logic, enabling caches to locally buffer and coalesce updates without requiring immediate synchronization or global traffic. When a read request occurs, Coup triggers a *reduction* phase, gathering all partial updates from private caches to produce the correct, up-to-date value before granting read permission. The protocol integrates seamlessly with existing coherence schemes (*e.g.*, MESI) but requires certain hardware changes, such as a *reduction* unit in shared caches and additional directory bits to track update types.

EIE – 2016 EIE [46] (Efficient Inference Engine) is a specialized ASIC accelerator designed for inference on compressed *Deep Neural Networks* (*DNNs*), optimizing the SpMV kernel with weight sharing and activation sparsity. It employs a scalable array of Processing Elements (PEs), each storing a partition of the compressed network in on-chip SRAM and performing computations locally, while a central control unit broadcasts non-zero input activations to all PEs. EIE leverages dynamic activation sparsity (skipping zero activations) and static weight sparsity (pruned connections), using a 4-bit index for shared weights to reduce storage and memory access energy. The accelerator uses a distributed leading non-zero detection network to efficiently collect and broadcast sparse activations, and a custom CSC format to store weights.

OuterSPACE – 2018 See Section 2.2.1 (Chapter 2).

GraphR – 2018 GraphR [107] is a ReRAM-based graph processing accelerator that leverages in-memory analog computation to efficiently execute graph algorithms by mapping them to the SpMV kernel. It consists of two main components: memory ReRAM, which stores graph data in compressed sparse format, and graph engines, composed of ReRAM crossbars that perform parallel Matrix-Vector (MV) operations on small subgraphs. GraphR employs a streaming-apply execution model to process subgraphs sequentially, converting compressed edge lists into sparse format for in-memory computation, while using simple logic units for *reduction* and convergence checks. The architecture supports a wide range of graph algorithms (*e.g.*, PageRank, BFS, SSSP) by expressing each problem in SpMV form, enabling high parallelism and energy efficiency.

SDE – 2019 SDE [51] (Streaming Dataflow Engine) is a high-performance, adaptable streaming dataflow accelerator designed for the SpMV kernel on FPGAs, optimized using high-level synthesis (HLS). It leverages loop pipelining, dataflow graphs, and data streaming to maximize memory bandwidth utilization, addressing the memory-bound nature of SpMV. The engine employs a modified CSR (MCSR) format to make loop iteration lengths predictable, enabling efficient pipelining and unrolling, and uses zero-padding to align row sizes for parallel processing. SDE supports multiport streaming to reduce memory port usage and load

balancing to evenly distribute workloads across parallel computing processes, ensuring high throughput and scalability.

ExTensor – 2019 See Section 2.2.2 (Chapter 2).

PHI – 2019 See Section 2.2.4 (Chapter 2).

SPiDRE – 2019 SPiDRE [10] (Sparse Data Rearrange Engine) is a near-main memory hardware accelerator designed to optimize sparse and irregular memory access patterns by transforming sparse data into dense, locality-friendly structures. It operates by decoupling data access from execution, allowing the host core to compute while SPiDRE reorganizes data in parallel. The accelerator uses a user-specified rearrange function to compact sparse data, improving cache block utilization and enabling efficient prefetching. SPiDRE maintains memory consistency by flushing and invalidating data as needed and synchronizes with the host core to ensure data is ready before access.

ITS – 2019 ITS [100] (Iteration Overlapped Two-Step SpMV) is a custom hardware accelerator designed for efficient SpMV on large and highly sparse matrices, leveraging a Two-Step algorithm that converts random memory accesses into sequential streaming accesses. The accelerator splits SpMV into two phases: partial SpMV between a source vector segment and a corresponding matrix stripe, followed by a multi-way merge of intermediate sparse vectors to produce the final result. To handle the computationally intensive merge operation, it employs a scalable, radix-based pre-sorter that parallelizes the merge process across multiple cores, each processing a subset of keys, ensuring high throughput and efficient use of off-chip bandwidth. Additional optimizations include Variable Length Delta Index (VLDI) compression to reduce off-chip traffic, iteration overlap to minimize data movement between iterations, and a Bloom Filter-based method to efficiently process nodes with exceptionally high degrees in power-law graphs.

SPU – 2019 SPU [22] (Sparse Processing Unit) is a general purpose accelerator designed to efficiently handle both data-dependent and regular algorithms by specializing in two common forms of data-dependence: stream-join and alias-free indirection. It integrates a systolic-style coarse-grained reconfigurable architecture with a compute-enabled scratchpad and aggressive stream reordering to support high-bandwidth indirect memory access and fully-pipelined stream-join operations. The accelerator employs decomposability across compute, network, and memory pipelines to flexibly support various data types, enabling independent flow control for finer-grain resources. SPU's stream-dataflow ISA and multicore mesh network-on-chip facilitate efficient communication, synchronization, and workload partitioning, making it adaptable across domains like machine learning, graph processing, and databases.

Tensaurus – 2020 Tensaurus [110] is a versatile hardware accelerator designed for mixed sparse-dense tensor computations, supporting tensor factorizations (*e.g.*, MTTKRP, TTMc) and matrix operations (*e.g.*, SpMM, SpMV, SpGEMM). It leverages a co-designed sparse storage format called CISS (Compressed Interleaved Sparse Slice), which enables vectorized, streaming access to sparse data, maximizing memory bandwidth utilization. The accelerator implements a common compute pattern (SF^3), scalar-*fiber* product followed by *fiber-fiber* products, allowing it to efficiently handle diverse tensor and matrix operations with a unified hardware

design. Tensaurus features a 2D array of Processing Elements (PEs), scratchpad memories, and specialized load/store units to manage data movement, while its systolic array architecture ensures high parallelism and efficient reuse of dense operand matrices.

MatRaptor – 2020 MatRaptor [109] (1) is a hardware accelerator designed for the SpGEMM kernel, leveraging row-wise *Gustavson*'s algorithm to optimize data reuse and on-chip memory efficiency. It introduces a novel Cyclic Channel Sparse Row (C^2SR) sparse format, which enables parallel, vectorized, and streaming memory access, maximizing bandwidth utilization. The accelerator uses a systolic array architecture with specialized loaders for input matrices and Processing Elements (PEs) that perform multiplication and merge operations using queues for efficient sorting and accumulation. MatRaptor avoids synchronization bottlenecks and minimizes on-chip memory requirements by processing rows asynchronously and using a round-robin allocation strategy for load balancing.

SpArch – 2020 SpArch [137](1) is a domain-specific accelerator designed to optimize the *Outer-Product* SpGEMM kernel by jointly improving input and output data reuse. It employs a highly parallelized, streaming-based merger to pipeline the multiply and merge stages, enabling on-chip merging of *partial output* matrices immediately after generation. SpArch introduces a condensed matrix representation to reduce the number of *partial output* matrices by up to three orders of magnitude, minimizing DRAM access. A Huffman tree scheduler optimizes the merge order of *partial output* matrices, further reducing memory traffic, while a row prefetcher with near-optimal buffer replacement recovers input reuse lost during matrix condensing.

Alrescha – 2020 Alrescha [3] is a lightweight, reconfigurable sparse-computation accelerator designed to address the challenge of data dependencies in sparse problems, which limit parallelism and memory bandwidth utilization. It employs mathematical transformations to break down computations into mostly parallel and minimally dependent parts, reordering these parts to maximize performance. The accelerator features a hybrid compute engine with a fixed unit for parallel operations and a small reconfigurable unit for data-dependent tasks, enabling fast runtime switching between different computation modes. Alrescha uses a locally-dense storage format that aligns the order of non-zero values with the computation sequence, eliminating metadata transfer and enabling uninterrupted streaming from memory.

SIGMA – 2020 SIGMA [95] is a flexible and scalable accelerator designed for the SpGEMM kernel, a core operation in deep learning training and inference. It features a Flexible Dot Product Engine (Flex-DPE) with a 1D substrate of multipliers, Benes network for non-blocking data distribution, and a Forwarding Adder Network (FAN) for efficient spatial *reduction*, enabling variable-sized dot-products and high compute utilization. SIGMA composes multiple Flex-DPEs into Flexible Dot Product Units (Flex-DPUs) via a mesh network-on-chip, allowing dynamic allocation of resources for different SpGEMM dimensions and sparsity levels. The accelerator supports unstructured sparsity using a bitmap compression format and a sparsity controller to map only non-zero elements, minimizing memory overhead and maximizing efficiency.

(1). See an extended description in Paper I.

SpaceA – 2021 SpaceA [126] is a processing-in-memory (PIM) accelerator designed for the SpMV kernel, leveraging 3D-stacked memory (*e.g.*, HMC/HBM) to overcome bandwidth bottlenecks in traditional architectures. It integrates Processing Elements (PEs) near each memory bank, enabling high-bandwidth, low-latency access to local data while hiding remote access latency through outstanding memory requests and memory-level parallelism. The accelerator employs Content Addressable Memory (CAM) buffers at both bank and vault levels to cache frequently accessed input vector elements, reducing inter-bank traffic and improving locality. A custom mapping scheme distributes non-zero matrix elements across PEs to balance workloads and maximize data reuse, while Product-PEs and Accumulation-PEs handle partial dot-products and result accumulation, respectively.

Gamma – 2021 See Section 2.2.3 (Chapter 2).

SPAGHETTI – 2021 SPAGHETTI [50] is an FPGA-optimized, open-source accelerator for the SpGEMM kernel, designed specifically for highly sparse inputs (density below 3%). It employs an *Outer-Product*-based algorithm, which splits the computation into a parallel multiplication phase followed by a merge phase, avoiding the index-matching overhead of *Inner-Product* methods. The accelerator features a pattern-aware software scheduler that partitions input matrices into tiles, maximizing output reuse and ensuring partial sums fit on-chip to eliminate DRAM refetching. Its streaming microarchitecture includes parallel multipliers, a round-robin allocator, and scalable sort-and-merge units that pipeline multiple rows of the output, adapting to FPGA resource constraints and memory bandwidth.

InnerSP – 2021 InnerSP [7][(1)] is a hardware accelerator designed to optimize sparse matrix multiplication by leveraging the row-wise *Gustavson*'s algorithm, avoiding the memory bloating issues common in *Outer-Product* methods. It uses a smart caching scheme to exploit locality in the second input matrix (B), reducing memory traffic and improving efficiency. InnerSP features an on-chip hash table for accumulating intermediate results, preventing overflows through pre-scanning and row splitting/merging techniques. The accelerator integrates B-caches (row and column-value of B) to further enhance data reuse, and employs a locality-aware replacement policy to minimize cache misses.

CoSPARSE – 2021 CoSPARSE [32] is a reconfigurable SpMV framework designed for graph analytics, combining software and hardware optimizations to adapt dynamically to varying input characteristics. It supports two SpMV algorithms, *Inner-Product* and *Outer-Product*, and automatically selects the optimal one based on the density of the input vector, while also reconfiguring the on-chip memory hierarchy (shared/private, cache/scratchpad) to maximize data reuse and minimize memory access overhead. The framework features workload-balancing strategies, including static matrix partitioning and dynamic task distribution, to handle sparse matrices efficiently, ensuring balanced workloads across Processing Elements (PEs). CoSPARSE is built on a programmable, many-core hardware substrate (Transmuter), enabling runtime reconfiguration of memory and dataflow with minimal overhead, and abstracts hardware complexity through a high-level software interface.

SortCache – 2021 SortCache [108][(1)] is a near-cache accelerator designed to optimize sparse data workloads by intelligently managing on-chip cache resources. It leverages Vectorized

Binary Search Trees (VBSTs) to dynamically reorder sparse data during *gather-scatter* operations, extracting sequential locality and maximizing cache bandwidth utilization. The system integrates specialized hardware for merging and preconditioning data, enabling efficient parallel *reductions* and minimizing redundant memory access. SortCache operates with minimal programmer intervention, requiring only simple annotations to specify parameters like cache size and *reduction* operations.

Sextans – 2022 Sextans [106] is a streaming accelerator designed for the SpGEMM kernel, optimized for HBM-equipped FPGAs. It employs a hierarchical architecture with processing engine groups and Processing Elements (PEs) to handle workload imbalance, using PE-aware non-zero scheduling to achieve an II=1 pipeline (Initiation Interval of 1 cycle) and balance workloads across PEs. The accelerator leverages multi-level memory optimizations, including on-chip BRAMs and URAMs for fast random access, and streams large matrices from off-chip HBM in batches to minimize latency. Sextans also features hardware flexibility, allowing it to support SpGEMMs of varying sizes without re-synthesis, by using a pointer list to dynamically schedule non-zero elements.

ASA – 2022 ASA [132][(1)] (Accelerating Sparse Accumulation) is a hardware accelerator designed to optimize the sparse accumulation (SPA) step in column-wise SpGEMM operations. It replaces traditional software-based hash table lookups (prone to branch mispredictions and collisions) with a dedicated, *set*-associative on-chip cache and a pipelined hardware accumulator. ASA extends the processor's instruction set with a Hardware Probing and Accumulation (HPA) instruction, eliminating data-dependent branches and enabling efficient, atomic search-and-accumulate operations. Overflowed entries are stored in a pre-allocated memory buffer and merged later, minimizing stalls.

Spada – 2023 Spada [74] is a hardware-software co-designed accelerator for the SpGEMM kernel, featuring a window-based adaptive dataflow based on *Gustavson*'s algorithm that dynamically adjusts execution modes to optimize for input or output data reuse based on local sparse patterns. The accelerator uses reconfigurable multiplication Processing Elements with dynamically partitioned lanes and addition Processing Elements for efficient merging of partial sums, supported by a global cache with an optimized replacement policy to maximize data locality. A profiling-guided window shape adaption algorithm detects sparse patterns in real-time, enabling Spada to switch between modes that prioritize either input reuse (like *Outer-Product*) or output reuse (like *Gustavson*) as needed.

Flexagon – 2023 Flexagon [85] is a reconfigurable SpMSpM accelerator designed to dynamically adapt to the most efficient algorithm (*Inner-Product*, *Outer-Product*, or *Gustavson*'s) for each layer of a *DNN* model. It features a novel Merger-Reduction Network (MRN), which unifies *reduction* and merging operations in a single, tree-based topology, allowing seamless switching between algorithms without hardware overhead. The accelerator includes a three-tier memory hierarchy, comprising a FIFO for stationary data, a cache for streaming data, and a specialized PSRAM for partial sums, tailored to the access patterns of each algorithm. Flexagon's unified memory controllers and flexible interconnects enable efficient handling of diverse sparsity patterns and matrix dimensions.

MOSCON – 2023 MOSCON [35] is a modified *Outer-Product*-based SpGEMM accelerator designed for FPGAs, optimized for deep learning applications. It employs configurable tiles with 16 Processing Elements (PEs) arranged in sub-tiles, allowing flexible allocation of rows or columns to maximize load balance and parallelism. The architecture uses bitonic parallel sorting for efficient merging of partial sums, reducing execution time compared to bubble sort-based implementations. MOSCON incorporates a sparsity checking unit to identify and skip zero rows/columns early, and an element parsing unit to dynamically configure sub-tiles based on row/column sparsity.

GSCSp – 2023 GSCSp [72] is an efficient *Gustavson*-based SpMM accelerator designed for FPGAs. It optimizes memory traffic and parallelism by employing element-wise parallelism, which evenly distributes workloads across Processing Elements (PEs) and reduces idle time caused by synchronization. The accelerator features SpCache, an access pattern-aware caching scheme that combines streaming and caching to handle irregular memory accesses and unpredictable row lengths in sparse matrices. GSCSp also introduces an optimized merge algorithm that processes only *partial outputs*, eliminating redundant merges and improving efficiency.

SDMA – 2023 SDMA [39] is a sparsity-aware SpMM accelerator designed for *Graph Neural Networks* (*GNNs*), leveraging an algorithm/hardware co-optimization approach to address load imbalance and memory access inefficiencies. It employs an equal-value partition method to evenly distribute sparse data, ensuring balanced workloads during data movement, and a vertex-clustering optimization to exploit data locality by aggregating vertices with shared neighbors, reducing global memory traffic. The accelerator uses an adaptive dataflow based on *Gustavson*'s algorithm, which decouples computation into multiplication and merging phases, enabling efficient parallelism and minimizing intermediate data overhead. SDMA's hardware architecture features multiple processing units with Processing Elements (PEs) and accumulation units, organized in a SIMD fashion, and ping-pong buffers to overlap communication and computation.

SSSR – 2023 Sparse Stream Semantic Registers [103] (SSSRs) is a lightweight, modular ISA extension for RISC-V that accelerates sparse linear algebra by handling indirection, *intersection*, and union operations in hardware. It extends Stream Semantic Registers (SSRs), which map memory streams to architectural registers, to support sparse tensor formats like CSR and CSF, enabling efficient streaming of both sparse-dense and sparse-sparse computations. SSSRs introduce indirection SSRs (ISSR) for reading index streams and generating indirect addresses, and egress SSRs (ESSR) for writing back indices, while an index comparator enables *intersection* and union of sparse operands. By offloading index processing and comparison to hardware, SSSRs reduce control overhead and maximize Floating-Point Unit (FPU) utilization, supporting flexible dataflows and a wide range of sparsity without requiring structural constraints.

Funnel – 2024 Funnel [82] is a sparse attention accelerator designed to dynamically predict and efficiently process unstructured sparse data in Transformer-based models. It uses a fast quantization method based on lookup tables to estimate the attention score matrix, followed by Top-k sorting to generate a sparse score matrix, minimizing prediction overhead. The

Funnel Computing Unit (FCU) unifies the computation of dense MM and SpMM through a multi-dataflow fusion approach, leveraging a reconfigurable one-dimensional systolic array and a grouped summation network to handle irregular data distribution. A lightweight buffer and data *tiling* strategy enhance data reuse, while a sensible partitioning method aligns matrix granularity with the accelerator array size.

GAS – 2024 GAS [136] is a general purpose in-memory computing (IMC) accelerator designed for sparse matrix multiplication, supporting SpMV, SpMSpV, SpMM, and SpMSpM kernels. It integrates FeFET-based Content Addressable Memory (CAM) arrays for index matching and multiply-add computation (MAC) arrays for in-situ floating-point operations, using a unified *Outer-Product*-based methodology to handle all four functions. The accelerator employs three working phases, write, load, and compute, to efficiently process sparse matrices, with optimization techniques like zero-padding adjustment, early termination, and workload balancing to enhance performance. GAS dynamically aligns mantissas and handles exponent sums to support double-precision floating-point arithmetic directly within the memory arrays.

ACES – 2024 ACES [78] is a SpMM accelerator designed to dynamically adapt its execution flow to diverse sparsity patterns, optimizing both data reuse and parallelism. It features an adaptive execution flow based on *Gustavson* that adjusts the condensing degree of input matrices, balancing input reuse and synchronization requirements, while a concurrency-aware cache management policy (PureFiber) optimizes global cache performance by considering both data locality and concurrent access patterns. The accelerator integrates a non-blocking buffer with the global cache to minimize stalls from cache misses, enhancing data access concurrency. ACES's hardware architecture pairs each multiplication Processing Element with an addition Processing Element, enabling fine-grained parallelism and reducing inter-PE dependencies.

FullSparse – 2024 FullSparse [129] is a SpGEMM accelerator designed to handle diverse sparsity levels (0.2% to 99%) in *Deep Neural Networks* (*DNNs*) by dynamically adapting its processing modes. It employs an online sparsity detector to switch between dense and sparse computation modes, optimizing resource usage for varying input sparsity. The accelerator features a multi-sparsity-compatible PE array that efficiently processes convolutions with irregular output data using collision-free methods, ensuring high computational efficiency. A result sparsity predictor, based on an XGBoost [19] model, dynamically adjusts data storage formats to enhance off-chip memory access efficiency.

Sparm – 2024 Sparm [80] is a SpGEMM accelerator designed to support multiple algorithms (*Inner-Product*, *Outer-Product*, and *Gustavson*) for diverse sparsity patterns. It features a Merge Manager and Bitindex SRAM to regularize the merging of partial sums, avoiding the "blockage" issues seen in prior designs like Flexagon [85] by precomputing index *intersections* and controlling FIFO reads. The accelerator retains Flexagon's StaFIFO, StrCache, and PSRAM for efficient data storage and employs a row/column prefetcher to proactively load streaming matrices, reducing DRAM access. Sparm's reconfigurable network-on-chip connects on-chip components, enabling adaptive dataflow execution while minimizing hardware overhead.

SPADES – 2024 SPADES [53] is a SpMV accelerator that integrates on-the-fly GZIP decompression to reduce off-chip data movement and memory access energy. It employs a segmented GZIP (SGZIP) format, dividing sparse matrices into independently compressed blocks that are decompressed in parallel by four dedicated decompression modules, each featuring a parallel Huffman decoder capable of decoding one symbol per cycle. The decompressed data is streamed to a Processing Element (PE) with a pipelined 64-bit floating-point multiply-accumulate compute unit, which performs SpMV using buffered input vectors and accumulates results in an output buffer. A custom microcontroller coordinates data flow, managing DMA transfers and dynamically configuring the PE based on decompressed block sizes.

DySpMM – 2024 DySpMM [118] is a dynamic FPGA-based accelerator for the SpMM kernel, designed to address the limitations of fixed-configuration approaches. It features a configurable data distributor that dynamically adjusts sub-matrix sizes to minimize memory access, and an element-wise allocator that balances workloads across Processing Elements (PEs) to improve utilization. The accelerator includes an interleaved reorder unit to avoid read-after-write hazards at runtime, eliminating the need for time-consuming preprocessing. DySpMM's merge tree efficiently combines results from adjacent PEs, while its scalable architecture supports parallel processing and memory bandwidth optimization.

Vesper – 2024 Vesper [61] is a versatile sparse linear algebra accelerator designed to support SpMV, SpMM, SpMSpV, and SpGEMM kernels, dynamically configuring its compute patterns (*Inner-Product*, *Outer-Product*, or *Gustavson*) to optimize performance for varying sparsity levels and workloads. It employs a distributed architecture with Processing Elements (PEs), operand access units, and HBM burst engines, connected via butterfly networks-on-chip, to maximize memory bandwidth utilization and minimize channel conflicts. Vesper uses a configurable on-chip buffer and double-buffered 4-way merge trees to handle data reuse and partial sum accumulation, while a specialized caching mechanism enhances SpGEMM performance for denser matrices. An analytical model guides runtime configuration, enabling rapid selection of optimal compute patterns, *tiling* schemes, and parallelization strategies.

Mentor – 2024 Mentor [77] is a memory-efficient SpMM accelerator that leverages the column-wise *Gustavson*'s algorithm to optimize data reuse and memory access patterns. It employs a streaming construction scheme at the software level, preprocessing input matrices to enable streaming memory access and resolve read-after-write conflicts by introducing control elements like Rest, Padding, and Blocking. The hardware architecture features a fully pipelined design with four stages, read, multiplication, accumulation, and write, using double-buffered on-chip scratchpads to avoid pipeline stalls and maximize throughput. Mentor's delayed feeding strategy balances bandwidth requirements and Processing Element (PE) utilization, while its analytical model guides hardware parameter selection for optimal performance.

IOPS – 2025 IOPS [111] is a unified SpMM accelerator that combines *Inner-Product* (`IP`) and *Outer-Product* (`OP`) approaches into a hybrid method (IOHP, Inner-Outer-Hybrid Product). It reuses input submatrices across Processing Elements (PEs) using `IP`, while skipping zero-element calculations within each PE using `OP`, balancing data reuse and computational efficiency. The accelerator supports both sparse-sparse and sparse-dense matrix multiplications, employing RP-CSC and CP-CSR encoding formats for submatrix reuse and an address

mapping method to handle irregular partial sum accumulation. An adaptive partition strategy dynamically tiles input matrices based on sparsity, optimizing on-chip storage and reducing DRAM access.

Leda – 2025 Leda [128] is a highly-tiled SpMM accelerator designed for HBM-equipped FPGAs, optimized for *Graph Neural Networks* (*GNNs*). It employs a *tiling* sparse format to balance workloads across Processing Elements (PEs) and enhance data locality, mitigating load imbalance and random memory access issues. Leda uses an improved *Outer-Product* dataflow to reduce memory bottlenecks and a minimum similarity reordering algorithm to optimize read-after-write dependencies, improving compute occupancy and throughput. The accelerator features a highly parallel hardware architecture with a matrix multiplication unit (MMU)-centric design, enabling efficient input reuse and high throughput.

DMSA – 2025 DMSA [88] is a SpMSpM accelerator that introduces the Distribute-Merge Product (DMP) dataflow, based on *Gustavson*'s algorithm, which evenly distributes workloads across multiple computation streams and merges *partial outputs* efficiently. It eliminates the need for explicit synchronization between partial matrix computations by distributing elements of matrix A into parallel streams, each computing a partial matrix using scalar-vector products. DMSA incorporates a multibank-based input data loading mechanism with a protocol to manage unpredictable data sizes for matrix B, and a credit-based flow control mechanism to handle dataflow irregularities during computation and merging. The architecture also features a two-stage accumulation process for *partial outputs*, simplifying the design of the postprocessing unit and ensuring high throughput.

C^{3}ache – 2025 C^{3}ache is a hierarchical cache-centric computing architecture designed for SpMM on GPGPUs, addressing the inefficiencies of traditional processor-centric optimizations. It introduces a hybrid dataflow that combines *Outer-Product* and *Gustavson*'s algorithms, decoupling SpMM into two phases, multiplication and merging, to align with memory access patterns and maximize data reuse. The architecture restructures the GPGPU cache hierarchy into large-scale data-parallel Processing-in-Memory (PIM) units in L1 cache for multiplication and in-situ merging PIM units in L2 cache for merging, reducing memory transactions and synchronization overhead. A memory-aware compressed format (C3SR) optimizes sparse matrix encoding by compressing multiple short sparse rows into a single cache-line, further reducing memory overhead. C^{3}ache also employs an efficient multi-precision matrix multiplication SRAM-PIM macro (MSPM) to support vector computations.

ScanNow – 2025 ScanNow [81] is a scan window-based SpMM accelerator, inspired by *Gustavson*'s algorithm, designed to dynamically balance input and output data reuse for diverse sparse patterns. It introduces a scan window-based task dispatch scheme that prioritizes tasks enhancing input reuse while ensuring output reuse, by constructing vertical scan windows to detect potential tasks in adjacent rows. The accelerator employs a hardware architecture featuring multiple memory components, including a scalar buffer, global cache, and partial vector buffer, along with multiplication Processing Elements and accumulation Processing Elements to handle scalar-vector multiplication and element-wise addition tasks, respectively. A task dispatcher dynamically schedules tasks based on sparse patterns, while an accumulation scheduler manages the merging of *partial outputs*.

Other Accelerators While numerous additional accelerators are referenced throughout the aforementioned works, many fall outside the scope of the classification criteria presented in Table 2.1. We provide brief descriptions of selected accelerators in the following paragraphs to complete this survey.

- Matrix-Vector (MV) Multiplication Accelerators
  - HiSparse [27]: An HBM-optimized SpMV accelerator using a custom sparse format for vectorized, streaming accesses to HBM channels. It employs a scalable on-chip buffer design combining replication and banking, and a pipelined PE with load-store forwarding to resolve read-after-write dependencies.
- Matrix-Matrix (MM) Multiplication Accelerators
  - HARP [63]: A user-transparent *tiling* accelerator for `OP` SpGEMM, using hardware-based pseudo-*tiling* to logically tile input matrices without generating compression formats for each tile. It employs Runtime Operand Descriptors (RODs) to point to effectual column-row pairs and dynamically adjusts *tiling* boundaries via super-*tiling* and sub-*tiling*.
  - AiSpGEMM [113]: An FPGA-based SpGEMM accelerator using an optimized C^2SR format to resolve memory bank conflicts and reuse matrix B. It features reconfigurable and parallel mergers for accelerating row computations.
  - RTSpMSpM [135]: Harnesses ray tracing hardware to accelerate SpMSpM by mapping SpMSpM operations to ray tracing problems. It leverages existing ray tracing units for *intersection* tests and introduces architectural optimizations to perform multiplications and accumulations directly in the ray tracing hardware.
- Sparse Matrix Accelerators for Graph Processing
  - HitGraph [139]: An FPGA graph accelerator generating separate bitstreams for each graph algorithm, focusing on edge-centric processing.
  - GraphLily [52]: A graph linear algebra overlay for HBM-equipped FPGAs, supporting SpMV with a cyclic packed streams of rows (CPSR) format for streaming accesses and a cluster-shared vector buffer (CSVecBuf) to exploit data reuse.
  - ThunderGP [20]: An FPGA-based graph processing framework using DDR memory, optimized for SpMV.
- Sparse Matrix Accelerators for Neural Networks
  - SCNN [92]: A *CNN* accelerator for compressed-sparse *CNNs*, exploiting both weight and activation sparsity with a novel dataflow and compressed storage.
  - SparTen [42]: A sparse tensor accelerator for *CNNs*, using inner join operations and index-matching logic to handle sparse activations and weights.
  - CANDLES [44]: A microarchitecture-dataflow co-design for sparse *CNNs*, combining pixel-first compression and channel-first dataflow to improve partial sum reuse and reduce index-matching complexity.
  - SparTA [138]: Targets unstructured weight sparsity in *DNNs*, partitioning matrices into structured and unstructured sparse components to leverage both sparse tensor cores and CUDA cores.
  - MaxK-GNN [93]: A GPU-based *GNN* training system that introduces MaxK nonlinearity to sparsify feature matrices. It uses a Compressed Balanced Sparse Row (CBSR) format and optimized SpGEMM kernel for forward and backward computations.
- Tensor Core (TC)-Based Sparse Matrix Accelerators

- Block-SpMM (cuSPARSE) [90]: Uses blocked-ELLPACK [64] (BELL) format to exploit TCs for block-sparse routines in SpMM.
- vectorSparse [37]: A fine-grained TC-based SpMM accelerator for Volta architecture (NVIDIA), using column vector sparse encoding (CVSE).
- Flash-LLM [125]: Targets unstructured weight sparsity in LLMs, employing load-as-sparse, compute-as-dense mode and double buffering for TC-based SpMM.
- TC-GNN [120]: Introduces TF32 TCs to general SpMM for *GNN* workloads, using sparse graph translation (SGT) to condense sparse matrices into TC blocks.
- DTC-SpMM [30]: A TC-optimized SpMM accelerator featuring memory-efficient formats, TCU-cache-aware reordering, and runtime optimizations like shared-memory bypassing and sparse double buffering.

B Sparse Matrices Used for the Evaluations

FOR transparency, we present in Table B.1 all matrices used in our simulations. This includes matrices from the SuiteSparse Matrix Collection [67] and 20 generated matrices, totaling 87. We provide their dimensions, density, density within in- and out-of-band regions (with half-*band-width* 1021), and d_{cd}. We also indicate which evaluations use each matrix (Python or RTL; the analytical model does not require real matrices).

Table B.1 – Matrices Used in Every Evaluations and Their Characteristics

Name	**Src**	***M***	***nnz***	***d***	d_{IB} (with $w_{hB} = 1021$)	d_{OB} (with $w_{hB} = 1021$)	d_{cd}	**Eval.**	
								Py.	**RTL**
west0381	SS	381	2.16 k	1.49 e-2	1.49 e-2	0.00 e-0	18%		✓
mhda416	SS	416	8.56 k	4.95 e-2	4.95 e-2	0.00 e-0	83%		✓
rotor2	SS	791	10.7 k	1.71 e-2	1.71 e-2	0.00 e-0	68%		✓
filter2D	SS	1.67 k	10.8 k	3.86 e-3	3.86 e-3	0.00 e-0	18%		✓
Journals	SS	124	12.1 k	7.85 e-1	7.85 e-1	0.00 e-0	99%		✓
Trefethen_700	SS	700	12.7 k	2.58 e-2	2.58 e-2	0.00 e-0	25%		✓
CAG_mat364	SS	364	13.6 k	1.03 e-1	1.03 e-1	0.00 e-0	46%		✓
Si2	SS	769	17.8 k	3.01 e-2	3.01 e-2	0.00 e-0	26%		✓
bcsstk10	SS	1.09 k	22.1 k	1.87 e-2	1.87 e-2	0.00 e-0	62%		✓
qc324	SS	324	26.7 k	2.55 e-1	2.55 e-1	0.00 e-0	83%		✓
ex2	SS	441	26.8 k	1.38 e-1	1.38 e-1	0.00 e-0	84%		✓
LeGresley_4908	SS	4.91 k	30.5 k	1.27 e-3	1.39 e-3	1.19 e-3	15%	✓	✓
mbeacxc	SS	496	49.9 k	2.03 e-1	2.03 e-1	0.00 e-0	59%		✓
bcsstk13	SS	2.00 k	83.9 k	2.09 e-2	2.73 e-2	7.21 e-4	67%		✓
wiki-Vote	SS	8.30 k	104 k	1.51 e-3	3.55 e-3	8.92 e-4	17%	✓	✓
p2p-Gnutella31	SS	62.6 k	148 k	3.78 e-5	2.60 e-4	3.03 e-5	13%	✓	✓
ca-CondMat	SS	23.1 k	187 k	3.49 e-4	3.96 e-4	3.45 e-4	13%	✓	✓
Alice_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	9.00 e-8	14%	✓	✓
Bob_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	1.70 e-7	14%	✓	✓
Charlie_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	3.20 e-7	14%	✓	✓
Diana_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	6.10 e-7	14%	✓	✓
Ethan_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	1.12 e-6	14%	✓	✓
Fiona_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	2.06 e-6	13%	✓	✓
George_1021	Gen	10.0 k	194 k	1.94 e-3	9.98 e-3	3.76 e-6	14%	✓	✓
Hannah_1021	Gen	10.0 k	194 k	1.94 e-3	9.98 e-3	6.88 e-6	14%	✓	✓
Isaac_1021	Gen	10.0 k	194 k	1.94 e-3	9.98 e-3	1.24 e-5	14%	✓	✓

Continued on next page

Table B.1 – Matrices Used in Every Evaluations and Their Characteristics (Continued)

Name	**Src**	***M***	***nnz***	***d***	d_{IB} (with $w_{hB} = 1021$)	d_{OB} (with $w_{hB} = 1021$)	d_{cd}	**Eval. Py.**	**Eval. RTL**
Julia_1021	Gen	10.0 k	195 k	1.95 e-3	9.98 e-3	2.07 e-5	13%	✓	✓
Kevin_1021	Gen	10.0 k	197 k	1.97 e-3	9.98 e-3	4.13 e-5	13%	✓	✓
Laura_1021	Gen	10.0 k	198 k	1.98 e-3	9.98 e-3	6.20 e-5	13%	✓	✓
Michael_1021	Gen	10.0 k	205 k	2.05 e-3	9.98 e-3	1.42 e-4	13%	✓	✓
Nina_1021	Gen	10.0 k	215 k	2.15 e-3	9.98 e-3	2.64 e-4	13%	✓	✓
Oliver_1021	Gen	10.0 k	228 k	2.28 e-3	9.98 e-3	4.34 e-4	13%	✓	✓
TSOPF_RS_b39_c7	SS	14.1 k	252 k	1.27 e-3	1.03 e-3	1.31 e-3	44%	✓	✓
Paula_1021	Gen	10.0 k	265 k	2.65 e-3	9.98 e-3	8.86 e-4	13%	✓	✓
Quentin_1021	Gen	10.0 k	324 k	3.24 e-3	9.98 e-3	1.62 e-3	13%	✓	✓
poisson3Da	SS	13.5 k	353 k	1.93 e-3	2.71 e-3	1.80 e-3	15%	✓	✓
cit-HepTh	SS	27.8 k	353 k	4.57 e-4	1.05 e-3	4.11 e-4	14%	✓	✓
email-Enron	SS	36.7 k	368 k	2.73 e-4	1.69 e-3	1.91 e-4	18%	✓	✓
Rachel_1021	Gen	10.0 k	428 k	4.28 e-3	9.98 e-3	2.91 e-3	13%	✓	✓
bcsstk17	SS	11.0 k	429 k	3.56 e-3	2.01 e-2	0.00 e-0	72%	✓	✓
soc-Epinions1	SS	75.9 k	509 k	8.84 e-5	7.00 e-4	7.16 e-5	17%	✓	✓
patents_main	SS	241 k	561 k	9.69 e-6	2.00 e-8	9.78 e-6	13%	✓	✓
Samuel_1021	Gen	10.0 k	628 k	6.28 e-3	9.98 e-3	5.39 e-3	13%	✓	✓
m133-b3	SS	200 k	801 k	2.00 e-5	9.97 e-6	2.01 e-5	17%	✓	✓
scircuit	SS	171 k	959 k	3.28 e-5	1.83 e-3	1.11 e-5	25%	✓	✓
Tina_1021	Gen	10.0 k	998 k	9.98 e-3	9.98 e-3	9.98 e-3	14%	✓	✓
EternityII_Etilde *	SS	10.1 k	1.17 M	5.70 e-4	2.22 e-5	2.81 e-5	13%	✓	
Maragal_7 *	SS	46.8 k	1.20 M	9.65 e-4	8.63 e-4	1.77 e-3	24%	✓	
msc10848	SS	10.8 k	1.23 M	1.05 e-2	2.10 e-2	8.15 e-3	73%	✓	✓
mac_econ_fwd500	SS	207 k	1.27 M	2.99 e-5	2.99 e-3	4.10 e-7	21%	✓	✓
NotreDame_actors *	SS	392 k	1.47 M	2.93 e-5	1.68 e-4	8.87 e-5	16%	✓	
raefsky3	SS	21.2 k	1.49 M	3.31 e-3	3.51 e-2	1.41 e-5	100%	✓	✓
lhr71	SS	70.3 k	1.53 M	3.09 e-4	2.02 e-3	2.58 e-4	61%		✓
2cubes_sphere	SS	101 k	1.65 M	1.60 e-4	6.16 e-3	3.73 e-5	23%	✓	✓
opt1	SS	15.4 k	1.93 M	8.09 e-3	5.44 e-2	1.31 e-3	86%	✓	✓
bcsstk32	SS	44.6 k	2.01 M	1.01 e-3	1.95 e-2	1.35 e-4	75%	✓	✓
cage12	SS	130 k	2.03 M	1.20 e-4	4.23 e-3	5.46 e-5	16%	✓	✓
sme3Db	SS	29.1 k	2.08 M	2.46 e-3	3.08 e-3	2.42 e-3	15%	✓	✓

Continued on next page

Table B.1 – Matrices Used in Every Evaluations and Their Characteristics (Continued)

Name	**Src**	***M***	***nnz***	***d***	d_{IB} (with $w_{hB} = 1021$)	d_{OB} (with $w_{hB} = 1021$)	d_{cd}	**Eval.**	
								Py.	**RTL**
mario002	SS	390 k	2.10 M	1.38 e-5	8.98 e-4	9.17 e-6	18%	✓	✓
Stanford	SS	282 k	2.31 M	2.91 e-5	2.61 e-5	2.91 e-5	13%		✓
rma10	SS	46.8 k	2.37 M	1.08 e-3	1.74 e-2	3.47 e-4	79%	✓	✓
cop20k_A	SS	121 k	2.62 M	1.79 e-4	2.80 e-3	1.34 e-4	31%	✓	✓
ct20stif	SS	52.3 k	2.70 M	9.85 e-4	1.96 e-2	2.38 e-4	74%		✓
filter3D	SS	106 k	2.71 M	2.39 e-4	8.15 e-3	8.50 e-5	22%	✓	✓
ramage02	SS	16.8 k	2.87 M	1.01 e-2	3.77 e-2	6.44 e-3	85%	✓	✓
webbase-1M	SS	1.00 M	3.11 M	3.11 e-6	6.55 e-4	1.77 e-6	25%	✓	✓
amazon0312	SS	401 k	3.20 M	1.99 e-5	6.88 e-4	1.65 e-5	18%	✓	✓
xenon2	SS	157 k	3.87 M	1.56 e-4	1.06 e-2	1.88 e-5	41%		✓
cant	SS	62.5 k	4.01 M	1.03 e-3	3.17 e-2	0.00 e-0	71%	✓	✓
vsp_bcsstk30_[...]	SS	58.3 k	4.03 M	1.18 e-3	1.18 e-3	1.18 e-3	13%	✓	✓
offshore	SS	260 k	4.24 M	6.29 e-5	5.15 e-3	2.26 e-5	19%	✓	✓
gupta2	SS	62.1 k	4.25 M	1.10 e-3	3.88 e-3	1.01 e-3	38%	✓	✓
pdb1HYS	SS	36.4 k	4.34 M	3.28 e-3	4.97 e-2	5.61 e-4	83%	✓	✓
ship_001	SS	34.9 k	4.64 M	3.81 e-3	3.35 e-2	1.99 e-3	83%	✓	✓
web-Google	SS	916 k	5.11 M	6.08 e-6	6.06 e-6	6.08 e-6	13%	✓	✓
roadNet-CA	SS	1.97 M	5.53 M	1.42 e-6	1.16 e-3	2.20 e-7	18%	✓	✓
consph	SS	83.3 k	6.01 M	8.65 e-4	2.27 e-2	3.21 e-4	58%	✓	✓
shipsec1	SS	141 k	7.81 M	3.94 e-4	1.12 e-2	2.35 e-4	76%	✓	✓
Ge87H76	SS	113 k	7.89 M	6.18 e-4	1.38 e-2	3.77 e-4	31%	✓	✓
degme *	SS	186 k	8.13 M	6.64 e-5	1.37 e-5	1.87 e-5	23%	✓	
Ge99H100	SS	113 k	8.45 M	6.62 e-4	1.42 e-2	4.14 e-4	32%	✓	✓
m_t1	SS	97.6 k	9.75 M	1.02 e-3	2.62 e-2	4.89 e-4	81%	✓	✓
x104	SS	108 k	10.2 M	8.66 e-4	2.60 e-2	3.86 e-4	79%	✓	✓
ohne2	SS	181 k	11.1 M	3.36 e-4	8.91 e-3	2.39 e-4	41%		✓
pwtk	SS	218 k	11.6 M	2.45 e-4	2.52 e-2	1.01 e-5	75%	✓	✓
crankseg_2	SS	63.8 k	14.1 M	3.47 e-3	3.70 e-2	2.37 e-3	73%		✓
cit_Patents	SS	3.77 M	16.5 M	1.16 e-6	0.00 e-0	1.16 e-6	13%	✓	

SS: SuiteSparse Matrix Collection [67]; Gen: Generated; *not square

C Thesis Statistics

To provide insight into the scope and duration of this thesis work, we present statistics and key information in this appendix. The work spanned multiple repositories managed through Git version control.

During the first year and a half, extensive exploration was conducted using Python scripts, culminating in the Python model. This work was maintained in the *Python Script* repository. The second half focused on the RTL implementation, distributed across several repositories: *SpDCache RTL* containing the core RTL implementation, *CVA6 RTL* with modifications to the CVA6 RISC-V core for SpDCache's new instructions, *RTL Platform* providing the compilation and simulation environment, and *Software* containing C programs for the *Outer-Product* SpMV kernel. In parallel, the *Benchmarks* repository was developed for efficient benchmarking and performance evaluation, contributed by student Jules DUBOIS during an internship.

Throughout the thesis, several publications were produced: a state-of-the-art survey [I], a paper on initial SpDCache results [II], a patent covering the development concepts around SpDCache [III], a paper on the RTL implementation and analytical model (currently under resubmission), and this manuscript. Figure C.1 illustrates the timeline of all commits across each repository.

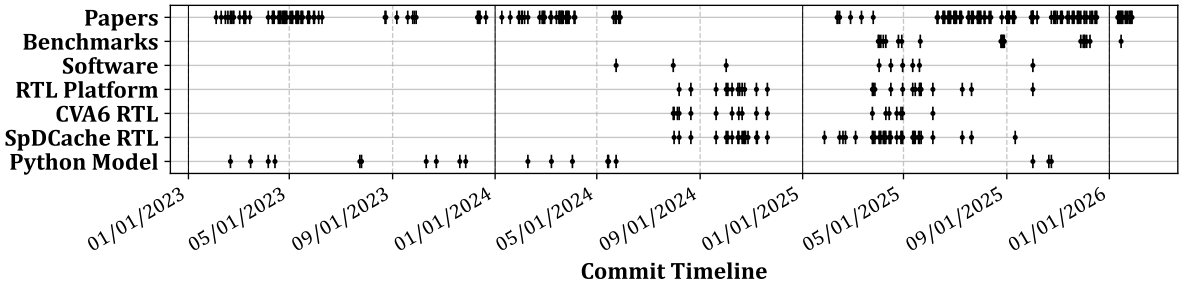


Figure C.1 – Timeline of Commits Across All Working Repositories and Publications

To provide a sense of the work delivered, we present the number of new lines of code for each repository (excluding papers) after final modifications:

- *Python Scripts*: 18,590 lines;
- *SpDCache RTL*: 5808 lines (33,696 → 39,504 total);
- *CVA6 RTL*: 255 lines (36,623 → 36,878 total);
- *RTL Platform*: 56 lines (6156 → 6212 total);
- *Software*: 1554 lines;
- *Benchmarks*: 7642 lines.

Additionally, Figure C.2 shows the programming language distribution across each repository.

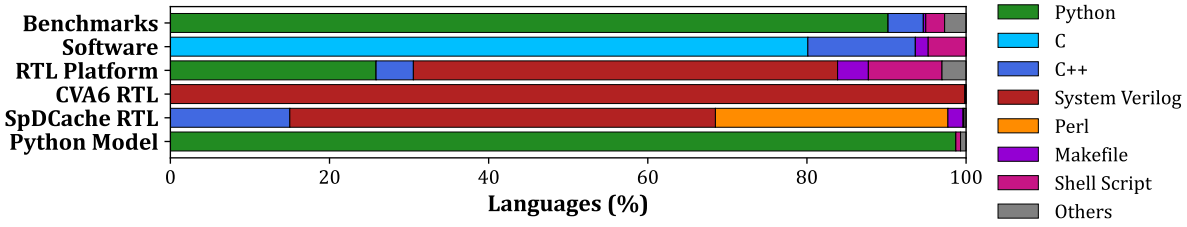


Figure C.2 – Programming Language Distribution Across All Working Repositories

LIST OF FIGURES

LIST OF TABLES

LIST OF ALGORITHMS

LIST OF SOURCE CODE

BIBLIOGRAPHY

[1] James Alfred Ang, Brian W. Barrett, Kyle Bruce Wheeler, and Richard C Murphy. 2010. *Introducing the graph 500.* Technical Report. United States.

[2] ARM IHI 0022. 2023. ARM – AMBA AXI Protocol Specification. (Sept. 2023). https://developer.arm.com/documentation/ihi0022/latest https://developer.arm.com/documentation/ihi0022/latest.

[3] Bahar Asgari, Ramyad Hadidi, Tushar Krishna, Hyesoon Kim, and Sudhakar Yalamanchili. 2020. ALRESCHA: A Lightweight Reconfigurable Sparse-Computation Accelerator. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 249–260. https://doi.org/10.1109/HPCA47549.2020.00029

[4] Venkitesh Ayyar, Evan Weinberg, Richard C. Brower, M.A. Clark, and Mathias Wagner. 2023. Optimizing Staggered Multigrid for Exascale performance. In *Proceedings of The 39th International Symposium on Lattice Field Theory — PoS(LATTICE2022)*, Vol. 430. 335. https://doi.org/10.22323/1.430.0335

[5] Ariful Azad, Aydin Buluç, and John Gilbert. 2015. Parallel Triangle Counting and Enumeration Using Matrix Algebra. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. 804–811. https://doi.org/10.1109/IPDPSW.2015.75

[6] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic Acids Research* 46, 6 (01 2018), e33–e33. https://doi.org/10.1093/nar/gkx1313

[7] Daehyeon Baek, Soojin Hwang, Taekyung Heo, Daehoon Kim, and Jaehyuk Huh. 2021. InnerSP: A Memory Efficient Sparse Matrix Multiplication Accelerator with Locality-Aware Inner Product Processing. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 116–128. https://doi.org/10.1109/PACT52795.2021.00016

[8] Ricardo Baeza-Yates and Alejandro Salinger. 2010. *Fast Intersection Algorithms for Sorted Sequences*. Springer Berlin Heidelberg, Berlin, Heidelberg, 45–61. https://doi.org/10.1007/978-3-642-12476-1_3

[9] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (jun 2017), 25 pages. https://doi.org/10.1145/3085572

[10] Adrián Barredo, Jonathan C. Beard, and Miquel Moretó. 2019. POSTER: SPiDRE: Accelerating Sparse Memory Access Patterns. In *2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 483–484. https://doi.org/10.1109/PACT.2019.00056

[11] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. 1994. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9781611971538

[12] Scott Beamer, Krste Asanović, and David Patterson. 2017. The GAP Benchmark Suite. (2017). https://doi.org/10.48550/arXiv.1508.03619 arXiv:1508.03619 [cs.DC]

[13] Nathan Bell and Michael Garland. 2009. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (Portland, Oregon) *(SC '09)*. Association for Computing Machinery, New York, NY, USA, Article 18, 11 pages. https://doi.org/10.1145/1654059.1654078

[14] Shekhar Borkar. 2007. Thousand core chips: a technology perspective. In *Proceedings of the 44th Annual Design Automation Conference* (San Diego, California) *(DAC '07)*. Association for Computing Machinery, New York, NY, USA, 746–749. https://doi.org/10.1145/1278480.1278667

[15] Achi Brandt and Dorit Ron. 2003. *Multigrid Solvers and Multilevel Optimization Strategies*. Springer US, Boston, MA, 1–69. https://doi.org/10.1007/978-1-4757-3748-6_1

[16] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30, 1 (1998), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X

[17] Aydın Buluç, John Gilbert, and Viral B. Shah. 2011. *13. Implementing Sparse Matrices for Graph Algorithms*. Society for Industrial and Applied Mathematics, Chapter 13, 287–313. https://doi.org/10.1137/1.9780898719918.ch13

[18] Timothy M. Chan. 2007. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing* (San Diego, California, USA) *(STOC '07)*. Association for Computing Machinery, New York, NY, USA, 590–598. https://doi.org/10.1145/1250790.1250877

[19] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery*

and Data Mining (San Francisco, California, USA) *(KDD '16)*. Association for Computing Machinery, New York, NY, USA, 785–794. https://doi.org/10.1145/2939672.2939785

[20] Xinyu Chen, Feng Cheng, Hongshi Tan, Yao Chen, Bingsheng He, Weng-Fai Wong, and Deming Chen. 2022. ThunderGP: Resource-Efficient Graph Processing Framework on FPGAs with HLS. *ACM Trans. Reconfigurable Technol. Syst.* 15, 4, Article 44 (Dec. 2022), 31 pages. https://doi.org/10.1145/3517141

[21] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format Abstraction for Sparse Tensor Algebra Compilers. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 123 (oct 2018), 30 pages. https://doi.org/10.1145/3276493

[22] Vidushi Dadu, Jian Weng, Sihao Liu, and Tony Nowatzki. 2019. Towards General Purpose Acceleration by Exploiting Common Data-Dependence Forms. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 924–939. https://doi.org/10.1145/3352460.3358276

[23] Steven Dalton, Nathan Bell, Luke Olson, and Michael Garland. 2014. Cusp: Generic Parallel Algorithms for Sparse Matrix and Graph Computations. Retrieved May 2022 from http://cusplibrary.github.io/ Version 0.5.1.

[24] Mehmet Deveci, Simon David Hammond, Michael M. Wolf, and Sivasankaran Rajamanickam. 2018. *Sparse Matrix-Matrix Multiplication on Multilevel Memory Architectures: Algorithms and Experiments*. Technical Report. United States. https://doi.org/10.2172/1435688

[25] Xiangyu Dong, Cong Xu, Yuan Xie, and Norman P. Jouppi. 2012. NVSim: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 31, 7 (July 2012), 994–1007. https://doi.org/10.1109/TCAD.2012.2185930

[26] Stijn Dongen. 2000. Graph Clustering by Flow Simulation. *PhD thesis, Center for Math and Computer Science (CWI)* (05 2000). https://www.researchgate.net/publication/27685884_Graph_Clustering_by_Flow_Simulation

[27] Yixiao Du, Yuwei Hu, Zhongchun Zhou, and Zhiru Zhang. 2022. High-Performance Sparse Linear Algebra on HBM-Equipped FPGAs Using HLS: A Case Study on SpMV. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) *(FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 54–64. https://doi.org/10.1145/3490422.3502368

[28] Iain S. Duff, Michael A. Heroux, and Roldan Pozo. 2002. An overview of the sparse basic linear algebra subprograms: The new standard from the BLAS technical forum. *ACM Trans. Math. Softw.* 28, 2 (June 2002), 239–267. https://doi.org/10.1145/567806.567810

[29] D. K. Faddeev and V. N. Faddeeva. 1981. Computational methods of linear algebra. *Journal of Soviet Mathematics* 15, 5 (1981), 531–650. https://doi.org/10.1007/BF01086544

[30] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 253–267. https://doi.org/10.1145/3620666.3651378

[31] Mirko Farina, Usman Ahmad, Ahmad Taha, Hussein Younes, Yusuf Mesbah, Xiao Yu, and Witold Pedrycz. 2024. Sparsity in transformers: A systematic literature review. *Neurocomputing* 582 (2024), 127468. https://doi.org/10.1016/j.neucom.2024.127468

[32] Siying Feng, Jiawen Sun, Subhankar Pal, Xin He, Kuba Kaszyk, Dong-hyeon Park, Magnus Morton, Trevor Mudge, Murray Cole, Michael O'Boyle, Chaitali Chakrabarti, and Ronald Dreslinski. 2021. CoSPARSE: A Software and Hardware Reconfigurable SpMV Framework for Graph Analytics. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 949–954. https://doi.org/10.1109/DAC18074.2021.9586114

[33] Sanford Friedenthal, Alan Moore, and Rick Steiner. 2012. Chapter 18 - Integrating SysML into a Systems Development Environment. In *A Practical Guide to SysML (Second Edition)* (second edition ed.), Sanford Friedenthal, Moore Alan, and Steiner Rick (Eds.). Morgan Kaufmann, Boston, 523–556. https://doi.org/10.1016/B978-0-12-385206-9.00018-1

[34] César Fuguet. 2023. HPDcache: Open-Source High-Performance L1 Data Cache for RISC-V Cores. In *Proceedings of the 20th ACM International Conference on Computing Frontiers* (, Bologna, Italy,) *(CF '23)*. Association for Computing Machinery, New York, NY, USA, 377–378. https://doi.org/10.1145/3587135.3591413

[35] Noble G, Nalesh S, and Kala S. 2023. MOSCON: Modified Outer Product based Sparse Matrix-Matrix Multiplication Accelerator with Configurable Tiles. In *2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID)*. 264–269. https://doi.org/10.1109/VLSID57277.2023.00061

[36] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) *(SC '20)*. IEEE Press, Article 17, 14 pages. https://dl.acm.org/doi/10.5555/3433701.3433723

[37] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU kernels for deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) *(SC '20)*. IEEE Press, Article 17, 14 pages. https://dl.acm.org/doi/10.5555/3433701.3433723

[38] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-Matrix Multiplication. *ACM Comput. Surv.* 55, 12, Article 244 (mar 2023), 36 pages. https://doi.org/10.1145/3571157

[39] Yingxue Gao, Lei Gong, Chao Wang, Teng Wang, Xi Li, and Xuehai Zhou. 2023. Algorithm/Hardware Co-Optimization for Sparsity-Aware SpMM Acceleration of GNNs.

IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 42, 12 (2023), 4763–4776. https://doi.org/10.1109/TCAD.2023.3281714

[40] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *Proceedings of the 8th International Conference on Applied Parallel Computing: State of the Art in Scientific Computing* (Umeå, Sweden) *(PARA'06)*. Springer-Verlag, Berlin, Heidelberg, 260–269. https://link.springer.com/chapter/10.1007/978-3-540-75755-9_32

[41] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2008. A Unified Framework for Numerical and Combinatorial Computing. *Computing in Science & Engineering* 10, 2 (March 2008), 20–25. https://doi.org/10.1109/MCSE.2008.45

[42] Ashish Gondimalla, Noah Chesnut, Mithuna Thottethodi, and T. N. Vijaykumar. 2019. SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO-52)*. Association for Computing Machinery, New York, NY, USA, 151–165. https://doi.org/10.1145/3352460.3358291

[43] Branko Grünbaum and Geoffrey C. Shephard. 1987. *Tilings and Patterns*. W. H. Freeman & Co., New York.

[44] Sumanth Gudaparthi, Sarabjeet Singh, Surya Narayanan, Rajeev Balasubramonian, and Visvesh Sathe. 2022. CANDLES: Channel-Aware Novel Dataflow-Microarchitecture Co-Design for Low Energy Sparse Neural Network Acceleration. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 876–891. https://doi.org/10.1109/HPCA53966.2022.00069

[45] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (sep 1978), 250–269. https://doi.org/10.1145/355791.355796

[46] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. *SIGARCH Comput. Archit. News* 44, 3 (jun 2016), 243–254. https://doi.org/10.1145/3007787.3001163

[47] Václav Hapla, David Horák, and Michal Merta. 2012. Use of Direct Solvers in TFETI Massively Parallel Implementation. In *Proceedings of the 11th International Conference on Applied Parallel and Scientific Computing* (Helsinki, Finland) *(PARA'12)*. Springer-Verlag, Berlin, Heidelberg, 192–205. https://doi.org/10.1007/978-3-642-36803-5_14

[48] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 319–333. https://doi.org/10.1145/3352460.3358275

[49] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture, Fifth Edition: A Quantitative Approach* (5th ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[50] Reza Hojabr, Ali Sedaghati, Amirali Sharifian, Ahmad Khonsari, and Arrvindh Shriraman. 2021. SPAGHETTI: Streaming Accelerators for Highly Sparse GEMM on FPGAs. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 84–96. https://doi.org/10.1109/HPCA51647.2021.00017

[51] Mohammad Hosseinabady and Jose Luis Nunez-Yanez. 2020. A Streaming Dataflow Engine for Sparse Matrix-Vector Multiplication Using High-Level Synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 6 (June 2020), 1272–1285. https://doi.org/10.1109/TCAD.2019.2912923

[52] Yuwei Hu, Yixiao Du, Ecenur Ustun, and Zhiru Zhang. 2021. GraphLily: Accelerating Graph Linear Algebra on HBM-Equipped FPGAs. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD51958.2021.9643582

[53] Paul Xuanyuanliang Huang, Yannis Tsividis, and Mingoo Seok. 2024. SPADES: A 0.54-GFLOPS/W Sparse Matrix Vector Multiplication Accelerator Featuring On-the-Fly GZIP Decompression for 3.36X Reduction in Off-Chip Data Movement. In *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design* (Newport Beach, CA, USA) *(ISLPED '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. https://doi.org/10.1145/3665314.3670811

[54] Eun-Jin Im, Katherine Yelick, and Richard Vuduc. 2004. Sparsity: Optimization Framework for Sparse Matrix Kernels. *The International Journal of High Performance Computing Applications* 18, 1 (2004), 135–158. https://doi.org/10.1177/1094342004041296

[55] Intel. 2003. Intel Math Kernel Library. Retrieved May 2022 from https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html#gs.ylo2j7

[56] Valentin Isaac--Chassande, Adrian Evans, and Yves Durand. 2025. DISPOSITIF DE CALCUL INFORMATIQUE A MEMOIRE CACHE OPTIMISEE POUR LE CALCUL MATRICIEL. https://data.inpi.fr/brevets/EP4650972

[57] Valentin Isaac--Chassande, Adrian Evans, and Yves Durand. 2025. Dispositif de calcul informatique à mémoire cache optimisée pour le calcul matriciel. https://data.inpi.fr/brevets/FR3162293

[58] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. https://doi.org/10.1145/3640542

[59] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. https://doi.org/10.1109/ASAP61560.2024.00012

[60] Satoshi Itoh, Pablo Ordejón, and Richard M. Martin. 1995. Order-N tight-binding molecular dynamics on parallel computers. *Computer Physics Communications* 88, 2 (1995), 173–185. https://doi.org/10.1016/0010-4655(95)00031-A

[61] Hanchen Jin, Zichao Yue, Zhongyuan Zhao, Yixiao Du, Chenhui Deng, Nitish Srivastava, and Zhiru Zhang. 2025. Vesper: A Versatile Sparse Linear Algebra Accelerator With Configurable Compute Patterns. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 5 (May 2025), 1731–1744. https://doi.org/10.1109/TCAD.2024.3496882

[62] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. 2015. Graphs, Matrices, and the GraphBLAS: Seven Good Reasons. *Procedia Computer Science* 51, International Conference On Computational Science, ICCS 2015 (2015), 2453–2462. https://doi.org/10.1016/j.procs.2015.05.353

[63] Jinkwon Kim, Myeongjae Jang, Haejin Nam, and Soontae Kim. 2023. HARP: Hardware-Based Pseudo-Tiling for Sparse Matrix Multiplication Accelerator. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 1148–1162. https://doi.org/10.1145/3613424.3623790

[64] David R. Kincaid, Thomas C. Oppe, and David M. Young. 1989. *ITPACKV 2D user's guide*. Technical Report. United States. https://doi.org/10.2172/7093021

[65] Vladimir Kiriansky, Yunming Zhang, and Saman Amarasinghe. 2016. Optimizing Indirect Memory References with Milk. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation* (Haifa, Israel) *(PACT '16)*. Association for Computing Machinery, New York, NY, USA, 299–312. https://doi.org/10.1145/2967938.2967948

[66] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (oct 2017), 29 pages. https://doi.org/10.1145/3133901

[67] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* 4, 35 (2019), 1244. https://doi.org/10.21105/joss.01244

[68] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. 1991. The Cache Performance and Optimizations of Blocked Algorithms. *SIGPLAN Not.* 26, 4 (apr 1991), 63–74. https://doi.org/10.1145/106973.106981

[69] Jure Leskovec and Andrej Krevl. 2014. *SNAP Datasets: Stanford Large Network Dataset Collection*. Retrieved May 2022 from http://snap.stanford.edu/data

[70] Ruipeng Li, Björn Sjögreen, and Ulrike Meier Yang. 2021. A New Class of AMG Interpolation Methods Based on Matrix-Matrix Multiplications. *SIAM Journal on Scientific Computing* 43, 5 (2021), S540–S564. https://doi.org/10.1137/20M134931X

[71] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 469–480. https://dl.acm.org/doi/10.1145/1669112.1669172

[72] Shiqing Li, Shuo Huai, and Weichen Liu. 2023. An Efficient Gustavson-Based Sparse Matrix–Matrix Multiplication Accelerator on Embedded FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (Dec 2023), 4671–4680. https://doi.org/10.1109/TCAD.2023.3281719

[73] Xiaojie Li, Mingyu Wang, Baiqing Zhong, Haiqiu Huang, Guangjie Cao, and Zhiyi Yu. 2025. C3ache: Towards Hierarchical Cache-Centric Computing for Sparse Matrix Multiplication on GPGPUs. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 1026–1039. https://doi.org/10.1145/3725843.3756077

[74] Zhiyao Li, Jiaxiang Li, Taijie Chen, Dimin Niu, Hongzhong Zheng, Yuan Xie, and Mingyu Gao. 2023. Spada: Accelerating Sparse Matrix Multiplication with Adaptive Dataflow. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 747–761. https://doi.org/10.1145/3575693.3575706

[75] Weifeng Liu and Brian Vinter. 2014. An Efficient GPU General Sparse Matrix-Matrix Multiplication for Irregular Data. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium*. 370–381. https://doi.org/10.1109/IPDPS.2014.47

[76] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. 2020. The gem5 Simulator: Version 20.0+. https://doi.org/10.48550/ARXIV.2007.03152

[77] Xiaobo Lu, Jianbin Fang, Lin Peng, Chun Huang, Zidong Du, Yongwei Zhao, and Zheng

Wang. 2024. Mentor: A Memory-Efficient Sparse-dense Matrix Multiplication Accelerator Based on Column-Wise Product. *ACM Trans. Archit. Code Optim.* 21, 4, Article 79 (Nov. 2024), 25 pages. https://doi.org/10.1145/3688612

[78] Xiaoyang Lu, Boyu Long, Xiaoming Chen, Yinhe Han, and Xian-He Sun. 2024. ACES: Accelerating Sparse Matrix Multiplication with Adaptive Execution Flow and Concurrency-Aware Cache Optimizations. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 71–85. https://doi.org/10.1145/3620666.3651381

[79] J.-C. Luo. 1992. Algorithms for Reducing the Bandwidth and Profile of a Sparse Matrix. *Computers & Structures* 44, 3 (1992), 535–548. https://doi.org/10.1016/0045-7949(92)90386-E

[80] Shengbai Luo, Bo Wang, Yihao Shi, Xueyi Zhang, Qingshan Xue, and Sheng Ma. 2024. Sparm: A Sparse Matrix Multiplication Accelerator Supporting Multiple Dataflows. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 122–130. https://doi.org/10.1109/ASAP61560.2024.00034

[81] Chaofang Ma, Lin Jiang, Zeyu Li, Hanwei Fan, Maolin Wang, Jiang Xu, and Wei Zhang. 2025. ScanNow: A Scan Window-Based Sparse Matrix Multiplication Accelerator Design. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD66269.2025.11240912

[82] Shenghong Ma, Jinwei Xu, Jingfei Jiang, Yaohua Wang, and Dongsheng Li. 2024. Funnel: An Efficient Sparse Attention Accelerator with Multi-Dataflow Fusion. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. 1311–1318. https://doi.org/10.1109/ISPA63168.2024.00176

[83] Liviu Mafteiu-Scai. 2014. The Bandwidths of a Matrix. A Survey of Algorithms. *West University of Timisoara Annals, Timisoara, Romania* LII (12 2014), 183– 223. https://doi.org/10.2478/awutm-2014-0019

[84] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse matrix-matrix multiplication on modern architectures. In *2012 19th International Conference on High Performance Computing*. 1–10. https://doi.org/10.1109/HiPC.2012.6507483

[85] Francisco Muñoz Martínez, Raveesh Garg, Michael Pellauer, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2023. Flexagon: A Multi-Dataflow Sparse-Sparse Matrix Multiplication Accelerator for Efficient DNN Processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 252–265. https://doi.org/10.1145/3582016.3582069

[86] Anurag Mukkara, Nathan Beckmann, and Daniel Sanchez. 2019. PHI: Architectural Support for Synchronization- and Bandwidth-Efficient Commutative Scatter Updates.

In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 1009–1022. https://doi.org/10.1145/3352460.3358254

[87] Francisco Muñoz-Martínez, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2021. STONNE: Enabling Cycle-Level Microarchitectural Simulation for DNN Inference Accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. 201–213. https://doi.org/10.1109/IISWC53511.2021.00028

[88] Yuta Nagahara, Jiale Yan, Kazushi Kawamura, Daichi Fujiki, Masato Motomura, and Thiem Van Chu. 2025. DMSA: An Efficient Architecture for Sparse–Sparse Matrix Multiplication Based on Distribute-Merge Product Dataflow. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 7 (July 2025), 1858–1871. https://doi.org/10.1109/TVLSI.2025.3558895

[89] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydın Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *Workshop Proceedings of the 47th International Conference on Parallel Processing* (Eugene, OR, USA) *(ICPP Workshops '18)*. Association for Computing Machinery, New York, NY, USA, Article 34, 10 pages. https://doi.org/10.1145/3229710.3229720

[90] NVIDIA. 2014. cuSPARSE Library. Retrieved May 2022 from https://developer.nvidia.com/cusparse

[91] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 724–736. https://doi.org/10.1109/HPCA.2018.00067

[92] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 27–40. https://doi.org/10.1145/3079856.3080254

[93] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 683–698. https://doi.org/10.1145/3620665.3640426

[94] Gerald Penn. 2006. Efficient transitive closure of sparse matrices over closed semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81. https://doi.org/10.1016/j.tcs.2005.11.008

[95] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. https://doi.org/10.1109/HPCA47549.2020.00015

[96] Michael O Rabin and Vijay V Vazirani. 1989. Maximum matchings in general graphs through randomization. *Journal of Algorithms* 10, 4 (1989), 557–567. https://doi.org/10.1016/0196-6774(89)90005-9

[97] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459. https://doi.org/10.1109/ISCA45697.2020.00045

[98] Oleg I. Ryabkov. 2022. Implementation of the Algebraic Multigrid Solver Designed for Graphics Processing Units Based on the AMGCL Framework. In *Parallel Computational Technologies*, Leonid Sokolinsky and Mikhail Zymbler (Eds.), Vol. 1618. Springer International Publishing, Cham, 131–142. https://doi.org/10.1007/978-3-031-11623-0_10

[99] Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (second ed.). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898718003

[100] Fazle Sadi, Joe Sweeney, Tze Meng Low, James C. Hoe, Larry Pileggi, and Franz Franchetti. 2019. Efficient SpMV Operation for Large and Highly Sparse Matrices Using Scalable Multi-Way Merge Parallelization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 347–358. https://doi.org/10.1145/3352460.3358330

[101] Daniel Sanchez and Christos Kozyrakis. 2013. ZSim: Fast and Accurate Microarchitectural Simulation of Thousand-Core Systems. In *Proceedings of the 40th Annual International Symposium on Computer Architecture* (Tel-Aviv, Israel) *(ISCA '13)*. Association for Computing Machinery, New York, NY, USA, 475–486. https://doi.org/10.1145/2485922.2485963

[102] Conrad Sanderson and Ryan Curtin. 2016. Armadillo: a template-based C++ library for linear algebra. *Journal of Open Source Software* 1, 2 (2016), 26. https://doi.org/10.21105/joss.00026

[103] P. Scheffler, F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini. 2023. Sparse Stream Semantic Registers: A Lightweight ISA Extension Accelerating General Sparse Linear

Algebra. *IEEE Transactions on Parallel and Distributed Systems* 34, 12 (dec 2023), 3147–3161. https://doi.org/10.1109/TPDS.2023.3322029

[104] Alan Jay Smith. 1982. Cache Memories. *ACM Comput. Surv.* 14, 3 (Sept. 1982), 473–530. https://doi.org/10.1145/356887.356892

[105] Shaden Smith, Jee W. Choi, Jiajia Li, Richard Vuduc, Jongsoo Park, Xing Liu, and George Karypis. 2017. *FROSTT: The Formidable Repository of Open Sparse Tensors and Tools*. Retrieved May 2022 from http://frostt.io/

[106] Linghao Song, Yuze Chi, Atefeh Sohrabizadeh, Young-kyu Choi, Jason Lau, and Jason Cong. 2022. Sextans: A Streaming Accelerator for General-Purpose Sparse-Matrix Dense-Matrix Multiplication. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) *(FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 65–77. https://doi.org/10.1145/3490422.3502357

[107] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. 2018. GraphR: Accelerating Graph Processing Using ReRAM. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 531–543. https://doi.org/10.1109/HPCA.2018.00052

[108] Sriseshan Srikanth, Anirudh Jain, Thomas M. Conte, Erik P. Debenedictis, and Jeanine Cook. 2021. SortCache: Intelligent Cache Management for Accelerating Sparse Data Workloads. *ACM Trans. Archit. Code Optim.* 18, 4, Article 56 (sep 2021), 24 pages. https://doi.org/10.1145/3473332

[109] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 766–780. https://doi.org/10.1109/MICRO50266.2020.00068

[110] Nitish Srivastava, Hanchen Jin, Shaden Smith, Hongbo Rong, David Albonesi, and Zhiru Zhang. 2020. Tensaurus: A Versatile Accelerator for Mixed Sparse-Dense Tensor Computations. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 689–702. https://doi.org/10.1109/HPCA47549.2020.00062

[111] Wenhao Sun, Wendi Sun, Song Chen, and Yi Kang. 2025. IOPS: A Unified SpMM Accelerator Based on Inner-Outer-Hybrid Product. *IEEE Trans. Comput.* 74, 7 (July 2025), 2210–2222. https://doi.org/10.1109/TC.2025.3558013

[112] Narayanan Sundaram, Nadathur Satish, Md Mostofa Ali Patwary, Subramanya R. Dulloor, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. 2015. GraphMat: high performance graph analytics made productive. *Proc. VLDB Endow.* 8, 11 (July 2015), 1214–1225. https://doi.org/10.14778/2809974.2809983

[113] Enhao Tang, Shun Li, Hao Zhou, Guohao Dai, Jun Lin, and Kun Wang. 2025. AiSpGEMM: Accelerating Imbalanced SpGEMM on FPGAs with Flexible Interconnect and Intra-row Parallel Merging. In *2025 Design, Automation & Test in Europe Conference (DATE)*. 1–7. https://doi.org/10.23919/DATE64628.2025.10993117

[114] O. Temam and W. Jalby. 1992. Characterizing the behavior of sparse algorithms on caches. In *Supercomputing '92:Proceedings of the 1992 ACM/IEEE Conference on Supercomputing*. 578–587. https://doi.org/10.1109/SUPERC.1992.236646

[115] Reginald P. Tewarson. 1973. *Sparse matrices*. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.

[116] Richard Vuduc, James W Demmel, and Katherine A Yelick. 2005. OSKI: A library of automatically tuned sparse matrix kernels. *Journal of Physics: Conference Series* 16, 1 (jan 2005), 521. https://doi.org/10.1088/1742-6596/16/1/071

[117] Chen Wang, Chuan Xu, and Abdel Lisser. 2014. Bandwidth Minimization Problem. In *10th International Conference on MOdeling, Optimization and SIMulation - MOSIM14*. Nancy, France. https://hal.science/hal-01166658

[118] Hongyi Wang, Kai Zhong, Haoyu Zhang, Shulin Zeng, Zhenhua Zhu, Xinhao Yang, Shuang Wang, Guohao Dai, Huazhong Yang, and Yu Wang. 2024. DySpMM: From Fix to Dynamic for Sparse Matrix-Matrix Multiplication Accelerators. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) *(DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 96, 6 pages. https://doi.org/10.1145/3649329.3657362

[119] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. 2016. Gunrock: a high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Barcelona, Spain) *(PPoPP '16)*. Association for Computing Machinery, New York, NY, USA, Article 11, 12 pages. https://doi.org/10.1145/2851141.2851145

[120] Yuke Wang, Boyuan Feng, Zheng Wang, Guyue Huang, and Yufei Ding. 2023. TC-GNN: Bridging Sparse GNN Computation and Dense Tensor Cores on GPUs. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 149–164. https://www.usenix.org/conference/atc23/presentation/wang-yuke

[121] Andrew Waterman, Yunsup Lee, David Patterson, and Krste Asanović. 2016. The RISC-V Instruction Set Manual, Volume I: User-Level ISA Version 2.1. (May 2016). https://riscv.org/specifications/ratified/ https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.pdf.

[122] Lucas Wilkinson, Kazem Cheshmi, and Maryam Mehri Dehnavi. 2023. Register Tiling for Unstructured Sparsity in Neural Network Inference. *Proc. ACM Program. Lang.* 7, PLDI, Article 188 (June 2023), 26 pages. https://doi.org/10.1145/3591302

[123] Samuel Williams, Leonid Oliker, Richard Vuduc, John Shalf, Katherine Yelick, and James Demmel. 2007. Optimization of sparse matrix-vector multiplication on emerging multicore platforms. In *SC '07: Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*. 1–12. https://doi.org/10.1145/1362622.1362674

[124] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the Memory Wall: Implications of the Obvious. *SIGARCH Comput. Archit. News* 23, 1 (mar 1995), 20–24. https://doi.org/10.1145/216585.216588

[125] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling Cost-Effective and Highly-Efficient Large Generative Model Inference with Unstructured Sparsity. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 211–224. https://doi.org/10.14778/3626292.3626303

[126] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. SpaceA: Sparse Matrix Vector Multiplication on Processing-in-Memory Accelerator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 570–583. https://doi.org/10.1109/HPCA51647.2021.00055

[127] Ichitaro Yamazaki and Xiaoye S. Li. 2011. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *High Performance Computing for Computational Science – VECPAR 2010*, José M. Laginha M. Palma, Michel Daydé, Osni Marques, and João Correia Lopes (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 421–434. https://doi.org/10.1007/978-3-642-19328-6_38

[128] Enxin Yi, Jiarui Bai, Yijie Nie, Dan Niu, Zhou Jin, and Weifeng Liu. 2025. Leda: Leveraging Tiling Dataflow to Accelerate SpMM on HBM-Equipped FPGAs for GNNs. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design* (Newark Liberty International Airport Marriott, New York, NY, USA) *(ICCAD '24)*. Association for Computing Machinery, New York, NY, USA, Article 215, 9 pages. https://doi.org/10.1145/3676536.3676773

[129] Jiangnan Yu, Yang Fan, Hanfei Wang, Yuxuan Qiao, Zheng Wu, Xiankui Xiong, Xiao Yao, Haidong Yao, and Yecheng Zhang. 2024. FullSparse: A Sparse-Aware GEMM Accelerator with Online Sparsity Prediction. In *Proceedings of the 21st ACM International Conference on Computing Frontiers* (Ischia, Italy) *(CF '24)*. Association for Computing Machinery, New York, NY, USA, 298–301. https://doi.org/10.1145/3649153.3649180

[130] Orestis Zachariadis, Nitin Satpute, Juan Gómez-Luna, and Joaquín Olivares. 2020. Accelerating sparse matrix–matrix multiplication with GPU Tensor Cores. *Computers & Electrical Engineering* 88 (2020), 106848. https://doi.org/10.1016/j.compeleceng.2020.106848

[131] Florian Zaruba and Luca Benini. 2019. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (Nov 2019), 2629–2640. https://doi.org/10.1109/TVLSI.2019.2926114

[132] Chao Zhang, Maximilian Bremer, Cy Chan, John Shalf, and Xiaochen Guo. 2022. ASA: Accelerating Sparse Accumulation in Column-Wise SpGEMM. *ACM Trans. Archit. Code Optim.* 19, 4, Article 49 (sep 2022), 24 pages. https://doi.org/10.1145/3543068

[133] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) *(ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 687–701. https://doi.org/10.1145/3445814.3446702

[134] Guowei Zhang, Webb Horn, and Daniel Sanchez. 2015. Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 13–25. https://doi.org/10.1145/2830772.2830774

[135] Hongrui Zhang, Yunan Zhang, and Hung-Wei Tseng. 2025. RTSpMSpM: Harnessing Ray Tracing for Efficient Sparse Matrix Computations. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 359–373. https://doi.org/10.1145/3695053.3731072

[136] Xiaoyu Zhang, Zerun Li, Rui Liu, Xiaoming Chen, and Yinhe Han. 2024. GAS: General-Purpose In-Memory-Computing Accelerator for Sparse Matrix Multiplication. *IEEE Trans. Comput.* 73, 6 (June 2024), 1427–1441. https://doi.org/10.1109/TC.2024.3371790

[137] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 261–274. https://doi.org/10.1109/HPCA47549.2020.00030

[138] Ningxin Zheng, Bin Lin, Quanlu Zhang, Lingxiao Ma, Yuqing Yang, Fan Yang, Yang Wang, Mao Yang, and Lidong Zhou. 2022. SparTA: Deep-Learning Model Sparsity via Tensor-with-Sparsity-Attribute. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 213–232. https://www.usenix.org/conference/osdi22/presentation/zheng-ningxin

[139] Shijie Zhou, Rajgopal Kannan, Viktor K. Prasanna, Guna Seetharaman, and Qing Wu. 2019. HitGraph: High-throughput Graph Processing Framework on FPGA. *IEEE Transactions on Parallel and Distributed Systems* 30, 10 (Oct 2019), 2249–2264. https://doi.org/10.1109/TPDS.2019.2910068

[140] Xiaowei Zhu, Wentao Han, and Wenguang Chen. 2015. GridGraph: large-scale graph processing on a single machine using 2-level hierarchical partitioning. In *Proceedings of the 2015 USENIX Conference on Usenix Annual Technical Conference* (Santa Clara, CA) *(USENIX ATC '15)*. USENIX Association, USA, 375–386. https://dl.acm.org/doi/10.5555/2813767.2813795

CONTENTS